

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO ĐỒ ÁN CHUYÊN NGÀNH - GD1

XÂY DỰNG HỆ THỐNG QUẢN LÝ THƯ VIỆN TRỰC TUYẾN
TÍCH HỢP AI HỖ TRỢ TÌM KIẾM VÀ GỢI Ý SÁCH

ProjectID: HK251-DAGD1-512
NHÓM: HAI ANH TRAI QUẢNG TRỊ

Giảng viên hướng dẫn: TS. Trương Tuấn Anh
CN. Dương Huỳnh Anh Đức
Thành viên: Võ Quang Thắng - 2213214
Hồ Sỹ Thắng - 2213188

TPHCM, tháng 10 năm 2025

LỜI CẢM ƠN

Trong suốt thời gian học tập và nghiên cứu tại **Trường Đại học Bách Khoa - Đại học Quốc gia Thành phố Hồ Chí Minh**, chúng em đã nhận được sự quan tâm, giảng dạy và hướng dẫn tận tình từ quý thầy cô trong khoa. Đồ án chuyên ngành này là minh chứng cho sự nỗ lực không ngừng của chúng em trong việc vận dụng những kiến thức đã được trang bị để giải quyết các vấn đề thực tiễn trong lĩnh vực công nghệ thông tin.

Trước tiên, chúng em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến **TS. Trương Tuấn Anh**, người thầy đã tận tâm hướng dẫn, định hướng và đồng hành cùng chúng em trong suốt quá trình thực hiện đồ án. Thầy không chỉ truyền đạt kiến thức chuyên môn mà còn định hướng phương pháp nghiên cứu khoa học, giúp chúng em có cái nhìn sâu sắc hơn về vấn đề đang giải quyết. Những lời khuyên, góp ý quý báu của thầy là động lực to lớn giúp chúng em vượt qua mọi khó khăn, thử thách để hoàn thành tốt đồ án này.

Chúng em cũng xin gửi lời cảm ơn chân thành đến **CN. Dương Huỳnh Anh Đức** đã đồng hành, hỗ trợ và đóng góp nhiều ý kiến quý báu trong quá trình thực hiện đồ án, giúp chúng em có thêm góc nhìn thực tiễn và áp dụng hiệu quả các giải pháp kỹ thuật.

Chúng em xin bày tỏ lòng biết ơn sâu sắc đến toàn thể quý thầy cô **Khoa Khoa học và Kỹ thuật Máy tính** đã tận tình giảng dạy, truyền đạt kiến thức và kinh nghiệm quý báu trong suốt những năm học vừa qua. Nền tảng kiến thức vững chắc mà các thầy cô đã trang bị là hành trang vô cùng quý giá để chúng em tự tin bước vào môi trường làm việc thực tế.

Chúng em cũng xin cảm ơn gia đình, bạn bè đã luôn bên cạnh, động viên và tạo điều kiện tốt nhất để chúng em có thể hoàn thành tốt chương trình học tập cũng như đồ án chuyên ngành này.

Do thời gian và kinh nghiệm có hạn, đồ án không tránh khỏi những thiếu sót. Chúng em rất mong nhận được sự góp ý, chỉ bảo của quý thầy cô để đồ án được hoàn thiện hơn và chúng em có thêm kinh nghiệm quý báu cho công việc sau này.

Xin chân thành cảm ơn!

Tp. Hồ Chí Minh, tháng 10 năm 2025

Nhóm sinh viên thực hiện

TÓM TẮT

Đồ án chuyên ngành này tập trung vào việc nghiên cứu, thiết kế và xây dựng **Hệ thống Quản lý Thư viện Trực tuyến tích hợp AI**, nhằm giải quyết các hạn chế của các hệ thống thư viện truyền thống như khả năng tìm kiếm kém hiệu quả và thiếu sự cá nhân hóa. Hệ thống được phát triển với mục tiêu số hóa toàn diện quy trình vận hành thư viện, đồng thời ứng dụng trí tuệ nhân tạo để nâng cao trải nghiệm đọc và khám phá tri thức của người dùng.

Về mặt kỹ thuật, hệ thống được xây dựng dựa trên kiến trúc **Modular Monolith** với Clean Architecture + Domain-Driven Design cho từng module, đảm bảo tính bảo mật và khả năng mở rộng. Backend sử dụng **Spring Boot (Java)** để xử lý các nghiệp vụ phức tạp, kết hợp với Frontend **React + Vite** mang lại giao diện hiện đại, tốc độ build nhanh và triển khai đơn giản qua NGINX. Các chức năng chính bao gồm: quản lý tài nguyên sách, quản lý độc giả, quy trình mượn trả tự động, kiểm soát phí phạt và hệ thống báo cáo thống kê trực quan cho thủ thư.

Điểm đột phá của đồ án là việc tích hợp sâu các công nghệ AI/NLP tiên tiến. Cụ thể, hệ thống ứng dụng mô hình **Sentence-BERT (SBERT)** để thực hiện tìm kiếm ngữ nghĩa (Semantic Search), giúp xử lý các truy vấn tiếng Việt phức tạp mà không phụ thuộc vào từ khóa chính xác. Song song đó, hệ thống gợi ý sách sử dụng phương pháp lai (**Hybrid Approach**), kết hợp giữa Lọc cộng tác (Collaborative Filtering) và Lọc dựa trên nội dung (Content-Based), giúp đưa ra các đề xuất sách phù hợp nhất dựa trên lịch sử và hành vi của từng độc giả.

Kết quả của đồ án là một hệ thống phần mềm hoàn chỉnh, có giao diện thân thiện, đáp ứng đầy đủ các yêu cầu nghiệp vụ của một thư viện đại học. Hệ thống đã chứng minh được tính khả thi trong việc kết hợp giữa quản lý truyền thống và công nghệ thông minh, mang lại giá trị thực tiễn cao trong việc thúc đẩy văn hóa đọc và chuyển đổi số trong môi trường giáo dục.

PHÂN CÔNG CÔNG VIỆC

STT	Nội dung công việc / Chương mục	Sinh viên thực hiện	Ghi chú
1	Nghiên cứu tổng quan và Cơ sở lý thuyết - Chương 1: Giới thiệu dự án - Chương 2: Kiến thức và công nghệ nền tảng - Chương 3: Công trình liên quan	Võ Quang Thắng Hồ Sỹ Thắng	Làm chung
2	Phân tích yêu cầu và Hệ thống đề xuất - Chương 4: Hệ thống đề xuất - Phân tích Use Case, Activity Diagram - Thiết kế cơ sở dữ liệu (ERD, Schema)	Võ Quang Thắng Hồ Sỹ Thắng	Làm chung
3	Thiết kế kiến trúc hệ thống (Chương 5) - Kiến trúc Modular Monolith + Clean Architecture - Thiết kế AI Service (FastAPI) - Luồng giao tiếp giữa các thành phần	Võ Quang Thắng Hồ Sỹ Thắng	Làm chung
4	Hiện thực Back-end (Spring Boot) - Xây dựng REST API và xử lý nghiệp vụ - Quản lý giao dịch, Domain Events - Xác thực và phân quyền (JWT)	Võ Quang Thắng	Phụ trách chính
5	Hiện thực module AI (Python FastAPI) - Tìm kiếm ngữ nghĩa (SBERT + pgvector) - Hệ thống gợi ý lai ghép (LightFM) - Pipeline OCR và Embedding tự động	Hồ Sỹ Thắng	Phụ trách chính
6	Hiện thực Front-end (Next.js) - Thiết kế giao diện người dùng (UI/UX) - Xây dựng các trang và luồng tương tác - Tích hợp API Back-end và module AI	Võ Quang Thắng Hồ Sỹ Thắng	Làm chung
7	Kiểm thử hệ thống (Chương 6) - Kiểm thử đơn vị: JUnit 5 + Mockito (Backend) - Kiểm thử tích hợp: Testcontainers - Kiểm thử API: Postman Collection - Kiểm thử hiệu năng: k6 Load Testing	Võ Quang Thắng	Phụ trách chính
8	Kiểm thử và Đánh giá module AI - Xây dựng bộ dữ liệu kiểm thử (Golden Dataset) - Đánh giá độ chính xác: Precision@K, Recall@K - Kiểm thử Jest cho Frontend	Hồ Sỹ Thắng	Phụ trách chính

Tiếp tục ở trang sau

Bảng – Tiếp theo từ trang trước

STT	Nội dung công việc / Chương mục	Sinh viên thực hiện	Ghi chú
9	Triển khai hệ thống (Chương 7) - Đóng gói Docker (Multi-stage Build) - Cấu hình Docker Compose, NGINX Reverse Proxy - Triển khai lên VPS, cấu hình SSL/TLS	Võ Quang Thắng	Phụ trách chính
10	Tổng kết và Hoàn thiện báo cáo - Chương 8: Tổng kết - Tổng hợp báo cáo, chỉnh sửa format - Chuẩn bị slide thuyết trình	Võ Quang Thắng Hồ Sỹ Thắng	Làm chung

Mục lục

1	Giới thiệu	12
1.1	Động cơ của đề tài	12
1.2	Mục tiêu	12
1.3	Phạm vi	13
1.4	Ý nghĩa	13
1.4.1	Ý nghĩa thực tiễn	13
1.4.2	Ý nghĩa khoa học	13
1.5	Cấu trúc báo cáo	14
2	Kiến thức và công nghệ nền tảng	15
2.1	Các Lĩnh vực Công nghệ Chính	15
2.2	Nền tảng Kiến thức Khoa học	15
2.3	Kết chương	16
3	Công trình liên quan	17
3.1	Các công trình nghiên cứu khoa học	17
3.2	Các hệ thống và phần mềm tương tự	17
3.3	Kết chương và xác định khoảng trống nghiên cứu	18
4	Hệ thống đề xuất	19
4.1	Mô tả hệ thống	19
4.2	Hướng tiếp cận thiết kế	19
4.3	Yêu cầu	20
4.3.1	Yêu cầu chức năng	20
4.3.2	Yêu cầu phi chức năng	20
4.3.3	Yêu cầu dữ liệu	21
4.4	Kết chương	21
5	Phân tích và Thiết kế	22
5.1	Phân tích	22
5.1.1	Mô hình hóa tương tác (Biểu đồ Usecase)	22
5.1.2	Mô hình hoá quy trình nghiệp vụ (Activity diagram)	26
5.2	Giải pháp công nghệ	37
5.2.1	Front-end	37
5.2.2	Back-end	38
5.3	Ứng dụng AI vào hệ thống	39
5.3.1	Module 1: Tìm kiếm ngữ nghĩa (Semantic Search)	40
5.3.2	Module 2: Hệ thống gợi ý (Recommendation)	42
5.4	Thiết kế	46
5.4.1	Kiến trúc hệ thống	46

5.4.2	Thiết kế cơ sở dữ liệu	49
5.4.3	Thiết kế chi tiết bên trong	51
5.4.4	Thiết kế giao diện	65
6	Hiện thực hệ thống	68
6.1	Cấu trúc mã nguồn	68
6.1.1	Mã nguồn Frontend (Giao diện người dùng)	68
6.1.2	Mã nguồn Backend (Máy chủ ứng dụng)	69
6.1.3	Mã nguồn AI Module (Trí tuệ nhân tạo)	70
6.2	Danh sách hiện thực chức năng theo API	71
6.2.1	Đăng ký và Xác thực tài khoản	71
6.2.2	Đăng nhập hệ thống	76
6.2.3	Đăng nhập bằng Google	78
6.2.4	Khôi phục mật khẩu	80
6.2.5	Thanh điều hướng	82
6.2.6	Quản lý thông tin cá nhân	83
6.2.7	Bảng điều khiển Độc giả	88
6.2.8	Khám phá và Tìm kiếm tài liệu	90
6.2.9	Danh sách sách yêu thích và Thông báo	93
6.2.10	Nghiệp vụ Mượn sách	95
6.2.11	Nghiệp vụ Đặt trước sách	97
6.2.12	Đánh giá và Nhận xét ấn phẩm	102
6.2.13	Lịch sử tìm kiếm	103
6.2.14	Hệ thống Thông báo và Nhắc nhở (Real-time Notifications)	105
6.2.15	Nghiệp vụ Trả sách và Xử lý sự cố	107
6.2.16	Khám phá và Đề xuất ấn phẩm	110
6.2.17	Bảng điều khiển Thủ thư	111
6.2.18	Quản lý Bản sao	113
6.2.19	Quản lý Ấn phẩm	113
6.2.20	Tạo mới ấn phẩm	116
6.2.21	Nghiệp vụ Trả sách và Xử lý sự cố	119
6.2.22	Nghiệp vụ Thu phí phạt	122
6.2.23	Quản lý và Xem danh sách mượn trả	125
7	Kiểm thử	129
7.1	Kiểm thử phần mềm và chiến lược kiểm thử	129
7.1.1	Kiểm thử đơn vị	129
7.1.2	Kiểm thử tích hợp	129
7.1.3	Kiểm thử API	129
7.1.4	Kiểm thử hiệu năng	129
7.2	Kiểm thử đơn vị	130
7.2.1	Framework kiểm thử Backend (JUnit 5 + Mockito)	130
7.2.2	Framework kiểm thử Frontend (Jest)	130
7.2.3	Thiết kế test case	130
7.2.4	Danh sách test case	131
7.2.5	Kết quả	131
7.3	Kiểm thử tích hợp	133
7.3.1	Chiến lược kiểm thử	133

7.3.2	Thiết kế test case	134
7.3.3	Kết quả	134
7.4	Kiểm thử API	135
7.4.1	Giới thiệu công cụ	135
7.4.2	Bộ test collection	135
7.4.3	Kết quả	136
7.5	Kiểm thử hiệu năng	136
7.5.1	Giới thiệu công cụ	136
7.5.2	Cấu hình kiểm thử	136
7.5.3	Kết quả	137
7.6	Tổng kết	137
8	Triển khai hệ thống	139
8.1	Container hóa với Docker	139
8.1.1	Giới thiệu Docker	139
8.1.2	Cấu hình môi trường phát triển cục bộ	139
8.2	Dockerfile đa giai đoạn	140
8.2.1	Backend Dockerfile	140
8.2.2	Frontend Dockerfile	140
8.3	Nginx — Reverse Proxy	140
8.3.1	Vai trò của Nginx	140
8.3.2	Cấu hình định tuyến	140
8.4	Môi trường sản xuất	141
8.4.1	Docker Compose sản xuất	141
8.4.2	Quản lý biến môi trường và bảo mật	141
8.4.3	Flyway Migration tự động	142
8.4.4	Sơ đồ kiến trúc triển khai	142
8.5	Quy trình triển khai	142
8.6	Hạ tầng mạng: VPS, Domain và HTTPS	143
8.6.1	Máy chủ ảo (VPS)	143
8.6.2	Tên miền (Domain)	144
8.6.3	HTTPS và SSL/TLS Certificate	144
8.6.4	Tổng chi phí vận hành	145
9	Tổng kết	146
9.1	Kết quả đạt được	146
9.1.1	Kiến trúc phần mềm	146
9.1.2	Tích hợp Trí tuệ nhân tạo	146
9.1.3	Kiểm thử	146
9.1.4	Triển khai hệ thống	147
9.2	Nhận xét và đánh giá	147
10	Tài nguyên và Mã nguồn dự án	148
	Tài liệu tham khảo	149

Danh sách hình vẽ

5.1	Biểu đồ Usecase tổng quát của Hệ thống Quản lý Thư viện	22
5.2	Đăng ký tài khoản — Activity Diagram	27
5.3	Đăng nhập tài khoản — Activity Diagram	28
5.4	Tìm kiếm sách — Activity Diagram	29
5.5	Gợi ý sách — Activity Diagram	30
5.6	Mượn sách — Activity Diagram	31
5.7	Xử lý hàng đợi — Activity Diagram	32
5.8	Gia hạn sách — Activity Diagram	33
5.9	Trả sách — Activity Diagram	34
5.10	Đánh giá và bình luận sách — Activity Diagram	35
5.11	Tạo sách với AI — Activity Diagram	36
5.12	Kiến trúc tích hợp hệ thống	40
5.13	Quy trình Vector hóa nội dung sách	41
5.14	Luồng thực thi tìm kiếm ngữ nghĩa	42
5.15	Lược đồ thuật toán phân luồng gợi ý (Switching Hybrid)	45
5.16	So sánh các đặc tính của các kiểu kiến trúc (Nguồn: Mark Richards, Neal Ford, <i>Fundamentals of Software Architecture</i> , O'Reilly, 2nd ed., 2020) [1].	47
5.17	Sơ đồ Quan hệ Thực thể (ERD) tổng thể của hệ thống	50
5.18	Ánh xạ cơ sở dữ liệu quan hệ ở dạng tổng quát	51
5.19	Sơ đồ kiến trúc tổng thể hệ thống	52
5.20	Các Bounded Contexts trong hệ thống	53
5.21	Clean architecture (R. C. Martin, <i>Clean Architecture: A Craftsman's Guide to Software Structure and Design</i> , Pearson Education, 2018 [2])	54
5.22	Cấu trúc Clean Architecture kết hợp Domain-Driven Design trong một Module	55
5.23	Sơ đồ luồng dữ liệu và Kiến trúc chi tiết AI Service	59
5.24	Kiến trúc chi tiết AI Service và Luồng dữ liệu	60
5.25	Giao diện Public: Trang chủ và Tìm kiếm	65
5.26	Giao diện Chi tiết sách	66
5.27	Giao diện Độc giả: Dashboard và Quản lý mượn sách	66
5.28	Giao diện Thủ thư: Quản lý danh mục và Lưu thông	67
6.1	Cấu trúc thư mục mã nguồn Frontend	68
6.2	Cấu trúc thư mục mã nguồn Backend (Modular Monolith)	69
6.3	Cấu trúc thư mục mã nguồn phân hệ AI	70
6.4	Giao diện Đăng ký tài khoản hệ thống SmartLibrary	71
6.5	Nội dung Email yêu cầu xác nhận tài khoản	74
6.6	Giao diện thông báo kích hoạt tài khoản thành công	75
6.7	Giao diện Đăng nhập hệ thống SmartLibrary	76
6.8	Giao diện lựa chọn tài khoản Google để xác thực	78

6.9	Giao diện hoàn tất hồ sơ cho lần đăng nhập đầu tiên	78
6.10	Giao diện nhập email yêu cầu khôi phục	80
6.11	Thông báo kiểm tra hộp thư	80
6.12	Nội dung thư điện tử chứa đường dẫn đặt lại mật khẩu	80
6.13	Giao diện thiết lập mật khẩu mới	81
6.14	Thông báo cập nhật thành công	81
6.15	Giao diện Thanh điều hướng chính của hệ thống	82
6.16	Giao diện Quản lý thông tin cá nhân (Profile Dashboard)	83
6.17	Menu tùy chọn thao tác với ảnh đại diện	84
6.18	Hộp thoại chọn tệp tin	85
6.19	Ảnh đại diện sau khi cập nhật	85
6.20	Giao diện Bảng điều khiển dành cho Sinh viên/Độc giả	88
6.21	Giao diện Tìm kiếm và Bộ lọc nâng cao	90
6.22	Giao diện Thông tin tổng quan và Tình trạng mượn trả	91
6.23	Khối dữ liệu Mô tả nội dung và Thông số xuất bản	92
6.24	Giao diện quản lý danh sách sách đã lưu của độc giả	93
6.25	Nút thao tác lưu sách tại trang chi tiết ấn phẩm	93
6.26	Thông báo đẩy trên hệ thống web	94
6.27	Thư điện tử nhắc nhở nhận sách	94
6.28	Giao diện chọn bản sao và thực hiện mượn sách	95
6.29	Phiếu mượn điện tử (Mã QR)	95
6.30	Thư điện tử thông báo lịch hẹn	95
6.31	Tra cứu qua Mã giao dịch / Quét QR	96
6.32	Tra cứu thủ công qua Mã SV và Mã vạch sách	96
6.33	Màn hình thông báo giao sách thành công	96
6.34	Giao diện tùy chọn cơ sở nhận sách khi đặt trước	98
6.35	Trạng thái Đang chờ xếp hàng	98
6.36	Trạng thái Sách sẵn sàng nhận	99
6.37	Thông báo cập nhật ngay trên trang web	99
6.38	Thư điện tử thông báo sách đã về	99
6.39	Phiếu nhận sách điện tử (Mã QR)	100
6.40	Giao diện Thủ thư xác nhận giao sách đặt trước	100
6.41	Giao diện Đánh giá và Nhận xét từ cộng đồng độc giả	102
6.42	Giao diện gợi ý từ khóa tìm kiếm gần đây tại Trang chủ	103
6.43	Danh sách thả xuống quản lý lịch sử tại Trang tìm kiếm	103
6.44	Giao diện quản lý Thông báo và Nhắc nhở dành cho Độc giả	105
6.45	Tập hợp các điểm cuối (Endpoints) của phân hệ Thông báo	106
6.46	Giao diện nhập thông tin để tra cứu sách đang mượn	107
6.47	Hộp thoại xác nhận hoàn trả tài liệu	108
6.48	Hộp thoại xử lý báo cáo hư hỏng sách	108
6.49	Hộp thoại xử lý báo cáo mất sách	108
6.50	Thư điện tử xác nhận giao dịch trả sách thành công	109
6.51	Giao diện trung tâm điều hành dành cho Thủ thư	111
6.52	Giao diện Danh sách bản sao trong hệ thống quản lý thư viện	113
6.53	Giao diện quản lý danh mục ấn phẩm tập trung	114
6.54	Giao diện hiệu chỉnh siêu dữ liệu của ấn phẩm	115
6.55	Quản lý ảnh bìa và tệp tin tài liệu số	116
6.56	Giao diện nhập thông tin chi tiết ấn phẩm	117

6.57	Giao diện chọn tệp đính kèm	118
6.58	Bảng điều khiển trạng thái tải lên	118
6.59	Giao diện nhập thông tin để tra cứu sách đang mượn	119
6.60	Hộp thoại xác nhận hoàn trả tài liệu	120
6.61	Hộp thoại xử lý báo cáo hư hỏng sách	120
6.62	Hộp thoại xử lý báo cáo mất sách	120
6.63	Thư điện tử xác nhận giao dịch trả sách thành công	121
6.64	Giao diện quản lý thu phí phạt tại quầy	122
6.65	Giao diện Thống kê tổng quát và danh sách các khoản phí chưa thanh toán	124
6.66	Giao diện hiển thị lịch sử các khoản phí đã hoàn tất thanh toán	124
6.67	Giao diện danh sách sách đang mượn và chờ lấy	125
6.68	Giao diện lịch sử mượn trả tài liệu	126
6.69	Chi tiết trạng thái giao dịch	126
6.70	Thông tin biên nhận mượn sách	126
6.71	Giao diện quản lý danh sách mượn trả tổng thể dành cho Thủ thư	127
7.1	Kết quả kiểm thử đơn vị Backend (<code>mvn test -pl library-circulation-module</code>)	132
7.2	Kết quả kiểm thử đơn vị Frontend (<code>npm test</code>)	133
7.3	Kết quả kiểm thử tích hợp — Surefire HTML Report (<code>library-bootstrap/target/reports/surefire.html</code>)	133
7.4	Kết quả kiểm thử API — Postman Collection Runner	136
7.5	Kết quả kiểm thử hiệu năng k6 — tất cả thresholds đạt yêu cầu	137
8.1	Docker	139
8.2	Kiến trúc triển khai hệ thống	141
8.3	Luồng traffic qua các container trong môi trường sản xuất	143
8.4	Luồng kết nối từ tên miền đến hệ thống	144

Danh sách bảng

4.1	Bảng đặc tả yêu cầu hệ thống	20
5.1	Bảng đặc tả Use Case UC-01: Tìm kiếm sách	23
5.2	Bảng đặc tả Use Case UC-02: Đăng ký tài khoản	24
5.3	Bảng đặc tả Use Case UC-08: Mượn sách	25
5.4	Bảng đặc tả Use Case UC-09: Trả sách	26
5.6	Bảng so sánh các công nghệ Front-end	37
5.8	Bảng so sánh các công nghệ Back-end	38
5.9	Bảng quy đổi điểm tương tác cho mô hình ALS	44
5.10	Đánh giá kiến trúc lại so với các chỉ số	49
6.1	Đặc tả dữ liệu đầu vào (Request Body)	72
6.3	Cấu trúc dữ liệu yêu cầu (Request Body) hoàn tất hồ sơ	79
6.5	Danh sách các API phân hệ Thông tin cá nhân	85
6.7	Danh sách API Quản lý sách yêu thích	94
6.9	Danh sách API Quản lý thông báo	106
6.11	Trạng thái bản sao và Phí phạt tương ứng sau khi trả sách	110
6.13	Trạng thái bản sao và Phí phạt tương ứng sau khi trả sách	122
6.15	Quy ước hiển thị trạng thái mượn trả	128
7.1	Danh sách test case kiểm thử đơn vị Backend	131
7.2	Danh sách test case kiểm thử đơn vị Frontend	132
7.3	Danh sách test case kiểm thử tích hợp — Luồng mượn sách	134
7.4	Tổng quan Postman Collection	135
7.5	Kết quả kiểm thử hiệu năng k6	137
7.6	Tổng hợp kết quả kiểm thử	137
8.1	Các nhóm biến môi trường của hệ thống	142
8.2	So sánh các nhà cung cấp VPS phổ biến	144
8.3	Chi phí vận hành hệ thống ước tính	145
10.1	Danh sách các tài nguyên trực tuyến của hệ thống	148

Chương 1: Giới thiệu

Chương này trình bày tổng quan về đề tài, bao gồm lý do lựa chọn, các mục tiêu cần đạt được, phạm vi nghiên cứu, ý nghĩa khoa học và thực tiễn của dự án, đồng thời giới thiệu cấu trúc tổng thể của báo cáo.

1.1 Động cơ của đề tài

Trong bối cảnh chuyển đổi số giáo dục đang diễn ra mạnh mẽ, việc hiện đại hóa các hệ thống thư viện tại trường đại học là một yêu cầu cấp thiết. Các hệ thống thư viện hiện tại, như Koha hay Opac-HCMUT, dù đáp ứng được nhu cầu quản lý cơ bản nhưng vẫn còn nhiều hạn chế. Cụ thể, các hệ thống này chủ yếu dựa trên tìm kiếm từ khóa đơn giản, chưa có khả năng phân tích hành vi người dùng để đưa ra gợi ý tài liệu phù hợp, và giao diện đã cũ, chưa tối ưu cho các thiết bị di động. Thêm vào đó, công tác thống kê và báo cáo xu hướng đọc vẫn còn thực hiện thủ công, gây tốn nhiều thời gian và công sức cho các thủ thư.

Từ thực tiễn đó, có một nhu cầu rõ rệt từ các bên liên quan:

- **Sinh viên và giảng viên:** Cần một công cụ tìm kiếm tài liệu nhanh chóng, chính xác và được gợi ý các tài liệu liên quan để phục vụ học tập, nghiên cứu.
- **Thủ thư và nhà quản lý:** Mong muốn có một hệ thống tự động hóa các công việc lặp lại, cung cấp báo cáo và thống kê trực quan để nâng cao hiệu quả quản lý.
- **Nhà trường:** Hướng tới việc thúc đẩy văn hóa đọc và năng lực số của sinh viên thông qua một nền tảng thư viện hiện đại.

Vì vậy, việc phát triển một "**Hệ thống quản lý thư viện trực tuyến tích hợp AI hỗ trợ tìm kiếm và gợi ý sách**" là hoàn toàn cần thiết, không chỉ để giải quyết các hạn chế của hệ thống cũ mà còn để đón đầu xu hướng công nghệ trong giáo dục đại học.

1.2 Mục tiêu

Đề tài đặt ra mục tiêu tổng quát là xây dựng một hệ thống quản lý thư viện trực tuyến thông minh, vừa đảm bảo các chức năng quản lý cốt lõi, vừa tích hợp AI/NLP để nâng cao trải nghiệm tìm kiếm và đọc sách cho người dùng.

Các mục tiêu cụ thể bao gồm:

- **Xây dựng hệ thống quản lý thư viện trực tuyến:** Cung cấp đầy đủ các chức năng cơ bản như quản lý sách, tác giả, thể loại, người dùng, và các hoạt động mượn - trả.
- **Ứng dụng AI/NLP để nâng cao trải nghiệm:**
 - Phát triển công cụ tìm kiếm ngữ nghĩa, hỗ trợ tiếng Việt, có khả năng hiểu ngữ cảnh và từ đồng nghĩa.
 - Xây dựng hệ thống gợi ý sách được cá nhân hóa dựa trên lịch sử mượn và sở thích của người dùng.
 - Tích hợp tính năng tự động sinh tóm tắt nội dung sách để người dùng tham khảo nhanh.
- **Hỗ trợ công tác quản lý:** Tự động hóa việc thống kê và tạo báo cáo về xu hướng đọc, sách được mượn

nhiều, giúp thủ thư dễ dàng theo dõi tình trạng mượn - trả và phí phạt.

- **Đảm bảo khả năng triển khai và mở rộng:** Xây dựng hệ thống với kiến trúc mở, sẵn sàng cho việc áp dụng thử nghiệm tại thư viện cấp trường/khoa và dễ dàng nâng cấp trong tương lai.

1.3 Phạm vi

Đề tài sẽ tập trung vào các phạm vi nghiên cứu và phát triển như sau:

- **Về chức năng quản lý:** Hệ thống sẽ thực hiện các chức năng cốt lõi bao gồm quản lý sách, tác giả, người dùng, xử lý mượn - trả, hạn trả, phí phạt và tạo các báo cáo thống kê cần thiết.
- **Về ứng dụng AI:**
 - Xây dựng một hệ thống tìm kiếm ngữ nghĩa hỗ trợ tiếng Việt, xử lý được các vấn đề như chính tả, từ đồng nghĩa và truy vấn không dấu.
 - Tích hợp một hệ thống gợi ý cá nhân hóa dựa trên lịch sử và hành vi mượn sách của người dùng.
 - Phát triển một mô-đun sinh tóm tắt nội dung sách.
- **Về nền tảng công nghệ:** Hệ thống được phát triển theo kiến trúc tách biệt giữa backend (xử lý logic nghiệp vụ) và frontend (giao diện người dùng), sử dụng cơ sở dữ liệu quan hệ và tích hợp các mô hình AI/NLP phù hợp.
- **Về phạm vi triển khai:** Sản phẩm của đề tài là một mô hình thử nghiệm (prototype), được kiểm chứng trên một bộ dữ liệu giới hạn (giáo trình, sách tham khảo) ở quy mô một thư viện cấp khoa hoặc trường.

1.4 Ý nghĩa

1.4.1 Ý nghĩa thực tiễn

- **Nâng cao trải nghiệm người dùng:** Giúp sinh viên và giảng viên tìm kiếm tài liệu nhanh hơn và tiếp cận nguồn tài nguyên tri thức một cách dễ dàng hơn.
- **Tối ưu hóa công tác quản lý:** Giảm tải công việc thủ công cho thủ thư thông qua các chức năng thống kê, báo cáo tự động, từ đó tiết kiệm thời gian và nguồn nhân lực.
- **Thúc đẩy chuyển đổi số:** Góp phần xây dựng mô hình thư viện số hiện đại, phù hợp với chủ trương chuyển đổi số trong giáo dục và phát triển năng lực số cho sinh viên.

1.4.2 Ý nghĩa khoa học

- **Ứng dụng AI/NLP vào bài toán thực tế:** Đề tài áp dụng các kỹ thuật xử lý ngôn ngữ tự nhiên và hệ thống gợi ý vào việc giải quyết bài toán tìm kiếm và gợi ý tài liệu trong môi trường thư viện tiếng Việt, một lĩnh vực còn nhiều không gian để cải tiến.
- **Kết hợp mô hình và hệ thống:** Dự án là cầu nối giữa việc nghiên cứu các mô hình AI và việc triển khai chúng thành một hệ thống phần mềm hoàn chỉnh, có tính ứng dụng cao, giải quyết vấn đề mà nhiều nghiên cứu lý thuyết bỏ qua.
- **Đề xuất giải pháp khả thi:** Hệ thống áp dụng mô hình gợi ý hybrid (kết hợp Content-Based và Collaborative Filtering) ở mức độ phù hợp với nguồn lực và dữ liệu của một dự án sinh viên, cho thấy hướng tiếp cận thực tế để xây dựng các hệ thống thông minh.

1.5 Cấu trúc báo cáo

Ngoài các phần đầu mục như Tóm tắt, Mục lục, Danh mục hình ảnh và bảng biểu, nội dung chính của báo cáo được cấu trúc thành 10 chương như sau:

- **Chương 1 - Giới thiệu:** Trình bày tổng quan về động cơ, mục tiêu, phạm vi và ý nghĩa của đề tài, đồng thời giới thiệu cấu trúc tổng thể của báo cáo.
- **Chương 2 - Kiến thức và công nghệ nền tảng:** Trình bày các cơ sở lý thuyết, kiến thức chuyên ngành và các công nghệ cốt lõi được sử dụng để phát triển dự án.
- **Chương 3 - Công trình liên quan:** Phân tích, so sánh các nghiên cứu và hệ thống tương tự để xác định khoảng trống nghiên cứu mà đề tài giải quyết.
- **Chương 4 - Hệ thống đề xuất:** Mô tả tổng quan hệ thống được đề xuất, bao gồm đối tượng người dùng, yêu cầu chức năng, phi chức năng và yêu cầu dữ liệu.
- **Chương 5 - Phân tích và Thiết kế:** Trình bày chi tiết quá trình phân tích và thiết kế hệ thống, bao gồm kiến trúc phần mềm, thiết kế cơ sở dữ liệu, thiết kế AI Service và thiết kế giao diện.
- **Chương 6 - Hiện thực hệ thống:** Mô tả quá trình hiện thực các chức năng theo từng API, giao diện cho Độc giả và Thủ thư, cùng với pipeline xử lý dữ liệu AI.
- **Chương 7 - Kiểm thử:** Trình bày chiến lược kiểm thử phân tầng gồm kiểm thử đơn vị, tích hợp, API và hiệu năng, kèm kết quả thực nghiệm.
- **Chương 8 - Triển khai hệ thống:** Mô tả quá trình đóng gói Docker, cấu hình NGINX, CI/CD và triển khai lên môi trường VPS thực tế.
- **Chương 9 - Tổng kết:** Tóm tắt kết quả đạt được, nhận xét ưu điểm và hạn chế còn tồn tại của hệ thống.
- **Chương 10 - Tài nguyên và Mã nguồn:** Cung cấp các đường dẫn đến mã nguồn (GitHub) và bản thiết kế giao diện (Figma) của dự án.

Chương 2: Kiến thức và công nghệ nền tảng

Chương này trình bày tổng quan về các khái niệm, kiến thức và các lĩnh vực công nghệ nền tảng được vận dụng để xây dựng "Hệ thống quản lý thư viện trực tuyến tích hợp AI hỗ trợ tìm kiếm và gợi ý sách".

2.1 Các Lĩnh vực Công nghệ Chính

Dự án được định hướng phát triển dựa trên kiến trúc phần mềm hiện đại, trong đó các thành phần chính được phân tách rõ ràng để thuận lợi cho việc bảo trì và mở rộng sau này.

- **Công nghệ phía Server (Back-End):** Đây là thành phần trung tâm, chịu trách nhiệm xử lý toàn bộ logic nghiệp vụ của hệ thống, cung cấp các giao diện lập trình ứng dụng (API), quản lý dữ liệu mượn-trả, và thực hiện các tác vụ thống kê.
- **Công nghệ phía Client (Front-End):** Thành phần này đảm nhiệm việc xây dựng giao diện người dùng, giúp người dùng cuối (thủ thư, bạn đọc) tương tác với hệ thống một cách trực quan và thân thiện trên nền tảng web.
- **Hệ quản trị Cơ sở dữ liệu:** Hệ thống sử dụng một hệ quản trị cơ sở dữ liệu quan hệ để tổ chức, lưu trữ và truy xuất toàn bộ dữ liệu của thư viện, bao gồm thông tin sách, người dùng, và lịch sử giao dịch.
- **Trí tuệ Nhân tạo (AI) và Học máy (Machine Learning):**
 - **Hệ thống gợi ý (Recommendation System):** Trọng tâm của dự án là áp dụng mô hình **Hybrid**, một phương pháp kết hợp ưu điểm của hai kỹ thuật chính:
 1. **Lọc dựa trên nội dung (Content-Based Filtering):** Gợi ý tài liệu dựa trên sự tương đồng về thuộc tính và nội dung của chúng.
 2. **Lọc cộng tác (Collaborative Filtering):** Đề xuất tài liệu dựa trên hành vi và sở thích của các nhóm người dùng tương tự nhau.
 - **Xử lý Ngôn ngữ Tự nhiên (NLP):** Các kỹ thuật NLP được ứng dụng để nâng cao trải nghiệm người dùng, bao gồm: phát triển công cụ **tìm kiếm ngữ nghĩa** cho tiếng Việt và tích hợp khả năng **tự động tóm tắt nội dung sách**.

2.2 Nền tảng Kiến thức Khoa học

Để triển khai các lĩnh vực công nghệ trên, nhóm nghiên cứu cần vận dụng các nền tảng kiến thức sau:

- **Kiến trúc và Phát triển Hệ thống:**
 - Hiểu biết về kiến trúc **Client-Server** và nguyên lý thiết kế **RESTful API** để đảm bảo sự giao tiếp hiệu quả giữa Front-End và Back-End.
 - Kiến thức về thiết kế và mô hình hóa cơ sở dữ liệu quan hệ (ví dụ: ERD) để xây dựng một cấu trúc lưu trữ tối ưu.
- **Khoa học Dữ liệu và Trí tuệ Nhân tạo:**

- Nguyên lý các thuật toán gợi ý: Nắm vững bản chất, ưu và nhược điểm của các phương pháp Content-Based, Collaborative Filtering, và cách chúng bổ trợ cho nhau trong một mô hình Hybrid.
- Các kỹ thuật xử lý ngôn ngữ tự nhiên: Có kiến thức về các phương pháp tiền xử lý văn bản, mô hình hóa ngôn ngữ (word embedding), và các thuật toán đo lường độ tương đồng ngữ nghĩa (ví dụ: cosine similarity).

2.3 Kết chương

Nội dung chương này đã hệ thống hóa các lĩnh vực công nghệ và nền tảng kiến thức khoa học cần thiết cho dự án. Các thành phần công nghệ chính bao gồm **Back-End**, **Front-End**, **Cơ sở dữ liệu quan hệ** và các module **Trí tuệ nhân tạo**. Trọng tâm của dự án nằm ở việc áp dụng các kiến thức về **Hệ thống gợi ý Hybrid** và **Xử lý Ngôn ngữ Tự nhiên** để tạo ra một sản phẩm thông minh và khác biệt.

Với các nền tảng kiến thức đã trình bày, nhóm xác nhận đã bao quát đầy đủ các yêu cầu lý thuyết cần thiết để triển khai dự án, từ việc xây dựng một ứng dụng web hoàn chỉnh đến tích hợp các tính năng AI nâng cao theo đúng mục tiêu đề ra.

Chương 3: Công trình liên quan

Chương này tập trung phân tích các công trình nghiên cứu khoa học và các hệ thống phần mềm đã có liên quan đến lĩnh vực thư viện số và hệ thống gợi ý. Từ đó, chương sẽ chỉ ra những khoảng trống nghiên cứu và định vị hướng đi của đề tài.

3.1 Các công trình nghiên cứu khoa học

Qua việc tổng hợp các công trình nghiên cứu, có thể thấy các hướng tiếp cận phổ biến trong việc xây dựng hệ thống gợi ý sách cho thư viện có những đặc điểm sau:

- **Ưu điểm và xu hướng chung:**

- Hầu hết các mô hình gợi ý đều theo hướng **hybrid**, kết hợp giữa Lọc dựa trên nội dung (Content-Based) và Lọc cộng tác (Collaborative Filtering) để cân bằng giữa độ cá nhân hóa và khả năng khai thác hành vi người dùng.
- Một số nghiên cứu cải tiến kết quả bằng cách thêm các cơ chế xếp hạng (ranking) hoặc gợi ý theo cụm người dùng (clustering), đặc biệt hiệu quả trong môi trường dữ liệu vừa và nhỏ.
- Việc đánh giá mô hình thường dựa trên dữ liệu thực tế từ các thư viện, giúp kiểm chứng rõ ràng hiệu quả của thuật toán.

- **Hạn chế phổ biến:**

- Nhiều công trình tập trung sâu vào phần mô hình thuật toán nhưng ít chú trọng đến việc triển khai thành một hệ thống hoàn chỉnh với đầy đủ backend và frontend.
- Các hướng đi hiện đại như tìm kiếm ngữ nghĩa (semantic search) hay các mô hình Transformer tuy tiềm năng nhưng đòi hỏi chi phí tính toán và nguồn lực lớn, vượt quá khả năng của nhiều dự án quy mô nhỏ.

3.2 Các hệ thống và phần mềm tương tự

Hiện nay, có nhiều hệ thống quản lý thư viện điện tử đã được triển khai rộng rãi, tiêu biểu có thể kể đến:

- **Opac-HCMUT-VNU:** Là hệ thống của Đại học Quốc Gia TP.HCM, đáp ứng tốt các nghiệp vụ tra cứu và mượn-trả cơ bản. Tuy nhiên, chức năng tìm kiếm vẫn chủ yếu dựa vào từ khóa và lọc và chưa tích hợp tính năng gợi ý thông minh, tìm kiếm theo ngữ nghĩa.
- **Koha:** Là một phần mềm quản lý thư viện mã nguồn mở rất phổ biến, cung cấp đầy đủ chức năng quản lý và báo cáo. Mặc dù vậy, giao diện của Koha đã cũ, chưa hỗ trợ gợi ý tự động và chưa có cơ chế phân tích hành vi người dùng.
- **Aleph (Ex Libris):** Đây là một hệ thống thương mại quy mô lớn, tích hợp nhiều cơ sở dữ liệu học thuật. Nhược điểm chính của hệ thống này là chi phí triển khai cao và không phù hợp cho các thư viện quy mô nhỏ như cấp khoa hoặc trường đại học nhỏ.

3.3 Kết chương và xác định khoảng trống nghiên cứu

Từ việc phân tích các công trình khoa học và hệ thống thực tế, nhóm nhận thấy một số **khoảng trống (gaps)** quan trọng mà đề tài này hướng đến giải quyết:

1. **Khoảng trống giữa Mô hình và Thực thi:** Nhiều nghiên cứu tập trung vào việc xây dựng các mô hình gợi ý phức tạp nhưng lại bỏ qua việc triển khai chúng thành một hệ thống phần mềm hoàn chỉnh, có tính ứng dụng thực tế. Đa số các mô hình hiện nay tập trung nhiều vào thuật toán gợi ý, trong khi ít chú trọng đến việc triển khai thành hệ thống hoàn chỉnh.
2. **Khoảng trống về Trải nghiệm Người dùng:** Các hệ thống hiện tại, kể cả mã nguồn mở như Koha, thường có giao diện cũ, chưa tối ưu cho các thiết bị hiện đại và thiếu các tính năng thông minh giúp nâng cao trải nghiệm tìm kiếm, khám phá tài liệu của người dùng.
3. **Khoảng trống về Tính khả thi:** Các giải pháp gợi ý tiên tiến thường yêu cầu nguồn dữ liệu lớn hoặc xử lý phức tạp, vượt quá phạm vi và nguồn lực của một nhóm sinh viên.

Hướng đi của dự án: Đề tài này không đặt nặng việc phát triển một mô hình AI quá chuyên sâu. Thay vào đó, dự án tập trung vào việc **bắc cầu cho các khoảng trống trên** bằng cách:

- Xây dựng một **hệ thống hoàn chỉnh** với kiến trúc frontend-backend tách biệt, dễ bảo trì và mở rộng.
- Ưu tiên **trải nghiệm người dùng** với giao diện hiện đại và tích hợp các tính năng gợi ý, tìm kiếm thông minh một cách liền mạch.
- Áp dụng một **giải pháp AI khả thi** (mô hình hybrid cơ bản) để tạo ra một sản phẩm hoạt động ổn định, có giá trị thực tiễn và phù hợp với bối cảnh của một đồ án tốt nghiệp.

Chương 4: Hệ thống đề xuất

4.1 Mô tả hệ thống

Hệ thống được đề xuất là một **hệ thống quản lý thư viện trực tuyến tích hợp AI hỗ trợ tìm kiếm và gợi ý sách**. Mục tiêu cốt lõi của dự án là xây dựng một nền tảng hiện đại, kết hợp hài hòa giữa hai yếu tố: (1) cung cấp đầy đủ các chức năng để quản lý thư viện một cách hiệu quả, và (2) ứng dụng Trí tuệ nhân tạo (AI) để mang lại trải nghiệm thông minh cho người dùng cuối. Thay vì chỉ dừng lại ở mức tra cứu từ khóa cơ bản như các hệ thống hiện tại, hệ thống đề xuất hướng tới việc trở thành một trợ lý ảo, giúp người dùng khám phá và tiếp cận nguồn tài nguyên tri thức một cách tối ưu nhất.

Hệ thống được thiết kế để phục vụ hai nhóm người dùng chính, mỗi nhóm có vai trò và nhu- cầu riêng biệt:

- **Bạn đọc (Sinh viên, Giảng viên, Nghiên cứu viên):** Là nhóm người dùng cuối, trực tiếp khai thác tài nguyên của thư viện. Nhu cầu chính của họ là tìm kiếm tài liệu nhanh chóng, đọc tóm tắt để đánh giá nội dung, thực hiện mượn sách và nhận được các gợi ý tài liệu liên quan để phục vụ cho việc học tập và nghiên cứu.
- **Thủ thư (Người quản lý thư viện):** Là nhóm người dùng quản trị, chịu trách nhiệm vận hành thư viện. Vai trò của họ bao gồm việc quản lý kho tài liệu, quản lý thông tin bạn đọc, theo dõi các hoạt động mượn - trả, và sử dụng các công cụ thống kê, báo cáo tự động để đánh giá hiệu quả hoạt động của thư viện.

4.2 Hướng tiếp cận thiết kế

Hệ thống Quản lý Thư viện sẽ được thiết kế theo phương pháp “từ dưới lên” (bottom-up design). Hướng tiếp cận này tập trung vào việc xây dựng và kiểm thử các thành phần cơ bản, độc lập trước khi tích hợp chúng lại thành một hệ thống lớn hơn. Quá trình này bao gồm các bước sau:

1. Xác định và thiết kế các thành phần nhỏ, cốt lõi nhất của hệ thống (ví dụ: các module quản lý sách, quản lý người dùng).
2. Xây dựng và kiểm thử riêng lẻ từng thành phần để đảm bảo chúng hoạt động chính xác.
3. Tích hợp các thành phần nhỏ đã hoàn thiện để xây dựng các chức năng hoặc hệ thống con lớn hơn.
4. Lặp lại quy trình này cho đến khi hoàn thành thiết kế của toàn bộ hệ thống.

4.3 Yêu cầu

4.3.1 Yêu cầu chức năng

Các yêu cầu chức năng của hệ thống được đặc tả chi tiết trong bảng dưới đây, phân loại theo từng nhóm chức năng và người dùng tương tác.

Bảng 4.1: Bảng đặc tả yêu cầu hệ thống

ID	Tên yêu cầu	Loại	Mô tả chi tiết
R1	Quản lý thực thể dữ liệu	Dữ liệu	Hệ thống phải có khả năng lưu trữ và quản lý ba thực thể dữ liệu cốt lõi: Sách , Người dùng , và Lịch sử Giao dịch (mượn/trả).
R2	Phân biệt Đầu sách và Bản sao	Dữ liệu	Hệ thống phải phân biệt giữa “Đầu sách” và “Bản sao”. Mỗi bản sao phải có một mã định danh duy nhất (barcode), thông tin trạng thái và vị trí kệ sách để xác định vị trí vật lý.
R3	Thuộc tính Đầu sách	Dữ liệu	Mỗi đầu sách phải có các thuộc tính cơ bản bao gồm: ISBN , tiêu đề, tác giả, thể loại và ngày xuất bản.
R4	Quản lý vai trò người dùng	Chức năng	Hệ thống phải hỗ trợ hai vai trò người dùng: Thủ thư (quản trị) và Bạn đọc (thành viên). Mỗi người dùng phải được định danh bởi một mã thẻ thư viện duy nhất.
R5	Cấu hình chính sách mượn/trả	Nghiệp vụ	Hệ thống phải cho phép Thủ thư cấu hình các chính sách mượn trả , ví dụ: giới hạn số lượng sách mượn tối đa là 5 cuốn và thời gian mượn tiêu chuẩn là 14 ngày .
R6	Tìm kiếm ngữ nghĩa	Chức năng	Hệ thống phải cung cấp chức năng tìm kiếm ngữ nghĩa , cho phép tìm kiếm sách theo tiêu đề, tác giả, hoặc chủ đề, và xử lý được các truy vấn tiếng Việt.
R7	Chức năng Đặt chỗ	Chức năng	Bạn đọc phải có khả năng đặt chỗ một đầu sách khi không có bản sao nào sẵn có. Mỗi đầu sách có thể được đặt chỗ bởi nhiều thành viên tại một thời điểm.
R8	Chức năng Gia hạn	Chức năng	Hệ thống phải cho phép Bạn đọc gia hạn thời gian mượn sách, với điều kiện sách đó không đang được người khác đặt chỗ.
R9	Hệ thống thông báo tự động	Chức năng	Hệ thống phải tự động gửi thông báo cho Bạn đọc về các sự kiện quan trọng, bao gồm: sách sắp đến hạn trả, sách đã quá hạn, và sách đặt chỗ đã có sẵn.

4.3.2 Yêu cầu phi chức năng

Hệ thống cần đảm bảo các yêu cầu phi chức năng trọng yếu sau:

- **Tính khả dụng (Usability):** Giao diện người dùng phải nhất quán, trực quan và dễ học cho cả bạn đọc

và thủ thư. Hệ thống phải có thiết kế đáp ứng (responsive), hoạt động tốt trên các trình duyệt web phổ biến và các kích thước màn hình khác nhau.

- **Hiệu năng (Performance):** Các chức năng tương tác phải có thời gian phản hồi nhanh. Cụ thể, thời gian tải trang chính không quá 3 giây và thời gian trả về kết quả của chức năng tìm kiếm và gợi ý không quá 2 giây trong điều kiện tải thông thường.
- **Độ tin cậy (Reliability):** Hệ thống phải hoạt động ổn định, sẵn sàng 24/7 với thời gian hoạt động (uptime) đạt tối thiểu 99.5%. Dữ liệu phải được sao lưu định kỳ để đảm bảo khả năng phục hồi khi có sự cố.
- **Bảo mật (Security):** Dữ liệu cá nhân của người dùng phải được mã hóa. Hệ thống cần có cơ chế xác thực và phân quyền rõ ràng, đảm bảo bạn đọc không thể truy cập các chức năng của thủ thư và ngược lại.
- **Khả năng bảo trì và mở rộng (Maintainability & Scalability):** Mã nguồn phải được viết rõ ràng, có tài liệu và tuân theo các chuẩn thiết kế. Kiến trúc tách biệt backend-frontend cho phép các nhóm phát triển độc lập và dễ dàng nâng cấp, tích hợp các tính năng mới trong tương lai.

4.3.3 Yêu cầu dữ liệu

Hệ thống yêu cầu và quản lý các nhóm dữ liệu chính sau:

- **Dữ liệu lõi (Core Data):** Bao gồm thông tin chi tiết về sách (metadata), tác giả, thể loại và thông tin tài khoản của người dùng (bạn đọc, thủ thư).
- **Dữ liệu nghiệp vụ (Transactional Data):** Gồm các bản ghi về hoạt động mượn và trả sách, lịch sử giao dịch, và thông tin về các khoản phí phạt (nếu có).
- **Dữ liệu hành vi (Behavioral Data):** Gồm lịch sử các truy vấn tìm kiếm, các lượt xem chi tiết sách, và các tương tác khác của người dùng trên hệ thống. Đây là nguồn dữ liệu quan trọng cho các thuật toán gợi ý.

4.4 Kết chương

Chương này đã trình bày một cách tổng quan và chi tiết về hệ thống quản lý thư viện thông minh được đề xuất. Hệ thống đã được mô tả rõ ràng về mục tiêu, định hướng và hai nhóm người dùng chính là Bạn đọc và Thủ thư.

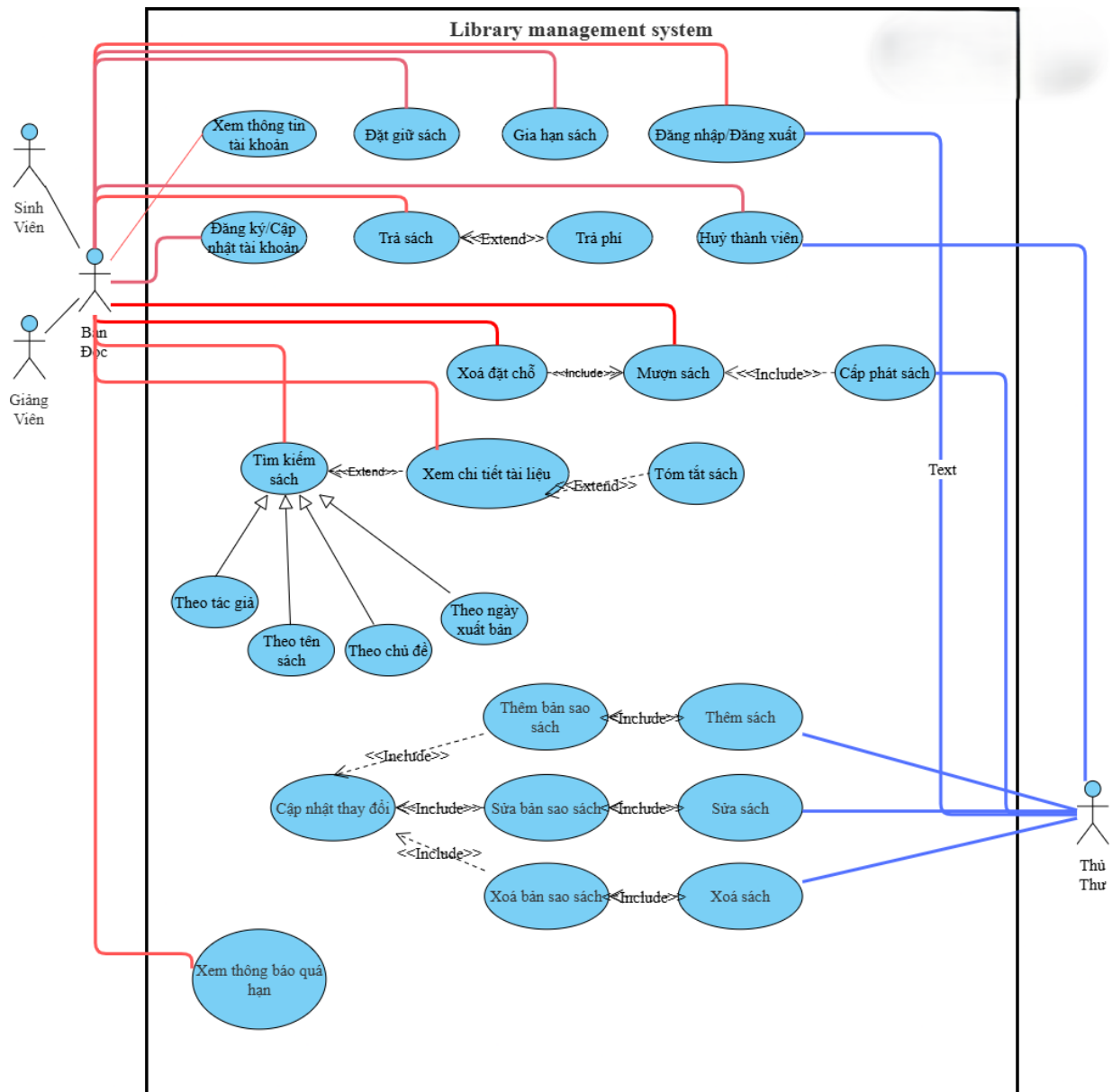
Phần trọng tâm của chương đã tập trung đặc tả các yêu cầu cốt lõi của hệ thống. Các **yêu cầu chức năng** đã được liệt kê chi tiết, bao quát từ các nghiệp vụ quản lý cơ bản cho đến các tính năng AI nâng cao như tìm kiếm ngữ nghĩa và gợi ý cá nhân hóa. Đồng thời, các **yêu cầu phi chức năng** quan trọng như hiệu năng, tính khả dụng, và bảo mật cũng đã được xác định để đảm bảo hệ thống hoạt động hiệu quả và an toàn. Cuối cùng, các **yêu cầu về dữ liệu** cần thiết cho việc vận hành và huấn luyện các mô hình AI cũng đã được làm rõ.

Những yêu cầu này tạo thành một nền tảng vững chắc, là cơ sở để tiến hành quá trình phân tích và thiết kế chi tiết hệ thống sẽ được trình bày ở Chương 5.

Chương 5: Phân tích và Thiết kế

5.1 Phân tích

5.1.1 Mô hình hóa tương tác (Biểu đồ Usecase)



Hình 5.1: Biểu đồ Usecase tổng quát của Hệ thống Quản lý Thư viện

5.1.1.1 Đặc tả chi tiết các Use Case chính

Bảng 5.1: Bảng đặc tả Use Case UC-01: Tìm kiếm sách

Thuộc tính	Mô tả chi tiết
Tên Use Case	Tìm kiếm sách
Mã số	UC-01
Tác nhân	Bạn đọc, Thủ thư
Mô tả ngắn gọn	Cho phép người dùng tìm kiếm các tài liệu trong thư viện dựa trên các tiêu chí khác nhau như tiêu đề, tác giả, chủ đề.
Tiền điều kiện	Người dùng đã truy cập vào giao diện tìm kiếm của hệ thống.
Hậu điều kiện	Thành công: Hệ thống hiển thị danh sách các tài liệu phù hợp với tiêu chí tìm kiếm. Thất bại: Hệ thống hiển thị thông báo "Không tìm thấy kết quả phù hợp".
Luồng chính	1. Người dùng nhập từ khóa vào ô tìm kiếm. 2. Người dùng có thể chọn bộ lọc (tác giả, chủ đề,...). 3. Người dùng nhấn nút "Tìm kiếm". 4. Hệ thống truy vấn cơ sở dữ liệu và trả về danh sách kết quả. 5. Hệ thống hiển thị danh sách kết quả cho người dùng. Use case kết thúc.
Luồng thay thế	4a. Không có kết quả phù hợp: 1. Hệ thống hiển thị thông báo "Không tìm thấy kết quả phù hợp". Use case kết thúc.

Bảng 5.2: Bảng đặc tả Use Case UC-02: Đăng ký tài khoản

Thuộc tính	Mô tả chi tiết
Tên Use Case	Đăng ký tài khoản
Mã số	UC-02
Tác nhân	Bạn đọc (người dùng mới)
Mô tả ngắn gọn	Cho phép người dùng mới tạo một tài khoản thành viên để sử dụng các dịch vụ của thư viện.
Tiền điều kiện	Người dùng đang ở trang đăng ký của hệ thống.
Hậu điều kiện	Thành công: Một tài khoản mới được tạo trong hệ thống và người dùng được đăng nhập tự động. Thất bại: Tài khoản không được tạo, hệ thống hiển thị thông báo lỗi.
Luồng chính	- Người dùng nhập thông tin cá nhân vào biểu mẫu đăng ký (Họ tên, email, mật khẩu,...). - Người dùng nhấn nút "Đăng ký". - Hệ thống kiểm tra tính hợp lệ của thông tin (định dạng email, mật khẩu đủ mạnh). - Hệ thống kiểm tra xem email đã tồn tại trong cơ sở dữ liệu chưa. - Hệ thống tạo tài khoản mới và lưu thông tin vào cơ sở dữ liệu. - Hệ thống tự động đăng nhập cho người dùng và chuyển hướng đến trang cá nhân. Use case kết thúc.
Luồng thay thế	3a. Thông tin không hợp lệ: - Hệ thống hiển thị thông báo lỗi tương ứng với từng trường thông tin. Người dùng được yêu cầu nhập lại. 4a. Email đã tồn tại: - Hệ thống hiển thị thông báo "Email này đã được sử dụng". Người dùng được yêu cầu sử dụng email khác.

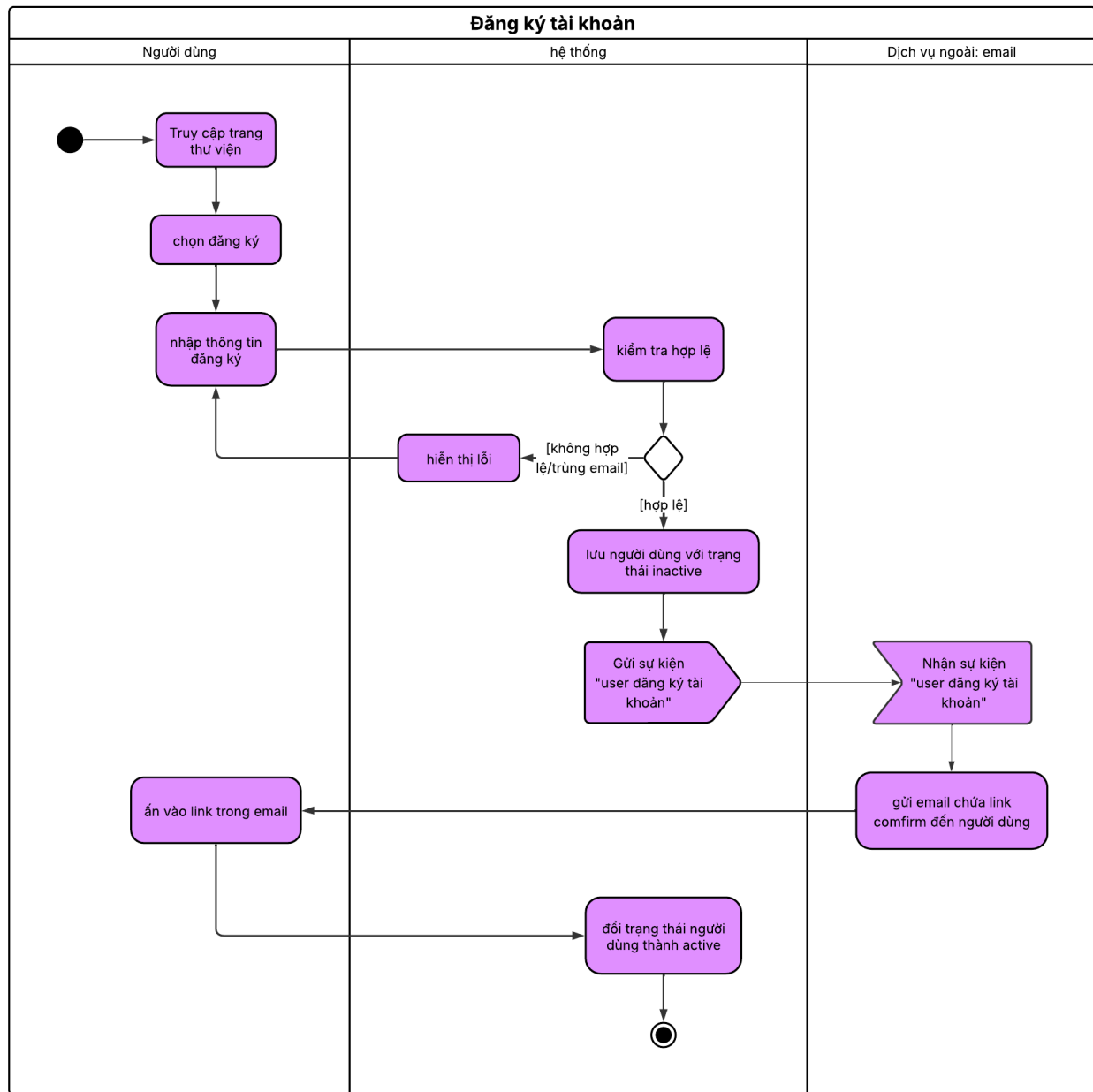
Bảng 5.3: Bảng đặc tả Use Case UC-08: Mượn sách

Thuộc tính	Mô tả chi tiết
Tên Use Case	Mượn sách
Mã số	UC-08
Tác nhân	Thủ thư
Mô tả ngắn gọn	Thủ thư ghi nhận thông tin một Bạn đọc mượn một bản sao sách cụ thể, cập nhật trạng thái của sách và hồ sơ của bạn đọc.
Tiền điều kiện	- Thủ thư đã đăng nhập vào hệ thống. - Bạn đọc có tài khoản đang ở trạng thái hoạt động. - Bản sao sách có trạng thái "Sẵn có".
Hậu điều kiện	Thành công: Một bản ghi mượn sách mới được tạo; Trạng thái của bản sao sách được cập nhật thành "Đã cho mượn"; Số lượng sách đang mượn của bạn đọc được cập nhật. Thất bại: Trạng thái hệ thống không thay đổi.
Luồng chính	- Thủ thư khởi tạo chức năng "Mượn sách". - Hệ thống yêu cầu nhập/quét mã vạch phiếu mượn của bạn đọc. - Thủ thư nhập/quét mã phiếu mượn của bạn đọc. - Hệ thống xác thực và hiển thị thông tin tóm tắt của Bạn đọc. - Hệ thống yêu cầu nhập/quét mã vạch của Bản sao sách. - Thủ thư nhập/quét mã Bản sao sách. - Hệ thống xác thực và kiểm tra trạng thái "Sẵn có" của sách. - Hệ thống kiểm tra các quy tắc nghiệp vụ (xem Luồng thay thế). - Use case này bao gồm use case "Xóa đặt chỗ" (nếu bạn đọc có đặt trước cuốn sách này). - Use case này bao gồm use case "Cấp phát sách". - Hệ thống xác nhận giao dịch thành công, hiển thị ngày hết hạn. Use case kết thúc.
Luồng thay thế	4a. Bạn đọc không tồn tại hoặc tài khoản bị khóa: - Hệ thống hiển thị thông báo lỗi. Use case kết thúc. 7a. Bản sao sách không tồn tại hoặc không ở trạng thái "Sẵn có": - Hệ thống hiển thị thông báo lỗi. Use case kết thúc. 8a. Bạn đọc đã mượn đủ số lượng sách tối đa cho phép: - Hệ thống hiển thị thông báo lỗi. Use case kết thúc.

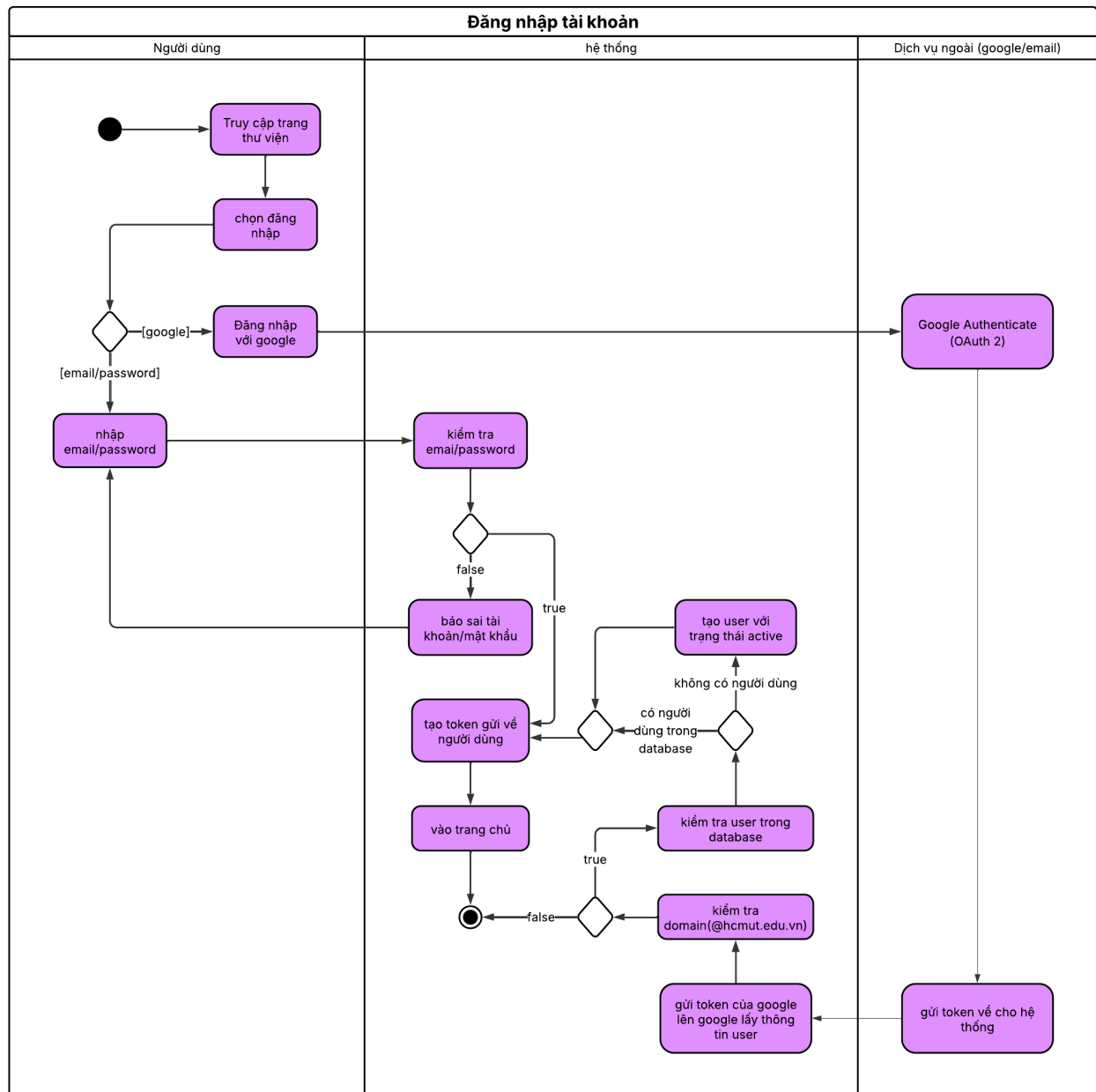
Bảng 5.4: Bảng đặc tả Use Case UC-09: Trả sách

Thuộc tính	Mô tả chi tiết
Tên Use Case	Trả sách
Mã số	UC-09
Tác nhân	Bạn đọc, Thủ thư
Mô tả ngắn gọn	Ghi nhận việc Bạn đọc trả lại một cuốn sách đã mượn. Hệ thống cập nhật trạng thái sách và kiểm tra các khoản phí phạt (nếu có).
Tiền điều kiện	Sách đang ở trạng thái "Đã cho mượn" bởi một bạn đọc cụ thể.
Hậu điều kiện	Thành công: Trạng thái bản sao sách được cập nhật thành "Sẵn có". Bản ghi mượn sách được đóng lại. Thất bại: Trạng thái hệ thống không thay đổi.
Luồng chính	- Thủ thư khởi tạo chức năng "Trả sách". - Hệ thống yêu cầu nhập/quét mã vạch của bản sao sách. - Người dùng nhập/quét mã vạch sách. - Hệ thống tìm bản ghi mượn sách tương ứng và kiểm tra ngày trả so với ngày hết hạn. - Hệ thống cập nhật trạng thái bản sao sách thành "Sẵn có". - Hệ thống đóng bản ghi mượn sách. Use case kết thúc.
Luồng thay thế	4a. Sách bị trả muộn (extend): Nếu ngày trả > ngày hết hạn, hệ thống sẽ kích hoạt use case "Trả phí".

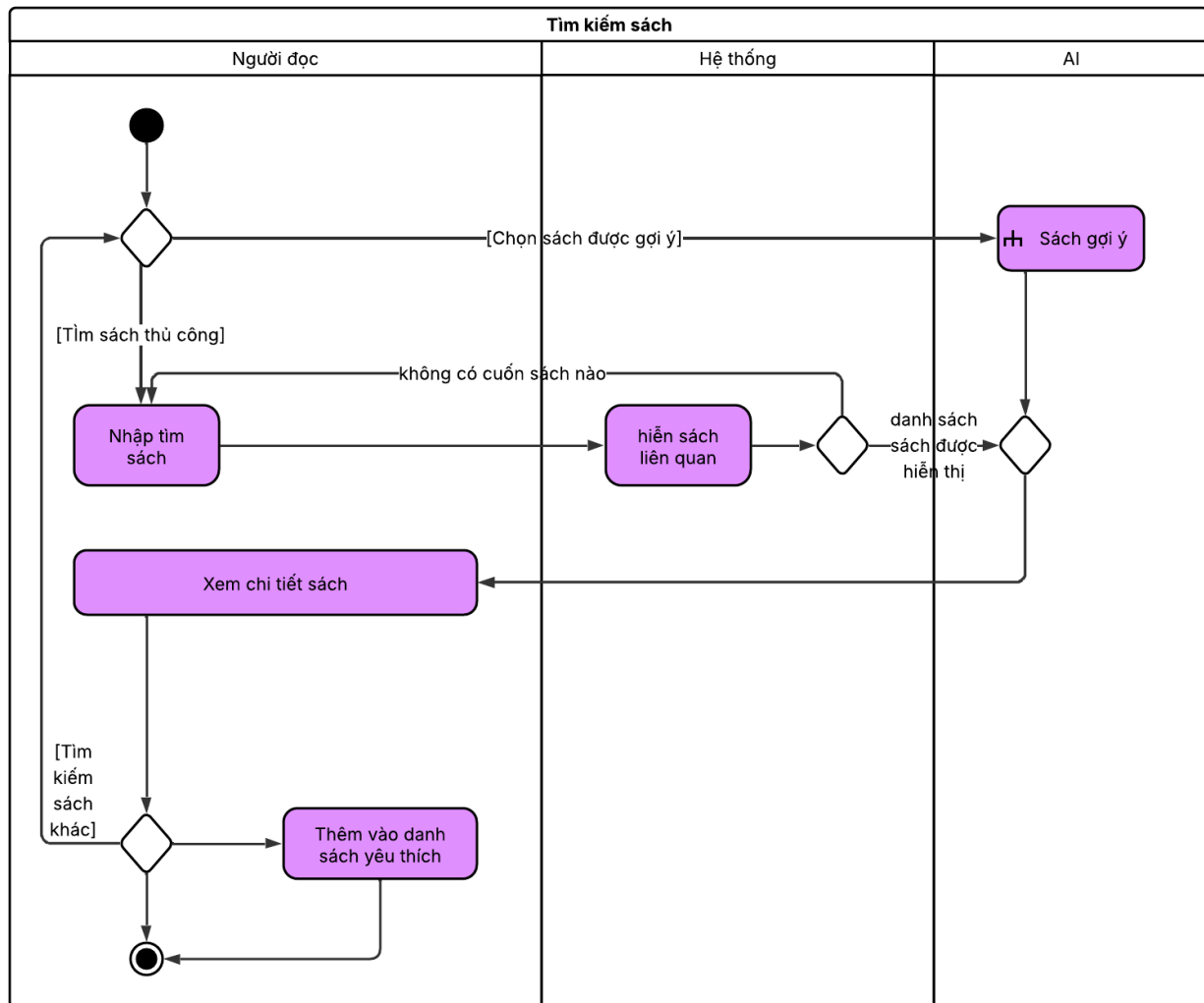
5.1.2 Mô hình hoá quy trình nghiệp vụ (Activity diagram)



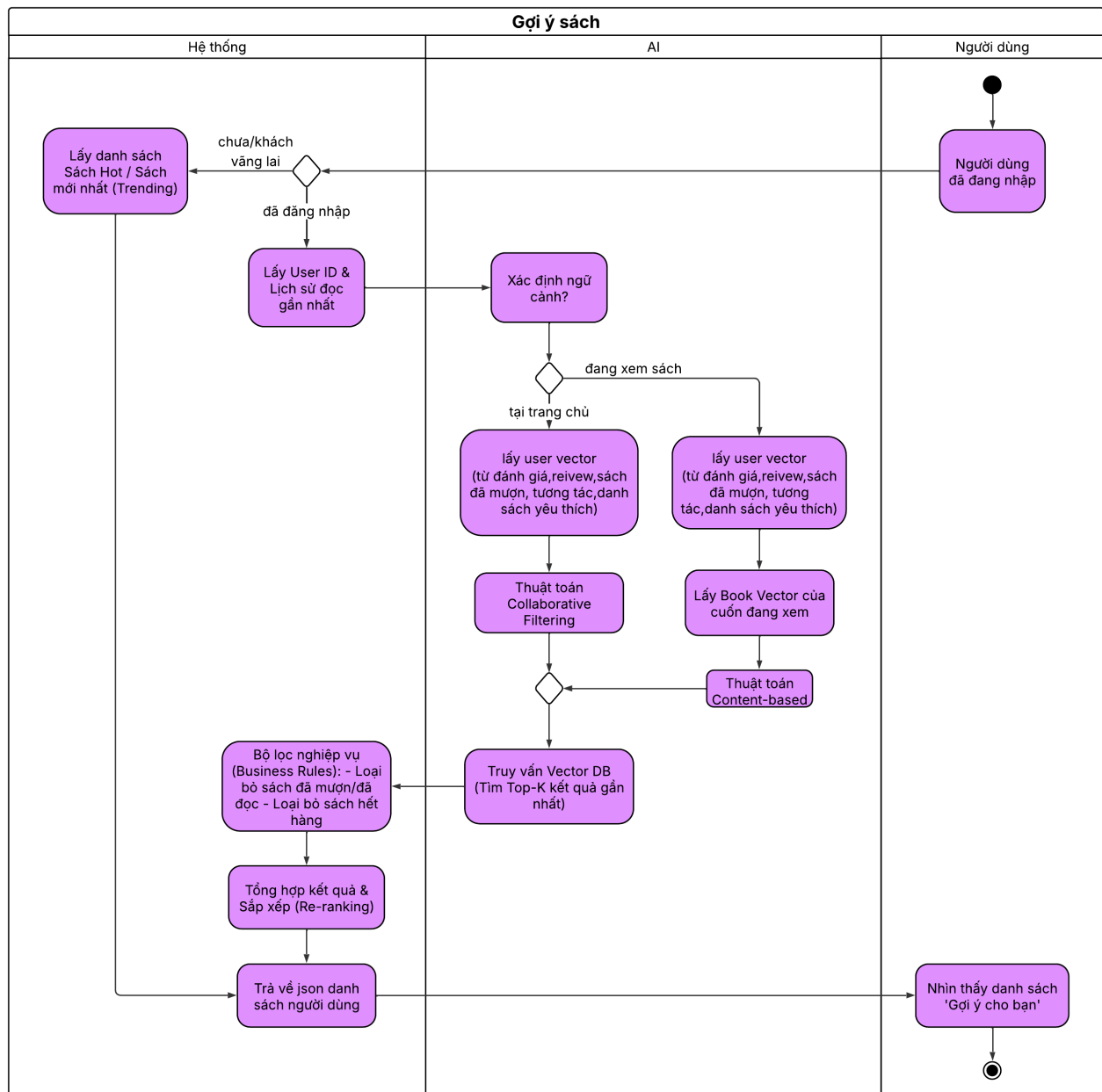
Hình 5.2: Đăng ký tài khoản — Activity Diagram



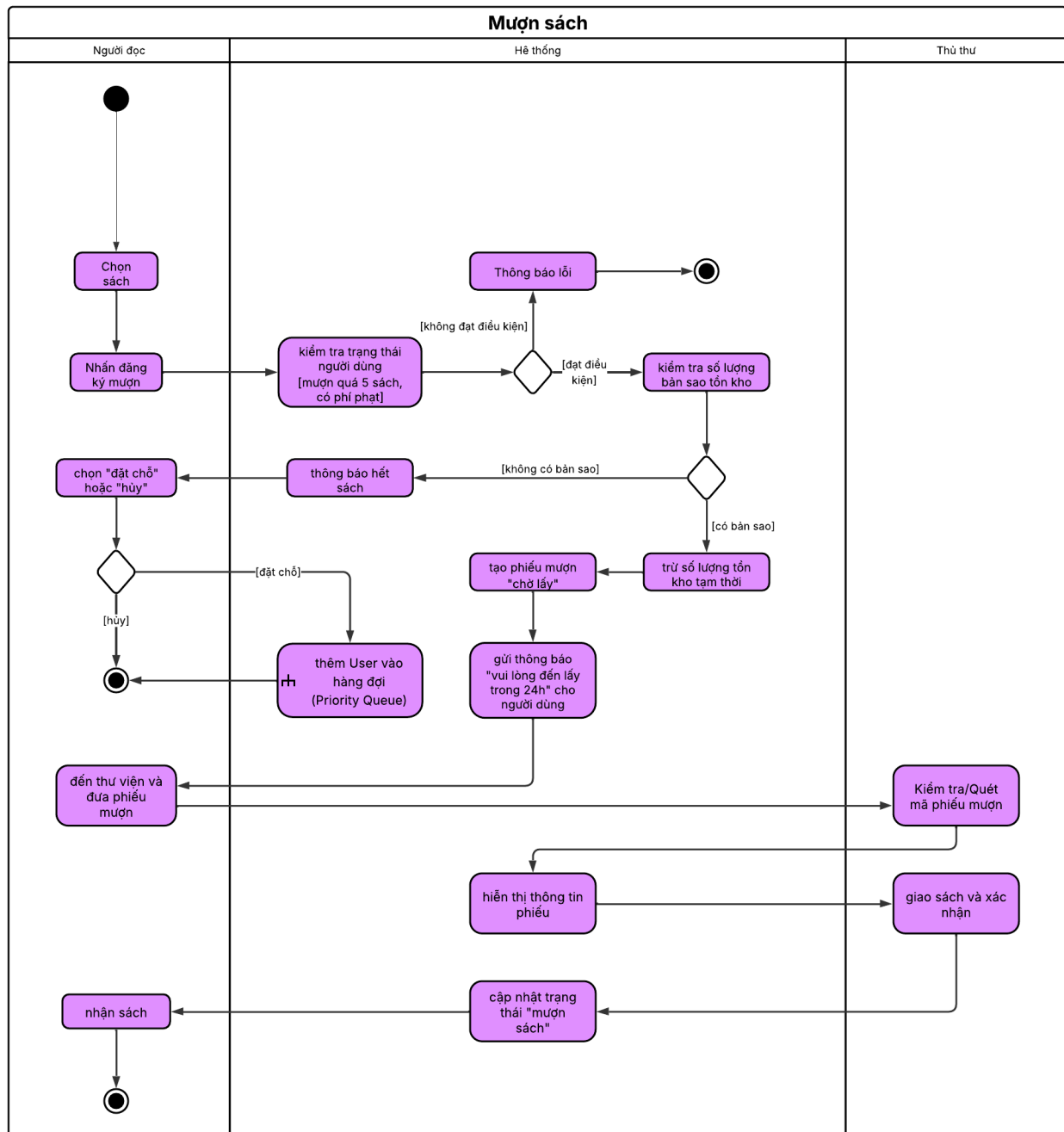
Hình 5.3: Đăng nhập tài khoản — Activity Diagram



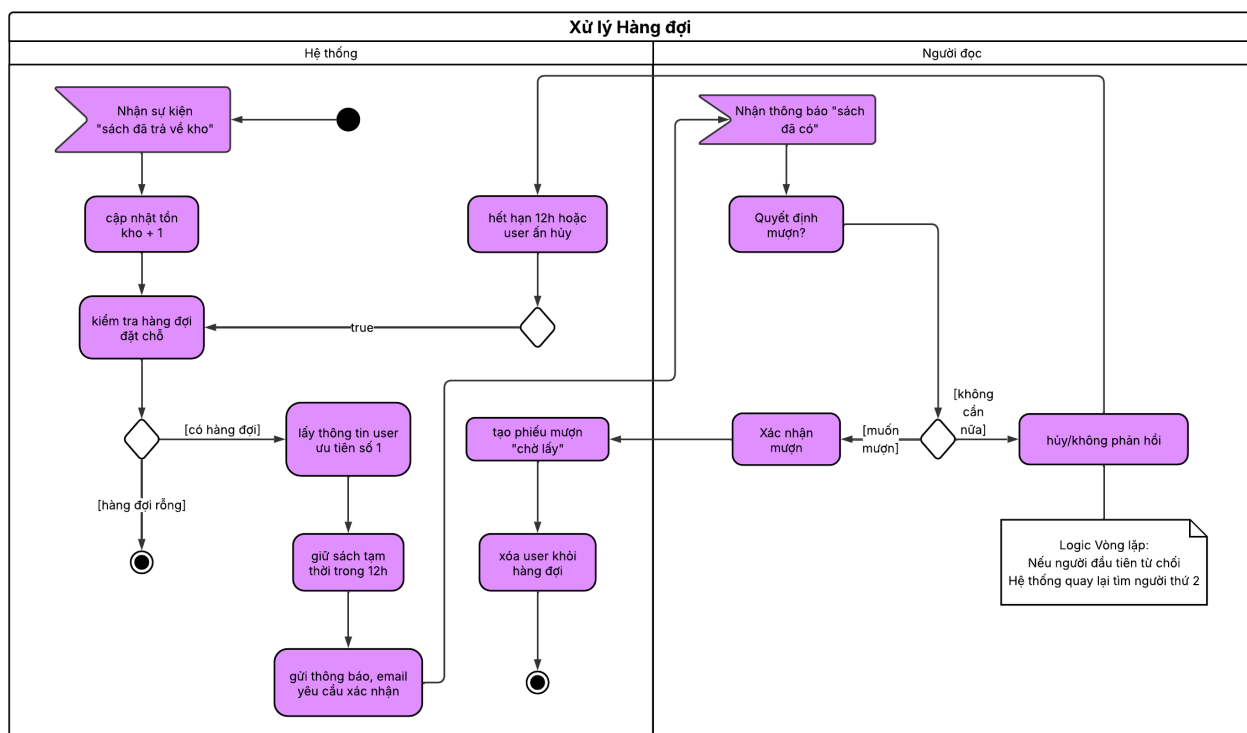
Hình 5.4: Tìm kiếm sách — Activity Diagram



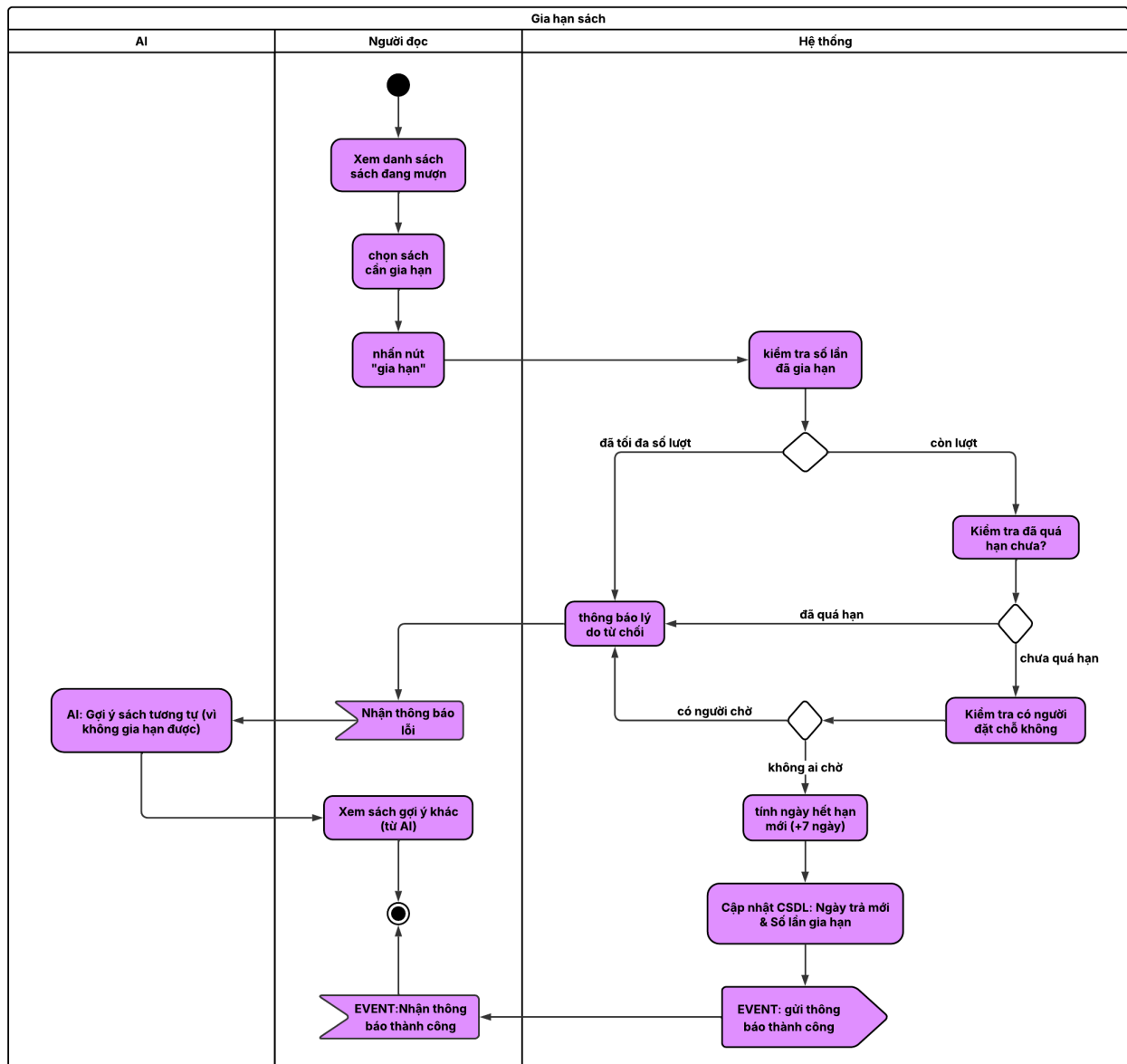
Hình 5.5: Gợi ý sách — Activity Diagram



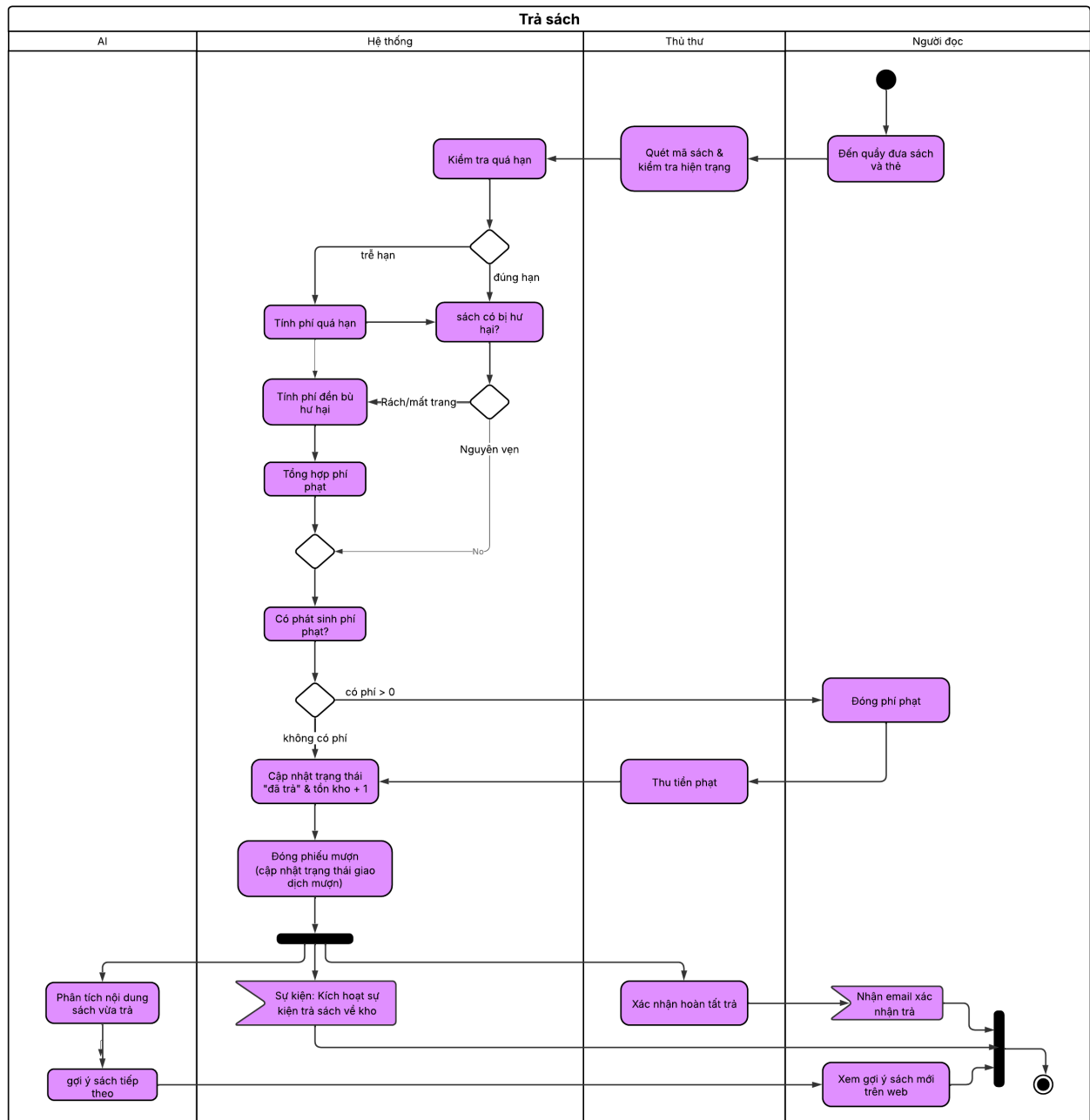
Hình 5.6: Mượn sách — Activity Diagram



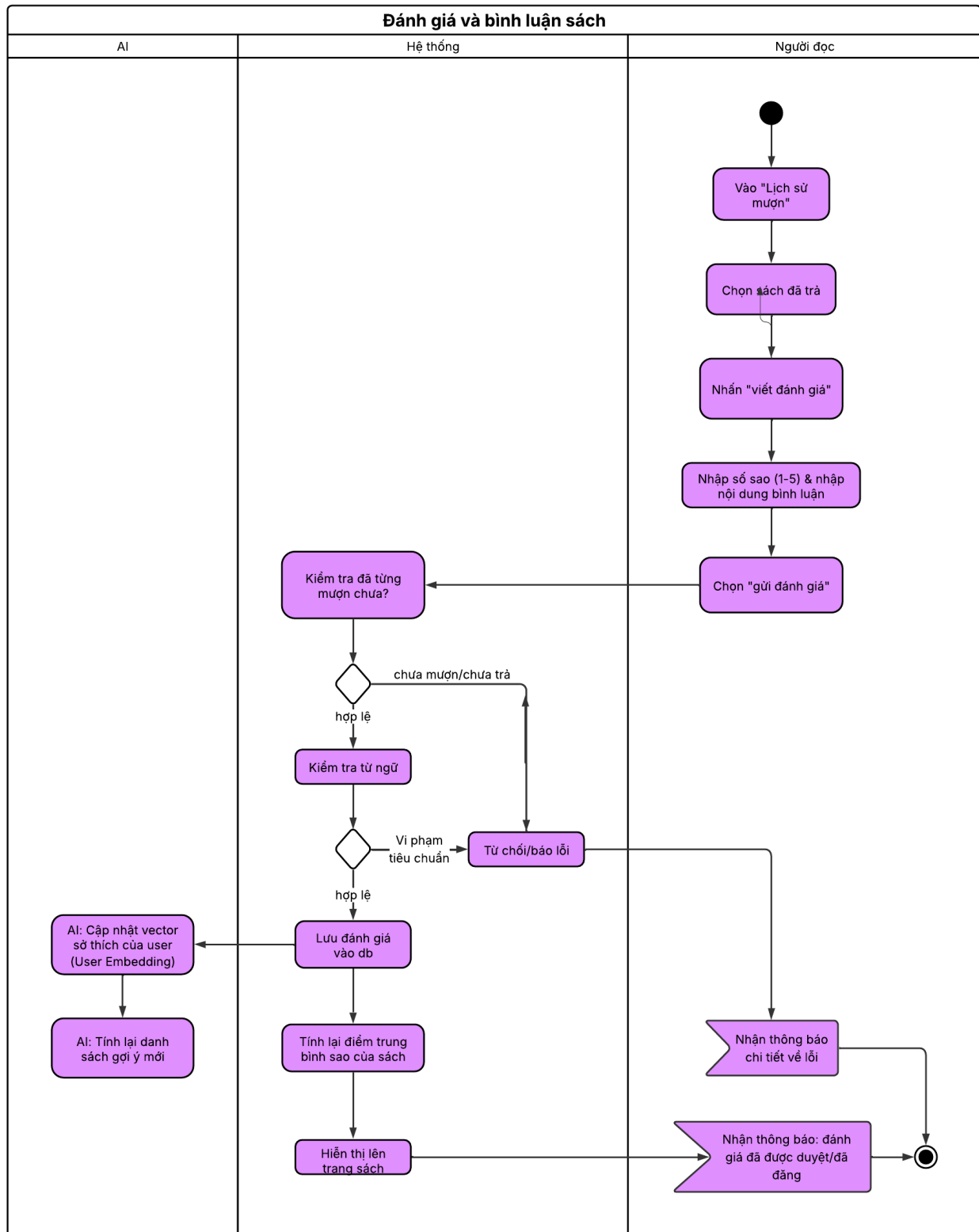
Hình 5.7: Xử lý hàng đợi — Activity Diagram



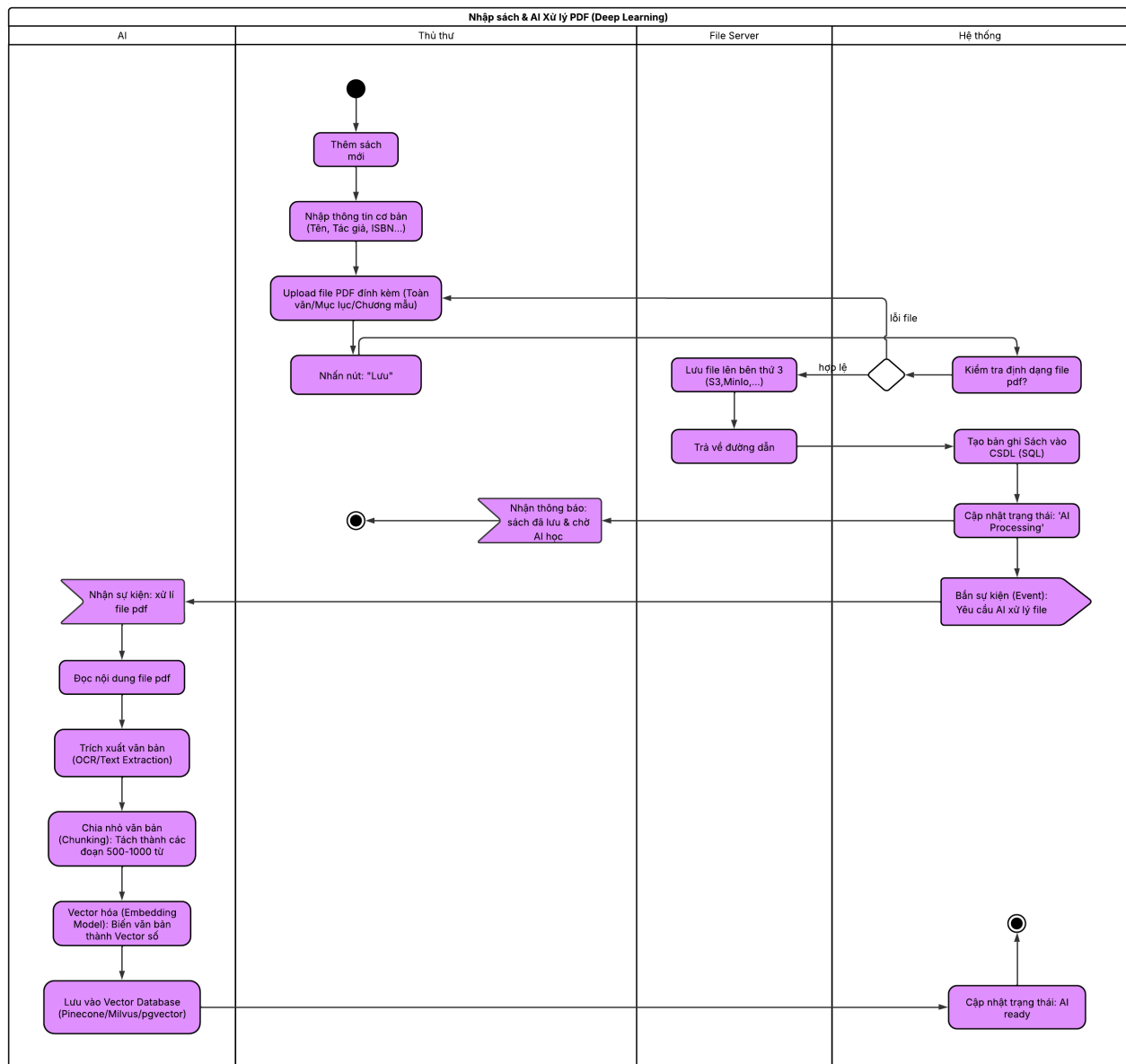
Hình 5.8: Gia hạn sách — Activity Diagram



Hình 5.9: Trả sách — Activity Diagram



Hình 5.10: Đánh giá và bình luận sách — Activity Diagram



Hình 5.11: Tạo sách với AI — Activity Diagram

5.2 Giải pháp công nghệ

Mỗi lựa chọn công nghệ được đưa ra dựa trên bộ tiêu chí: (1) Hiệu năng & độ trễ, (2) Khả năng mở rộng & bảo trì, (3) Bảo mật & phân quyền, (4) Trải nghiệm phát triển (DX) & hệ sinh thái, (5) Chi phí & mức độ phù hợp học thuật.

5.2.1 Front-end

Kiến trúc Front-end được xây dựng dựa trên **React + Vite** — kết hợp giữa thư viện giao diện phổ biến nhất hiện nay (React) và công cụ build thể hệ mới với tốc độ cực nhanh (Vite). Sự lựa chọn này ưu tiên trải nghiệm phát triển (Developer Experience) vượt trội, tốc độ phản hồi tức thì trong quá trình lập trình, và khả năng triển khai đơn giản dưới dạng file tĩnh thông qua NGINX.

Để đưa ra quyết định công nghệ phù hợp nhất, nhóm đã tiến hành so sánh các nền tảng phổ biến dựa trên đặc thù của hệ thống. Dưới đây là bảng phân tích chi tiết.

Nền tảng	Ưu điểm	Nhược điểm	Lý do chọn/không chọn
React + Vite (Được chọn)	Tốc độ HMR và build cực nhanh nhờ Vite (ESBuild), triển khai file tĩnh đơn giản qua NGINX, hệ sinh thái React phong phú, dễ tích hợp với mọi Backend API.	Client-side rendering thuần, SEO phụ thuộc vào cấu hình thêm, phải tự cấu hình Routing (React Router).	Được chọn vì hệ thống ưu tiên hiệu năng phát triển, triển khai nhẹ qua NGINX, và phần lớn giao diện là dashboard nội bộ không cần SEO.
Next.js	Hybrid Rendering (SSR/CSR/SSG), tối ưu SEO, hoạt động như BFF bảo mật.	Cần môi trường Node.js runtime để chạy, phức tạp hơn React thuần, tốn tài nguyên server hơn.	Không được chọn vì yêu cầu Node.js server riêng, phức tạp hóa việc triển khai khi hệ thống đã có Backend Spring Boot đảm nhận API.
Nuxt (Vue)	Mạnh mẽ tương đương Next.js trong hệ sinh thái Vue.	Cộng đồng doanh nghiệp nhỏ hơn React, ít thư viện UI hiện đại tương thích hơn.	Nhóm ưu tiên hệ sinh thái React do sự phổ biến và thư viện phong phú hơn.
Angular	Cấu trúc chặt chẽ, phù hợp Enterprise cực lớn.	Độ phức tạp cao , nhiều boilerplate, đường cong học tập dốc.	Quá cồng kềnh và cứng nhắc so với yêu cầu linh hoạt và phát triển nhanh của đồ án.

Bảng 5.6: Bảng so sánh các công nghệ Front-end

Lựa chọn: *React + Vite* kết hợp với *TypeScript*, *Tailwind CSS*, *Shadcn/ui*, *React Router* và *React Hook Form + Zod*.

Lý do chọn theo tiêu chí chuyên môn

- **Tốc độ phát triển vượt trội (Developer Experience):** Vite sử dụng ESBuild (viết bằng Go) để biên dịch, cho tốc độ khởi động server dev gần như tức thì và Hot Module Replacement (HMR) dưới 50ms bất kể kích thước dự án. So với Webpack truyền thống, thời gian build production giảm đáng kể, cho phép nhóm lập trình lập nhanh hơn trong quá trình phát triển.

- **Triển khai đơn giản qua NGINX:** React + Vite tạo ra các file tĩnh (HTML/CSS/JS) sau khi build, có thể phục vụ trực tiếp bởi NGINX mà không cần Node.js server runtime. Điều này giúp giảm tải nguyên máy chủ, đơn giản hóa cấu hình Docker và tận dụng khả năng cache tĩnh của NGINX để tăng hiệu năng.
- **Phù hợp với đặc thù hệ thống:** Phần lớn giao diện của hệ thống là các trang Dashboard nội bộ (quản lý mượn/trả, quản lý độc giả, thống kê) dành cho người dùng đã đăng nhập — những trang này không cần SEO. Bảo mật API được đảm bảo ở phía Backend Spring Boot (JWT), nên không cần thêm một lớp Node.js BFF trung gian.
- **Hệ sinh thái React và tính an toàn dữ liệu:** Sự kết hợp giữa **TypeScript** (định kiểu tĩnh) và **Zod** (validation runtime) đảm bảo tính toàn vẹn dữ liệu khi giao tiếp với Backend API. Thư viện **Shadcn/ui** (dựa trên Radix UI) và **Tailwind CSS** giúp xây dựng giao diện đẹp, nhất quán và dễ tiếp cận (Accessible) với lượng code tối thiểu.
- **Quản lý trạng thái và Routing:** **React Router v6** cung cấp hệ thống routing linh hoạt với Data Loaders và Error Boundaries tích hợp. Kết hợp với **TanStack Query** để quản lý server state và cache dữ liệu từ API, giúp giảm số lần gọi API trùng lặp và đồng bộ trạng thái giao diện tự động.

5.2.2 Back-end

Kiến trúc Back-end được xây dựng tập trung vào sự ổn định, an toàn và khả năng xử lý các nghiệp vụ phức tạp của thư viện.

Nền tảng	Ưu điểm	Nhược điểm	Lý do không được chọn
Spring Boot (Được chọn)	Ổn định, bảo mật cao , hệ sinh thái Java mạnh mẽ, quản lý giao dịch và dữ liệu quan hệ (JPA) rất tốt.	Có thể hơi “dài dòng” hơn so với các framework khác.	N/A
NestJS / Express	Linh hoạt, hiệu năng tốt, sử dụng chung ngôn ngữ JavaScript/TypeScript với front-end.	Việc xử lý các nghiệp vụ quan hệ phức tạp và giao dịch không mạnh mẽ bằng Spring Boot.	Spring Boot phù hợp hơn với bản chất dữ liệu quan hệ và giao dịch chặt chẽ của nghiệp vụ thư viện.
Django / DRF	Phát triển CRUD cực nhanh, hệ sinh thái Python mạnh.	Nhóm phát triển quen thuộc với hệ sinh thái Java hơn.	Stack công nghệ Java đã quen thuộc và đồng bộ hơn với kinh nghiệm của nhóm.
MongoDB (NoSQL)	Schema linh hoạt, dễ dàng mở rộng theo chiều ngang.	Không phù hợp với các mối quan hệ dữ liệu chặt chẽ và yêu cầu giao dịch cao của hệ thống thư viện.	Bản chất nghiệp vụ của dự án phù hợp hơn với cơ sở dữ liệu quan hệ (RDBMS).

Bảng 5.8: Bảng so sánh các công nghệ Back-end

Lựa chọn: *Spring Boot 3 (Java 21), Spring Security (JWT), Spring Data JPA, PostgreSQL, Redis, và OpenAPI/Swagger. Mô-đun AI/NLP được tách thành một service riêng sử dụng Python (FastAPI).*

Lý do chọn theo tiêu chí chuyên môn

- **Độ tin cậy và Hiệu năng:** Spring Boot là một framework đã được kiểm chứng về độ ổn định và hiệu năng trong các hệ thống enterprise. Kết hợp với **Java Virtual Machine (JVM)**, nó cung cấp khả năng

xử lý đồng thời (concurrency) mạnh mẽ, phù hợp với các tác vụ CRUD và giao dịch (transactions) phức tạp. Việc sử dụng **Redis** làm lớp cache giúp giảm tải cho cơ sở dữ liệu đối với các truy vấn tốn kém hoặc thường xuyên (ví dụ: gợi ý, top sách).

- **Mô hình Dữ liệu và Khả năng mở rộng:** Nghiệp vụ thư viện có bản chất là quan hệ chặt chẽ. **Spring Data JPA** và **Hibernate** cung cấp một tầng ORM (Object-Relational Mapping) mạnh mẽ, cho phép mô hình hóa các thực thể và mối quan hệ phức tạp một cách tự nhiên. Việc sử dụng **Specification Pattern** giúp xây dựng các truy vấn động một cách an toàn và dễ bảo trì, phục vụ cho các yêu cầu lọc và tìm kiếm phức tạp.
- **Bảo mật Toàn diện:** **Spring Security** là một framework bảo mật hàng đầu, cho phép triển khai cơ chế xác thực và phân quyền một cách linh hoạt. Việc sử dụng **JWT (JSON Web Tokens)** với cặp **Access/Refresh Token** đảm bảo một cơ chế xác thực stateless, phù hợp với kiến trúc API hiện đại. Luồng bảo mật được thiết kế để token không bao giờ được lưu trữ ở client-side storage, thay vào đó là sử dụng **HttpOnly cookies** để giảm thiểu rủi ro từ các cuộc tấn công **Cross-Site Scripting (XSS)**.
- **Kiến trúc Microservice cho AI:** Việc tách mô-đun AI/NLP (tìm kiếm ngữ nghĩa, gợi ý) thành một service riêng (Python/FastAPI) là một quyết định kiến trúc chiến lược. Nó cho phép hai hệ thống có thể được phát triển, triển khai và mở rộng quy mô một cách độc lập. Python và hệ sinh thái của nó là lựa chọn tối ưu cho các tác vụ Machine Learning, trong khi Java/Spring Boot mạnh hơn về xử lý nghiệp vụ.

Kết luận về Sự phối hợp giữa hai nền tảng

Kiến trúc tổng thể được thiết kế theo mô hình **đầu-cuối (end-to-end) an toàn và hiệu quả**. **Next.js** hoạt động như một lớp BFF, chịu trách nhiệm render giao diện và giao tiếp an toàn với **Spring Boot API**. Luồng xác thực sử dụng **HttpOnly cookies** để vận chuyển JWT, đảm bảo token không thể bị truy cập bởi mã JavaScript độc hại trên trình duyệt. Sự phân tách rõ ràng giữa các thành phần không chỉ giúp tối ưu hiệu năng mà còn tạo ra một hệ thống linh hoạt, dễ bảo trì và sẵn sàng cho việc mở rộng trong tương lai.

5.3 Ứng dụng AI vào hệ thống

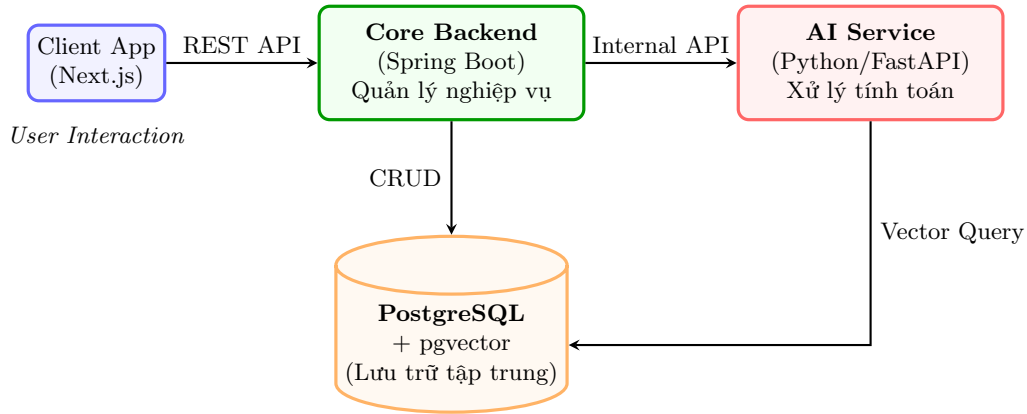
Bối cảnh và Mục tiêu

Mục tiêu cốt lõi là chuyển đổi trải nghiệm từ **"Tìm kiếm" (Searching)** sang **"Khám phá" (Discovering)**. Hệ thống sử dụng AI để thấu hiểu hành vi người dùng và nội dung sách, từ đó đưa ra các gợi ý phù hợp ngay cả khi người dùng chưa xác định rõ nhu cầu.

Để giải quyết bài toán gợi ý và tìm kiếm ngữ nghĩa trong thời gian thực, hệ thống được thiết kế theo kiến trúc Microservices lai. Logic AI được tách biệt hoàn toàn khỏi Core Backend để đảm bảo hiệu năng và khả năng mở rộng.

Đặc tả luồng dữ liệu:

- **Spring Boot (Business Layer):** Chịu trách nhiệm ghi nhận tương tác người dùng (User Interaction) và gửi yêu cầu phân tích sang AI Service. Việc tách biệt này giúp hệ thống nghiệp vụ vẫn hoạt động ổn định ngay cả khi AI Service đang quá tải [3].
- **AI Service (Intelligence Layer):** Được xây dựng bằng Python/FastAPI để tận dụng hệ sinh thái thư viện AI mạnh mẽ (PyTorch, Scikit-learn). Service này chạy các Batch Job định kỳ để huấn luyện mô hình và cung cấp API thời gian thực cho tính toán gợi ý.
- **PostgreSQL + pgvector (Data Layer):** Nhóm lựa chọn giải pháp lưu trữ "All-in-one" thay vì sử dụng Vector Database chuyên biệt (như Pinecone hay Milvus). Lý do là để đơn giản hóa kiến trúc và tận dụng khả năng truy vấn hỗn hợp (Hybrid Search) - kết hợp giữa lọc theo siêu dữ liệu (metadata filtering) và



Hình 5.12: Kiến trúc tích hợp hệ thống

tìm kiếm vector trong cùng một câu lệnh SQL [4].

5.3.1 Module 1: Tìm kiếm ngữ nghĩa (Semantic Search)

5.3.1.1 Cơ sở Lý thuyết: Text Embedding

Hệ thống sử dụng kỹ thuật **Dense Retrieval** thay vì Keyword Matching truyền thống (như SQL LIKE hay BM25). Các phương pháp cũ như TF-IDF hay GloVe [5] gặp hạn chế lớn trong việc hiểu ngữ cảnh của câu (context-aware), vì chúng chỉ biểu diễn từ đơn lẻ hoặc không thay đổi theo ngữ cảnh (static embeddings) [6].

Công nghệ lõi nhóm lựa chọn là **Sentence-BERT (SBERT)** [7], một biến thể của BERT [8] sử dụng kiến trúc mạng Siamese (Siamese Networks) để tinh chỉnh (fine-tune) việc tạo ra các sentence embedding. **Lý giải lựa chọn SBERT (Sentence-BERT):**

- Mặc dù BERT [8] đạt hiệu quả cao trong các tác vụ NLP, nhưng nó sử dụng kiến trúc Cross-Encoder, yêu cầu đưa cả hai câu vào mạng cùng lúc để so sánh. Điều này dẫn đến độ phức tạp tính toán khổng lồ khi tìm kiếm trong cơ sở dữ liệu lớn.
- Theo Reimers và Gurevych [7], nhóm sử dụng kiến trúc **Bi-Encoder** của SBERT để tính toán vector độc lập cho từng văn bản. Điều này cho phép tính toán trước (Pre-compute) vector cho toàn bộ kho sách, chuyển bài toán tìm kiếm về bài toán tìm láng giềng gần nhất (ANN), đảm bảo thời gian phản hồi thời gian thực.

Mô hình toán học: Mô hình biến đổi một đoạn văn bản T (Text) thành một vector số học cố định $v \in \mathbb{R}^{768}$:

$$f_{\text{SBERT}}(T) \rightarrow v = [x_1, x_2, \dots, x_{768}] \quad (5.1)$$

Trong đó, độ tương đồng giữa câu truy vấn q và tài liệu d được tính bằng **Cosine Similarity**:

$$\text{sim}(q, d) = \frac{v_q \cdot v_d}{\|v_q\| \|v_d\|} = \frac{\sum_{i=1}^n q_i d_i}{\sqrt{\sum_{i=1}^n q_i^2} \sqrt{\sum_{i=1}^n d_i^2}} \quad (5.2)$$

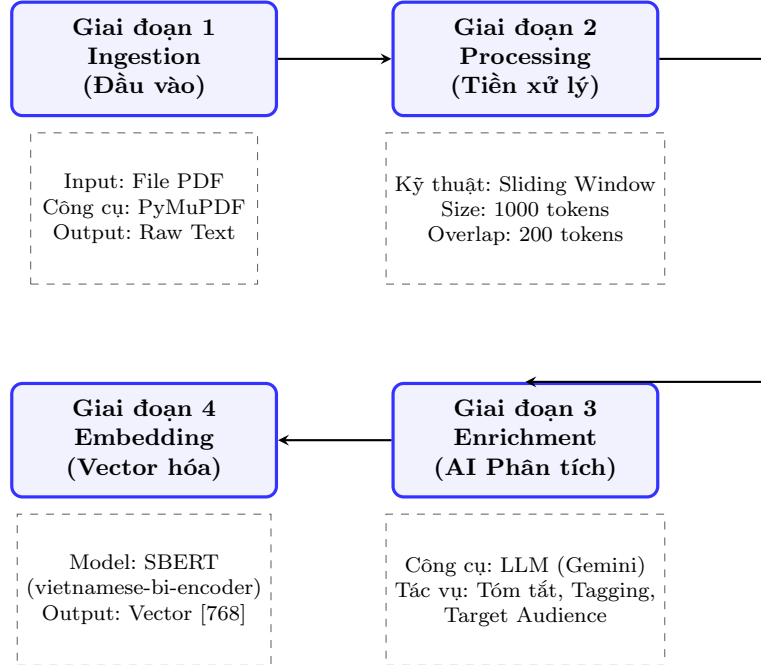
Ví dụ: Mô hình biến đổi một đoạn văn bản (Text) thành một vector số học cố định (Vector) trong không gian nhiều chiều (768 chiều).

$$f(\text{"Lập trình C++"}) \rightarrow [0.12, -0.59, 0.21, \dots, 0.05] \quad (5.3)$$

Các văn bản có ngữ nghĩa gần nhau sẽ có vector nằm gần nhau trong không gian này.

5.3.1.2 Quy trình Xử lý Dữ liệu Sách (ETL Pipeline)

Để chuyển đổi một file PDF "vô tri" thành các thực thể có ý nghĩa trong cơ sở dữ liệu, nhóm thiết kế một quy trình ETL gồm 4 giai đoạn: **Ingestion** (Thu thập) → **Processing** (Xử lý) → **Enrichment** (Làm giàu) → **Embedding** (Vector hóa).



Hình 5.13: Quy trình Vector hóa nội dung sách

Giai đoạn 1: Ingestion (Trích xuất) Sử dụng thư viện PyMuPDF để đọc nội dung file PDF. Hệ thống áp dụng Regular Expression (Regex) để loại bỏ nhiễu (header, footer, số trang) nhằm đảm bảo văn bản sạch trước khi đưa vào xử lý.

Giai đoạn 2: Processing (Cắt đoạn) Vì giới hạn đầu vào của các mô hình ngôn ngữ (Context Window), chúng ta không thể đưa cả cuốn sách vào một lần.

- **Kỹ thuật:** *Sliding Window* (Cửa sổ trượt).
- **Thông số:** Kích thước đoạn (Chunk size) là 1000 tokens, với độ chồng lấn (Overlap) là 200 tokens.
- **Mục đích:** Đảm bảo không làm đứt gãy mạch văn hoặc mất ngữ cảnh quan trọng tại các điểm cắt.

Giai đoạn 3: Enrichment (Làm giàu dữ liệu) Nhóm sử dụng **Large Language Model (LLM)** để trích xuất thông tin ẩn mà các phương pháp NLP truyền thống khó làm được. Prompt Engineering được thiết kế để trả về cấu trúc JSON nghiêm ngặt, giúp dễ dàng tích hợp vào Database.

Mẫu Prompt gửi cho LLM:

PROMPT = ""

Đọc nội dung sau từ cuốn sách và thực hiện các tác vụ:

1. Tóm tắt nội dung chính (Summary) trong khoảng 150 từ.
2. Xác định 5 Tags (Thẻ) quan trọng nhất mô tả chủ đề.
3. Xác định Đối tượng độc giả phù hợp (Target Audience).
4. Trả về định dạng JSON (không kèm markdown):

```
{
  "summary": "...",
  "tags": ["...", "..."],
```

```
"target_audience": ["...", "..."]
}
```

Ví dụ kết quả thực tế (Sách "Clean Code"):

- **Summary:** "Cuốn sách hướng dẫn các nguyên tắc và thực hành để viết mã sạch, dễ bảo trì và mở rộng. Tác giả nhấn mạnh vào việc đặt tên biến, refactoring và kiểm thử đơn vị."
- **Tags:** ["Software Engineering", "Refactoring", "Java", "Best Practices", "Agile"]
- **Target Audience:** ["Lập trình viên", "Kỹ sư phần mềm", "Sinh viên CNTT năm cuối"]

Giai đoạn 4: Embedding và Lưu trữ Cuối cùng, phần summary (đã được cô đọng) sẽ được vector hóa.

- **Model:** Sử dụng `bkai-foundation-models/vietnamese-bi-encoder` (hoặc mô hình tương đương hỗ trợ tiếng Việt tốt).
- **Output:** Vector $v \in \mathbb{R}^{768}$.
- **Lưu trữ:** Vector này được lưu vào cột embedding kiểu vector (768) trong PostgreSQL.
- **Tối ưu:** Sử dụng chỉ mục **HNSW** (Hierarchical Navigable Small World) để tăng tốc độ truy vấn vector thay vì quét toàn bộ bảng.

5.3.1.3 Thuật toán Tìm kiếm

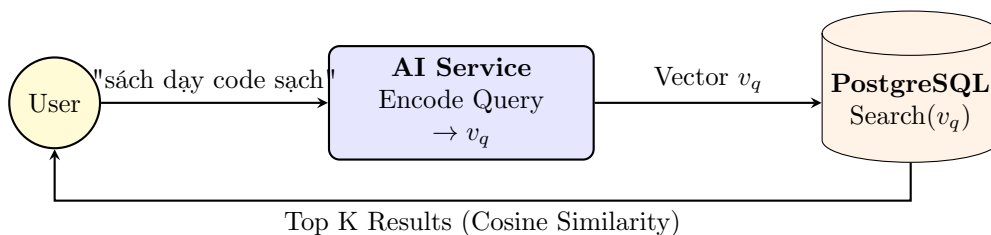
Khi người dùng nhập câu truy vấn q :

1. AI Service chuyển q thành vector v_q .
2. Tính độ tương đồng Cosine giữa v_q và tất cả vector sách v_d trong Database.

$$\text{Similarity}(v_q, v_d) = \frac{v_q \cdot v_d}{\|v_q\| \|v_d\|} \quad (5.4)$$

3. Trả về danh sách sách có độ tương đồng cao nhất.

Khi người dùng thực hiện tìm kiếm, hệ thống không so khớp từ khóa đơn thuần mà so khớp ý định (Semantic Match).



Hình 5.14: Luồng thực thi tìm kiếm ngữ nghĩa

5.3.2 Module 2: Hệ thống gợi ý (Recommendation)

Chiến lược Lai ghép (Hybridization Strategy): Theo phân loại của **Burke** (2002) [9], hệ thống áp dụng chiến lược **Switching Hybrid** (Lai ghép chuyển đổi).

- Hệ thống chọn lựa thuật toán dựa trên trạng thái dữ liệu của người dùng (tính sẵn có của Profile).
- Chiến lược này giải quyết trực tiếp điểm yếu lớn nhất của Collaborative Filtering là bài toán *Cold-start*

(Khởi động lạnh). Khi người dùng mới (U_{new}) chưa có vector tương tác, nhánh CF sẽ thất bại vì ma trận thưa. Lúc này, hệ thống "chuyển mạch" sang CBF để gợi ý dựa trên thuộc tính sách (Item attributes) mà không cần lịch sử người dùng.

Cụ thể, mô hình lai ghép này được vận hành bởi sự phối hợp giữa hai cơ chế cốt lõi sau:

- **Collaborative Filtering (CF - Lọc cộng tác):** Sử dụng kỹ thuật phân rã ma trận (Matrix Factorization) với thuật toán tối ưu **ALS (Alternating Least Squares)** dành cho dữ liệu phản hồi ẩn theo đề xuất của **Hu et al. [10]**. Phương pháp này khai thác các mẫu hành vi tiềm ẩn của cộng đồng để tạo ra tính "ngẫu nhiên thú vị" (Serendipity) - gợi ý những sách người dùng *có thể thích* dù nội dung hoàn toàn khác với lịch sử đọc trước đó.
- **Content-Based Filtering (CBF - Lọc nội dung):** Ứng dụng biểu diễn **Vector Embedding** (dựa trên không gian vector của Mikolov [6] và kiến trúc SBERT [7]) để đo lường độ tương đồng ngữ nghĩa giữa các cuốn sách. Phương pháp này đóng vai trò "lưới an toàn", đảm bảo hệ thống luôn đưa ra được gợi ý liên quan ngay cả khi chưa có dữ liệu hành vi người dùng.

5.3.2.1 Mô hình IMPLICIT MATRIX FACTORIZATION

Trong môi trường thư viện, người dùng hiếm khi chủ động đánh giá (rating 1-5 sao). Dữ liệu chủ yếu là **Phản hồi ẩn (Implicit Feedback)** như: xem chi tiết, thêm vào wishlist, mượn sách. Do đó, nhóm sử dụng thuật toán **ALS (Alternating Least Squares)** được tối ưu cho dữ liệu ẩn.

Định nghĩa bài toán

Giả sử chúng ta có tập người dùng U và tập sách I .

- r_{ui} : Giá trị thô (raw count) thể hiện mức độ tương tác của người dùng u với sách i (Ví dụ: số lần xem, số lần mượn).
- p_{ui} : Biến nhị phân thể hiện "sở thích" (Preference):

$$p_{ui} = \begin{cases} 1 & \text{nếu } r_{ui} > 0 \\ 0 & \text{nếu } r_{ui} = 0 \end{cases}$$

- c_{ui} : Độ tin cậy (Confidence Level) vào sở thích đó. Tương tác càng nhiều, độ tin cậy càng cao:

$$c_{ui} = 1 + \alpha \cdot r_{ui} \quad (5.5)$$

(Trong đó α là siêu tham số, thực nghiệm chọn $\alpha = 40$).

Nguồn gốc và Ý nghĩa công thức: Mô hình và công thức độ tin cậy c_{ui} này được đề xuất bởi **Hu, Koren và Volinsky** trong nghiên cứu "*Collaborative Filtering for Implicit Feedback Datasets*" (2008) [10].

- Khác với Explicit Feedback (người dùng chấm điểm 1-5 sao rõ ràng), Implicit Feedback (click, xem) chứa nhiều nhiễu. Việc người dùng *không* tương tác với một cuốn sách ($r_{ui} = 0$) không đồng nghĩa với việc họ *ghét* nó, mà có thể do họ chưa nhìn thấy.
- Công thức trên giả định rằng: Mọi tương tác dương ($r_{ui} > 0$) đều phản ánh sở thích ($p_{ui} = 1$), nhưng với các mức độ chắc chắn khác nhau. Hệ số α kiểm soát tốc độ tăng của độ tin cậy. Việc sử dụng hàm tuyến tính đơn giản này đã được chứng minh là hiệu quả và đủ tốt để biểu diễn mối quan hệ giữa tần suất hành vi và sở thích người dùng.

Hàm mục tiêu (Loss Function)

Mục tiêu là tìm ra hai ma trận tiềm ẩn (Latent Factors):

- Ma trận người dùng $X \in \mathbb{R}^{m \times f}$ (mỗi hàng là vector x_u).
- Ma trận sách $Y \in \mathbb{R}^{n \times f}$ (mỗi hàng là vector y_i).

Sao cho tích vô hướng $x_u^T y_i$ xấp xỉ được sở thích p_{ui} . Hàm mất mát cần tối thiểu hóa là:

$$L(X, Y) = \sum_{u,i} c_{ui} (p_{ui} - x_u^T y_i)^2 + \lambda \left(\sum_u \|x_u\|^2 + \sum_i \|y_i\|^2 \right) \quad (5.6)$$

Trong đó λ là hệ số Regularization để chống Overfitting.

Phân tích thành phần hàm Loss:

- **Weighted Squared Error (Thành phần đầu):** Không giống như Matrix Factorization cổ điển chỉ tính trên các ô có dữ liệu (Observed ratings), mô hình này tính toán trên *toàn bộ* cặp (u, i) bao gồm cả các ô trống. Trọng số c_{ui} đóng vai trò cốt yếu: nó phạt nặng sai số trên các sách người dùng đã tương tác (nơi ta có độ tin cậy cao), và phạt nhẹ hơn trên các sách chưa tương tác (nơi ta không chắc chắn).
- **Regularization (Thành phần sau - λ):** Do ma trận người dùng-sách rất thưa (Sparsity > 99%), mô hình rất dễ bị Overfitting (học vẹt các nhiễu). Thành phần chuẩn hóa L_2 giúp kiểm soát độ lớn của các vector tiềm ẩn x_u, y_i , đảm bảo mô hình có khả năng tổng quát hóa tốt hơn cho các dữ liệu mới.

Phương pháp giải tối ưu - ALS: Vì hàm mục tiêu trên không lồi (non-convex) khi xét chung cả hai biến X, Y , nhóm sử dụng giải thuật **ALS (Alternating Least Squares)**. Bằng cách cố định X để tối ưu Y và ngược lại, bài toán trở thành quy hoạch toàn phương (Quadratic) lồi, có thể giải chính xác và dễ dàng song song hóa trên môi trường phân tán.

5.3.2.2 Kỹ thuật Dữ liệu (Feature Engineering)

Dữ liệu được trích xuất từ bảng UserInteraction trong PostgreSQL và được quy hoạch thành các trọng số như sau:

Loại tương tác (Event)	Ý nghĩa hành vi	Điểm (r_{ui})
VIEW_DETAILS	Tò mò, tìm hiểu	1
ADD_TO_WISHLIST	Quan tâm, lưu lại	5
BORROW_SUCCESS	Hành động thực tế	10

Bảng 5.9: Bảng quy đổi điểm tương tác cho mô hình ALS

SQL Query trích xuất dữ liệu huấn luyện:

Listing 5.1: Truy vấn tổng hợp điểm tương tác User-Item

```
SELECT
  u.id as user_idx,
  b.id as item_idx,
  SUM(
    CASE
      WHEN interaction_type = 'VIEW' THEN 1
      WHEN interaction_type = 'WISHLIST' THEN 5
      WHEN interaction_type = 'BORROW' THEN 10
      ELSE 0
    )
```

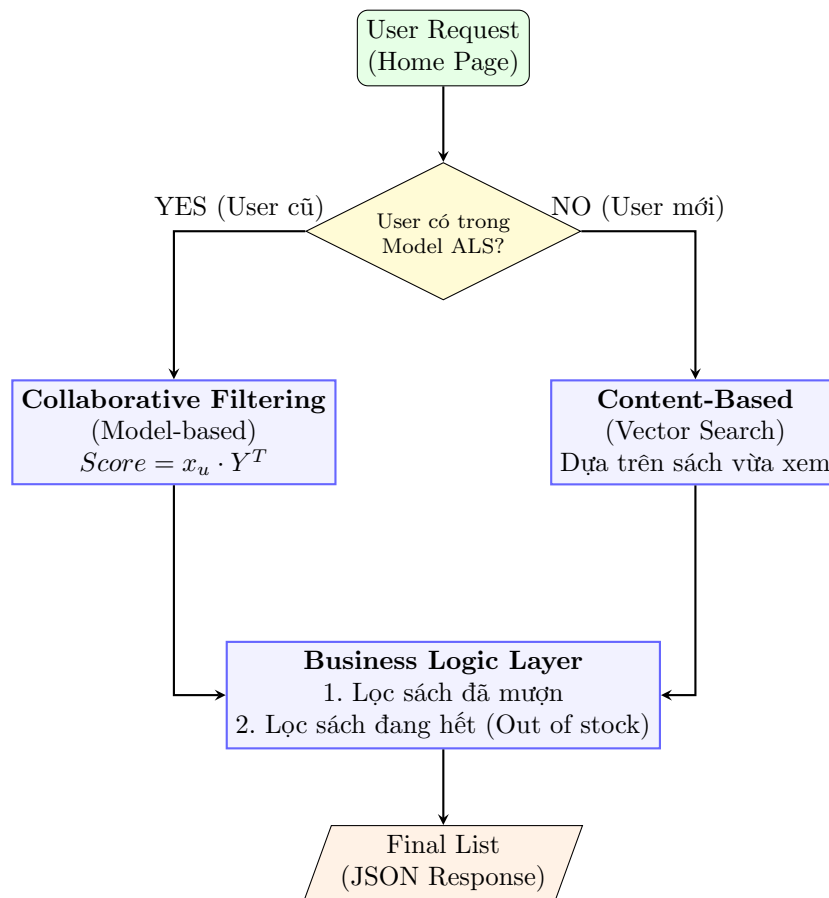
```

END
) as raw_score
FROM user_interactions ui
JOIN users u ON ui.user_id = u.id
JOIN publications b ON ui.publication_id = b.id
GROUP BY u.id, b.id;

```

5.3.2.3 Luồng xử lý Gợi ý (Recommendation Pipeline)

Hệ thống hoạt động theo cơ chế thời gian thực (Real-time Inference) với chiến lược phân nhánh.



Hình 5.15: Lược đồ thuật toán phân luồng gợi ý (Switching Hybrid)

5.3.2.4 Chi tiết logic phân luồng:

1. Bước 1: Nhận diện Người dùng.

- Hệ thống kiểm tra User ID trong từ điển ánh xạ của mô hình ALS đã huấn luyện.

2. Bước 2: Chọn Chiến lược (Strategy Selection).

- Nhánh CF (User cũ):** Sử dụng vector tiềm ẩn x_u của người dùng nhân với toàn bộ ma trận item Y^T . Kết quả là danh sách điểm dự đoán cho tất cả các sách. Lấy Top-N sách có điểm cao nhất.
- Nhánh CBF (User mới/Vãng lai):**
 - Nếu người dùng đang xem một cuốn sách cụ thể → Gợi ý các sách có vector nội dung (SBERT) gần nhất (More like this).

– Nếu người dùng chưa xem gì → Gợi ý các sách "Trending" (mượn nhiều nhất trong 7 ngày qua).

3. Bước 3: Hậu xử lý (Post-Processing).

- **Bloom Filter:** Loại bỏ các sách người dùng *đã mượn* hoặc *đã xem* để tránh gợi ý lại nội dung cũ.
- **Availability Check:** Kiểm tra tồn kho thực tế trong Database để đảm bảo sách gợi ý có thể mượn được ngay.

5.4 Thiết kế

5.4.1 Kiến trúc hệ thống

5.4.1.1 So sánh và lựa chọn kiến trúc hệ thống

Kiến trúc hệ thống là nền móng quyết định khả năng mở rộng, bảo trì và tiến hóa của toàn bộ sản phẩm phần mềm, việc lựa chọn kiến trúc cần cân nhắc đồng thời ba nhóm yêu cầu then chốt:

1. **Tính nhất quán giao dịch (Transactional Consistency):** Các nghiệp vụ mượn/trả sách đòi hỏi cập nhật đồng thời nhiều thực thể (trạng thái sách, giao dịch mượn, tồn kho, hàng đợi) trong một transaction ACID để tránh inconsistency.
2. **Tích hợp AI phức tạp:** Hệ thống cần xử lý các tác vụ machine learning resource-intensive như OCR từ PDF, tạo vector embeddings, và tìm kiếm ngữ nghĩa - các workload này có đặc điểm công nghệ và scaling requirements hoàn toàn khác biệt so với CRUD nghiệp vụ.
3. **Nguồn lực hạn chế:** Với bối cảnh đồ án sinh viên, cần tối ưu hóa chi phí vận hành (infrastructure cost) và độ phức tạp triển khai, đồng thời vẫn đảm bảo khả năng mở rộng trong tương lai.

A. Các kiểu kiến trúc phổ biến:

Hiện nay, có nhiều kiểu kiến trúc hệ thống phổ biến, mỗi loại có những ưu và nhược điểm riêng:

- **Kiến trúc Phân lớp (Layered Architecture):** Một kiểu monolith truyền thống, phân tách logic theo các lớp kỹ thuật (Presentation, Business, Data Access).
- **Kiến trúc Monolith dạng Mô-đun (Modular Monolith):** Một khối ứng dụng duy nhất nhưng được cấu trúc nội bộ thành các mô-đun nghiệp vụ (domain) rõ ràng, độc lập tương đối với nhau.
- **Kiến trúc Hướng dịch vụ (Service-Based Architecture):** Kiến trúc lai giữa Monolith và Microservices, hệ thống được chia thành một số ít các dịch vụ nghiệp vụ lớn (coarse-grained), thường sử dụng chung cơ sở dữ liệu nhưng có thể triển khai độc lập.
- **Kiến trúc Vi dịch vụ (Microservices):** Chia nhỏ hệ thống thành nhiều dịch vụ nhỏ, độc lập, mỗi dịch vụ đảm nhận một chức năng nghiệp vụ cụ thể và có thể được triển khai, mở rộng độc lập.
- **Kiến trúc Hướng sự kiện (Event-Driven Architecture):** Các thành phần trong hệ thống giao tiếp với nhau một cách bất đồng bộ thông qua việc phát và lắng nghe các sự kiện (events).
- **Các kiến trúc khác:** Microkernel, Space-Based Architecture.

B. So sánh các kiến trúc:

Để lựa chọn kiến trúc phù hợp nhất, nhóm đã phân tích các kiểu kiến trúc dựa trên các tiêu chí quan trọng được thể hiện trong biểu đồ so sánh dưới đây.

C. Phân tích loại trừ: Từ biểu đồ trên kết hợp với các yêu cầu nghiệp vụ đã xác định, nhóm tiến hành loại trừ các kiến trúc không phù hợp:

								
partitioning	technical	domain	domain	domain	domain	technical	technical	technical
cost	\$	\$	\$	\$\$\$\$	\$\$	\$\$\$\$	\$\$\$	\$\$\$\$
maintainability	★	★★	★★★	★★★★★	★★★★★	★	★★★	★★★
testability	★★	★★	★★★	★★★★★	★★★★★	★	★★	★
deployability	★	★★	★★★	★★★★★	★★★★★	★	★★★	★★★
simplicity	★★★★★	★★★★★	★★★★	★	★★★	★	★★	★
scalability	★	★	★	★★★★★	★★★★	★★★	★★★★★	★★★★★
elasticity	★	★	★	★★★★★	★★	★★★	★★★★	★★★★★
responsiveness	★★★	★★★	★★★	★★	★★★	★★	★★★★★	★★★★★
fault-tolerance	★	★	★	★★★★★	★★★★★	★★★	★★★★★	★★★
evolvability	★	★	★★★	★★★★★	★★★★★	★	★★★★★	★★★
abstraction	★	★	★★★	★	★	★★★★★	★★★★★	★
interoperability	★	★	★★★	★★★	★★	★★★★★	★★★	★★

Hình 5.16: So sánh các đặc tính của các kiểu kiến trúc (Nguồn: Mark Richards, Neal Ford, *Fundamentals of Software Architecture*, O'Reilly, 2nd ed., 2020) [1].

• **Loại bỏ Kiến trúc Vi dịch vụ (Microservices):**

- *Ưu điểm từ biểu đồ:* Đạt điểm tuyệt đối về khả năng mở rộng (scalability: 5 sao), triển khai độc lập (deployability: 5 sao), và khả năng chịu lỗi (fault-tolerance: 5 sao).
- *Nhược điểm nghiêm trọng:* Chi phí vận hành ở mức cao nhất (\$\$\$\$\$) và độ phức tạp có điểm thấp nhất (simplicity: 1 sao).
- *Mâu thuẫn với yêu cầu:* Với nguồn lực hạn chế của đồ án sinh viên, việc vận hành hạ tầng phân tán (service discovery, distributed tracing, container orchestration) là không khả thi. Hơn nữa, các nghiệp vụ mượn/trả sách đòi hỏi cập nhật đồng thời nhiều aggregate (giao dịch, tồn kho, hàng đợi) - nếu tách thành microservices sẽ phải xử lý distributed transactions phức tạp (Saga pattern, 2PC:Two-Phase Commit) với rủi ro eventual consistency.

• **Loại bỏ Kiến trúc Phân lớp (Layered) truyền thống:**

- *Ưu điểm từ biểu đồ:* Đơn giản nhất (simplicity: 5 sao) và chi phí thấp nhất (cost: \$).
- *Nhược điểm nghiêm trọng:* Các chỉ số quan trọng cho sự phát triển dài hạn đều ở mức thấp: khả năng bảo trì (maintainability: 1 sao), khả năng kiểm thử (testability: 2 sao), và tính linh hoạt (evolvability: 1 sao).
- *Mâu thuẫn với yêu cầu:* Kiến trúc phân lớp theo kỹ thuật (technical layers) dễ dẫn đến tight coupling giữa các logic nghiệp vụ. Đối với hệ thống có yêu cầu tích hợp AI phức tạp (xử lý bất đồng bộ PDF, cập nhật vector embeddings, trigger recommendation engine), kiến trúc này dễ dẫn đến "big ball of mud" với business logic rải rác khắp các layer.

• **Cân nhắc về Kiến trúc Hướng dịch vụ (Service-Based):**

- *Ưu điểm từ biểu đồ:* Sự cân bằng tốt với hầu hết chỉ số đạt 3–4 sao (maintainability: 4 sao, testability: 4 sao, deployability: 4 sao) và chi phí vừa phải (cost: \$\$).
- *Hạn chế trong bối cảnh dự án:* Việc tách ngay từ đầu toàn bộ các nghiệp vụ thư viện thành nhiều service riêng biệt (User Service, Catalog Service, Lending Service, Fine Service) là **over-engineering** cho giai đoạn MVP. Các service này có phụ thuộc lẫn nhau cao (mượn sách cần kiểm tra user credit score, catalog availability, và fine status) dẫn đến nhiều inter-service calls, tăng network latency và độ phức tạp debugging.

- *Kết luận:* Service-Based là hướng đi tương lai hợp lý khi hệ thống cần scale, nhưng chưa phù hợp cho giai đoạn đầu.

D. Quyết định: Kiến trúc Modular Monolith kết hợp Dịch vụ vệ tinh AI

Dựa trên phân tích trên, nhóm quyết định áp dụng một cách tiếp cận **thực dụng và có định hướng phát triển**, kết hợp ưu điểm của Modular Monolith với tính linh hoạt của Service-Based Architecture:

1. Lõi hệ thống: Modular Monolith (Spring Boot)

Đây là kiến trúc chủ đạo cho các nghiệp vụ quản lý thư viện (User Management, Catalog, Lending, Fine Management, Engagement).

Lý do lựa chọn dựa trên biểu đồ so sánh:

- **Cân bằng tối ưu:** Modular Monolith cung cấp maintainability (3 sao) và evolvability (3 sao) tốt hơn hẳn Layered Architecture (1 sao) nhưng vẫn giữ được chi phí thấp (cost: \$) và độ đơn giản cao (simplicity: 5 sao).
- **Testability:** Đạt 3 sao - cao hơn đáng kể so với Layered (2 sao) nhờ tách biệt rõ ràng giữa các domain module.
- **Deployability:** Dù chỉ đạt 3 sao (thấp hơn Microservices), nhưng lại phù hợp với nguồn lực đồ án: chỉ cần deploy một artifact duy nhất, không cần orchestration phức tạp.

Lợi ích nghiệp vụ cụ thể:

- **Tính nhất quán giao dịch (ACID):** Các use case phức tạp như trả sách (cần atomic update: trạng thái giao dịch + tính phạt + cập nhật tồn kho + trigger hàng đợi) được đảm bảo trong một transaction duy nhất mà không cần distributed transaction protocols.
- **Tính module hóa cao:** Mặc dù là monolith, code được tổ chức theo **domain boundaries** (không phải technical layers). Mỗi module (User, Lending, Catalog...) có domain model, use cases, và repositories riêng, giao tiếp thông qua well-defined interfaces. Điều này tạo "exit strategy" - dễ dàng tách module ra thành service riêng trong tương lai nếu cần.
- **DevOps đơn giản:** Chỉ cần một deployment pipeline, một database connection pool, không cần service mesh hay API gateway phức tạp.

2. Dịch vụ vệ tinh: AI Service (Python/FastAPI)

Mô-đun AI/Recommendation System được tách ra thành một dịch vụ độc lập, giao tiếp với Monolith qua REST API và Message Queue (RabbitMQ).

Lý do tách riêng:

- **Công nghệ khác biệt:** Hệ sinh thái Python (LangChain, HuggingFace Transformers, Sentence-BERT) vượt trội cho ML/NLP so với Java. Việc nhúng Python vào Spring Boot (qua Jython) sẽ gây overhead và mất đi performance của native libraries.
- **Scaling requirements khác biệt:** AI workload (PyMuPDF, vector similarity search, model inference) tiêu tốn CPU/GPU nhiều hơn CRUD nghiệp vụ. Tách riêng cho phép scale độc lập (horizontal scaling với GPU instances) mà không tốn tài nguyên cho monolith.
- **Bất đồng bộ tự nhiên:** Các tác vụ AI (xử lý PDF sau khi thêm sách, cập nhật user embeddings sau khi rating, mượn sách, tương tác) không cần real-time response, có thể chạy background qua message queue. Điều này không block main business flow.
- **Tính module hóa:** AI Service có database riêng (Vector DB: pgvector), không share schema với

monolith. Giao tiếp qua API contract rõ ràng, cho phép thay đổi AI stack (model architecture, vector database) mà không ảnh hưởng hệ thống chính.

Migration path: Đây là bước đệm hướng tới Service-Based Architecture. Trong tương lai, nếu cần scale, có thể tách dần các module khác (User Service, Notification Service) ra khỏi monolith theo pattern tương tự.

E. Đánh giá kiến trúc lại theo các chỉ số: Kiến trúc Modular Monolith + AI Satellite này đạt được sự cân bằng tốt nhất cho bối cảnh dự án:

Bảng 5.10: Đánh giá kiến trúc lại so với các chỉ số

Tiêu chí	Điểm	Nhận xét
Cost	\$\$	Giữa Modular Monolith (\$) và Service-Based (\$\$), phù hợp ngân sách
Simplicity	3-4 sao	Đơn giản hơn Microservices nhưng cần quản lý 2 components
Maintainability	3-4 sao	Module hóa domain tốt, dễ sửa lỗi và refactor
Testability	3 sao	Domain logic tách biệt, dễ unit test
Deployability	2-3 sao	Deploy 2 artifacts (Monolith + AI), CI/CD đơn giản
Scalability	3 sao	AI service scale độc lập, monolith scale theo nhu cầu
Evolvability	4 sao	Có thể tách dần modules ra thành services khi cần

Kết luận: Kiến trúc Modular Monolith + AI Satellite Service đạt được ba mục tiêu then chốt: (1) Đảm bảo tính nhất quán giao dịch cho nghiệp vụ core, (2) Tối ưu hóa AI workload với công nghệ và scaling phù hợp, (3) Giữ độ phức tạp và chi phí ở mức chấp nhận được cho đồ án, đồng thời tạo nền tảng để mở rộng thành Service-Based Architecture hoặc Microservice Architecture trong tương lai.

5.4.2 Thiết kế cơ sở dữ liệu

Thiết kế cơ sở dữ liệu là quá trình chuyển đổi từ mô hình phân tích yêu cầu và các thực thể đã được đặc tả ở các phần trước thành một cấu trúc lưu trữ logic và vật lý. Mục tiêu là xây dựng một nền tảng dữ liệu nhất quán, toàn vẹn, hiệu quả và có khả năng mở rộng để hỗ trợ toàn bộ nghiệp vụ của hệ thống, từ các giao dịch mượn trả cơ bản đến các tính năng AI phức tạp.

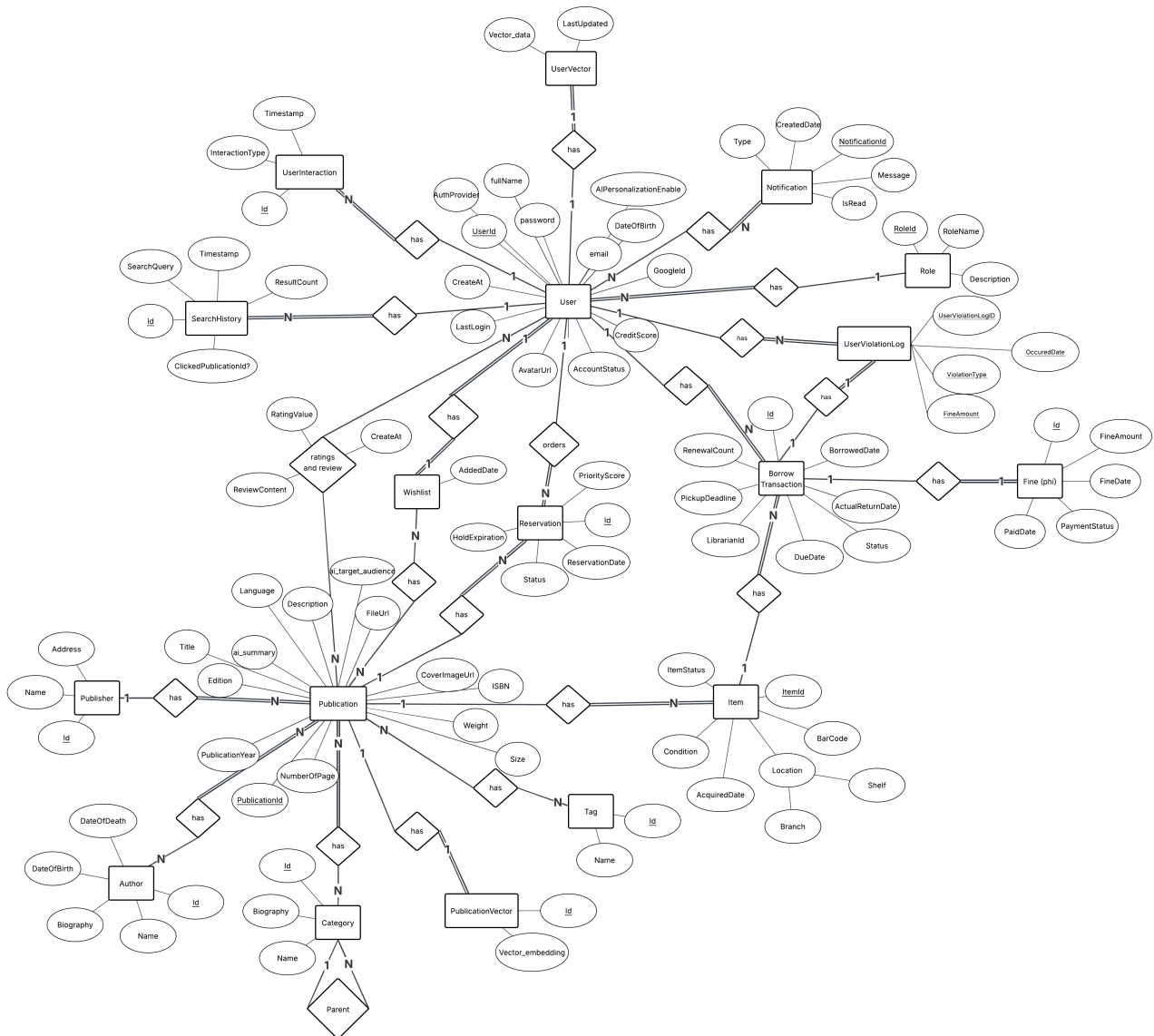
5.4.2.1 Lựa chọn Hệ Quản trị Cơ sở dữ liệu (HQTCSDL)

Dựa trên bản chất quan hệ chặt chẽ của dữ liệu (người dùng, sách, lượt mượn), hệ thống lựa chọn **PostgreSQL** làm HQTCSDL chính.

- **Tính tin cậy và toàn vẹn dữ liệu:** PostgreSQL nổi tiếng với sự tuân thủ nghiêm ngặt các tiêu chuẩn ACID (Atomicity, Consistency, Isolation, Durability), đảm bảo các giao dịch nghiệp vụ quan trọng như mượn, trả, và tính phí phạt luôn được thực hiện một cách an toàn.
- **Hỗ trợ các kiểu dữ liệu nâng cao:** PostgreSQL hỗ trợ các kiểu dữ liệu phức tạp như JSONB, rất hữu ích để lưu trữ các metadata linh hoạt. Đặc biệt, thông qua các extension như `pgvector`, nó có khả năng lưu trữ và truy vấn các vector embedding hiệu quả, phục vụ trực tiếp cho tính năng tìm kiếm ngữ nghĩa.
- **Khả năng mở rộng:** Đây là một hệ quản trị CSDL mã nguồn mở mạnh mẽ, đã được chứng minh về hiệu năng trong các hệ thống quy mô lớn.

5.4.2.2 Mô hình Quan hệ Thực thể (ERD)

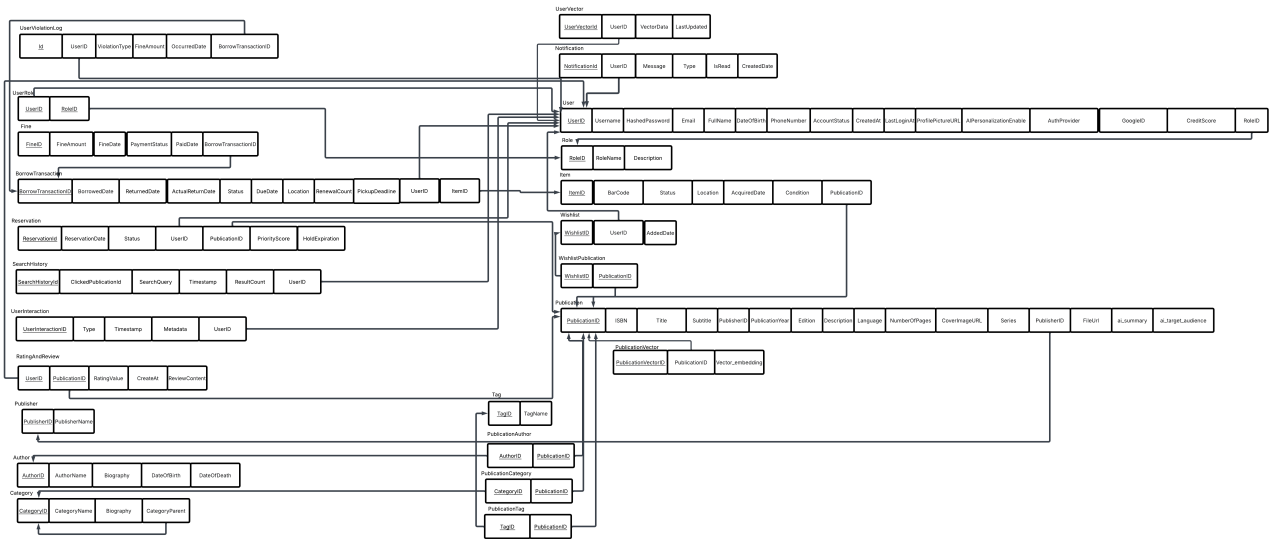
Bên dưới là sơ đồ ERD tổng quan của toàn bộ hệ thống.



Hình 5.17: Sơ đồ Quan hệ Thực thể (ERD) tổng thể của hệ thống

5.4.2.3 Ánh xạ lược đồ cơ sở dữ liệu quan hệ

Sau khi thực hiện quá trình ánh xạ từ mô hình ERD ở trên, ta thu được một lược đồ cơ sở dữ liệu quan hệ như trình bày bên dưới. Trong lược đồ này, các thuộc tính chi tiết đã được lược bớt nhằm tập trung làm rõ mối quan hệ giữa các bảng trong hệ thống.



Hình 5.18: Ảnh xạ cơ sở dữ liệu quan hệ ở dạng tổng quát

5.4.3 Thiết kế chi tiết bên trong

5.4.3.1 Mô hình kiến trúc chi tiết

Sau khi quyết định sử dụng kiến trúc Modular Monolith kết hợp AI Satellite Service, phần này trình bày chi tiết cấu trúc nội bộ của từng thành phần và các nguyên tắc thiết kế được áp dụng.

A. Tổng quan kiến trúc hệ thống: Hệ thống bao gồm ba thành phần chính:

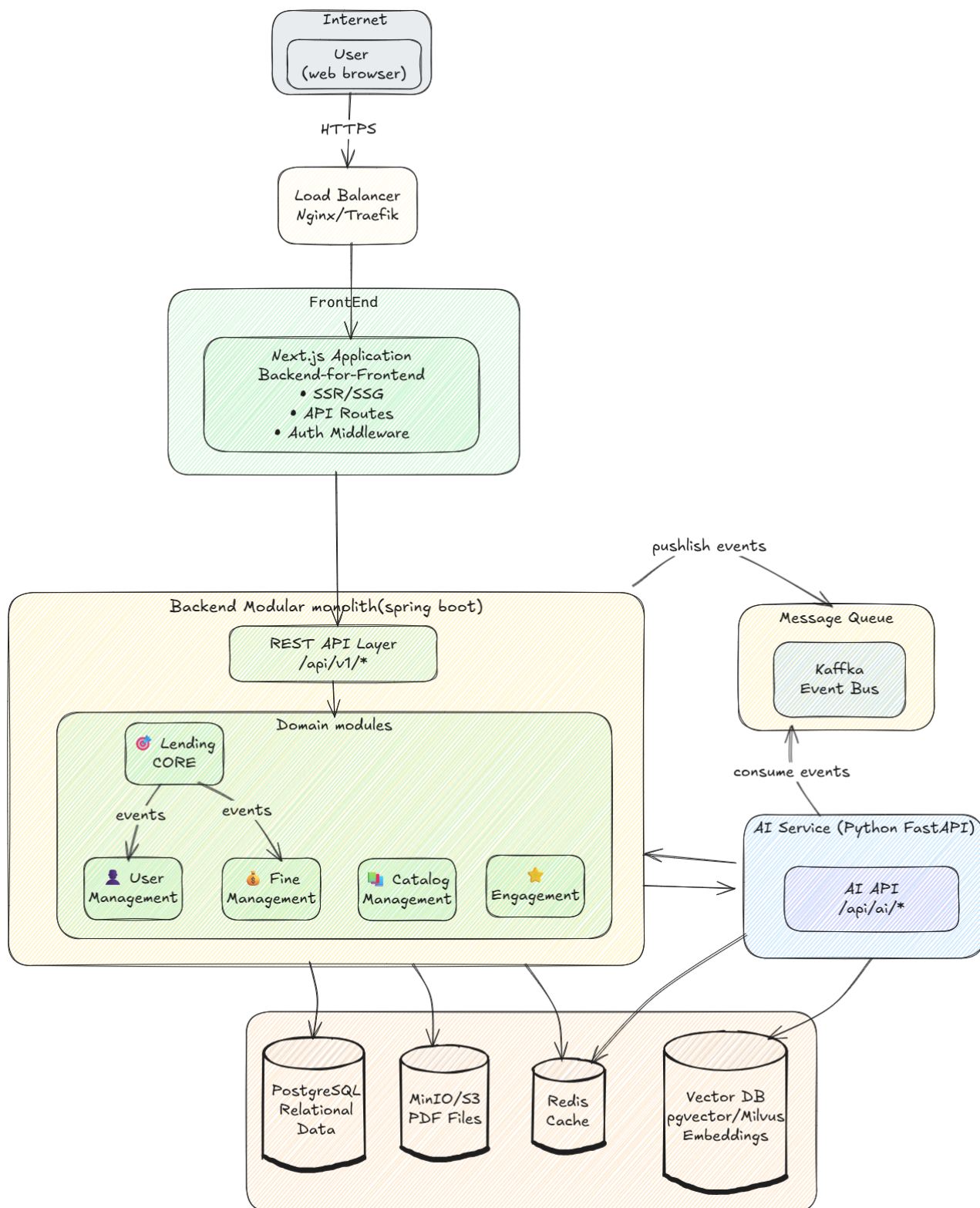
- Frontend (Next.js):** Giao diện người dùng đóng vai trò Backend-for-Frontend (BFF), xử lý SSR và tổng hợp API calls.
- Backend Monolith (Spring Boot):** Lõi nghiệp vụ áp dụng Domain-Driven Design và Clean Architecture.
- AI Service (Python FastAPI):** Dịch vụ độc lập xử lý các tác vụ Machine Learning và Vector Search.

B.1). Kiến trúc Backend Monolith (Spring Boot) Tổ chức theo Bounded Contexts (DDD):

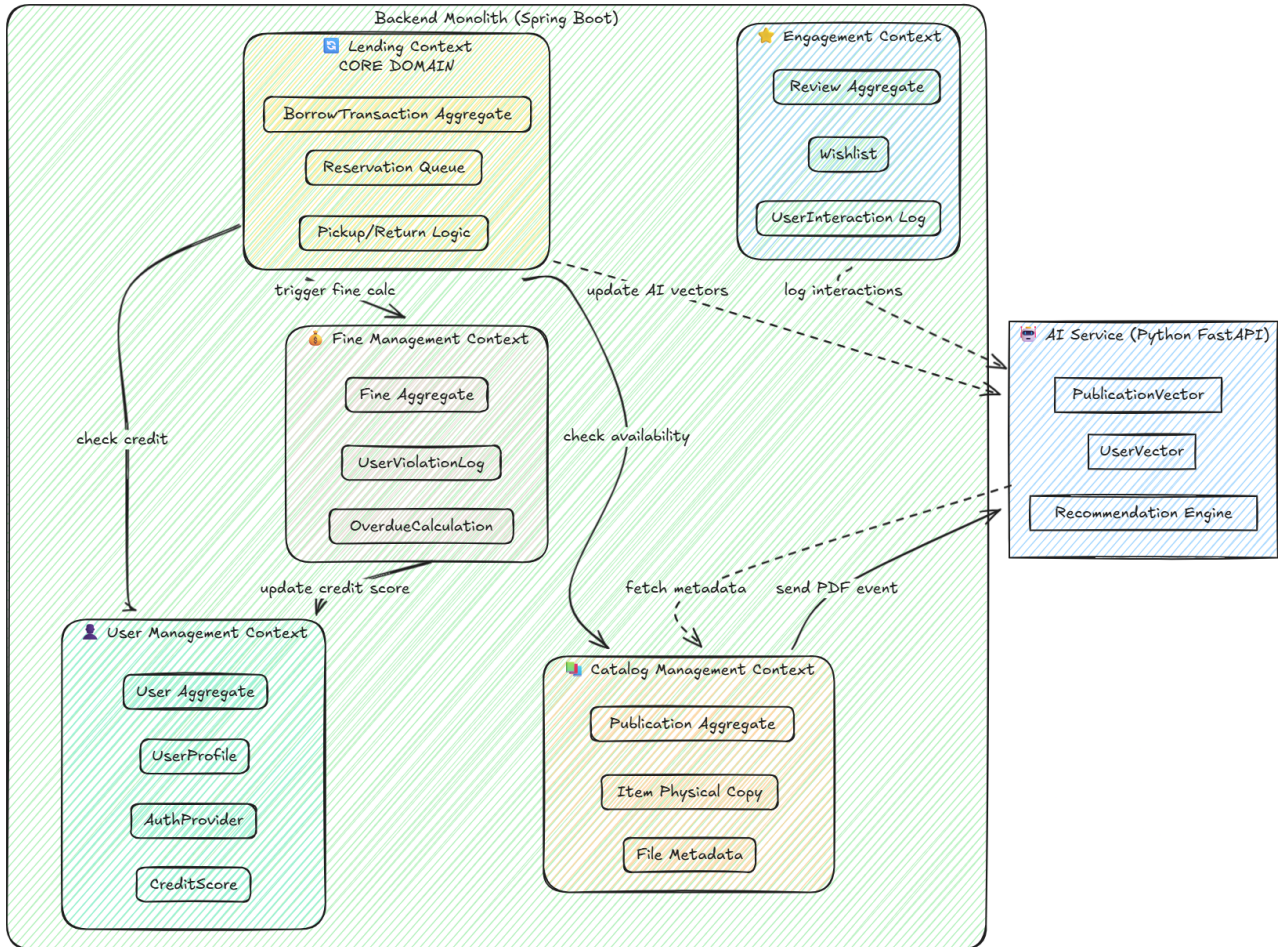
Thay vì tổ chức code theo các lớp kỹ thuật ngang (technical layers), hệ thống được chia thành các module theo ranh giới nghiệp vụ (domain boundaries), mỗi module tương ứng với một Bounded Context trong Domain-Driven Design:

- User Management Context:** Quản lý người dùng, xác thực (Local + Google OAuth), phân quyền, và credit scoring.
- Catalog Management Context:** Quản lý thông tin xuất bản phẩm (Publication), các bản sao vật lý (Item), metadata sách, và file storage.
- Lending Context:** (*Core Domain*) - Quản lý vòng đời mượn/trả sách (BorrowTransaction), hàng đợi đặt chỗ (Reservation), và các business rules phức tạp về deadline, pickup, renewal.
- Fine Management Context:** Quản lý tiền phạt, vi phạm người dùng (UserViolationLog), tính toán credit score.
- Engagement Context:** Quản lý đánh giá sách (Review), danh sách yêu thích (Wishlist), và log tương tác người dùng (UserInteraction).

Mỗi Bounded Context được thiết kế như một "mini-service" bên trong monolith, có thể tách ra thành service



Hình 5.19: Sơ đồ kiến trúc tổng thể hệ thống



Hình 5.20: Các Bounded Contexts trong hệ thống

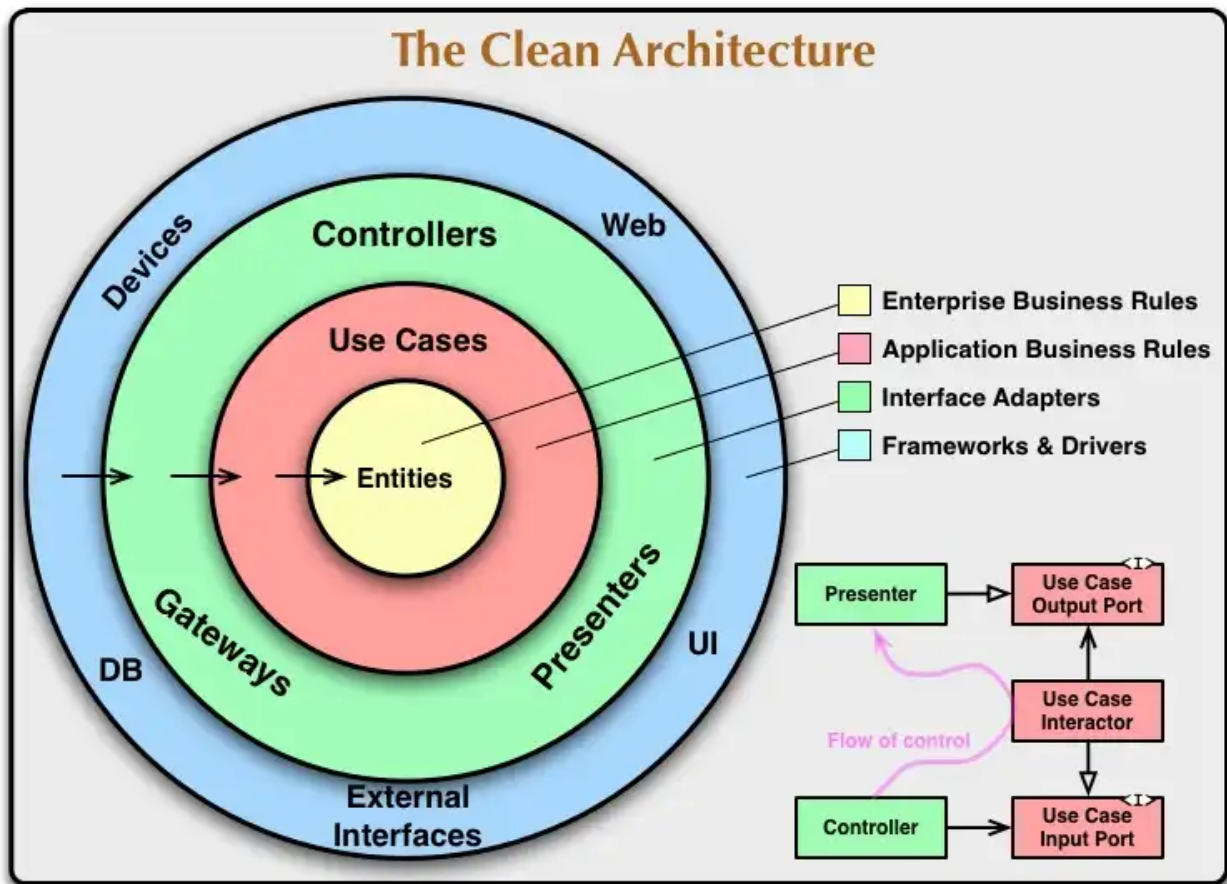
độc lập trong tương lai mà không cần refactor lớn.

Clean Architecture bên trong mỗi Module:

Bên trong mỗi Bounded Context, code được tổ chức theo 4 layers của Clean Architecture để đảm bảo tách biệt trách nhiệm và independence từ framework/database:

1. Domain Layer (Lõi nghiệp vụ - Innermost):

- **Entities và Aggregates:** Chứa các đối tượng nghiệp vụ cốt lõi với business invariants. Ví dụ:
 - BorrowTransaction Aggregate: Đảm bảo các quy tắc như "không được pickup sau 24h", "tối đa gia hạn 2 lần", "phải kiểm tra hư hại khi trả".
 - Publication Aggregate: Đảm bảo "ít nhất một Item để có thể cho mượn", "ISBN không trùng".
 - User Aggregate: Đảm bảo "credit score ≥ 0 ", "không mượn quá 5 cuốn cùng lúc".
- **Value Objects:** Các đối tượng bất biến đại diện cho khái niệm nghiệp vụ không có identity riêng (ví dụ: ISBN, DateRange, Address).
- **Domain Services:** Logic nghiệp vụ không thuộc về một Entity/Aggregate cụ thể. Ví dụ:
 - OverdueCalculationService: Tính tiền phạt dựa trên số ngày trễ và chính sách thư viện.
 - PriorityQueueService: Tính điểm ưu tiên cho hàng đợi đặt chỗ.



Hình 5.21: Clean architecture (R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*, Pearson Education, 2018 [2])

- **Domain Events:** Biểu diễn các sự kiện nghiệp vụ quan trọng để các module khác có thể phản ứng (loose coupling):
 - BookBorrowedEvent, BookReturnedEvent
 - FineIncurredEvent, UserBannedEvent
 - ReservationExpiredEvent, AIProcessingCompletedEvent
- **Repository Interfaces:** Domain định nghĩa các interface (contract) mà Infrastructure sẽ implement. Domain không quan tâm dùng JPA, JDBC hay MongoDB.
- **Ràng buộc nghiêm ngặt:** Domain Layer KHÔNG ĐƯỢC phụ thuộc vào bất kỳ layer nào khác, không import Spring annotations, không biết HTTP hay Database. Đây là "business logic in pure Java".

2. Application Layer (Use Cases - Orchestrator):

- **Use Case Services:** Mỗi use case là một entry point cho một chức năng nghiệp vụ hoàn chỉnh. Ví dụ:
 - BorrowBookUseCase: Điều phối luồng mượn sách (validate user → check availability → create transaction → publish event).
 - ReturnBookUseCase: Điều phối luồng trả sách (update transaction → check damage → calculate fine → trigger queue → notify AI).
 - ProcessReservationQueueUseCase: Xử lý hàng đợi (find next user → notify → set hold expi-



Hình 5.22: Cấu trúc Clean Architecture kết hợp Domain-Driven Design trong một Module

ration).

- **Trách nhiệm chính:**

- Orchestrate workflow: Gọi repositories để load aggregates, gọi domain services, và save kết quả.
- Quản lý transaction boundaries (sử dụng `@Transactional` annotation của Spring).
- Publish domain events sau khi transaction commit thành công.
- Transform exceptions: Chuyển domain exceptions thành application-level error codes.

- **Application Services:** Các facade cho phép module khác gọi vào (public API giữa các modules).

- **DTOs (Data Transfer Objects):** Chuyển đổi giữa domain objects và external representations (JSON, DTO cho API).

- **Lưu ý:** Application Layer được phép phụ thuộc vào Domain Layer và định nghĩa interfaces cho Infrastructure (Dependency Inversion Principle).

3. Infrastructure Layer (Technical Implementation - Outermost):

- **Repository Implementations:** Hiện thực các repository interfaces từ Domain bằng JPA/Hibernate, JDBC, hoặc bất kỳ persistence technology nào. Ví dụ:

- `BorrowTransactionRepositoryJpa`: Implements `BorrowTransactionRepository` interface.
- Mapping giữa Domain Aggregate (`BorrowTransaction`) và JPA Entity (`BorrowTransactionJpaEntity`).

- **External Service Adapters:** Tích hợp với các dịch vụ bên ngoài:

- Email/SMS Service (gửi notification)
- File Storage (MinIO/S3 để lưu PDF sách)
- Payment Gateway (thanh toán phạt trực tuyến)
- OAuth Provider (Google Authentication)

- **Event Infrastructure:** Hiện thực event publishing/subscribing:

- In-memory event bus (Spring `ApplicationEventPublisher`) cho giao tiếp nội bộ giữa các modules.
- Message Queue client (RabbitMQ) cho giao tiếp với AI Service.

- **Technical Services:**

- Scheduler (Spring `@Scheduled`) cho background jobs (check overdue daily).
- Caching (Redis/Caffeine) cho frequently accessed data.
- Security (JWT token generation/validation, password hashing).

- **Configuration:** Database connection pools, message queue configuration, security settings.

4. Presentation Layer (Interface Adapters):

- **REST Controllers:** Expose các use cases qua HTTP/REST API. Mỗi Bounded Context có controller riêng:

- `UserController (/api/v1/users/*)`
- `LendingController (/api/v1/lending/*)`
- `CatalogController (/api/v1/books/*)`

- **Trách nhiệm:**

- Nhận HTTP requests, validate syntax (JSON format, required fields, regex).
- Chuyển đổi Request DTOs → Commands/Queries cho Application Layer.
- Authentication & Authorization: Extract user info từ JWT, check permissions.
- Chuyển đổi kết quả từ Application Layer → HTTP Response (JSON).
- Error handling: Chuyển exceptions thành HTTP status codes (400, 401, 403, 404, 500).

- **Nguyên tắc:** Controllers phải "thin" - chỉ làm việc chuyển đổi và delegation, KHÔNG chứa business logic.

Dependency Rule (Nguyên tắc phụ thuộc): Clean Architecture tuân thủ nguyên tắc: **Dependencies chỉ được point inward** (từ ngoài vào trong):

Presentation → Application → Domain ← Infrastructure

- Domain Layer: Không phụ thuộc vào ai (pure business logic).
- Application Layer: Chỉ phụ thuộc Domain, định nghĩa interfaces cho Infrastructure.
- Infrastructure & Presentation: Phụ thuộc cả Domain và Application, implement interfaces.

Điều này đảm bảo business logic (Domain) không bị "ô nhiễm" bởi framework, database hay HTTP concerns.

B. Kiến trúc AI Service (Python FastAPI) Hệ thống AI Service được thiết kế như một *Dịch vụ Vệ tinh Độc lập* (Autonomous Satellite Microservice), tách biệt hoàn toàn khỏi Monolith (Spring Boot). Quyết định này nhằm tận dụng sức mạnh của hệ sinh thái Python trong các tác vụ Học máy (Machine Learning) và Xử lý ngôn ngữ tự nhiên (NLP), đồng thời đảm bảo khả năng mở rộng độc lập cho các tác vụ tính toán nặng.

B.1. Chiến lược thiết kế và Lựa chọn công nghệ Khác với Backend chính tập trung vào tính toàn vẹn dữ liệu giao dịch, AI Service tập trung vào hiệu năng tính toán ma trận và xử lý bất đồng bộ.

- **Web Framework - FastAPI:** Được lựa chọn thay vì Flask/Django nhờ khả năng xử lý bất đồng bộ (Native Async/Await) vượt trội, cho phép phục vụ hàng nghìn yêu cầu suy luận (inference) đồng thời mà không bị tắc nghẽn I/O. Tích hợp sẵn Pydantic giúp kiểm soát chặt chẽ kiểu dữ liệu đầu vào cho các mô hình ML.
- **Kiến trúc Phi trạng thái (Stateless):** AI Service không lưu trữ session người dùng. Mọi trạng thái cần thiết được truyền qua payload của request hoặc truy xuất từ Database/Cache, giúp dễ dàng mở rộng theo chiều ngang (Horizontal Scaling) bằng cách tăng số lượng container.

B.2. Cấu trúc Phân tầng (Layered Architecture)

Kiến trúc nội bộ của AI Service được tinh gọn thành 3 tầng chính để tối ưu hóa đường đi của dữ liệu:

a. API Layer (Giao tiếp & Định tuyến):

- *Semantic Search Router:* POST /api/v1/search/semantic
 - Xử lý truy vấn tìm kiếm bằng ngôn ngữ tự nhiên.
 - Pipeline: Tiền xử lý text → Encoding → Vector Search.
- *Recommendation Router:* GET /api/v1/recommendations/{user_id}
 - Trả về danh sách gợi ý cá nhân hóa dựa trên ngữ cảnh (Trang chủ, Chi tiết sách).
- *Data Ingestion Router:* POST /api/v1/ingest/pdf
 - Endpoint bất đồng bộ (Async) để nhận yêu cầu xử lý tài liệu mới (OCR, Embedding).

b. **Service Layer (Logic nghiệp vụ ML):** Đây là trái tim của hệ thống, chứa các pipeline xử lý dữ liệu phức tạp:

- **Embedding Service:** Sử dụng mô hình `bkai-foundation-models/vietnamese-bi-encoder` (SBERT) để chuyển đổi văn bản tiếng Việt thành các vector 768 chiều. Mô hình này được huấn luyện trên tập dữ liệu pháp lý Zalo Legal, giúp hiểu tốt văn phong học thuật.
- **PDF Processing Pipeline:**
 - *Extraction:* Sử dụng **PyMuPDF** để trích xuất văn bản tốc độ cao (nhanh hơn 10x so với `pypdf`).
 - *Chunking:* Áp dụng chiến lược Cửa sổ trượt (Sliding Window) với kích thước chunk 256 tokens và độ chồng lấn (overlap) 50 tokens để bảo toàn ngữ cảnh giữa các đoạn cắt.
 - *OCR Fallback:* Tích hợp **Tesseract** để xử lý các tài liệu dạng scan khi PyMuPDF không tìm thấy lớp văn bản.
- **Recommendation Engine:** Sử dụng thư viện **LightFM** để cài đặt thuật toán Lai ghép (Hybrid), kết hợp giữa Lọc cộng tác (Collaborative Filtering) và Lọc nội dung (Content-based).

c. **Data & Storage Layer:**

- **Vector Database (PostgreSQL + pgvector):** Lưu trữ embeddings của sách và người dùng. Sử dụng chỉ mục **HNSW** (Hierarchical Navigable Small World) để tối ưu hóa tốc độ tìm kiếm láng giềng gần nhất (ANN) thay vì quét toàn bộ bảng (IVFFlat).
- **Model Store:** Các mô hình nặng (SBERT) được tải sẵn vào RAM khi khởi động (Pre-loaded) để giảm độ trễ (Latency).
- **Cache (Redis):** Lưu trữ kết quả của các truy vấn phổ biến và vector người dùng (User Vectors) để giảm tải tính toán lại.

Cấu trúc thư mục của AI service

5.4.3.2 Luồng giao tiếp (Communication Flows) nâng cao

Hệ thống sử dụng hai pattern giao tiếp chính: **Synchronous Communication** (REST API) cho các tương tác real-time cần phản hồi ngay lập tức, và **Asynchronous Communication** (Domain Events + Message Queue) cho các tác vụ có thể xử lý sau hoặc cần loose coupling giữa các modules.

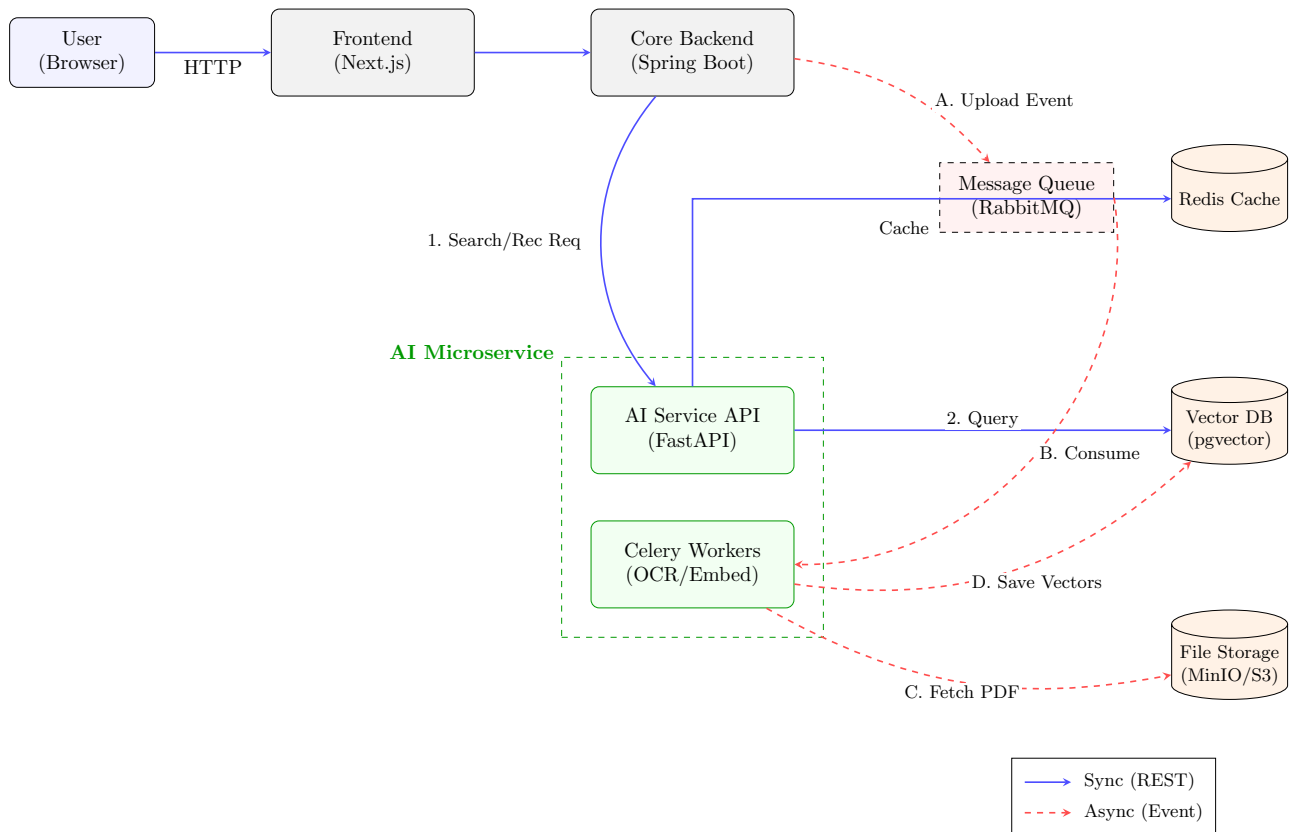
1. Giao tiếp đồng bộ (Synchronous Communication - REST API):

Được sử dụng khi:

- Frontend cần phản hồi ngay lập tức để hiển thị cho người dùng.
- Thao tác đọc dữ liệu (queries) không có side effects.
- Validation errors cần trả về trực tiếp cho người dùng.
- Cần đảm bảo transaction hoàn thành trước khi tiếp tục workflow.

Ví dụ 1: Semantic Search Flow Luồng tìm kiếm ngữ nghĩa yêu cầu phản hồi nhanh để UX tốt:

1. User nhập query "sách về machine learning" trên giao diện tìm kiếm.
2. `Next.js` Frontend gửi `POST /api/v1/search/semantic` đến Backend Monolith với payload JSON chứa query string, limit, và filters.).
3. Backend (Search Gateway Service) xử lý:



Hình 5.23: Sơ đồ luồng dữ liệu và Kiến trúc chi tiết AI Service

- Validate JWT token và extract user info.
 - Check cache trong Redis với key: hash(query + filters).
 - Nếu cache hit → return cached results ngay (< 50ms).
 - Nếu cache miss → forward request đến AI Service qua internal network.
4. Backend gọi POST `http://ai-service:8000/api/ai/semantic-search` (internal URL, không expose ra internet).
 5. AI Service xử lý:
 - Embedding Service chuyển query thành vector 768 chiều.
 - Vector Search Service thực hiện ANN search trong Vector DB.
 - Trả về danh sách book IDs với similarity scores.
 6. Backend nhận response từ AI Service:
 - Cache kết quả trong Redis với Time-To-Live 1 hour.
 - Log request (userId, query, resultCount, responseTime) để phân tích.
 7. Backend query PostgreSQL để lấy metadata chi tiết (title, author, cover, availability) của các book IDs.
 8. Backend merge AI results + metadata, trả về Frontend.
 9. Frontend render kết quả tìm kiếm.

Đặc điểm kỹ thuật:

```

1  ai-service/
2  └─ app/
3      └─ api/                                # API Layer (Giao tiếp)
4          └─ __init__.py
5          └─ router.py                        # Định nghĩa các route chính
6          └─ v1/
7              └─ endpoints/
8                  └─ search.py                # Endpoint: /semantic-search
9                  └─ recommend.py             # Endpoint: /recommendations
10                 └─ ingest.py                # Endpoint: /process-document
11                 └─ dependencies.py           # Auth dependencies (API Key/JWT)
12
13      └─ core/                                # Core Layer (Cấu hình & Setting)
14          └─ config.py                       # Load biến môi trường (DB URL, Redis URL)
15          └─ logging.py                      # Cấu hình structured logging
16          └─ exceptions.py                   # Custom exception handlers
17
18      └─ db/                                  # Data Layer (Kết nối CSDL)
19          └─ session.py                      # PostgreSQL/pgvector session management
20          └─ redis.py                        # Redis cache connection
21
22      └─ schemas/                             # Pydantic Models (Data Validation)
23          └─ search_schema.py                # Input/Output cho tìm kiếm
24          └─ book_schema.py                 # Metadata sách
25          └─ user_schema.py                 # User interaction payload
26
27      └─ services/                           # Service Layer (Logic nghiệp vụ ML cốt lõi)
28          └─ __init__.py
29          └─ embedding_service.py            # Wrapper cho mô hình SBERT (bkai-foundation)
30          └─ ocr_service.py                  # Wrapper cho PyMuPDF & Tesseract
31          └─ chunking_service.py             # Logic cắt văn bản (Sliding window)
32          └─ vector_search.py                # Logic truy vấn HNSW với pgvector
33          └─ recommender/
34              └─ collaborative.py            # Thuật toán ALS (LightFM)
35              └─ content_based.py           # Cosine similarity logic
36              └─ hybrid.py                  # Logic Switching Hybrid
37
38      └─ workers/                             # Background Tasks (Celery)
39          └─ celery_app.py                   # Cấu hình Celery instance
40          └─ tasks.py                        # Định nghĩa task: process_pdf_task, retrain_model_task
41
42      └─ models_store/                       # Nơi chứa file model (nếu không load từ HuggingFace)
43          └─ sbert/
44              └─ lightfm_model.pkl           # Model gợi ý đã huấn luyện
45
46      └─ tests/                              # Unit & Integration Tests
47          └─ test_api.py
48          └─ test_embedding.py
49          └─ test_ocr.py
50
51      └─ .env                                # Biến môi trường (không commit lên git)
52      └─ .gitignore
53      └─ docker-compose.yml                 # Setup dev env (Redis, Postgres)
54      └─ Dockerfile                         # Build image cho AI service
55      └─ main.py                            # Entry point (Khởi chạy Uvicorn)
56      └─ poetry.lock / requirements.txt      # Quản lý thư viện (torch, fastapi, pymupdf...)
57      └─ README.md
58

```

Hình 5.24: Kiến trúc chi tiết AI Service và Luồng dữ liệu

- **Request-Response pattern:** Client chờ response trước khi tiếp tục (blocking call).
- **Timeout handling:** Frontend set timeout 5 giây, hiển thị error message nếu service không phản hồi.
- **Retry logic:** Tự động retry 2 lần với exponential backoff (1s, 2s) nếu gặp lỗi network transient.
- **Circuit breaker:** Sau 5 requests liên tiếp fail, tạm ngừng gọi AI Service 30 giây, fallback sang keyword search.

2. Giao tiếp bất đồng bộ - Sự kiện nội bộ (Asynchronous Communication - Internal Events) (In-Memory Event Bus):

Sử dụng Spring ApplicationEventPublisher cho giao tiếp giữa các modules trong Backend Monolith.

Đặc điểm:

- Sự kiện được phát (publish) ngay sau khi giao dịch (transaction) hoàn tất thành công (sử dụng @TransactionalEvent).
- Nhiều bộ lắng nghe (listeners) có thể cùng đăng ký (subscribe) một sự kiện (mô hình publish-subscribe).
- Các bộ lắng nghe được thực thi trong các giao dịch riêng biệt (separate transactions) để tránh ảnh hưởng lẫn nhau.

Ví dụ 3: Quy trình Trả sách và Tính toán Phạt (Return Book with Fine Calculation)

Khi người dùng trả sách, hàng loạt các tác vụ phụ (side effects) cần được thực hiện. Tuy nhiên, để đảm bảo trải nghiệm người dùng, các tác vụ này không được phép làm chặn (block) phản hồi của API chính:

1. Người dùng thực hiện trả sách thông qua API POST `/api/v1/lending/return`.
2. `ReturnBookUseCase` bên trong **Mô-đun Mượn trả (Lending Module)** thực hiện:
 - Tải `BorrowTransaction` (Giao dịch mượn) từ cơ sở dữ liệu.
 - Cập nhật trạng thái giao dịch: `BORROWING` → `RETURNED`.
 - Kiểm tra hư hại (Check damage) → Xác nhận không hư hại.
 - Tính toán trễ hạn (nếu `returnDate > dueDate`).
 - Lưu giao dịch vào cơ sở dữ liệu.
 - Phát sự kiện (Publish) `BookReturnedEvent` chứa thông tin: `transactionId`, `userId`, `itemId`, `returnDate`, `isOverdue`, `daysOverdue`.
3. Giao dịch cơ sở dữ liệu hoàn tất (Transaction commit) → Sự kiện được điều phối (dispatch) đến các bộ lắng nghe (listeners).
4. **Mô-đun Phạt (Fine Module)** lắng nghe sự kiện `BookReturnedEvent`:
 - Kiểm tra điều kiện: `event.isOverdue() == true`.
 - Gọi `CalculateFineUseCase` với số ngày quá hạn.
 - Tạo đối tượng tổng hợp Fine (Aggregate) với số tiền = `daysOverdue × dailyFineRate`.
 - Lưu thông tin phạt vào cơ sở dữ liệu.
 - Phát sự kiện `FineIncurredEvent` (Sự kiện phát sinh phí phạt).
5. **Mô-đun Người dùng (User Module)** lắng nghe sự kiện `BookReturnedEvent`:
 - Nếu quá hạn (overdue): Giảm điểm tín nhiệm (credit score) -5 điểm.
 - Nếu đúng hạn (on-time): Tăng điểm tín nhiệm +1 điểm.

- Cập nhật `user.creditScore` trong cơ sở dữ liệu.

6. **Mô-đun Người dùng** cũng lắng nghe sự kiện `FineIncurredEvent`:

- Cộng dồn số tiền phạt vào `user.totalOutstandingFines` (Tổng nợ phạt hiện tại).
- Nếu `totalOutstandingFines > 500,000 VND` → Cấm người dùng (chuyển trạng thái = BANNED).

7. **Mô-đun Danh mục (Catalog Module)** lắng nghe sự kiện `BookReturnedEvent`:

- Cập nhật trạng thái bản sao (Item status): BORROWED → AVAILABLE.
- Tăng biến đếm số lượng khả dụng (`availableCount`).
- Phát sự kiện `ItemReturnedEvent`.

8. **Mô-đun Mượn trả (Lending Module)** lắng nghe sự kiện `ItemReturnedEvent`:

- Kích hoạt `ProcessReservationQueueUseCase` (Xử lý hàng đợi đặt chỗ).
- Tìm người dùng ưu tiên đầu tiên trong hàng đợi đặt chỗ cho ấn phẩm này.
- Gửi thông báo: "Sách bạn đặt đã có sẵn, vui lòng đến lấy trong vòng 24 giờ".
- Thiết lập thời gian hết hạn giữ sách (`hold expiration`) = thời điểm hiện tại + 24 giờ.

Lợi ích của Internal Events:

- **Decoupling:** Lending Module không biết Fine Module, User Module, Catalog Module tồn tại. Chỉ cần publish event, không quan tâm ai consume.
- **Single Responsibility:** Mỗi module chỉ quan tâm đến domain logic của mình.
- **Extensibility:** Thêm listener mới (ví dụ: Notification Module gửi email) không cần sửa code Lending Module.
- **Transaction safety:** Events chỉ publish sau khi transaction commit thành công. Nếu transaction rollback, events không được trigger.
- **Testability:** Có thể test từng listener độc lập bằng cách publish fake events.

3. Giao tiếp Bất đồng bộ - Sự kiện Bên ngoài (Hàng đợi Thông điệp):

Sử dụng **RabbitMQ** cho giao tiếp giữa Backend Monolith và Dịch vụ AI (giao tiếp liên tiến trình - cross-process communication).

Đặc điểm kỹ thuật:

- **Thông điệp bền vững (Persistent Messages):** Dữ liệu không bị mất ngay cả khi máy chủ khởi động lại.
- **Đảm bảo giao nhận (Guaranteed Delivery):** Tích hợp cơ chế thử lại (retry mechanism) khi có lỗi.
- **Hàng đợi Thư chết (Dead Letter Queue - DLQ):** Nơi lưu trữ các thông điệp thất bại sau nhiều lần thử lại để xử lý sau.
- **Xác nhận từ người tiêu thụ (Consumer Acknowledgment):** Đảm bảo thông điệp chỉ được xóa khỏi hàng đợi khi đã xử lý thành công.

Ví dụ 4: Quy trình Thêm sách mới kèm Xử lý PDF (Add Book with PDF Processing)

Quá trình xử lý PDF tốn nhiều thời gian (OCR 100 trang có thể mất 5-10 phút), do đó không thể sử dụng cơ chế chặn (blocking) API:

1. Thủ thư tải lên sách mới thông qua API POST `/api/v1/books` với định dạng form-data chứa tệp PDF và siêu dữ liệu (tiêu đề, tác giả, ISBN).
2. AddBookUseCase trong **Mô-đun Danh mục (Catalog Module)** thực hiện:
 - Kiểm tra hợp lệ siêu dữ liệu (định dạng ISBN, các trường bắt buộc).
 - Tải tệp PDF lên MinIO/S3, nhận về pdfUrl.
 - Tạo đối tượng tổng hợp Publication với trạng thái `processingStatus = PENDING`.
 - Tạo Item cho mỗi bản sao vật lý.
 - Lưu dữ liệu vào PostgreSQL.
 - Phát sự kiện BookAddedEvent lên RabbitMQ exchange `book.events` với khóa định tuyến (routing key) `book.added`.
3. API trả về mã HTTP 201 Created ngay lập tức ($< 500\text{ms}$) kèm phản hồi: `publicationId, processingStatus: PENDING, estimatedCompletionTime: "5-10 minutes"`.
4. Thủ thư nhận thông báo "Sách đã được thêm. Đang xử lý nội dung PDF..." và có thể tiếp tục thực hiện công việc khác mà không phải chờ đợi.
5. **Dịch vụ AI** có các Celery worker tiêu thụ thông điệp từ hàng đợi `book.processing.queue`:
 - Worker nhận thông điệp chứa: `publicationId, pdfUrl, metadata`.
 - Tải PDF từ MinIO (dạng luồng - streaming để tiết kiệm bộ nhớ).
 - Dịch vụ OCR trích xuất văn bản từng trang (sử dụng Tesseract/PaddleOCR).
 - Dịch vụ Phân mảnh văn bản (Text Chunking) chia văn bản thành các đoạn 512 tokens, độ chồng lấn (overlap) 50 tokens.
 - Dịch vụ Vector hóa (Embedding Service) tạo vector cho mỗi đoạn (xử lý theo lô - batch processing 10 chunks/lần).
 - Lưu vào CSDL Vector: bảng `publication_vectors` với các cột (`publication_id, chunk_id, embedding, chunk_text, page_num`).
 - Nếu bất kỳ bước nào thất bại $\rightarrow$ ném ra ngoại lệ (throw exception), thông điệp tự động được thử lại (tối đa 3 lần).
6. Sau khi hoàn tất (hoặc thất bại sau tất cả lần thử):
 - Worker gọi REST API phản hồi (callback) về Backend: POST `/api/v1/books/{id}/processing-complete` với trạng thái: SUCCESS hoặc FAILED.
 - Backend cập nhật `processingStatus` trong cơ sở dữ liệu.
 - Gửi thông báo cho thủ thư: "PDF đã xử lý xong" hoặc "Xử lý PDF thất bại".
7. Kết quả: Nếu xử lý thành công, sách xuất hiện trong kết quả tìm kiếm ngữ nghĩa. Nếu thất bại, sách vẫn có trong danh mục nhưng chỉ hỗ trợ tìm kiếm theo từ khóa truyền thống.

Đặc điểm của Mẫu thiết kế Hàng đợi Thông điệp (Message Queue Pattern):

- **Xử lý bất đồng bộ (Async processing):** Backend không bị chặn (non-blocking), trả phản hồi ngay lập tức cho người dùng để tối ưu trải nghiệm.
- **Cơ chế thử lại (Retry mechanism):** RabbitMQ tự động thử lại nếu Dịch vụ AI gặp lỗi tạm thời (hết

thời gian chờ mạng, dịch vụ bận).

- **Hàng đợi Thư chết (DLQ):** Các thông điệp bị lỗi sau 3 lần thử lại được chuyển sang DLQ để quản trị viên điều tra và xử lý thủ công (ví dụ: tệp PDF bị hỏng).
- **Giao nhận ít nhất một lần (At-least-once delivery):** Thông điệp có thể được giao nhiều lần (do mạng thử lại). Dịch vụ AI phải đảm bảo tính lũy đẳng (idempotent): kiểm tra xem `publicationId` đã được xử lý chưa trước khi thực hiện.
- **Cân bằng tải (Load balancing):** Nếu có nhiều instance của Dịch vụ AI, RabbitMQ tự động phân phối thông điệp (theo thuật toán round-robin).
- **Áp lực ngược (Backpressure):** Nếu Dịch vụ AI quá tải, thông điệp sẽ xếp hàng trong queue, Backend vẫn nhận yêu cầu bình thường mà không bị sập.

4. Chuỗi sự kiện lan truyền - Quy trình nghiệp vụ đa mô-đun phức tạp (Event Cascade)

Một số quy trình công việc (workflows) đòi hỏi sự kết hợp giữa cả gọi đồng bộ (sync calls) và sự kiện bất đồng bộ (async events) để đạt được sự cân bằng tối ưu giữa tính nhất quán dữ liệu (consistency) và hiệu năng (performance).

Ví dụ 5: Quy trình Mượn sách hoàn chỉnh (Borrow Book - Complete Workflow)

1. Giai đoạn 1: Kiểm tra hợp lệ Đồng bộ (Synchronous Validation)

- Người dùng nhấn "Mượn sách" → Frontend gửi yêu cầu POST `/api/v1/lending/borrow`.
- `LendingController` kiểm tra tính hợp lệ của yêu cầu (Validation).
- `BorrowBookUseCase` gọi đồng bộ sang **Mô-đun Người dùng**: kiểm tra điểm tín nhiệm ≥ 50 .
- `BorrowBookUseCase` gọi đồng bộ sang **Mô-đun Danh mục**: kiểm tra sách có sẵn (availability).
- Tạo `BorrowTransaction`, lưu vào cơ sở dữ liệu, và cam kết giao dịch (commit transaction).
- API trả về mã 201 Created (tổng thời gian: $\sim 200\text{ms}$).

2. Giai đoạn 2: Sự kiện Nội bộ (In-Memory Internal Events)

- **Mô-đun Mượn trả** phát sự kiện `BookBorrowedEvent`.
- **Mô-đun Người dùng** lắng nghe → tăng số lượng sách đang mượn (`currentBorrowCount`), cập nhật điểm tín nhiệm (nếu có chính sách).
- **Mô-đun Danh mục** lắng nghe → cập nhật trạng thái bản sao: AVAILABLE → BORROWED.
- **Mô-đun Tương tác (Engagement)** lắng nghe → ghi nhật ký hành vi người dùng (`userId`, `publicationId`, hành động: BORROW).

3. Giai đoạn 3: Sự kiện Bên ngoài (External Events - RabbitMQ)

- Backend phát sự kiện `BookBorrowedEvent` lên RabbitMQ.
- **Dịch vụ AI** tiêu thụ sự kiện → cập nhật vector sở thích người dùng (học tăng cường - incremental learning).
- Lên lịch huấn luyện lại mô hình gợi ý (retrain schedule) nếu tích lũy đủ 1000 tương tác mới.

Dòng thời gian xử lý (Timeline):

- **T+0ms:** Người dùng nhấn nút "Mượn sách".
- **T+200ms:** API phản hồi 201 Created → Người dùng thấy thông báo "Mượn sách thành công".

- **T+300ms:** Các sự kiện nội bộ hoàn tất → Cơ sở dữ liệu đạt trạng thái nhất quán (Consistent).
- **T+5s:** Dịch vụ AI cập nhật vector người dùng → Bộ máy gợi ý có dữ liệu mới nhất.

Lợi ích kiến trúc:

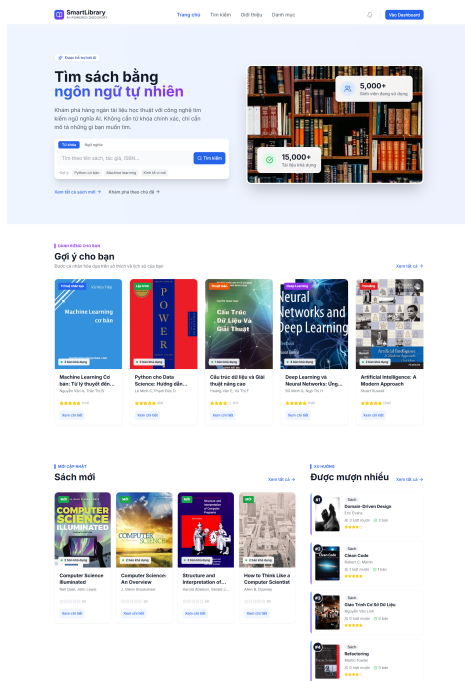
- **Phản hồi nhanh (Fast response):** Người dùng không phải chờ AI xử lý xong mới mượn được sách.
- **Tính nhất quán mạnh (Strong consistency):** Giao dịch cốt lõi (bản ghi mượn sách, kho hàng) đảm bảo nhất quán tuyệt đối nhờ ACID transaction.
- **Tính nhất quán cuối cùng (Eventual consistency):** Dữ liệu gợi ý của AI đạt trạng thái nhất quán sau một khoảng thời gian ngắn (chấp nhận được trong nghiệp vụ).
- **Khả năng phục hồi (Resilience):** Nếu Dịch vụ AI gặp sự cố (down), chức năng mượn sách cốt lõi vẫn hoạt động bình thường, hệ thống chỉ tạm thời mất tính năng cập nhật gợi ý theo thời gian thực.

5.4.4 Thiết kế giao diện

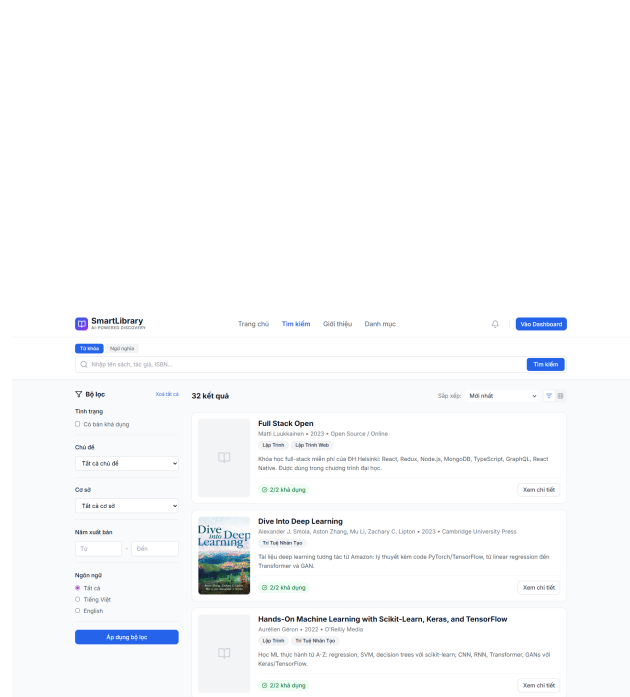
Toàn bộ giao diện hệ thống được thiết kế trên **Figma** trước khi hiện thực, đảm bảo sự thống nhất về phong cách (Design System) và luồng tương tác (User Flow) giữa các màn hình. Bản thiết kế đầy đủ có thể xem tại đường dẫn trong Phụ lục 10.

Hệ thống tuân thủ các nguyên tắc thiết kế cốt lõi: **tính nhất quán** (màu sắc, typography, hành vi component thống nhất toàn ứng dụng), **phản hồi trạng thái** (loading skeleton khi AI xử lý, toast notification sau thao tác), **tối giản** (nội dung trọng tâm là sách và thanh tìm kiếm), và **responsive** (Tailwind CSS Grid đảm bảo hiển thị tốt trên mọi thiết bị). Thư viện component **Shadcn/ui** được sử dụng để xây dựng các thành phần tái sử dụng nhất quán.

5.4.4.1 Giao diện Public (Trang chủ và Tìm kiếm)

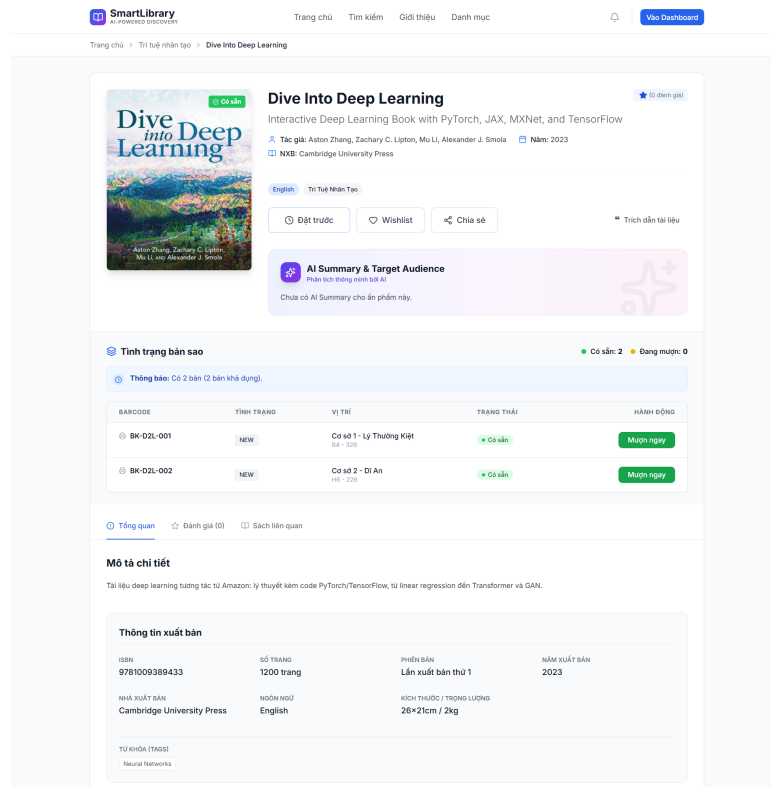


(a) Trang chủ



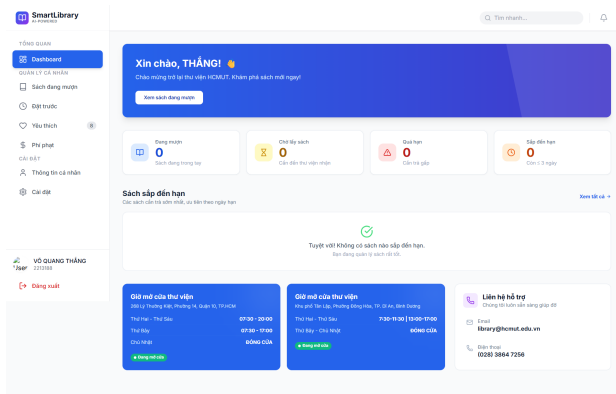
(b) Tìm kiếm sách với bộ lọc

Hình 5.25: Giao diện Public: Trang chủ và Tìm kiếm

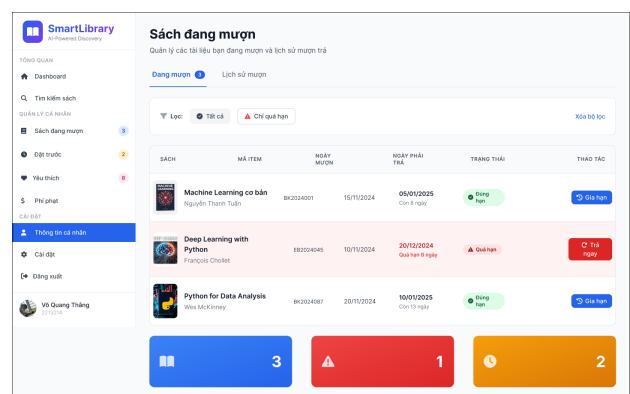


Hình 5.26: Giao diện Chi tiết sách

5.4.4.2 Giao diện Đọc giả (User Pages)



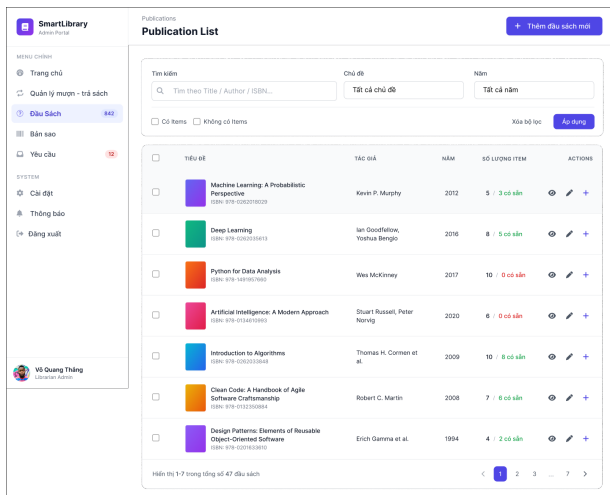
(a) Dashboard tổng quan



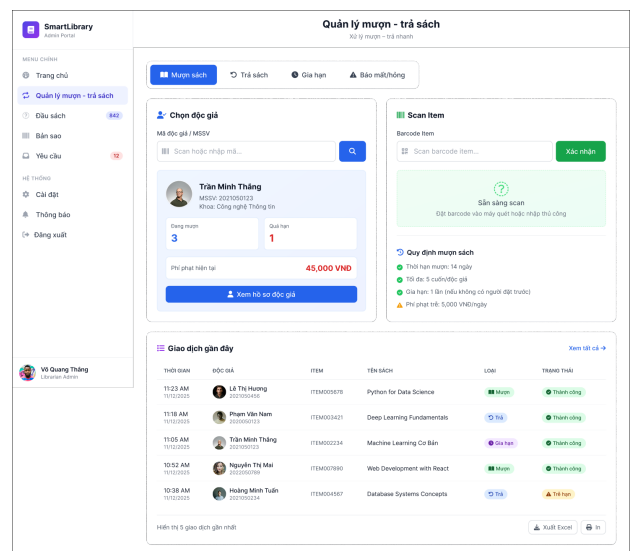
(b) Quản lý sách đang mượn

Hình 5.27: Giao diện Đọc giả: Dashboard và Quản lý mượn sách

5.4.4.3 Giao diện Thủ thư (Librarian Pages)



(a) Quản lý danh mục sách



(b) Ghi nhận mượn sách (Check-out)

Hình 5.28: Giao diện Thủ thư: Quản lý danh mục và Lưu thông

Chương 6: Hiện thực hệ thống

6.1 Cấu trúc mã nguồn

Hệ thống được phân chia thành 3 thành phần độc lập: Frontend (Giao diện người dùng), Backend (Máy chủ ứng dụng) và AI Module (Khối xử lý Trí tuệ nhân tạo). Việc tổ chức thư mục được thiết kế rõ ràng nhằm đảm bảo tính mở rộng và dễ dàng bảo trì.

6.1.1 Mã nguồn Frontend (Giao diện người dùng)

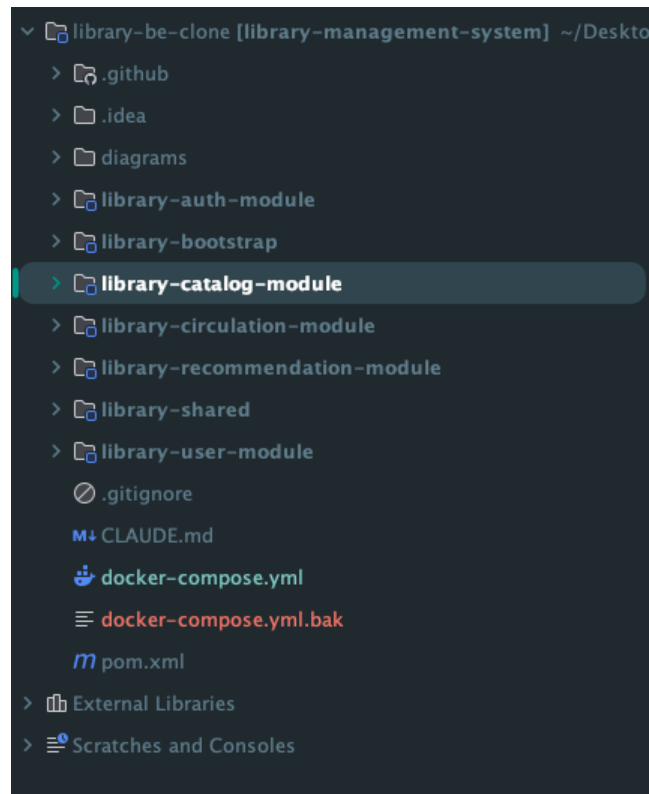


Hình 6.1: Cấu trúc thư mục mã nguồn Frontend

Mô tả cấu trúc: Frontend được xây dựng dựa trên ReactJS và trình đóng gói Vite. Các thành phần chính bao gồm:

- `api/`: Chứa các cấu hình Axios và các hàm gọi API giao tiếp với Backend.
- `components/`: Nơi lưu trữ các thành phần giao diện (UI components) có thể tái sử dụng trên nhiều trang.
- `contexts/`: Quản lý trạng thái toàn cục của ứng dụng (Context API) như thông tin người dùng, trạng thái đăng nhập, giỏ sách/wishlist.
- `pages/`: Chứa mã nguồn của các trang màn hình chính (Trang chủ, Đăng nhập, Chi tiết sách, Dashboard...).

6.1.2 Mã nguồn Backend (Máy chủ ứng dụng)

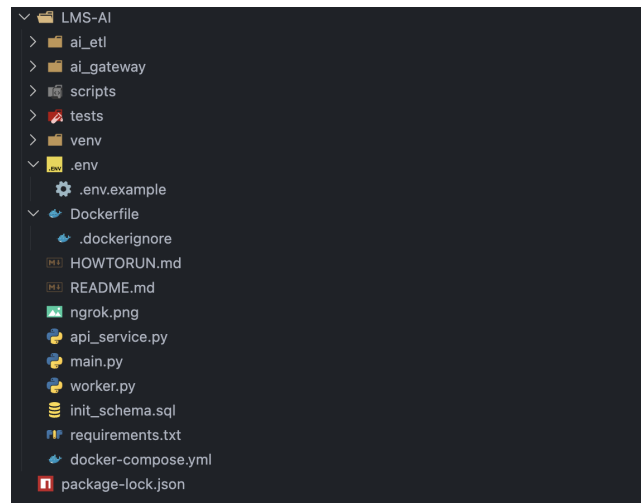


Hình 6.2: Cấu trúc thư mục mã nguồn Backend (Modular Monolith)

Mô tả cấu trúc: Backend được phát triển bằng Java Spring Boot, áp dụng kiến trúc Modular Monolith để chia tách các nghiệp vụ lõi thành từng mô-đun riêng biệt:

- `library-auth-module`: Xử lý toàn bộ logic xác thực (Authentication), phân quyền (Authorization) và tích hợp SSO (Google Login).
- `library-catalog-module`: Quản lý thông tin metadata của ấn phẩm, sách vật lý, tác giả và danh mục.
- `library-circulation-module`: Xử lý các nghiệp vụ vòng đời tài liệu như mượn sách, trả sách, đặt trước và tính phí phạt.
- `library-recommendation-module`: Cung cấp các giao tiếp và logic nền tảng để kết nối với AI Module.
- `library-user-module`: Quản lý thông tin hồ sơ (Profile) của độc giả và thủ thư.

6.1.3 Mã nguồn AI Module (Trí tuệ nhân tạo)



Hình 6.3: Cấu trúc thư mục mã nguồn phân hệ AI

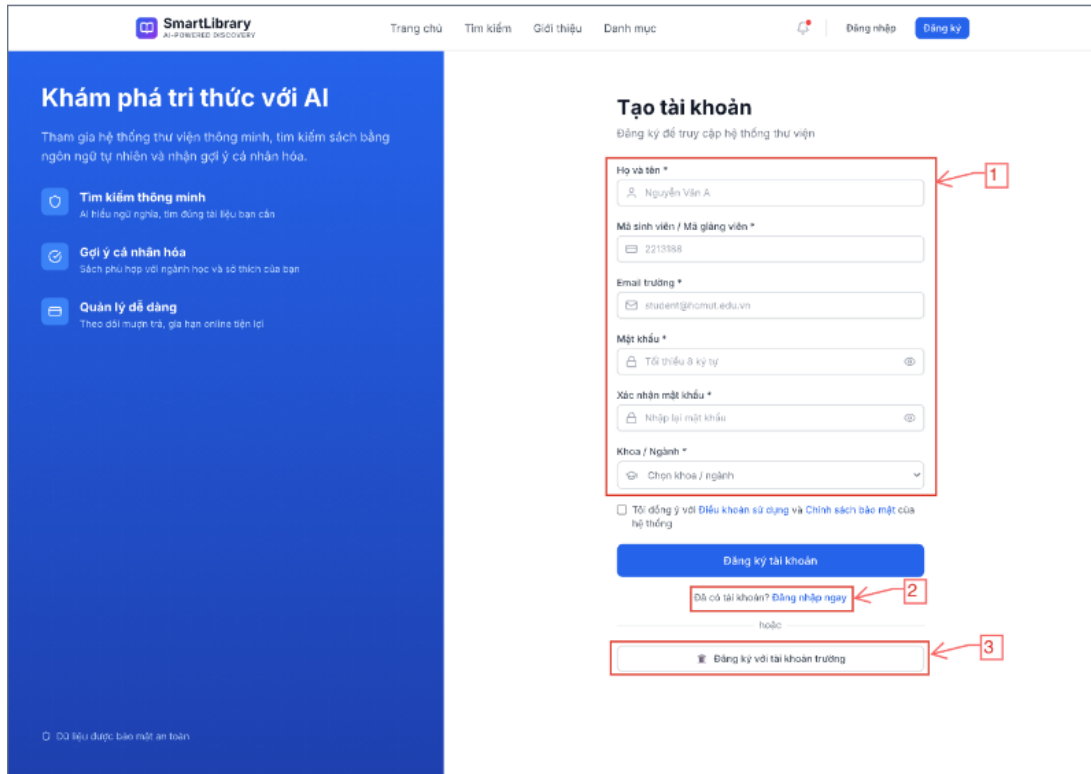
Mô tả cấu trúc: Phân hệ Trí tuệ nhân tạo được viết bằng Python, cung cấp sức mạnh tìm kiếm ngữ nghĩa và gợi ý sách:

- `ai_etl/`: Chứa các đoạn mã thực thi ETL (Extract, Transform, Load) để trích xuất dữ liệu từ cơ sở dữ liệu chính, tiền xử lý và nhúng (embedding) để đưa vào Vector Database.
- `ai_gateway/`: Khối giao tiếp API (thường dùng FastAPI) để tiếp nhận yêu cầu từ Backend Java.
- `scripts/`: Các tập lệnh tiện ích hỗ trợ huấn luyện (train) mô hình hoặc cập nhật dữ liệu.
- `api_service.py` & `worker.py`: Các tệp tin thực thi chính (entry points) để chạy dịch vụ AI và xử lý các tác vụ nền (background jobs).
- `docker-compose.yml` & `Dockerfile`: Cấu hình đóng gói ứng dụng (Containerization) giúp việc triển khai lên môi trường máy chủ diễn ra tự động và đồng nhất.

6.2 Danh sách hiện thực chức năng theo API

6.2.1 Đăng ký và Xác thực tài khoản

6.2.1.1 Giao diện Đăng ký tài khoản



Hình 6.4: Giao diện Đăng ký tài khoản hệ thống SmartLibrary

Chú thích các vùng chức năng:

- **Vùng 1 (Biểu mẫu đăng ký):** Khu vực người dùng nhập các thông tin cá nhân và định danh bao gồm: Họ và tên, Mã sinh viên (hoặc Mã giảng viên), Email trường, Mật khẩu, và Khoa/Ngành trực thuộc.
- **Vùng 2 (Chuyển hướng đăng nhập):** Liên kết hỗ trợ người dùng chuyển nhanh sang trang Đăng nhập nếu đã có tài khoản từ trước.
- **Vùng 3 (Đăng nhập SSO):** Tùy chọn đăng ký/dăng nhập nhanh thông qua hệ thống tài khoản chung (SSO) của nhà trường.

Đặc tả API Đăng ký tài khoản (User Registration):

- **Endpoint:** POST `/api/v1/auth/register`
- **Định dạng dữ liệu:** application/json
- **Mô tả:** Tiếp nhận thông tin đăng ký từ sinh viên, khởi tạo tài khoản và kích hoạt quy trình xác thực email. Hệ thống áp dụng kiểm soát dữ liệu (Validation) chặt chẽ tại tầng Application.

Bảng 6.1: Đặc tả dữ liệu đầu vào (Request Body)

Trường dữ liệu	Kiểu	Ràng buộc kỹ thuật (Validation)	Ý nghĩa nghiệp vụ
fullName	String	@Pattern (Chỉ chứa chữ và khoảng trắng)	Tên đầy đủ của sinh viên.
studentId	String	@Pattern (Đúng 7 chữ số)	Mã số sinh viên chính quy.
email	String	@Pattern (Đuôi @hcmut.edu.vn)	Email định danh do nhà trường cấp.
password	String	Tối thiểu 4 ký tự	Mật khẩu truy cập hệ thống.
faculty	Enum	Thuộc danh sách Khoa hợp lệ	Khoa mà sinh viên đang theo học.

Luồng xử lý nghiệp vụ (Business Logic):

- Kiểm tra điều kiện:** Hệ thống kiểm tra xem email đã được đăng ký trước đó hay chưa.
- Khởi tạo tài khoản:**
 - Tài khoản mới mặc định được gán vai trò STUDENT.
 - Trạng thái ban đầu là INACTIVE (Chưa kích hoạt).
 - Thiết lập thông số mặc định: Điểm tín nhiệm (Credit Score) = 100 và bật tính năng đề xuất cá nhân hóa bằng AI.
- Xác thực:** Phát hành yêu cầu gửi email chứa mã kích hoạt (Token) đến địa chỉ sinh viên vừa cung cấp.

Giải pháp kỹ thuật và Kiến trúc (Technical Implementation):

- Kiến trúc Domain-Driven Design (DDD):** Logic đăng ký được đóng gói gọn trong SignUpUseCase. Thực thể User tự quản lý trạng thái và phát sinh Domain Event (UserRegisteredEvent) khi có thay đổi quan trọng.
- Bảo mật dữ liệu:** Sử dụng PasswordHasher (Interface) để trừu tượng hóa thuật toán mã hóa (BCrypt/Argon2), đảm bảo không lưu mật khẩu dưới dạng văn bản thuần túy (Plaintext).
- Xử lý bất đồng bộ với Kafka (Event-Driven):** Sau khi lưu User thành công, một sự kiện được đẩy vào Kafka topic user.registered. Dịch vụ Email (Consumer) sẽ lắng nghe để gửi thư, giúp giảm thời gian phản hồi (Response time) và tách biệt logic (Decoupling).
- Quản lý giao dịch (Transaction):** Dùng @Transactional đảm bảo tính nhất quán (ACID). Nếu lưu DB lỗi, sự kiện gửi email sẽ không được phát đi.

Cấu trúc phản hồi (Response Format):

- Thành công (200 OK):**

```
{
  "status": "success",
  "message": "Register successful",
  "data": null
}
```

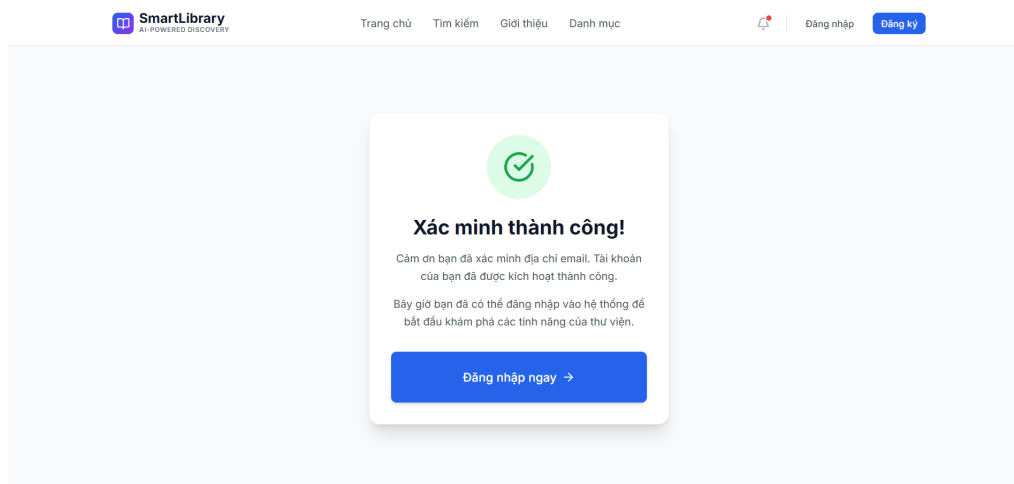
- Lỗi nghiệp vụ (409 Conflict - Trùng Email):**

```
{
```

```
"status": "error",  
"message": "Email already exists",  
"code": "EMAIL_ALREADY_EXISTS"  
}
```


Chú thích giao diện:

- **Vùng 1 (Nút thao tác chính):** Nút bấm chứa đường dẫn đính kèm mã token bảo mật (JWT/UUID) để kích hoạt tài khoản một cách tự động.
- **Vùng 2 (Đường dẫn dự phòng):** Đường dẫn thô được cung cấp phòng trường hợp nút bấm ở Vùng 1 bị trình duyệt hoặc ứng dụng mail chặn chuyển hướng.



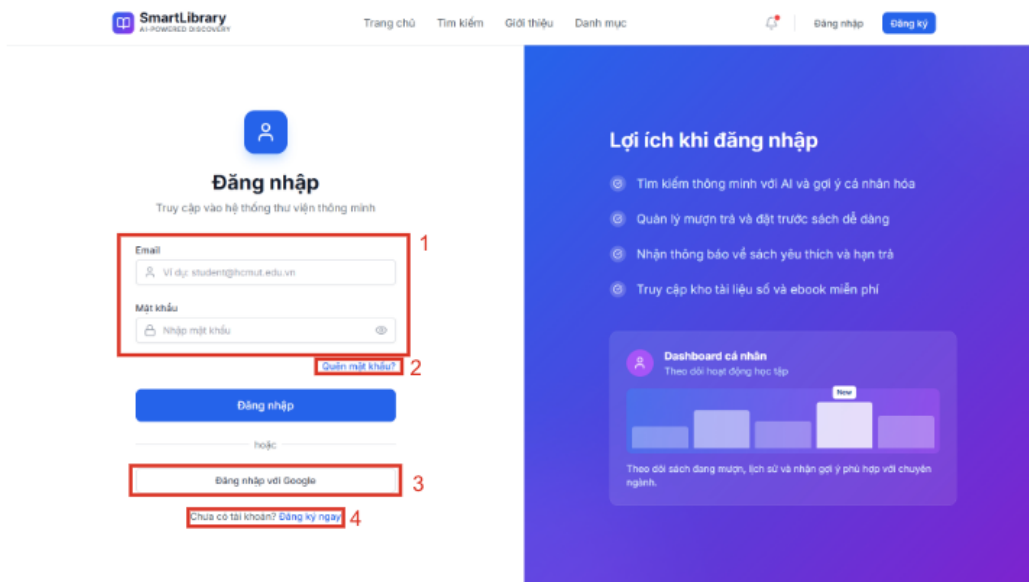
Hình 6.6: Giao diện thông báo kích hoạt tài khoản thành công

Mô tả nghiệp vụ: Khi người dùng truy cập vào liên kết xác thực hợp lệ, hệ thống sẽ kiểm tra chữ ký và hạn mức thời gian của token. Nếu hợp lệ, trạng thái của User trong Database sẽ được chuyển từ INACTIVE sang

ACTIVE, và giao diện thông báo thành công sẽ hiển thị cùng với nút điều hướng người dùng đăng nhập.

6.2.2 Đăng nhập hệ thống

6.2.2.1 Giao diện Đăng nhập



Hình 6.7: Giao diện Đăng nhập hệ thống SmartLibrary

Chú thích các vùng chức năng:

- **Vùng 1 (Biểu mẫu đăng nhập):** Khu vực người dùng nhập thông tin xác thực bao gồm Email trường (theo định dạng @hcmut.edu.vn) và Mật khẩu.
- **Vùng 2 (Quên mật khẩu):** Liên kết hỗ trợ người dùng thao tác khôi phục lại mật khẩu trong trường hợp thất lạc hoặc quên.
- **Vùng 3 (Đăng nhập SSO):** Tùy chọn đăng nhập nhanh thông qua hệ thống tài khoản Google của nhà trường.
- **Vùng 4 (Chuyển hướng đăng ký):** Liên kết điều hướng người dùng sang trang Đăng ký nếu chưa có tài khoản trên hệ thống.

6.2.2.2 Đặc tả luồng nghiệp vụ Đăng nhập

Mục tiêu của luồng nghiệp vụ này là đảm bảo xác thực đúng danh tính người dùng, kiểm tra trạng thái hoạt động của tài khoản và tiến hành phân quyền truy cập ngay sau khi đăng nhập thành công. Luồng đăng nhập được chia thành 4 giai đoạn chính:

1. Giai đoạn 1: Tiếp nhận và Kiểm tra định dạng (Frontend)

- Người dùng cung cấp thông tin gồm Email và Mật khẩu thông qua giao diện đăng nhập.
- Hệ thống thực hiện kiểm tra sơ bộ: Email phải đúng định dạng của trường. Nếu không hợp lệ, hệ thống sẽ báo lỗi ngay tại giao diện mà chưa gửi yêu cầu về máy chủ nhằm tối ưu tài nguyên.

2. Giai đoạn 2: Xác thực và Kiểm tra điều kiện tài khoản (Backend)

- **Tìm kiếm tài khoản:** Hệ thống truy vấn thông tin người dùng trong cơ sở dữ liệu dựa trên Email.
- **Kiểm tra trạng thái tài khoản (Nghệ thuật quan trọng):** Trước khi kiểm tra mật khẩu, hệ thống xác minh trạng thái hoạt động:
 - *Tài khoản chưa kích hoạt:* Hệ thống từ chối đăng nhập và yêu cầu người dùng kiểm tra hộp thư để xác thực.
 - *Tài khoản bị khóa/cấm:* Ngăn chặn truy cập và thông báo lý do cụ thể (áp dụng cho tài khoản vi phạm nội quy hoặc bị khóa thủ công bởi quản trị viên).
- **Xác thực mật khẩu:** Hệ thống sử dụng thuật toán mã hóa một chiều để so sánh mật khẩu nhập vào với mã băm (hash) đã lưu trong cơ sở dữ liệu.

3. Giai đoạn 3: Khởi tạo phiên làm việc và Phân quyền

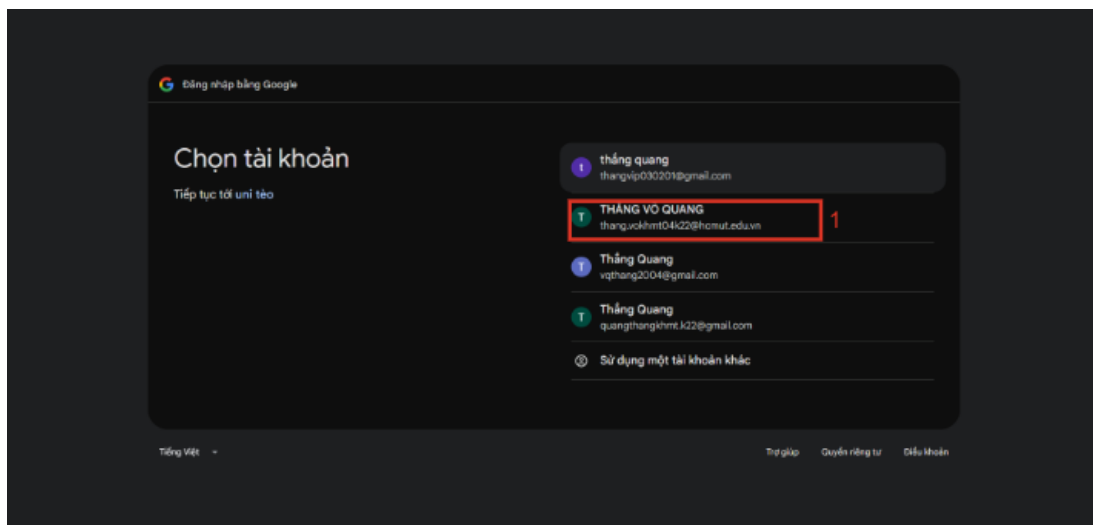
- **Cấp phát Token:** Sau khi xác thực thành công, máy chủ tạo ra bộ định danh bảo mật (gồm Access Token và Refresh Token) để duy trì trạng thái đăng nhập.
- **Phân quyền người dùng (Role-based):** Thông tin quyền hạn được đính kèm vào token:
 - *Quyền Thủ thư (Librarian):* Có toàn quyền quản lý kho sách, duyệt yêu cầu mượn trả và xem báo cáo.
 - *Quyền Sinh viên (Student):* Có quyền tìm kiếm sách, đặt trước và quản lý thông tin mượn trả cá nhân.

4. Giai đoạn 4: Điều hướng người dùng (Frontend)

- Hệ thống client nhận bộ định danh và lưu trữ bảo mật tại trình duyệt.
- Dựa trên quyền hạn đã được phân tích, hệ thống tự động điều hướng: Thủ thư được dẫn trực tiếp đến Bảng điều khiển quản lý (Librarian Dashboard), Sinh viên được dẫn đến Trang cá nhân và Khám phá sách (User Dashboard).

6.2.3 Đăng nhập bằng Google

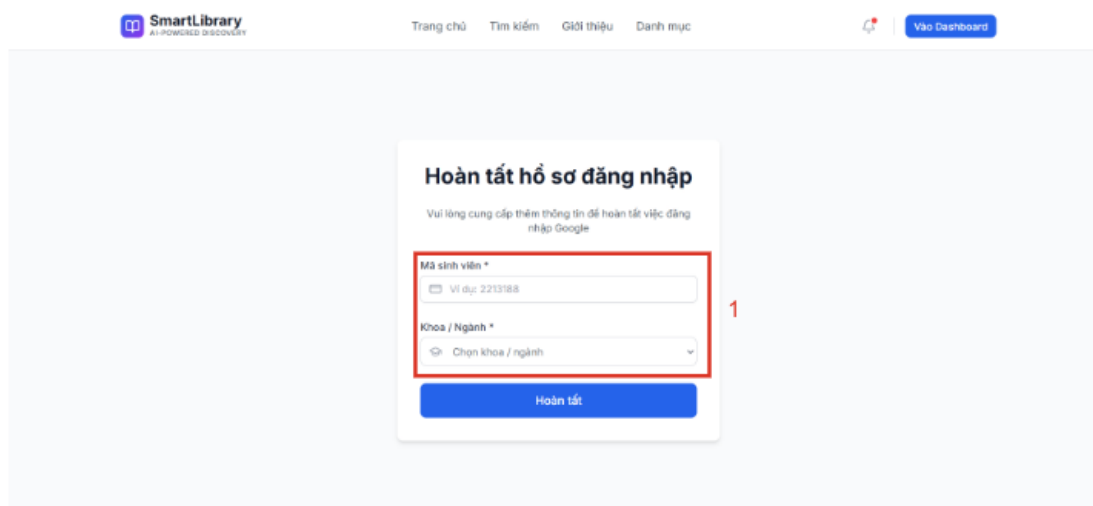
6.2.3.1 Giao diện Đăng nhập qua tài khoản Google



Hình 6.8: Giao diện lựa chọn tài khoản Google để xác thực

Chú thích giao diện:

- **Vùng 1 (Danh sách tài khoản):** Cửa sổ pop-up từ Google OAuth2 hiển thị các tài khoản đang lưu trên trình duyệt. Để vào được hệ thống, người dùng bắt buộc phải chọn tài khoản email có đuôi mở rộng hợp lệ của trường (ví dụ: @hcmut.edu.vn).



Hình 6.9: Giao diện hoàn tất hồ sơ cho lần đăng nhập đầu tiên

Chú thích giao diện:

- **Vùng 1 (Biểu mẫu bổ sung thông tin):** Trong trường hợp người dùng đăng nhập bằng Google lần đầu, hệ thống chỉ lấy được Email, Tên và Ảnh đại diện. Màn hình này bắt buộc người dùng cung cấp thêm **Mã sinh viên** và **Khoa/Ngành** trước khi được cấp quyền truy cập vào bảng điều khiển (Dashboard).

6.2.3.2 Đặc tả luồng nghiệp vụ và API

Luồng đăng nhập bằng Google chia làm 2 nhánh chính: Đăng nhập thông thường (tài khoản đã tồn tại) và Khởi tạo tài khoản qua Google (lần đầu đăng nhập).

1. API Xác thực Token từ Google (Xử lý Đăng nhập)

- **Endpoint:** POST /api/v1/auth/google
- **Quyền truy cập:** Public
- **Dữ liệu đầu vào:** idToken (Chuỗi token do Google SDK trả về sau khi người dùng chọn tài khoản).
- **Mô tả logic:**
 1. Backend nhận idToken và xác minh chữ ký với Google Server (để chống giả mạo).
 2. Lấy địa chỉ email từ token. Kiểm tra định dạng email (phải là @hcmut.edu.vn).
 3. Kiểm tra email trong cơ sở dữ liệu (Database):
 - Trường hợp 1 (Đã có tài khoản): Hệ thống trực tiếp cấp phát Access Token / Refresh Token và trả về mã 200 OK. Người dùng vào thẳng Dashboard.
 - Trường hợp 2 (Tài khoản chưa tồn tại): Hệ thống lưu tạm thông tin (Email, Tên, Avatar) vào bộ nhớ đệm (Redis) kèm một temporaryToken, trả về mã 202 Accepted và yêu cầu Frontend điều hướng sang màn hình **Hoàn tất hồ sơ**.

2. API Hoàn tất hồ sơ (Dành cho lần đăng nhập đầu tiên)

- **Endpoint:** POST /api/v1/auth/google/complete-profile
- **Quyền truy cập:** Yêu cầu Bearer Token là temporaryToken đã cấp ở bước trên.

Bảng 6.3: Cấu trúc dữ liệu yêu cầu (Request Body) hoàn tất hồ sơ

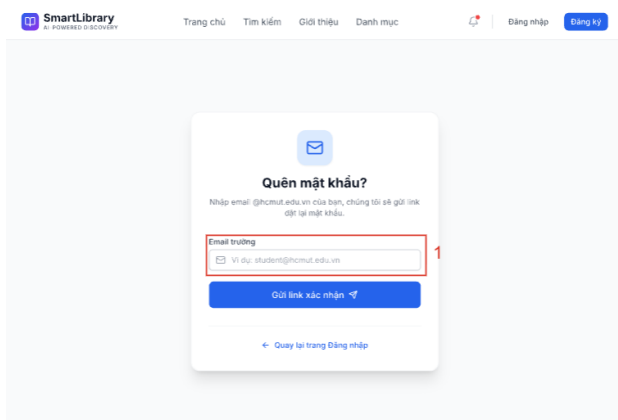
Trường dữ liệu	Kiểu	Bắt buộc	Mô tả
studentId	String	✓	Mã số sinh viên (Kiểm tra trùng lặp trong DB).
faculty	Enum	✓	Khoa/Ngành trực thuộc.

Logic xử lý hoàn tất hồ sơ:

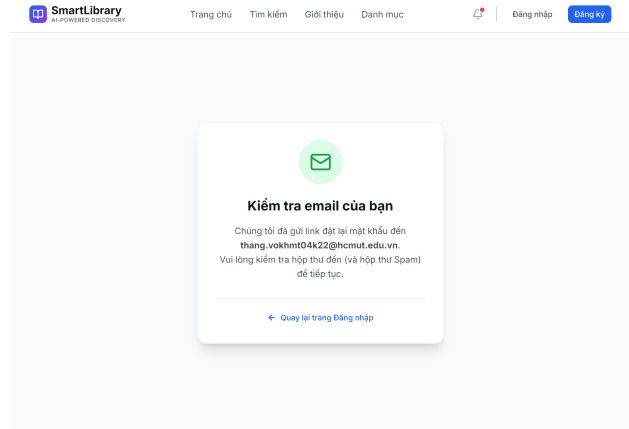
- Hệ thống trích xuất thông tin Google từ bộ nhớ đệm (dựa vào temporaryToken).
- Kết hợp thông tin Google với studentId và faculty do người dùng cung cấp.
- Khởi tạo bản ghi User mới trong cơ sở dữ liệu với trạng thái ACTIVE (không cần gửi email xác thực vì Google đã đảm bảo email là thật).
- Cấp phát Access Token / Refresh Token chính thức.

6.2.4 Khôi phục mật khẩu

6.2.4.1 Yêu cầu khôi phục mật khẩu



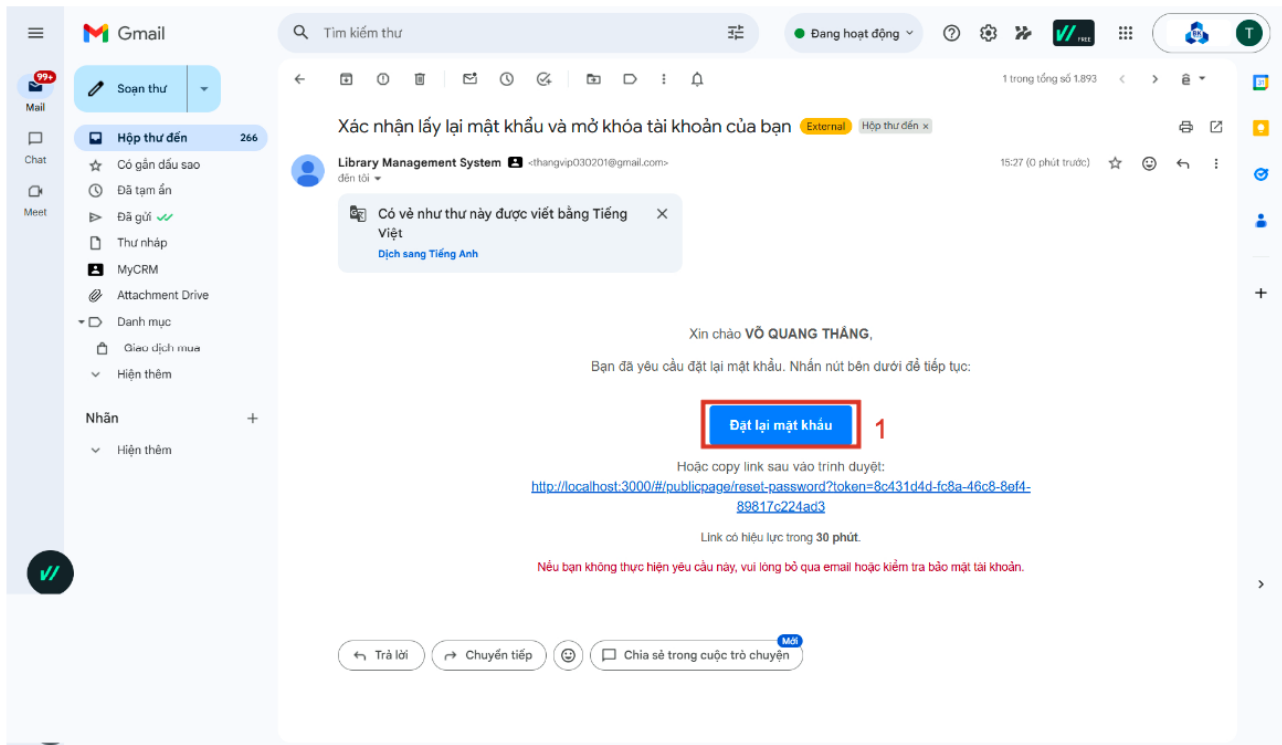
Hình 6.10: Giao diện nhập email yêu cầu khôi phục



Hình 6.11: Thông báo kiểm tra hộp thư

Mô tả giao diện: Khi người dùng quên thông tin truy cập, hệ thống cung cấp một biểu mẫu để nhập địa chỉ email định danh. Để đảm bảo tính hợp lệ, email đầu vào bắt buộc phải tuân thủ đúng định dạng tên miền do nhà trường cấp. Ngay sau khi gửi yêu cầu thành công, giao diện sẽ chuyển sang màn hình thông báo, hướng dẫn người dùng kiểm tra hộp thư đến hoặc thư mục rác để nhận đường dẫn xác thực.

6.2.4.2 Xác thực hệ thống qua Email

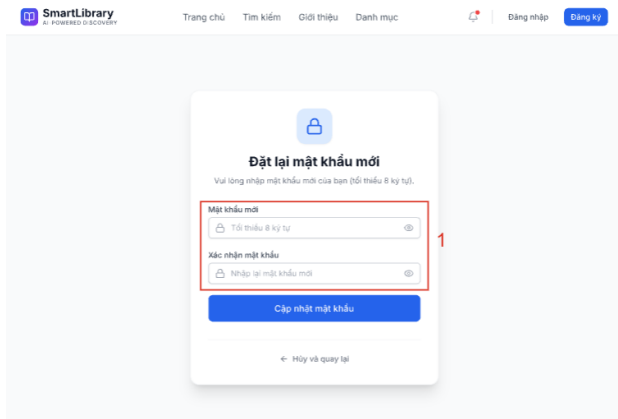


Hình 6.12: Nội dung thư điện tử chứa đường dẫn đặt lại mật khẩu

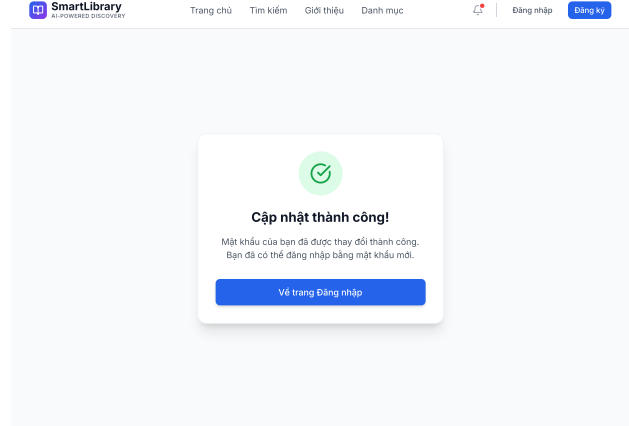
Mô tả giao diện: Hệ thống tự động gửi một thư điện tử đến địa chỉ mà người dùng vừa cung cấp. Nội dung thư bao gồm một nút bấm chứa đường dẫn bảo mật để chuyển hướng đến trang thiết lập mật khẩu mới. Nhằm

đảm bảo an toàn thông tin, đường dẫn này chỉ có hiệu lực sử dụng trong vòng 30 phút. Hệ thống cũng cung cấp đường dẫn dạng thô để người dùng sao chép trực tiếp vào trình duyệt trong trường hợp nút bấm bị lỗi.

6.2.4.3 Thiết lập mật khẩu mới và Hoàn tất



Hình 6.13: Giao diện thiết lập mật khẩu mới



Hình 6.14: Thông báo cập nhật thành công

Mô tả giao diện: Khi truy cập vào đường dẫn hợp lệ, hệ thống sẽ yêu cầu người dùng nhập và xác nhận lại mật khẩu mới. Mật khẩu mới phải đáp ứng tiêu chuẩn bảo mật cơ bản là có độ dài tối thiểu 8 ký tự. Sau khi lưu thông tin, một màn hình thông báo thành công sẽ xuất hiện, xác nhận quá trình thay đổi đã hoàn tất và hỗ trợ người dùng quay lại trang chủ để đăng nhập bằng thông tin vừa tạo.

6.2.4.4 Đặc tả luồng nghiệp vụ

Quy trình khôi phục mật khẩu được thiết kế chặt chẽ thông qua hai giai đoạn tương ứng với hai điểm cuối giao tiếp API như sau:

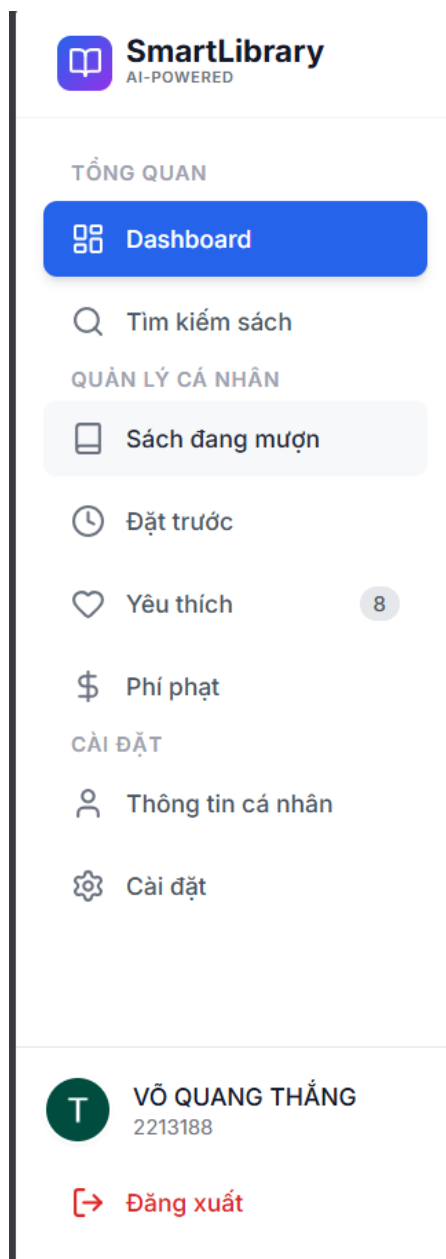
1. Giai đoạn yêu cầu cấp lại mật khẩu

- **Endpoint:** POST /api/v1/auth/forgot-password
- **Dữ liệu đầu vào:** Địa chỉ email của người dùng.
- **Logic xử lý:** Máy chủ kiểm tra sự tồn tại của email trong cơ sở dữ liệu. Nếu tài khoản hợp lệ, hệ thống sẽ khởi tạo một mã xác thực bảo mật dạng chuỗi (Token), lưu trữ mã này cùng với hạn sử dụng (30 phút) và kích hoạt dịch vụ gửi thư điện tử. Để tránh việc kẻ gian lợi dụng dò tìm tài khoản, hệ thống luôn trả về thông báo thành công ngay cả khi email không tồn tại.

2. Giai đoạn thiết lập mật khẩu mới

- **Endpoint:** POST /api/v1/auth/reset-password
- **Dữ liệu đầu vào:** Mã xác thực (Token) trích xuất từ đường dẫn và chuỗi Mật khẩu mới.
- **Logic xử lý:** Hệ thống tiến hành xác minh tính hợp lệ và thời hạn của mã xác thực. Nếu mã vượt qua kiểm tra, máy chủ sẽ tiến hành băm (hash) mật khẩu mới để thay thế dữ liệu cũ trong cơ sở dữ liệu, đồng thời vô hiệu hóa ngay lập tức mã xác thực vừa dùng để ngăn chặn việc tái sử dụng.

6.2.5 Thanh điều hướng



Hình 6.15: Giao diện Thanh điều hướng chính của hệ thống

Mô tả các vùng chức năng:

- **Khối Tổng quan:** Nơi cung cấp các lối tắt truy cập nhanh vào Bảng điều khiển (Dashboard) và công cụ Tìm kiếm sách.
- **Khối Quản lý cá nhân:** Tập trung các tính năng theo dõi hoạt động của người dùng tại thư viện. Bao gồm quản lý danh sách sách đang mượn, theo dõi tiến trình đặt trước, xem lại danh mục sách yêu thích (hỗ trợ hiển thị trực quan số lượng thông qua huy hiệu đếm) và tra cứu các khoản phí phạt nếu có.
- **Khối Cài đặt & Tài khoản (Nửa dưới):** Chứa liên kết điều hướng đến trang quản lý hồ sơ cá nhân và thiết lập hệ thống. Điểm nhấn ở cuối thanh điều hướng là thẻ thông tin định danh tĩnh (bao gồm ảnh đại diện, họ tên, mã số sinh viên) giúp người dùng luôn nhận biết được tài khoản đang sử dụng, đi kèm

với nút thao tác Đăng xuất nhanh.

Logic kỹ thuật cốt lõi:

- **Cơ chế hiển thị:** Thanh điều hướng được thiết kế tĩnh bên lề trái, áp dụng kiến trúc ứng dụng trang đơn (Single Page Application - SPA) giúp người dùng chuyển đổi mượt mà giữa các phân hệ mà không phải tải lại toàn bộ trang web.
- **Đồng bộ dữ liệu:** Thông tin hiển thị ở thẻ người dùng (tên, mã số) được trích xuất và đồng bộ trực tiếp từ trạng thái lưu trữ của Token (JWT) hiện tại.
- **Xử lý Đăng xuất:** Khi kích hoạt, hệ thống Frontend sẽ tiến hành xóa bỏ toàn bộ Token khỏi bộ nhớ cục bộ (Local Storage/Session) và ngay lập tức điều hướng người dùng quay trở lại màn hình Đăng nhập.

6.2.6 Quản lý thông tin cá nhân

6.2.6.1 Giao diện Tổng quan hồ sơ

The screenshot displays the 'SmartLibrary' Profile Dashboard for user 'VÕ QUANG THẮNG'. The interface includes a sidebar with navigation options like 'Dashboard', 'Tìm kiếm sách', and 'Thống tin cá nhân'. The main content area is divided into three sections, each highlighted with a red box and a number:

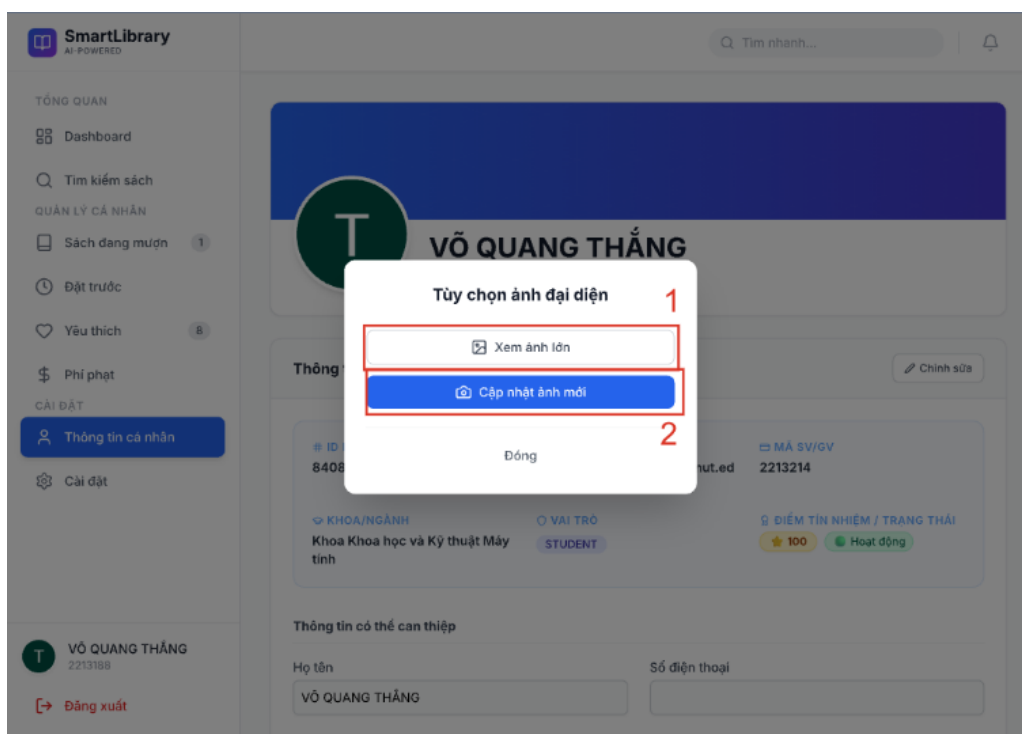
- 1. Thông tin cơ bản (Basic Information):** Displays user details including ID (84086582373400562), Email (thang.vokhmt04k22@hcmut.edu.vn), and Student ID (2213214). It also shows the department (Khoa Khoa học và Kỹ thuật Máy tính) and role (STUDENT).
- 2. Thông tin có thể can thiệp (Editable Information):** A form for updating personal details such as Name (VÕ QUANG THẮNG), Phone Number, Date of Birth, Address, and Department (currently set to Khoa Khoa học và Kỹ thuật Máy tính).
- 3. Cài đặt tài khoản (Account Settings):** Includes a 'Đổi mật khẩu' (Change Password) section with fields for current, new, and confirm passwords, and a 'Thông báo' (Notifications) section with checkboxes for email alerts and wishlist updates.

Hình 6.16: Giao diện Quản lý thông tin cá nhân (Profile Dashboard)

Chú thích các vùng chức năng:

- **Vùng 1 (Thông tin cơ bản):** Hiển thị các thông tin định danh cốt lõi không thể tự ý thay đổi (chỉ đọc) bao gồm: ID Hệ thống, Email trường, Mã SV/GV, Khoa/Ngành, Vai trò (STUDENT/LIBRARIAN), Điểm tín nhiệm (mặc định 100) và Trạng thái hoạt động.
- **Vùng 2 (Thông tin có thể can thiệp):** Biểu mẫu cho phép người dùng tự cập nhật các thông tin cá nhân như: Họ tên, Số điện thoại, Ngày sinh, Địa chỉ. Ngoài ra còn có công tắc bật/tắt tính năng "Cá nhân hóa AI" (sử dụng lịch sử mượn để AI gợi ý sách).
- **Vùng 3 (Cài đặt tài khoản):** Khu vực hỗ trợ bảo mật và tùy chọn hệ thống, bao gồm biểu mẫu đổi mật khẩu (yêu cầu mật khẩu hiện tại) và các tùy chọn nhận thông báo qua email.

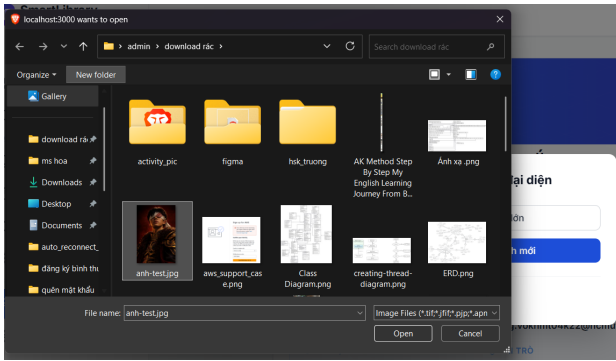
6.2.6.2 Giao diện Cập nhật ảnh đại diện



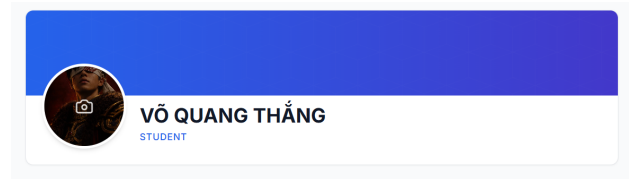
Hình 6.17: Menu tùy chọn thao tác với ảnh đại diện

Chú thích giao diện (Hình 6.17): Khi người dùng tương tác (click) vào ảnh đại diện hiện tại, hệ thống hiển thị một menu với hai lựa chọn:

- **Vùng 1 (Xem ảnh lớn):** Phóng to ảnh đại diện hiện tại.
- **Vùng 2 (Cập nhật ảnh mới):** Kích hoạt luồng tải lên ảnh đại diện mới từ thiết bị cá nhân.



Hình 6.18: Hộp thoại chọn tệp tin



Hình 6.19: Ảnh đại diện sau khi cập nhật

Mô tả nghiệp vụ cập nhật ảnh: Khi chọn "Cập nhật ảnh mới", hệ thống gọi hộp thoại chọn tệp của hệ điều hành (Hình 6.18). File ảnh sau khi chọn sẽ được mã hóa dạng multipart/form-data và đẩy lên máy chủ. Giao diện sau đó tự động cập nhật hình ảnh hiển thị mới ngay lập tức mà không cần tải lại trang (Hình 6.19).

6.2.6.3 Đặc tả API và Kỹ thuật

Tất cả các API trong phân hệ quản lý hồ sơ cá nhân đều yêu cầu xác thực bảo mật bằng JWT. Hệ thống tự động trích xuất mã định danh người dùng (userId) từ JWT token thông qua SecurityEvaluator, tuyệt đối không nhận userId truyền từ phía client để tránh rủi ro bảo mật. Base endpoint cho phân hệ này là /api/v1/users.

Bảng 6.5: Danh sách các API phân hệ Thông tin cá nhân

Method	Endpoint	Mô tả nghiệp vụ
GET	/my-profile	Lấy toàn bộ thông tin cá nhân của người dùng đang đăng nhập để hiển thị lên giao diện.
PUT	/my-profile	Cập nhật thông tin (Họ tên, ngày sinh, sdt, địa chỉ).
POST	/avatar	Tải lên ảnh đại diện mới (multipart/form-data).
PUT	/my-profile/change-password	Đổi mật khẩu tài khoản.

Ràng buộc dữ liệu tại Backend:

- **fullName):** Giới hạn @Pattern (chỉ chứa chữ cái và khoảng trắng).
- **phoneNumber :** Giới hạn @Pattern (từ 10 đến 15 chữ số).
- **dateOfBirth :** Ràng buộc @Past (ngày sinh phải nằm trong quá khứ).
- **address :** Giới hạn @Size (tối đa 200 ký tự).

Luồng xử lý Upload Ảnh đại diện (Avatar): Máy chủ Backend tiếp nhận file qua định dạng multipart/form-data, sau đó trực tiếp đẩy file lưu trữ lên dịch vụ điện toán đám mây Amazon S3 theo đường dẫn avatars/{userId}. Dịch vụ S3 sẽ trả về một đường dẫn (URL) lưu trữ tĩnh. Backend lưu URL này vào cơ sở dữ liệu (profilePictureUrl) và trả URL về cho Frontend để cập nhật giao diện.

Luồng xử lý Đổi mật khẩu: Người dùng nhập 3 trường: mật khẩu hiện tại, mật khẩu mới, và xác nhận mật khẩu mới. Tại Frontend, hệ thống xác thực mật khẩu mới không rỗng và khớp với trường xác nhận. Tại

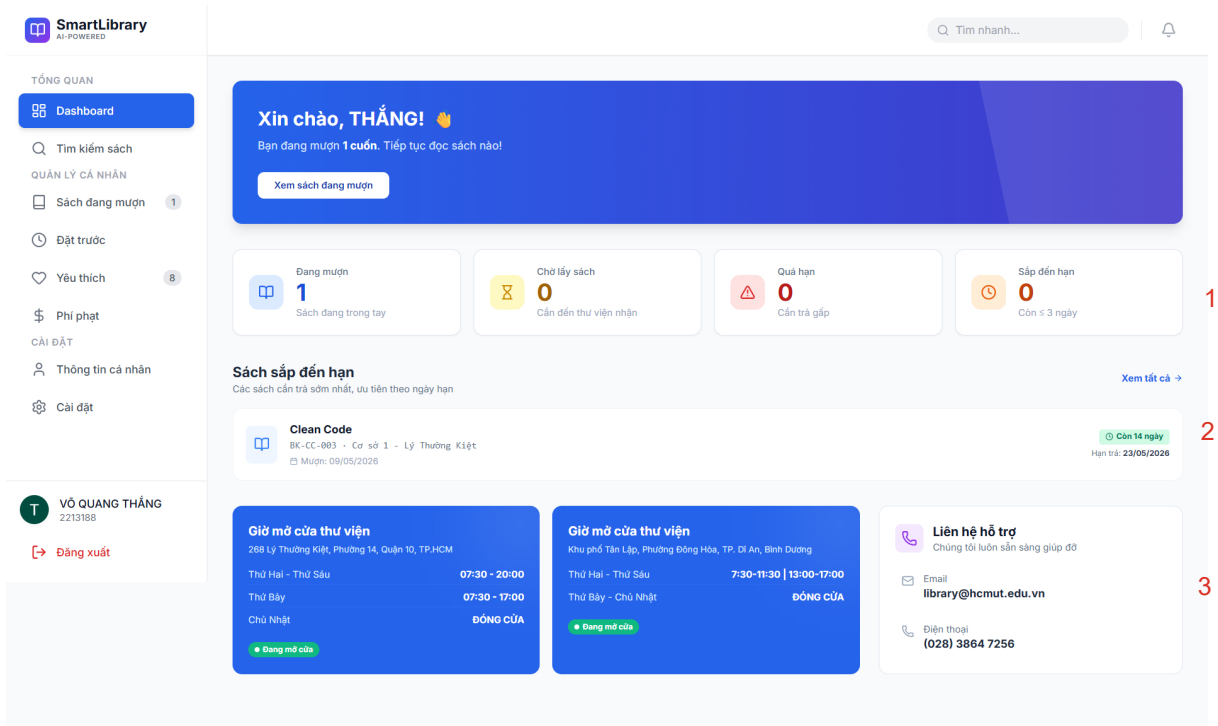


Backend, máy chủ đối chiếu mật khẩu hiện tại trong cơ sở dữ liệu. Nếu chính xác, hệ thống tiến hành băm (hash) mật khẩu mới và lưu lại.



6.2.7 Bảng điều khiển Độc giả

6.2.7.1 Giao diện Tổng quan



The screenshot displays the SmartLibrary user dashboard. On the left is a sidebar with navigation links: TÓNG QUAN, Dashboard, Tìm kiếm sách, QUẢN LÝ CÁ NHÂN, Sách đang mượn (1), Đặt trước, Yêu thích (8), Phí phạt, Cài đặt, Thông tin cá nhân, and Cài đặt. The main content area features a welcome banner for user THẮNG!, a summary of loan statistics (1 book borrowed, 0 books to be returned, 0 overdue books, 0 books due soon), a list of books due soon (Clean Code), and a section for library hours (Giờ mở cửa thư viện) for two locations. A contact support section is also present. Red numbers 1, 2, and 3 are overlaid on the right side of the dashboard, pointing to the loan statistics, due books, and contact support sections respectively.

Hình 6.20: Giao diện Bảng điều khiển dành cho Sinh viên/Độc giả

Chú thích các vùng chức năng:

- **Khối Lời chào (Phía trên cùng):** Lời chào được cá nhân hóa dựa trên tên của người dùng đang đăng nhập. Khu vực này cũng cung cấp một thông điệp tóm tắt nhanh (ví dụ: "Bạn đang mượn 1 cuốn") kèm theo nút điều hướng trực tiếp đến danh sách sách đang mượn để độc giả tiện theo dõi.
- **Vùng 1 (Thống kê hoạt động cá nhân):** Hiển thị các chỉ số cốt lõi về tình trạng giao dịch hiện tại của độc giả thông qua 4 thẻ (card) trực quan:
 - *Đang mượn:* Số lượng sách thực tế đang được giữ bởi độc giả.
 - *Chờ lấy sách:* Số lượng yêu cầu đặt trước đã được thư viện chuẩn bị xong và đang chờ người dùng đến nhận.
 - *Quá hạn:* Cảnh báo đỏ về số lượng sách đã vượt quá thời hạn hoàn trả, yêu cầu độc giả xử lý gấp để tránh phát sinh thêm phí phạt.
 - *Sắp đến hạn:* Số sách chuẩn bị đến hạn trả trong một vài ngày tới.
- **Vùng 2 (Danh sách sắp đến hạn):** Khu vực cảnh báo chi tiết, liệt kê cụ thể những cuốn sách mà người dùng cần trả sớm. Thông tin hiển thị bao gồm tiêu đề sách, vị trí cơ sở mượn (ví dụ: Cơ sở Lý Thường Kiệt), số ngày còn lại và mốc thời gian hoàn trả chính xác. Mục này giúp độc giả chủ động sắp xếp thời gian đến thư viện.
- **Vùng 3 (Thông tin vận hành thư viện):** Bảng thông tin tĩnh cung cấp lịch hoạt động của cả hai cơ sở thư viện (Cơ sở Lý Thường Kiệt và Cơ sở Dĩ An). Tùy thuộc vào thời gian thực của hệ thống, giao diện sẽ tự động hiển thị nhãn "Đang mở cửa" (màu xanh) hoặc "Đóng cửa" để hỗ trợ độc giả lên kế hoạch di chuyển. Bên cạnh đó là các kênh liên hệ đường dây nóng và email hỗ trợ của nhà trường.

6.2.7.2 Đặc tả luồng nghiệp vụ Bảng điều khiển

Bảng điều khiển của độc giả đóng vai trò là trạm thông tin trung tâm. Để tải toàn bộ dữ liệu cho màn hình này, hệ thống Frontend sẽ gọi song song các API để tối ưu hóa thời gian tải trang.

1. Truy xuất thống kê giao dịch

- **Endpoint:** GET /api/v1/transactions/me/summary (hoặc tập hợp các lệnh đếm từ API giao dịch cá nhân).
- **Quyền truy cập:** STUDENT / LECTURER.
- **Logic xử lý:** Máy chủ truy vấn cơ sở dữ liệu dựa trên userId của người dùng hiện tại, tính toán và gom nhóm (GROUP BY) số lượng giao dịch theo các trạng thái: Đang mượn (BORROWED), Chờ lấy (RESERVED), Quá hạn (OVERDUE) và Sắp đến hạn (các giao dịch BORROWED có dueDate nhỏ hơn 3 ngày tới).

2. Truy xuất danh sách cần chú ý (Sắp đến hạn)

- **Endpoint:** GET /api/v1/transactions/me?status=BORROWED&sort=dueDate,asc&size=5
- **Logic xử lý:** Hệ thống trả về danh sách tối đa 5 giao dịch đang mượn, được sắp xếp ưu tiên theo ngày hạn trả tăng dần (những cuốn sắp đến hạn nhất sẽ nằm trên cùng) để hiển thị ra Vùng 2 của giao diện.

6.2.8 Khám phá và Tìm kiếm tài liệu

6.2.8.1 Giao diện Tìm kiếm tổng hợp

The screenshot displays the BK library search interface. At the top, there is a search bar with a dropdown menu for 'Từ khóa' (Keyword) and 'Ngữ nghĩa' (Semantics). Below the search bar, there is a search button labeled 'Tìm kiếm'. To the left of the search results, there is a sidebar with filters for 'Bộ lọc' (Filter), 'Tình trạng' (Status), 'Chủ đề' (Topic), 'Cơ sở' (Source), 'Năm xuất bản' (Year of publication), and 'Ngôn ngữ' (Language). The main area shows 12 search results, each with a book cover, title, author, publisher, and a 'Xem chi tiết' (View details) button. The results are sorted by 'Mới nhất' (Newest).

Hình 6.21: Giao diện Tìm kiếm và Bộ lọc nâng cao

Chú thích các vùng chức năng:

- **Vùng 1 (Thanh tìm kiếm trung tâm):** Khu vực nhập từ khóa tự do. Khung tìm kiếm hỗ trợ tra cứu linh hoạt qua nhiều tiêu chí như tên sách, tên tác giả, hoặc mã số chuẩn quốc tế (ISBN).
- **Vùng 2 (Công cụ hiển thị):** Thanh công cụ hỗ trợ người dùng sắp xếp kết quả (theo độ mới, lượt mượn, đánh giá) và linh hoạt chuyển đổi chế độ xem giữa dạng danh sách (List) hoặc dạng lưới (Grid).
- **Vùng 3 (Bộ lọc đa chiều):** Hệ thống cung cấp các bộ lọc sâu để thu hẹp kết quả, bao gồm: tình trạng (chỉ hiện sách còn bản sao khả dụng), chủ đề chuyên ngành, cơ sở lưu trữ (Lý Thường Kiệt hoặc Dĩ An), khoảng thời gian xuất bản và ngôn ngữ.
- **Vùng 4 (Danh sách kết quả):** Khu vực trình bày các ấn phẩm thỏa mãn tiêu chí tìm kiếm. Mỗi thẻ

ấn phẩm hiển thị ảnh bìa, tiêu đề, tác giả, năm xuất bản và nhãn trạng thái (ví dụ: "Hết sách" khi toàn bộ bản sao đang được mượn).

Đặc tả luồng nghiệp vụ Tìm kiếm:

- **Điểm cuối (Endpoint):** GET /api/v1/publications/search
- **Quyền truy cập:** Công khai (Không yêu cầu đăng nhập).
- **Chiến lược truy vấn (SQL Strategy):** Để tối ưu hóa hiệu suất tìm kiếm linh hoạt mà không cần đến các engine bên ngoài như Elasticsearch, máy chủ Backend sử dụng NamedParameterJdbcTemplate. Truy vấn tìm kiếm từ khóa (keyword) được thực thi theo dạng không phân biệt hoa thường (ILIKE) quét qua 6 bảng dữ liệu liên kết: bảng Tiêu đề sách, Tiêu đề phụ, Mã ISBN, bảng Tác giả, bảng Từ khóa (Tags) và bảng Thể loại.
- **Logic Sắp xếp (Sorting):** Nếu người dùng tìm theo từ khóa, kết quả được sắp xếp theo độ liên quan (mức độ khớp của tiêu đề được ưu tiên cao nhất). Nếu sắp xếp theo "Lượt mượn nhiều nhất", hệ thống sử dụng truy vấn con đếm tổng số giao dịch mượn. Nếu sắp xếp theo "Đánh giá", hệ thống dựa vào điểm đánh giá trung bình.

6.2.8.2 Giao diện Chi tiết ấn phẩm

Khi người dùng chọn một ấn phẩm từ kết quả tìm kiếm, hệ thống sẽ điều hướng sang trang Chi tiết ấn phẩm. Màn hình này được chia thành nhiều khối thông tin chuyên sâu.

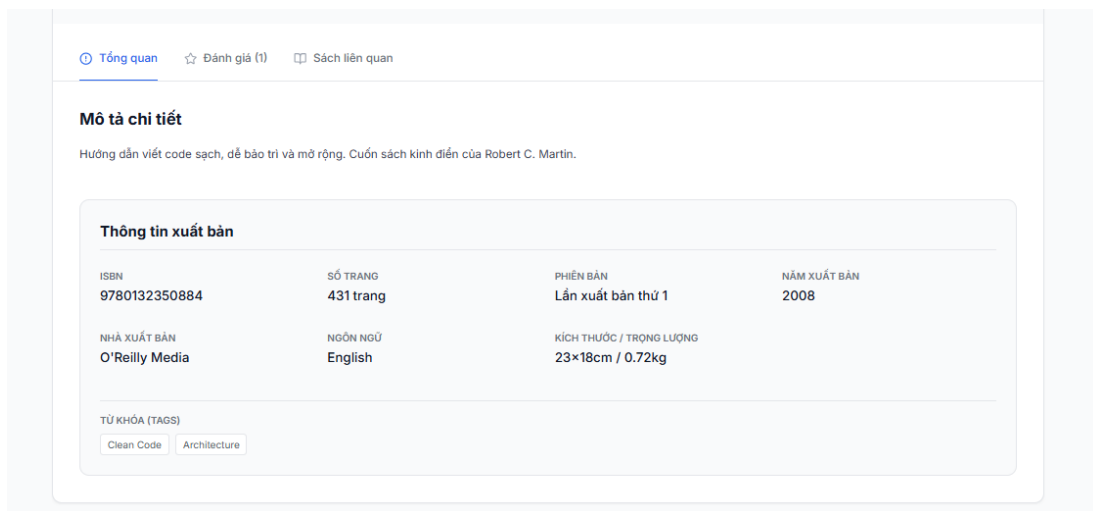
The screenshot displays the 'Clean Code' book detail page. It includes a book cover image, title, author (Robert C. Martin), year (2008), and publisher (O'Reilly Media). Below this are buttons for 'Đặt trước' (Pre-order), 'Đã lưu' (Saved), and 'Chia sẻ' (Share). An 'AI Summary & Target Audience' section is also present. At the bottom, a 'Tình trạng bản sao' (Copy Status) section shows a table of available and borrowed copies.

BARCODE	TÌNH TRẠNG	VỊ TRÍ	TRẠNG THÁI	HÀNH ĐỘNG
BK-CC-001	NEW	Cơ sở 1 - Lý Thường Kiệt B4 - 301	Đã có người đặt	Đặt trước
BK-CC-002	OLD	Cơ sở 1 - Lý Thường Kiệt B4 - 301	Đang được mượn Trả dự kiến: 13/5/2026	Đặt trước
BK-CC-003	OLD	Cơ sở 1 - Lý Thường Kiệt B4 - 304	Đang được mượn Trả dự kiến: 23/5/2026	Đặt trước
BK-CC-004	OLD	Cơ sở 1 - Lý Thường Kiệt B4 - 304	Đang được mượn	Đặt trước

Hình 6.22: Giao diện Thông tin tổng quan và Tình trạng mượn trả

Chú thích giao diện (Hình 6.22):

- **Vùng 1 (Thông tin nền tảng):** Hiển thị ảnh bìa, nhan đề chính, nhan đề phụ, tác giả, năm và nhà xuất bản, kèm theo các thẻ phân loại (Tags).
- **Vùng 2 (Chỉ số đánh giá):** Điểm số trung bình dựa trên phản hồi của độc giả hệ thống.
- **Vùng 3 (Thanh thao tác):** Các hành động người dùng có thể thực hiện bao gồm Đặt trước sách, Lưu vào danh sách yêu thích và Lấy liên kết chia sẻ.
- **Vùng 4 (Phân tích Trí tuệ nhân tạo):** Khu vực đặc biệt tích hợp AI nhằm tự động tóm tắt nội dung sách và đề xuất tệp đọc giả mục tiêu phù hợp.
- **Vùng 5 (Tình trạng bản sao vật lý):** Danh sách thời gian thực các quyển sách vật lý đang tồn tại trong thư viện. Bảng này cung cấp mã vạch, tình trạng sách (Mới/Cũ), vị trí kệ, trạng thái (Có sẵn/Đang mượn) và nút hành động tương ứng. Nếu một bản sao đang được mượn, hệ thống sẽ dự báo ngày có thể trả lại.



Hình 6.23: Khối dữ liệu Mô tả nội dung và Thông số xuất bản

Chú thích giao diện (Hình 6.23): Khu vực tab mở rộng cung cấp đoạn văn mô tả chi tiết nội dung sách và một bảng ma trận các thông số kỹ thuật xuất bản (Mã ISBN, Số trang, Phiên bản, Kích thước vật lý, Trọng lượng).

Đặc tả luồng nghiệp vụ Chi tiết ấn phẩm:

Trang chi tiết yêu cầu một khối lượng dữ liệu khổng lồ từ nhiều phân hệ khác nhau. Để tải trang nhanh chóng, hệ thống sử dụng cơ chế chia nhỏ truy vấn thông qua hai API song song.

1. Truy xuất thông tin metadata ấn phẩm

- **Điểm cuối:** GET /api/v1/publications/{id}
- **Logic xử lý dữ liệu:** Thay vì sử dụng một truy vấn kết nối (JOIN) khổng lồ dễ gây thất cổ chai, Backend được thiết kế thực thi 6 truy vấn độc lập (Lấy thông tin gốc; Lấy danh sách Tác giả; Lấy bộ Từ khóa; Lấy Danh mục; Tính trung bình Đánh giá; Thống kê số lượng sách). Kết quả từ 6 luồng này được máy chủ tổng hợp (Aggregate) lại thành một gói phản hồi duy nhất gửi về Frontend.

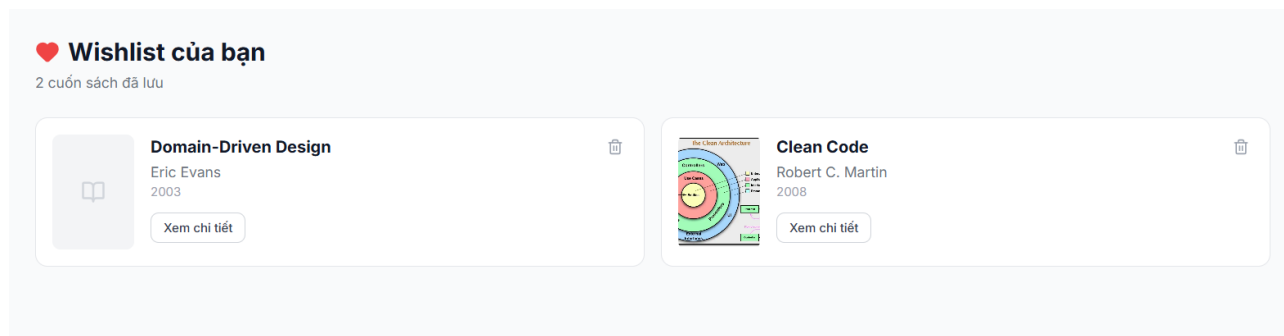
2. Truy xuất tình trạng bản sao vật lý (Vùng 5)

- **Điểm cuối:** GET /api/v1/publications/{id}/items

- **Logic xử lý dữ liệu:** API này trả về danh sách các quyền sách vật lý phục vụ cho việc đặt trước. Backend sử dụng ngôn ngữ truy vấn JPQL thực hiện phép kết nối trái (LEFT JOIN) với bảng Lịch sử Giao dịch. Kỹ thuật này giúp hệ thống tìm ra mức ngày hạn trả lớn nhất ($\text{MAX}(\text{dueDate})$) của các giao dịch đang diễn ra ($\text{status} = \text{'BORROWING'}$), từ đó tính toán và hiển thị chính xác ngày dự kiến sách được trả về kho cho độc giả.

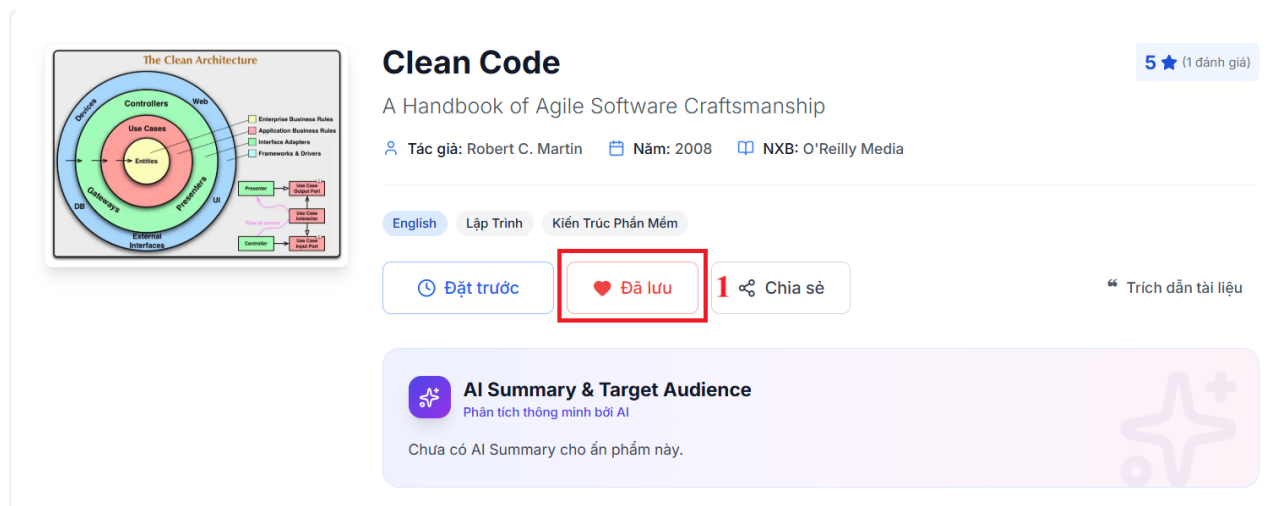
6.2.9 Danh sách sách yêu thích và Thông báo

6.2.9.1 Giao diện quản lý danh sách yêu thích



Hình 6.24: Giao diện quản lý danh sách sách đã lưu của độc giả

Chú thích giao diện: Trang quản lý danh sách yêu thích được trình bày dưới dạng lưới trực quan. Mỗi thẻ thông tin sách cung cấp đầy đủ ảnh bìa, tiêu đề, tên tác giả và năm xuất bản. Từ đây, độc giả có thể dùng nút "Xem chi tiết" để chuyển hướng nhanh đến trang thông tin của ấn phẩm, hoặc nhấn vào biểu tượng thùng rác để gỡ bỏ sách khỏi danh sách lưu trữ.



Hình 6.25: Nút thao tác lưu sách tại trang chi tiết ấn phẩm

Chú thích giao diện: Tại trang chi tiết của mỗi ấn phẩm, hệ thống bố trí một nút thao tác lưu sách nổi bật. Trạng thái của nút này (đã lưu hoặc chưa lưu) được hệ thống đánh giá và hiển thị theo thời gian thực dựa trên dữ liệu cá nhân hóa của từng người dùng.

6.2.9.2 Đặc tả API và Luồng xử lý dữ liệu

Hệ thống cung cấp một nhóm giao tiếp (API) chuyên trách cho tính năng lưu sách với đường dẫn gốc là /api/v1/wishlist. Để đảm bảo bảo mật, toàn bộ các điểm cuối đều yêu cầu người dùng phải xác thực hệ thống (Authentication) trước khi sử dụng.

Bảng 6.7: Danh sách API Quản lý sách yêu thích

Phương thức	Điểm cuối	Mô tả nghiệp vụ
GET	/	Truy xuất toàn bộ danh sách sách đã lưu của người dùng hiện tại.
POST	/ {publicationId}	Thêm một ấn phẩm mới vào danh sách yêu thích.
DELETE	/ {publicationId}	Gỡ bỏ một ấn phẩm ra khỏi danh sách.
GET	/ {publicationId}/status	Kiểm tra trạng thái lưu của một ấn phẩm cụ thể.

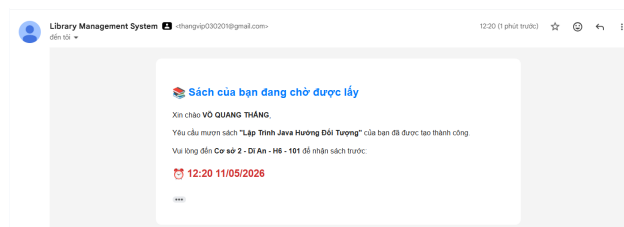
Logic nghiệp vụ cốt lõi:

- **Thao tác thêm sách:** Khi nhận yêu cầu, hệ thống sẽ kiểm tra danh sách của người dùng và tự động tạo mới nếu chưa có. Điểm đặc biệt của quy trình này là tính độc lập (Idempotent): nếu sách đã tồn tại trong danh sách, hệ thống vẫn ghi nhận thành công mà không phát sinh lỗi. Đồng thời, mỗi lượt lưu sách sẽ ngầm tạo ra một bản ghi tương tác để cung cấp dữ liệu huấn luyện cho hệ thống Trí tuệ Nhân tạo (AI) gợi ý sách sau này.
- **Thao tác gỡ bỏ:** Khi người dùng yêu cầu xóa, hệ thống tiến hành gỡ bỏ ấn phẩm và trả về kết quả. Giao diện chỉ làm mới danh sách và hiển thị thông báo thành công sau khi nhận được phản hồi chính thức từ máy chủ (hệ thống không sử dụng kỹ thuật cập nhật lạc quan - Optimistic Update nhằm đảm bảo tính đồng nhất dữ liệu).
- **Truy xuất dữ liệu:** Thông tin trả về được tối ưu hóa hiệu suất bằng các truy vấn cơ sở dữ liệu nguyên thủy (Native SQL). Dữ liệu tác giả được gom nhóm gọn gàng thông qua hàm STRING_AGG. Danh sách trả về luôn được sắp xếp theo thời gian ưu tiên những cuốn sách mới được thêm vào gần nhất.

6.2.9.3 Hệ thống thông báo và nhắc nhở mượn sách



Hình 6.26: Thông báo đẩy trên hệ thống web



Hình 6.27: Thư điện tử nhắc nhở nhận sách

Mô tả nghiệp vụ tương tác: Ngoài việc theo dõi sách yêu thích, hệ thống còn hỗ trợ độc giả thông qua luồng thông báo đa kênh. Ngay khi một yêu cầu mượn sách được xử lý thành công, một thông báo đẩy sẽ xuất hiện trên giao diện trang web, báo cáo chi tiết về cơ sở thư viện và giới hạn 24 giờ để đến nhận sách. Song song đó, một thư điện tử tự động cũng được phát hành gửi thẳng vào hộp thư cá nhân của độc giả, giúp họ dễ dàng lưu trữ và tra cứu thông tin lịch hẹn.

6.2.10 Nghiệp vụ Mượn sách

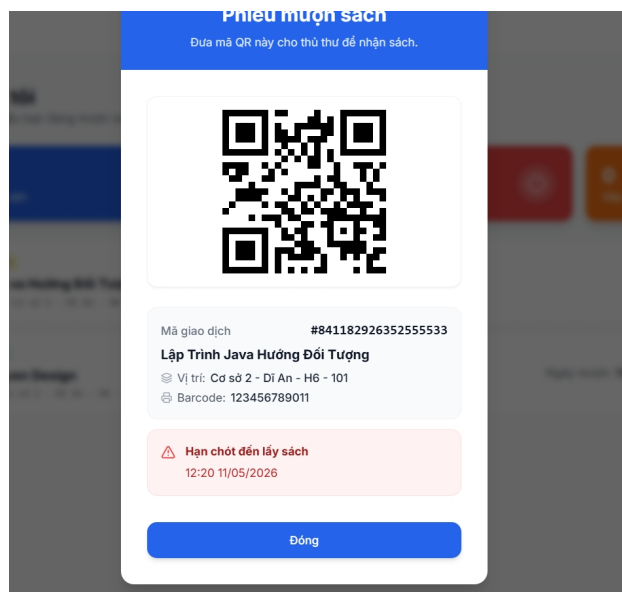
Phân hệ mượn sách được thiết kế linh hoạt với hai luồng thao tác chính: Độc giả tự đặt trước qua hệ thống trực tuyến (Web Flow) và Thủ thư xử lý trực tiếp tại quầy (Direct Flow).

6.2.10.1 Luồng Độc giả tự đặt mượn trực tuyến

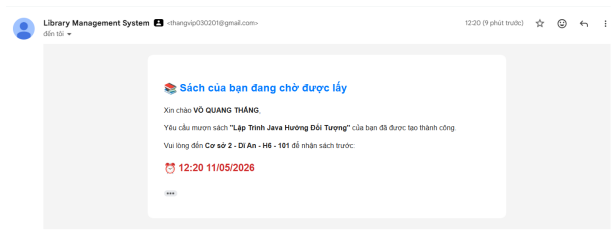
Tinh trạng bản sao ● Có sẵn: 1 ● Đang mượn: 1				
Thông báo: Có 3 bản (1 bản khả dụng).				
BARCODE	TÌNH TRẠNG	VỊ TRÍ	TRẠNG THÁI	HÀNH ĐỘNG
BK-JAVA-001	OLD	Cơ sở 2 - Dĩ An H6 - 202	● Đang bảo trì	Không khả dụng
BK-JAVA-002	NEW	Cơ sở 2 - Dĩ An H6 - 203	● Đang được mượn	Đặt trước
123456789011	NEW	Cơ sở 2 - Dĩ An H6 - 101	● Có sẵn	1 Mượn ngay

Hình 6.28: Giao diện chọn bản sao và thực hiện mượn sách

Mô tả giao diện: Tại trang chi tiết ấn phẩm, độc giả có thể xem danh sách các bản sao vật lý hiện có tại từng cơ sở. Với những bản sao đang ở trạng thái "Có sẵn", hệ thống cung cấp nút thao tác "Mượn ngay". Nút này chỉ khả dụng khi hệ thống đánh giá người dùng đủ điều kiện mượn sách.



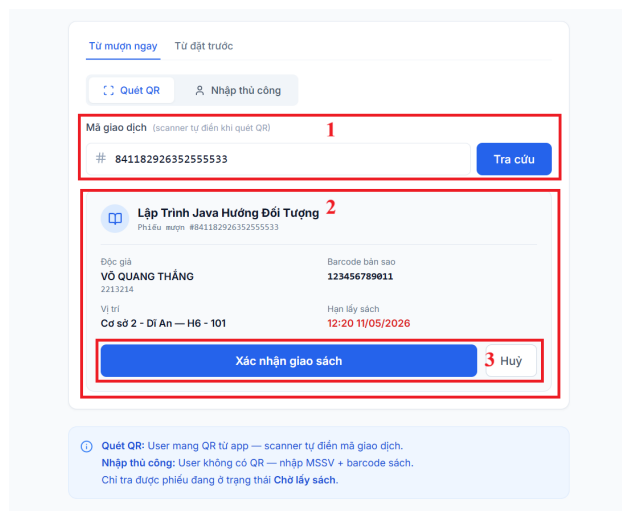
Hình 6.29: Phiếu mượn điện tử (Mã QR)



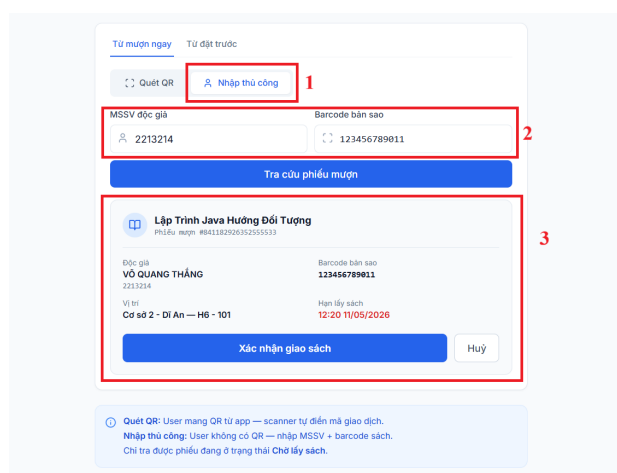
Hình 6.30: Thư điện tử thông báo lịch hẹn

Mô tả nghiệp vụ: Ngay khi độc giả nhấn "Mượn ngay", hệ thống sẽ khởi tạo một giao dịch đặt trước. Giao diện trả về một Phiếu mượn điện tử chứa mã QR, mã giao dịch, và hạn chót đến lấy sách (thường là 24 giờ kể từ lúc đặt). Đồng thời, một thư điện tử và thông báo đẩy sẽ được phát hành tự động để nhắc nhở người dùng lịch hẹn đến thư viện nhận sách.

6.2.10.2 Luồng Thủ thư xử lý và giao sách tại quầy

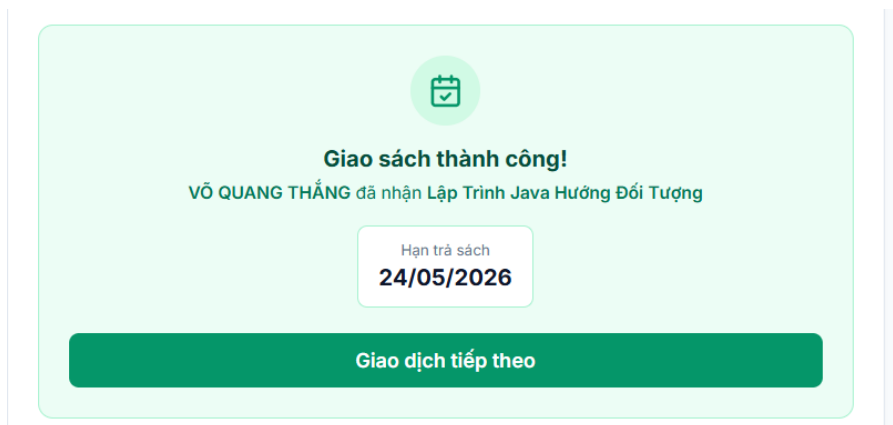


Hình 6.31: Tra cứu qua Mã giao dịch / Quét QR



Hình 6.32: Tra cứu thủ công qua Mã SV và Mã vạch sách

Mô tả giao diện tra cứu: Khi đọc giả đến quầy, thủ thư sử dụng công cụ tra cứu để tìm lại giao dịch đặt trước. Hệ thống hỗ trợ hai phương thức linh hoạt: sử dụng máy quét mã vạch để quét trực tiếp mã QR trên ứng dụng của đọc giả, hoặc nhập thủ công tổ hợp Mã số sinh viên và Mã vạch của cuốn sách. Nếu thông tin khớp, phiếu mượn sẽ hiển thị kèm nút "Xác nhận giao sách".



Hình 6.33: Màn hình thông báo giao sách thành công

Mô tả nghiệp vụ giao sách: Sau khi thủ thư kiểm tra vật lý và nhấn xác nhận, trạng thái giao dịch chính thức chuyển sang "Đang mượn". Màn hình hiển thị thông báo thành công kèm theo mốc thời gian hạn trả chính xác để thủ thư có thể nhắc nhở trực tiếp đọc giả.

6.2.10.3 Đặc tả API và Kỹ thuật

Phân hệ mượn sách được vận hành bởi 4 giao tiếp (API) lõi, đảm bảo tính toàn vẹn dữ liệu thông qua các vòng kiểm duyệt chặt chẽ.

1. Khởi tạo mượn sách trực tuyến (Web Flow)

- **Endpoint:** POST /api/v1/transactions/borrow (Quyền: Đọc giả)

- **Kiểm duyệt điều kiện (Validation):** Hệ thống thực hiện 4 rào chắn tuần tự: Bản sao phải có sẵn (AVAILABLE); Người dùng chưa đạt giới hạn mượn (tối đa 5 cuốn); Không có khoản phạt nào chưa thanh toán; và không được phép mượn trùng lặp một đầu sách đã đặt trước đó.
- **Chuyển đổi trạng thái:** Bản sao chuyển thành RESERVED. Giao dịch ở trạng thái WAITING_FOR_PICKUP.
- **Sự kiện (Event-driven):** Đẩy thông báo qua Kafka để kích hoạt dịch vụ gửi Email và App Notification.

2. Tra cứu giao dịch tại quầy

- **Endpoint:** GET /api/v1/transactions/lookup (Quyền: Thủ thư)
- **Logic:** Tìm kiếm các giao dịch đang ở trạng thái WAITING_FOR_PICKUP thông qua mã giao dịch (ID) hoặc kết hợp Mã sinh viên và Mã vạch sách.

3. Xác nhận giao sách (Hoàn tất Web Flow)

- **Endpoint:** POST /api/v1/transactions/{id}/confirm-pickup (Quyền: Thủ thư)
- **Logic:** Kiểm tra ID giao dịch hợp lệ. Cập nhật trạng thái giao dịch từ WAITING_FOR_PICKUP sang BORROWING. Cập nhật trạng thái bản sao từ RESERVED sang BORROWED. Ghi nhận thời gian mượn thực tế, mã số thủ thư giải quyết và thiết lập hạn trả (mặc định 14 ngày).

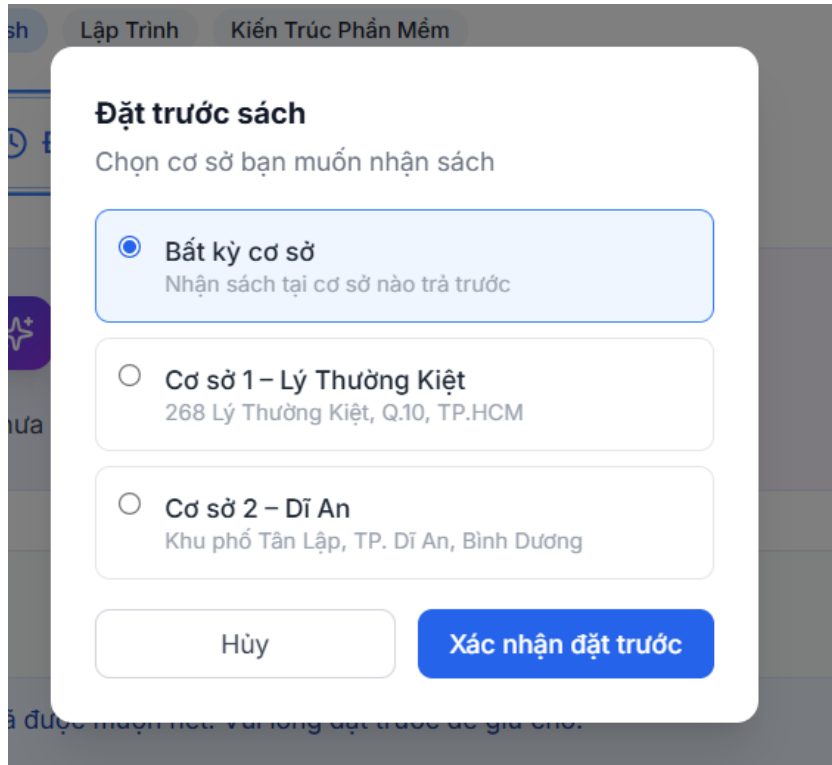
4. Mượn trực tiếp tại quầy (Direct Flow)

- **Endpoint:** POST /api/v1/transactions/borrow-direct (Quyền: Thủ thư)
- **Logic:** Dành cho trường hợp độc giả lấy sách trực tiếp từ kệ ra quầy mà không đặt trước. Thủ thư nhập Mã sinh viên và Mã vạch. Hệ thống kiểm tra điều kiện tương tự luồng trực tuyến, nhưng bỏ qua giai đoạn chờ. Giao dịch được tạo và chuyển thẳng sang trạng thái BORROWING, bản sao chuyển sang BORROWED.

6.2.11 Nghiệp vụ Đặt trước sách

Phân hệ Đặt trước sách (Reservation) là giải pháp giúp độc giả "xếp hàng" điện tử đối với những ấn phẩm đang tạm hết bản sao khả dụng tại thư viện.

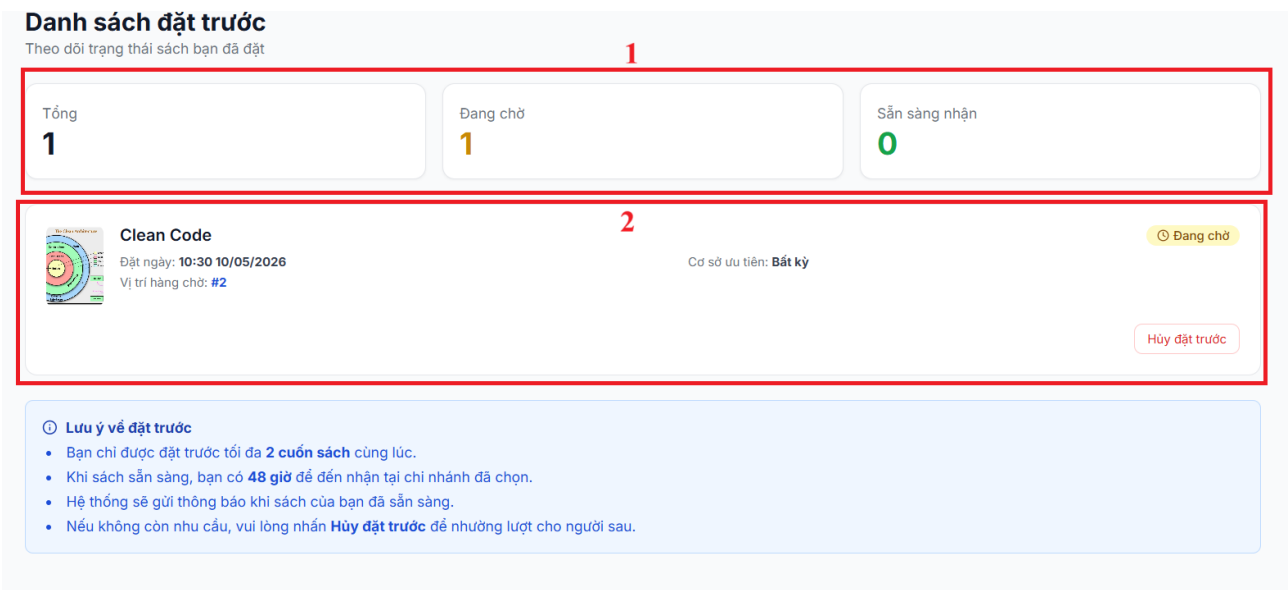
6.2.11.1 Khởi tạo yêu cầu đặt trước



Hình 6.34: Giao diện tùy chọn cơ sở nhận sách khi đặt trước

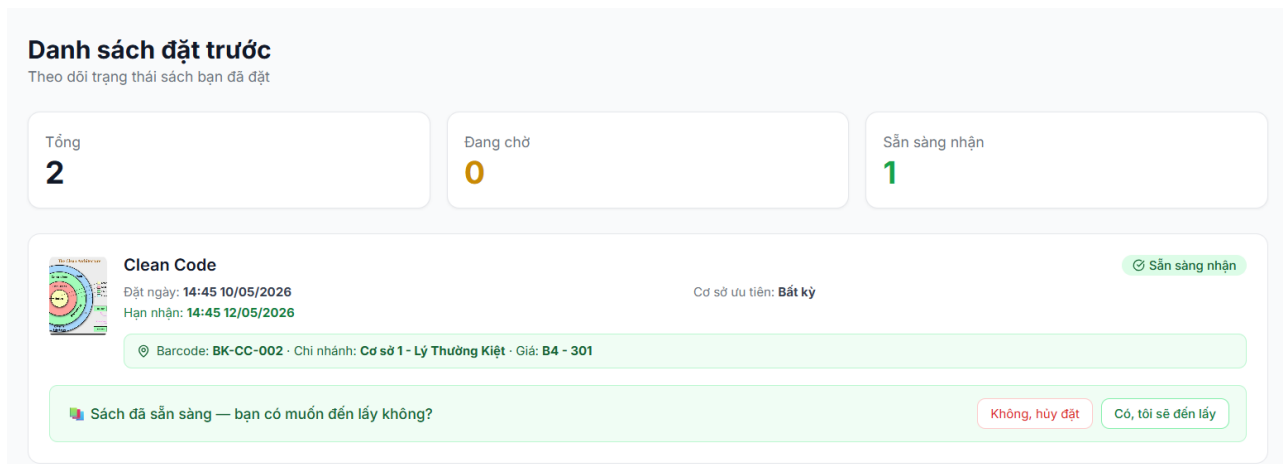
Mô tả giao diện: Khi toàn bộ bản sao của một quyển sách đang được mượn, nút chức năng "Mượn ngay" sẽ được thay thế bằng "Đặt trước". Một hộp thoại sẽ xuất hiện yêu cầu độc giả chọn cơ sở mong muốn nhận sách. Người dùng có thể chọn một cơ sở cụ thể (như Lý Thường Kiệt hoặc Dĩ An) hoặc chọn "Bất kỳ cơ sở" để tối ưu hóa thời gian chờ đợi (nhận sách ở cơ sở nào có người trả sớm nhất).

6.2.11.2 Quản lý danh sách đặt trước



Hình 6.35: Trạng thái Đang chờ xếp hàng

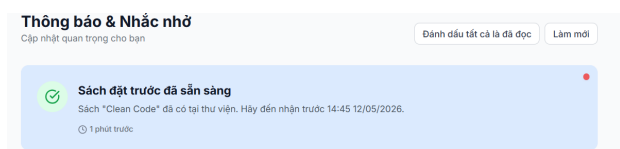
Mô tả trạng thái Đang chờ: Màn hình quản lý cung cấp một bộ đếm tổng quan về số lượng sách đang chờ và sẵn sàng. Ở trạng thái "Đang chờ", thẻ thông tin sẽ hiển thị rõ **vị trí hàng chờ** của độc giả (ví dụ: #2 nghĩa là có 1 người khác đang xếp hàng trước). Hệ thống cũng liệt kê minh bạch các quy định: Mỗi người chỉ được đặt tối đa 2 cuốn cùng lúc và có thời hạn 48 giờ để đến nhận sách khi sách sẵn sàng. Tại đây, độc giả có thể chủ động hủy yêu cầu để nhường lượt cho người sau.



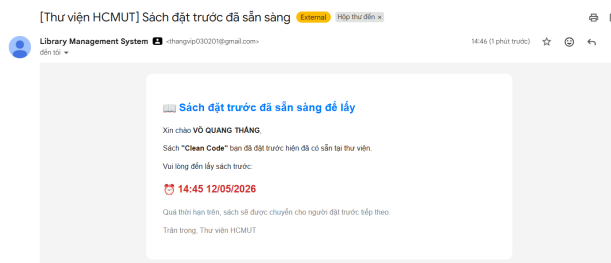
Hình 6.36: Trạng thái Sách sẵn sàng nhận

Mô tả trạng thái Sẵn sàng nhận: Khi đến lượt, trạng thái thẻ chuyển sang "Sẵn sàng nhận" (màu xanh). Lúc này, hệ thống sẽ tiết lộ cụ thể mã vạch (barcode) của cuốn sách được phân bổ, chi nhánh và vị trí kệ sách chi tiết. Giao diện yêu cầu người dùng xác nhận lại việc đến lấy sách hoặc hủy bỏ.

6.2.11.3 Hệ thống thông báo và Phiếu nhận

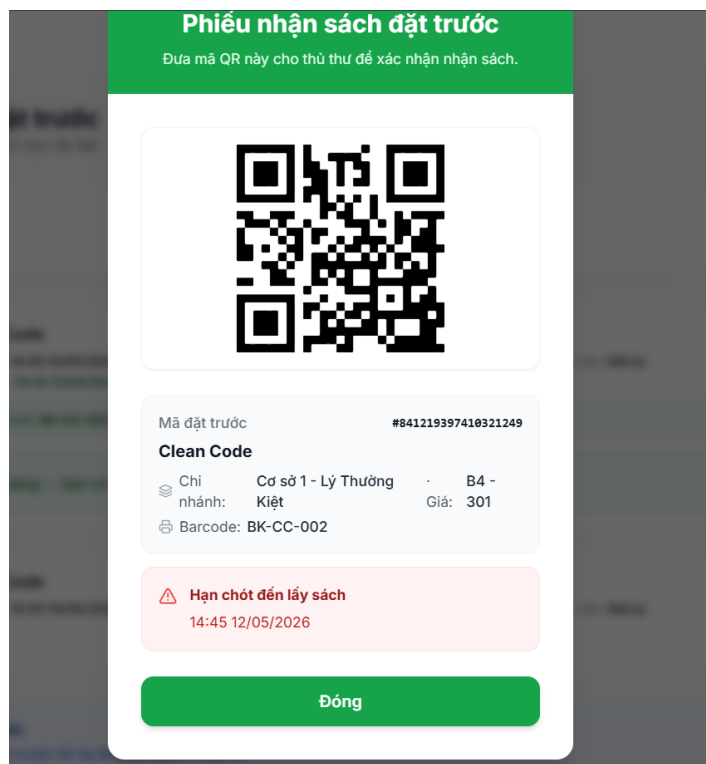


Hình 6.37: Thông báo cập nhật ngay trên trang web



Hình 6.38: Thư điện tử thông báo sách đã về

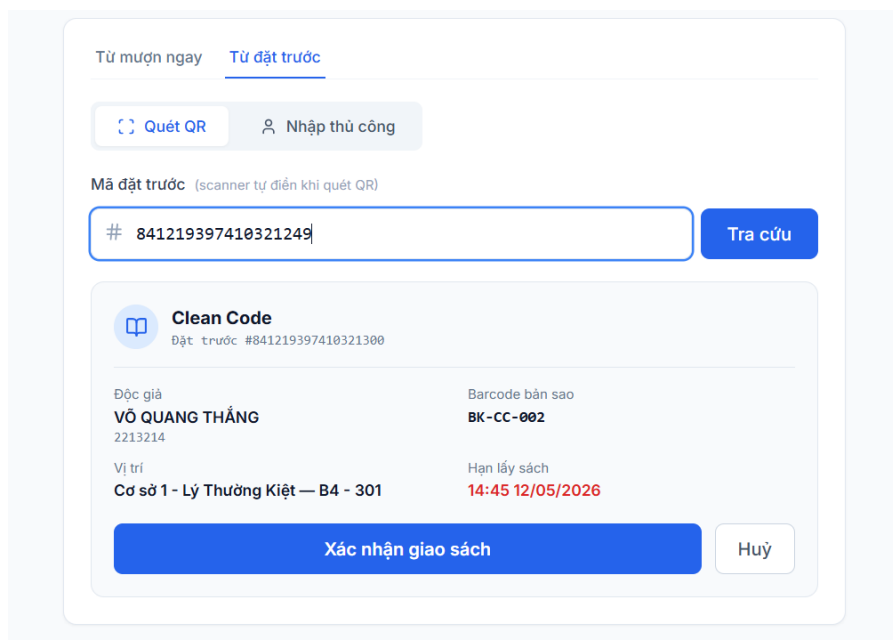
Luồng thông báo: Ngay khi hệ thống cấp phát sách thành công, một thông báo đầy thời gian thực và một thư điện tử sẽ được gửi song song đến độc giả, nhấn mạnh mốc thời gian hạn chót 48 giờ.



Hình 6.39: Phiếu nhận sách điện tử (Mã QR)

Phiếu nhận sách: Nếu độc giả xác nhận sẽ đến lấy ở Hình 6.36, một Phiếu nhận sách điện tử chứa mã QR sẽ được tạo ra, dùng để xuất trình cho thủ thư khi đến quầy.

6.2.11.4 Xử lý giao sách tại quầy



Hình 6.40: Giao diện Thủ thư xác nhận giao sách đặt trước

Luồng thủ thư: Tại hệ thống dành cho nhân viên, thủ thư chuyển sang phân hệ "Tủ đặt trước". Bằng cách quét mã QR của độc giả, hệ thống sẽ đối chiếu mã giao dịch và hiển thị thông tin bản sao đã được giữ sẵn. Thủ

thư chỉ việc lấy đúng quyển sách trên kệ (dựa theo barcode hiển thị) và nhấn "Xác nhận giao sách". Giao dịch lúc này chính thức chuyển thành "Đang mượn".

6.2.11.5 Đặc tả API và Kỹ thuật điều phối

Tính năng đặt trước yêu cầu cơ chế xử lý đồng thời (Concurrency) rất khắt khe để tránh tình trạng tranh chấp dữ liệu khi nhiều sách được trả về cùng lúc.

1. Khởi tạo yêu cầu đặt trước

- **Endpoint:** POST /api/v1/reservations
- **Kiểm duyệt (Validation):** Hệ thống áp dụng 5 tầng kiểm tra trước khi nhận đơn: Người dùng không nợ phí phạt; Chưa vượt hạn mức (tối đa 2 cuốn); Chưa đặt cuốn này trước đó; Thư viện phải sở hữu ít nhất 1 bản sao của sách này; và **quan trọng nhất**, toàn bộ các bản sao hiện tại đều không có sẵn (nếu có sẵn, hệ thống bắt lỗi và yêu cầu độc giả dùng tính năng Mượn ngay).
- **Xử lý hàng đợi:** Tính toán vị trí xếp hàng bằng công thức $\text{MAX}(\text{queue_position}) + 1$. Trạng thái giao dịch khởi tạo là PENDING.

2. Cơ chế phân bổ sách tự động (Assignment Service)

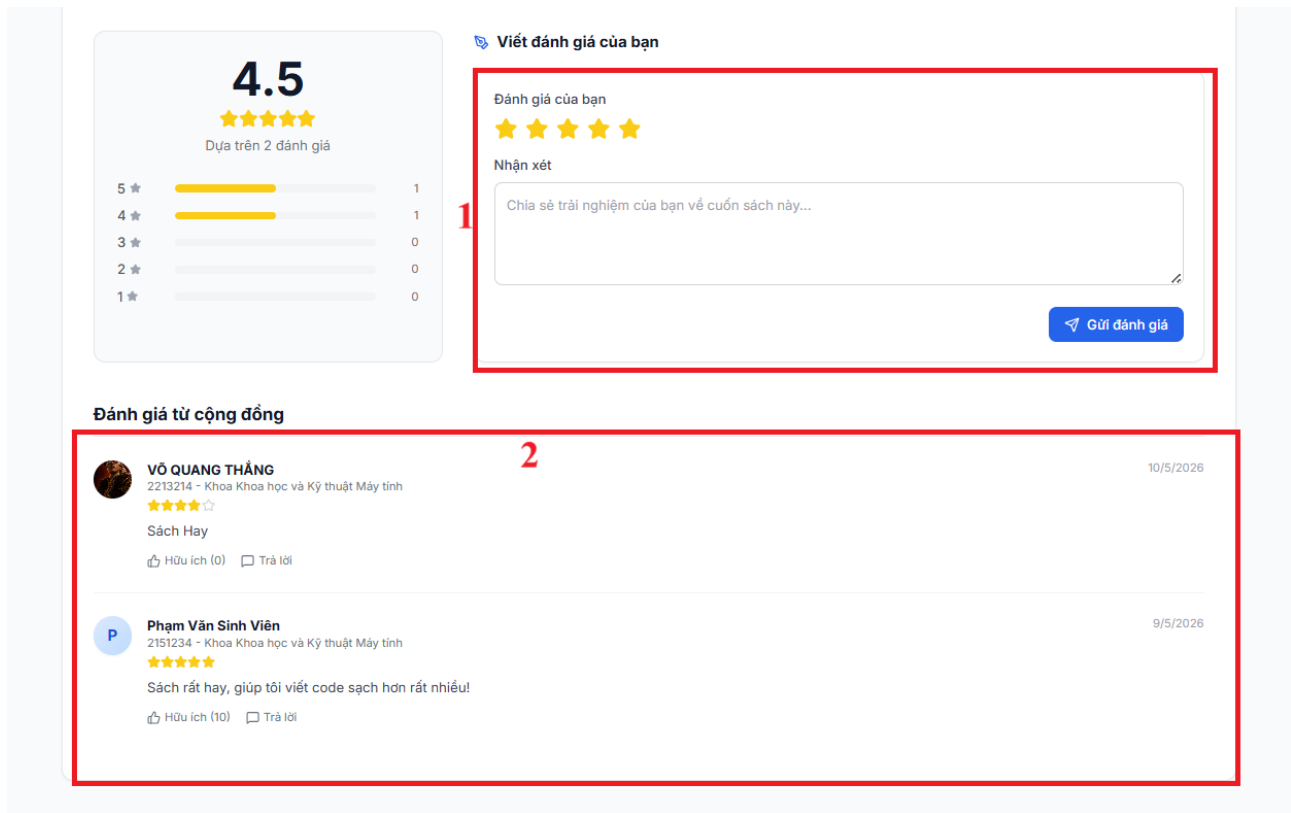
- Khi một bản sao được người dùng trước hoàn trả, hệ thống kích hoạt luồng kiểm tra hàng đợi đặt trước.
- **Tránh xung đột (Race Condition):** Truy vấn cơ sở dữ liệu sử dụng kỹ thuật khóa hàng FOR UPDATE SKIP LOCKED. Điều này đảm bảo nếu hai bản sao được trả lại cùng một lúc, hai tiến trình máy chủ sẽ không phân bổ sách cho cùng một người đầu hàng đợi.
- **Cập nhật trạng thái:** Người đầu tiên trong hàng đợi (ưu tiên theo chi nhánh) sẽ được gán assignedItemId. Trạng thái chuyển sang READY_FOR_PICKUP. Hệ thống thiết lập holdExpirationTime là thời điểm hiện tại cộng thêm 48 giờ, đồng thời bắn sự kiện qua Kafka để gửi thư thông báo.

3. Hủy hoặc quá hạn đặt trước

- **Endpoint:** DELETE /api/v1/reservations/{id}
- **Logic:** Nếu độc giả chủ động hủy, hoặc hệ thống quét thấy đã quá 48 giờ mà chưa nhận, giao dịch hiện tại sẽ chuyển sang CANCELLED hoặc EXPIRED. Ngay lập tức, bản sao vừa được giải phóng sẽ lại kích hoạt luồng Phân bổ tự động (bước 2) để nhường lượt cho người đang đứng thứ 2 trong hàng đợi.

6.2.12 Đánh giá và Nhận xét ấn phẩm

6.2.12.1 Giao diện Hệ thống đánh giá



Viết đánh giá của bạn

Đánh giá của bạn: ★★★★★

Nhận xét: Chia sẻ trải nghiệm của bạn về cuốn sách này...

Đánh giá từ cộng đồng

VŨ QUANG THẮNG
2213214 - Khoa Khoa học và Kỹ thuật Máy tính
★★★★☆
Sách Hay
👍 Hữu ích (0) 🗨️ Trả lời 10/5/2026

Phạm Văn Sinh Viên
2151234 - Khoa Khoa học và Kỹ thuật Máy tính
★★★★★
Sách rất hay, giúp tôi viết code sạch hơn rất nhiều!
👍 Hữu ích (10) 🗨️ Trả lời 9/5/2026

Hình 6.41: Giao diện Đánh giá và Nhận xét từ cộng đồng độc giả

Chú thích các vùng chức năng:

- **Khối Thống kê tổng quan (Bên trái):** Trình bày điểm đánh giá trung bình của ấn phẩm và thanh tiến trình phân bổ số lượng chi tiết cho từng mức sao (từ 1 đến 5).
- **Vùng 1 (Biểu mẫu tương tác):** Khu vực cho phép độc giả chủ động để lại nhận xét. Hệ thống yêu cầu người dùng chọn mức độ hài lòng thông qua hệ thống 5 sao và bắt buộc phải nhập nội dung chia sẻ trải nghiệm đọc.
- **Vùng 2 (Danh sách cộng đồng):** Hiển thị dòng thời gian các đánh giá từ những độc giả khác. Mỗi nhận xét đều được minh bạch hóa thông qua việc đính kèm thông tin định danh của người viết, bao gồm ảnh đại diện, họ tên, mã số sinh viên, khoa trực thuộc và thời gian đăng tải.

6.2.12.2 Đặc tả API và Ràng buộc nghiệp vụ

Hệ thống đánh giá được thiết kế như một mô-đun độc lập nhằm tối ưu hóa hiệu suất truy vấn. Đường dẫn gốc phục vụ giao tiếp nghiệp vụ này là `/api/v1/publications/{publicationId}/ratings`.

1. Ràng buộc cốt lõi khi tạo đánh giá

- **Xác thực quyền hạn:** Chỉ những người dùng đã xác thực (đăng nhập) mới được phép sử dụng điểm cuối của phân hệ này để tạo mới đánh giá.
- **Điều kiện thực tiễn:** Để đảm bảo tính khách quan của nền tảng, thuật toán kiểm duyệt sẽ đối chiếu

lịch sử giao dịch. Độc giả **bắt buộc phải từng mượn** ấn phẩm này thì mới có quyền để lại nhận xét. Nếu vi phạm, hệ thống sẽ từ chối và trả về mã lỗi USER_NOT_BORROWED_PUBLICATION.

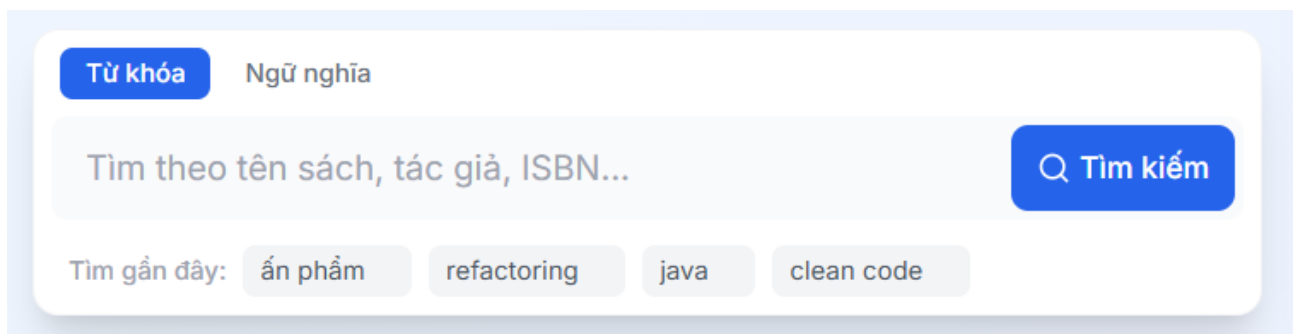
- **Tính duy nhất:** Mỗi độc giả chỉ được phép đánh giá một lần duy nhất cho mỗi đầu sách. Việc cố tình gửi yêu cầu trùng lặp sẽ bị chặn lại bởi ràng buộc dữ liệu kèm mã lỗi RATING_ALREADY_EXISTS.

2. Tối ưu hóa tải trang giao diện

- Khi độc giả truy cập vào tab Đánh giá, giao diện mạng (Frontend) sẽ tiến hành gọi đồng thời (song song) hai giao tiếp: một truy vấn để lấy dữ liệu thống kê tổng hợp và một truy vấn khác để lấy danh sách bình luận phân trang. Điều này giúp giảm thiểu độ trễ tải trang.
- Ngay sau khi người dùng gửi đánh giá mới thành công, toàn bộ giao diện (bao gồm cả khối thống kê và danh sách bình luận) sẽ tự động làm mới để phản ánh dữ liệu tức thời.

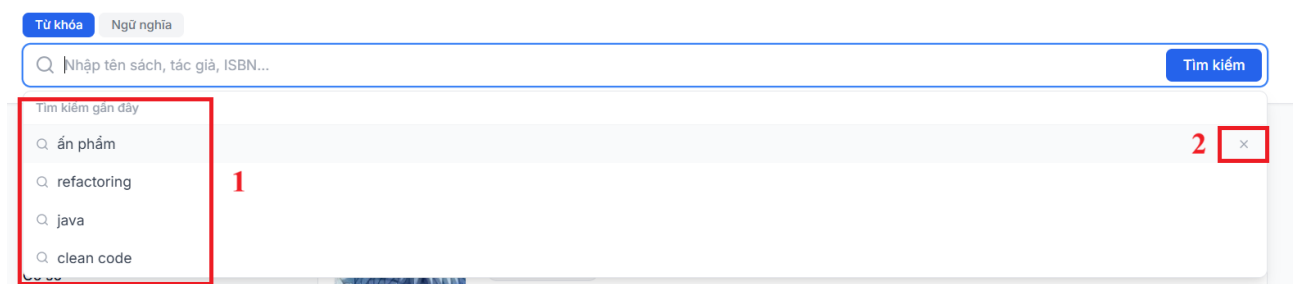
6.2.13 Lịch sử tìm kiếm

6.2.13.1 Giao diện hiển thị



Hình 6.42: Giao diện gợi ý từ khóa tìm kiếm gần đây tại Trang chủ

Mô tả giao diện tại Trang chủ: Ngay bên dưới thanh tìm kiếm chính, hệ thống cung cấp một danh sách các từ khóa mà độc giả đã tìm kiếm gần đây nhất, được trình bày dưới dạng các thẻ (chips) trực quan. Khi người dùng nhấn vào một thẻ, hệ thống sẽ tự động điền từ khóa đó vào ô tìm kiếm và thực hiện lệnh tra cứu ngay lập tức. Ngoài ra, khi di chuột lên từng thẻ, một biểu tượng dấu tắt (×) sẽ xuất hiện cho phép người dùng xóa nhanh từ khóa đó khỏi lịch sử.



Hình 6.43: Danh sách thả xuống quản lý lịch sử tại Trang tìm kiếm

Chú thích các vùng chức năng tại Trang tìm kiếm:

- **Vùng 1 (Danh sách từ khóa):** Khi người dùng nhấp chuột (focus) vào thanh tìm kiếm, một danh sách thả xuống sẽ xuất hiện, hiển thị tối đa 10 từ khóa tra cứu gần nhất. Quá trình này diễn ra mượt mà và tự động lọc các kết quả tương đồng (gợi ý) ngay khi người dùng bắt đầu gõ từng ký tự.

- **Vùng 2 (Công cụ gỡ bỏ):** Nút thao tác nhanh giúp độc giả chủ động xóa từng lịch sử riêng lẻ. Thao tác này sẽ làm mới danh sách hiển thị ngay lập tức.

6.2.13.2 Đặc tả giao tiếp hệ thống và Xử lý nghiệp vụ

Phân hệ này hoạt động dựa trên 3 luồng giao tiếp dữ liệu chính. Toàn bộ các điểm cuối đều yêu cầu người dùng phải đăng nhập hợp lệ để đảm bảo tính cá nhân hóa.

1. Ghi nhận và Lưu trữ từ khóa

- **Điểm cuối:** POST /api/v1/search-history
- **Mô tả luồng xử lý (Upsert):** Khi người dùng nhấn "Tìm kiếm", hệ thống sẽ âm thầm gửi từ khóa về máy chủ mà không làm gián đoạn trải nghiệm (không chặn giao diện). Thuật toán lưu trữ được thiết kế tối ưu như sau:
 - Hệ thống bỏ qua các từ khóa rỗng.
 - Nếu từ khóa đã từng được tìm kiếm trước đó, hệ thống chỉ cập nhật lại mốc thời gian và đẩy từ khóa này lên vị trí đầu tiên.
 - Nếu là từ khóa mới, hệ thống sẽ kiểm tra giới hạn lưu trữ. Mỗi tài khoản chỉ được lưu tối đa 10 từ khóa. Nếu vượt quá, máy chủ sẽ tự động tìm và xóa bản ghi cũ nhất trước khi thêm bản ghi mới.

2. Truy xuất danh sách lịch sử

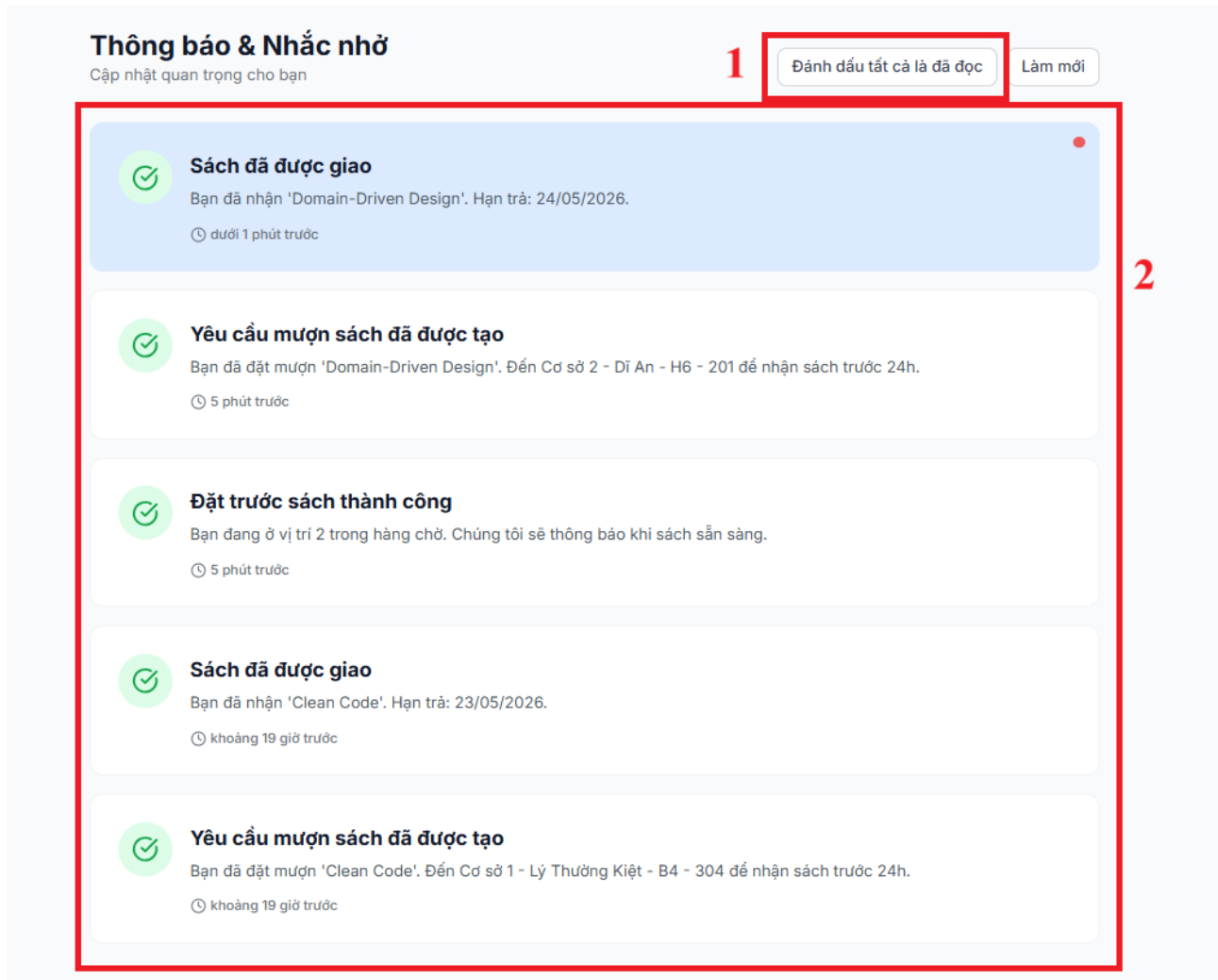
- **Điểm cuối:** GET /api/v1/search-history
- **Mô tả luồng xử lý:** Giao tiếp này phục vụ cả việc lấy 10 từ khóa gần nhất (khi ô tìm kiếm trống) và tính năng tự động gợi ý (khi người dùng đang gõ). Dữ liệu trả về luôn được sắp xếp ưu tiên theo mốc thời gian cập nhật mới nhất.

3. Gỡ bỏ từ khóa

- **Điểm cuối:** DELETE /api/v1/search-history/{id}
- **Mô tả luồng xử lý:** Người dùng có thể xóa một lịch sử cụ thể thông qua mã định danh. Máy chủ sẽ xác minh quyền sở hữu (đảm bảo người dùng không xóa lịch sử của người khác) trước khi thực thi lệnh.

6.2.14 Hệ thống Thông báo và Nhắc nhở (Real-time Notifications)

6.2.14.1 Giao diện Trung tâm thông báo



Hình 6.44: Giao diện quản lý Thông báo và Nhắc nhở dành cho Độc giả

Chú thích các vùng chức năng:

- **Vùng 1 (Thanh công cụ):** Cung cấp các thao tác quản lý nhanh, bao gồm nút "Đánh dấu tất cả là đã đọc" giúp làm sạch giao diện ngay lập tức và nút "Làm mới" để tải lại dữ liệu thủ công nếu cần.
- **Vùng 2 (Danh sách thông báo):** Hiển thị dòng thời gian các sự kiện liên quan trực tiếp đến người dùng. Mỗi thẻ thông báo được thiết kế trực quan với biểu tượng tương ứng với loại sự kiện (ví dụ: Tạo yêu cầu mượn, Giao sách thành công, Đặt trước sẵn sàng). Những thông báo chưa đọc sẽ được làm nổi bật bằng lớp nền xanh nhạt và một chấm đỏ ở góc phải, giúp độc giả dễ dàng nhận diện.

6.2.14.2 Đặc tả Kiến trúc và Luồng xử lý thời gian thực

Để mang lại trải nghiệm liền mạch, hệ thống thông báo được thiết kế dựa trên kiến trúc hướng sự kiện (Event-driven Architecture) kết hợp với công nghệ truyền tải hai chiều (WebSocket).

1. Hạ tầng kỹ thuật cốt lõi

- **Cơ chế hàng đợi (Kafka):** Bất cứ khi nào có một nghiệp vụ phát sinh từ các phân hệ khác (như mượn trả sách, bộ lập lịch quét quá hạn), máy chủ sẽ đóng gói một thông điệp và đẩy vào chủ đề (topic) `notification.send` của Kafka.
- **Truyền tải thời gian thực (WebSocket STOMP):** Hệ thống sử dụng giao thức STOMP trên nền SockJS để duy trì kết nối liên tục với trình duyệt của người dùng. Tính bảo mật được đảm bảo tuyệt đối bằng cách đính kèm chuỗi định danh (JWT) ngay trong phần tiêu đề (headers) khi khởi tạo kết nối.
- **Lưu trữ tối ưu:** Cơ sở dữ liệu được tách làm hai bảng riêng biệt: `notifications` dùng để lưu trữ nội dung thông báo chung (tiêu đề, nội dung, đường dẫn), và `user_notifications` để ánh xạ và theo dõi trạng thái "Đã đọc / Chưa đọc" của từng cá nhân.

2. Vòng đời của một Thông báo

1. **Phát sinh:** Phân hệ xử lý trung tâm (Consumer) nhận thông điệp từ Kafka, lưu nội dung vào cơ sở dữ liệu và ngay lập tức đẩy (push) dữ liệu qua kênh WebSocket cá nhân hóa (`/queue/notifications`) của người dùng.
2. **Tiếp nhận hiển thị:** Trình duyệt của người dùng nhận dữ liệu, tự động đẩy thông báo mới lên đầu danh sách, tăng bộ đếm số lượng chưa đọc, đồng thời hiển thị một thẻ thông báo nổi (Toast) với màu sắc được phân loại theo mức độ quan trọng.
3. **Khởi động hệ thống:** Ngay khi người dùng đăng nhập thành công, ứng dụng mạng (Frontend) sẽ thiết lập kết nối WebSocket có hỗ trợ tự động kết nối lại (auto-reconnect) sau 5 giây nếu mạng gián đoạn. Song song đó, hệ thống gọi API để lấy 20 thông báo gần nhất và cập nhật bộ đếm.

6.2.14.3 Đặc tả Giao tiếp dữ liệu (API)

notification-controller			
PUT	<code>/api/v1/users/notifications/{id}/read</code>	Mark a notification as read	🔒
PUT	<code>/api/v1/users/notifications/read-all</code>	Mark all notifications as read	🔒
GET	<code>/api/v1/users/notifications</code>	Get paginated notifications for current user	🔒
GET	<code>/api/v1/users/notifications/unread-count</code>	Get unread notification count for current user	🔒

Hình 6.45: Tập hợp các điểm cuối (Endpoints) của phân hệ Thông báo

Hệ thống quản lý trạng thái thông báo thông qua 4 điểm cuối chính thuộc `/api/v1/users/notifications`:

Bảng 6.9: Danh sách API Quản lý thông báo

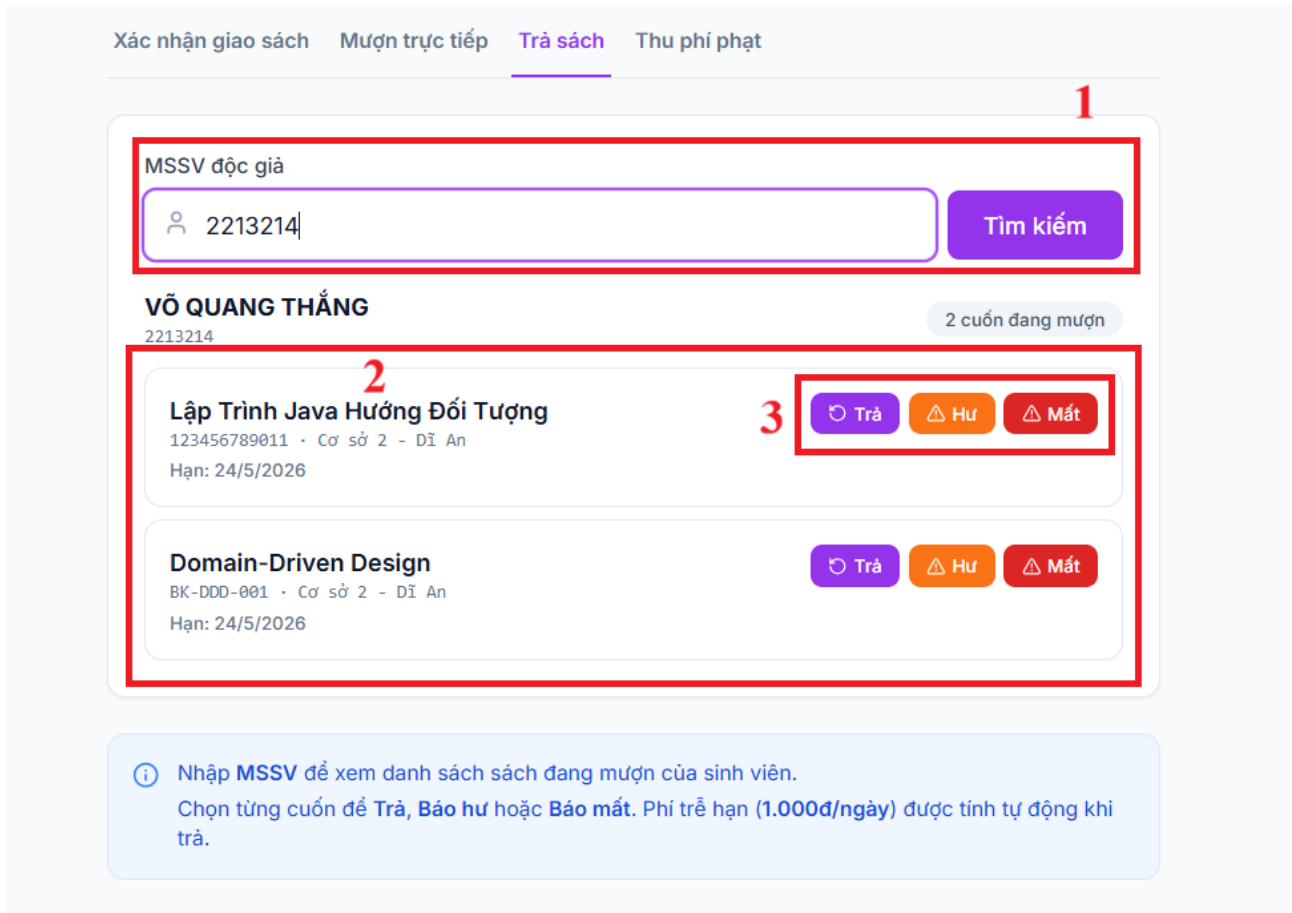
Phương thức	Đường dẫn (Path)	Mô tả nghiệp vụ
GET	<code>/</code>	Truy xuất danh sách thông báo của người dùng kèm phân trang.
GET	<code>/unread-count</code>	Trả về con số thống kê các thông báo chưa được đọc.
PUT	<code>/ {id} /read</code>	Đánh dấu một thông báo cụ thể là "Đã đọc".
PUT	<code>/read-all</code>	Đánh dấu toàn bộ thông báo của người dùng là "Đã đọc".

Tối ưu trải nghiệm: Khi người dùng nhấp vào một thông báo hoặc chọn "Đánh dấu tất cả là đã đọc", giao diện (Frontend) sẽ ngay lập tức chuyển trạng thái hiển thị (tắt chấm đỏ, giảm bộ đếm) mà không cần chờ phản

hồi từ máy chủ. Yêu cầu cập nhật cơ sở dữ liệu sẽ được gửi ngầm song song ở chế độ nền thông qua các lệnh PUT, giúp thao tác của người dùng luôn đạt độ trễ tiệm cận bằng không.

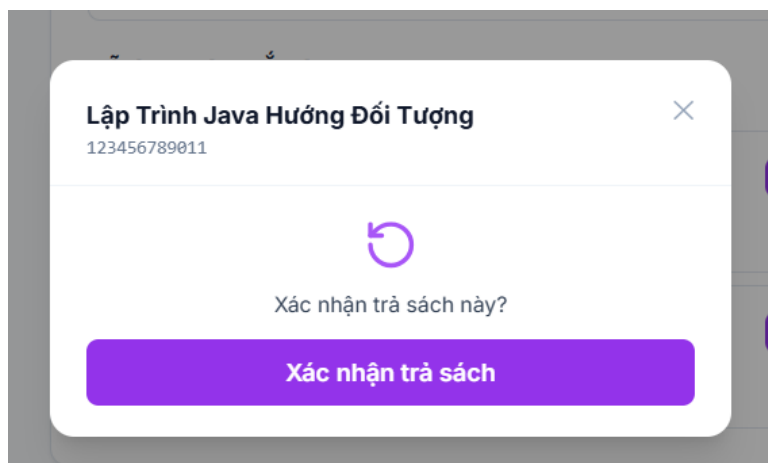
6.2.15 Nghiệp vụ Trả sách và Xử lý sự cố

6.2.15.1 Giao diện Tra cứu và Trả sách



Hình 6.46: Giao diện nhập thông tin để tra cứu sách đang mượn

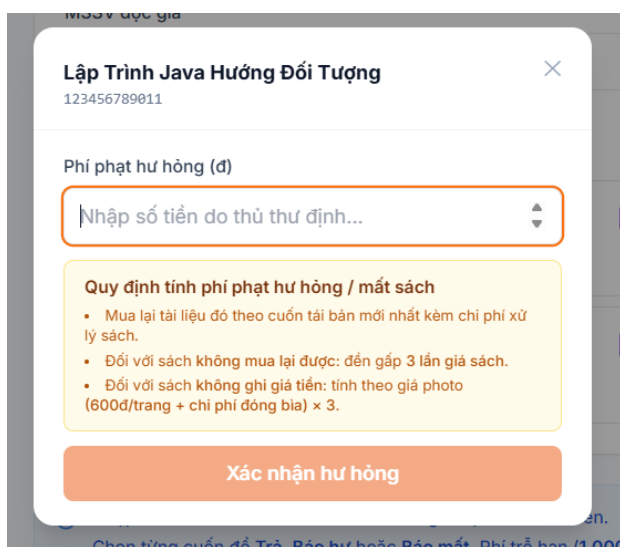
Chú thích giao diện: Tại phân hệ dành cho thủ thư, hệ thống cho phép tra cứu nhanh các giao dịch đang mượn thông qua Mã số sinh viên (MSSV). Danh sách trả về hiển thị rõ các ấn phẩm mà độc giả đang giữ, đi kèm với các nút thao tác tương ứng: "Trả" (hoàn trả bình thường), "Hư" (báo cáo hư hỏng) và "Mất" (báo cáo thất lạc).



Hình 6.47: Hộp thoại xác nhận hoàn trả tài liệu

Nghiệp vụ hoàn trả cơ bản: Khi thủ thư chọn "Trả", một hộp thoại xác nhận sẽ xuất hiện để đối chiếu thông tin sách và mã vạch lần cuối. Nếu giao dịch hoàn tất thành công, trạng thái của bản sao sẽ ngay lập tức được chuyển về "Có sẵn" (AVAILABLE). Trong trường hợp độc giả trả sách quá hạn, hệ thống sẽ tự động tính toán phí phạt dựa trên công thức số ngày trễ nhân với 1.000 VNĐ/ngày.

6.2.15.2 Xử lý sự cố Hư hỏng và Mất sách



Hình 6.48: Hộp thoại xử lý báo cáo hư hỏng sách



Hình 6.49: Hộp thoại xử lý báo cáo mất sách

Quy trình xử lý ngoại lệ: Khi xảy ra sự cố vật lý với bản sao, thủ thư sẽ sử dụng tính năng "Hư" hoặc "Mất". Hệ thống cung cấp một biểu mẫu để thủ thư nhập số tiền phạt cụ thể dựa trên quy định bồi thường của thư viện (ví dụ: đền gấp 3 lần giá sách đối với sách không mua lại được). Trạng thái bản sao sẽ lập tức bị khóa, chuyển thành Đang bảo trì (IN_MAINTENANCE) hoặc Thất lạc (LOST), đảm bảo không có giao dịch đặt trước nào được phép khởi tạo trên bản sao này.

6.2.15.3 Hệ thống thông báo và Xác nhận



Hình 6.50: Thư điện tử xác nhận giao dịch trả sách thành công

Mô tả luồng thông báo: Ngay sau khi thủ thư xác nhận trả sách trên hệ thống, một sự kiện (event) sẽ được kích hoạt thông qua Kafka. Độc giả sẽ lập tức nhận được một thư điện tử biên nhận, ghi rõ thời gian hoàn tất giao dịch để lưu trữ và đối soát.

6.2.15.4 Đặc tả API và Kỹ thuật điều phối

Phân hệ trả sách yêu cầu xử lý logic phức tạp để đảm bảo tính nhất quán dữ liệu, đặc biệt là cơ chế khóa chống tranh chấp (Pessimistic Lock).

1. Truy xuất thông tin giao dịch

- **Điểm cuối (Endpoint):** GET /api/v1/transactions/active
- **Mô tả:** Hệ thống tiến hành liên kết (JOIN) các bảng dữ liệu để tìm kiếm giao dịch đang ở trạng thái Đang mượn (BORROWING) hoặc Quá hạn (OVERDUE) tương ứng với mã vạch được cung cấp.

2. Hoàn trả tài liệu bình thường

- **Điểm cuối (Endpoint):** POST /api/v1/transactions/return
- **Cơ chế xử lý (Logic):** Máy chủ áp dụng khóa dữ liệu (Pessimistic Lock) trên bản sao để ngăn chặn các tiến trình đồng thời. Sau khi chuyển trạng thái giao dịch thành Đã trả (RETURNED), hệ thống sẽ gọi Dịch vụ Điều phối (ReservationAssignmentService) để tự động phân bổ quyền sách này cho độc giả tiếp theo đang xếp hàng đợi.

3. Báo cáo và xử lý sự cố

- **Điểm cuối (Endpoint):** POST /api/v1/transactions/{id}/report-issue
- **Mô tả:** API tiếp nhận yêu cầu cập nhật sự cố kèm theo mức phạt bồi thường do thủ thư quyết định. Nếu độc giả đồng thời trả trễ hạn và làm mất sách, hệ thống sẽ gộp cả hai khoản phạt: phí trễ hạn (OVERDUE_RETURN) và phí bồi thường (LOST_BOOK) thành danh sách nợ (UNPAID) trên hồ sơ sinh viên. Quy trình phân bổ sách tự động sẽ bị chặn lại hoàn toàn đối với ấn phẩm này.

4. Bảng tổng kết trạng thái

Bảng 6.11: Trạng thái bản sao và Phí phạt tương ứng sau khi trả sách

Trường hợp thực tế	Trạng thái bản sao	Loại phí phạt phát sinh
Trả sách đúng hạn	AVAILABLE	Không có
Trả sách trễ hạn	AVAILABLE	1 khoản: OVERDUE_RETURN
Báo hỏng (Đúng hạn)	IN_MAINTENANCE	1 khoản: DAMAGED_BOOK
Báo mất (Đúng hạn)	LOST	1 khoản: LOST_BOOK
Báo hỏng hoặc mất + Trễ hạn	IN_MAINTENANCE / LOST	2 khoản: Phí sự cố + OVERDUE_RETURN

6.2.16 Khám phá và Đề xuất ấn phẩm

6.2.16.1 Giao diện hiển thị Trang chủ

Chú thích các vùng chức năng:

- **Vùng 1 (Mới cập nhật):** Trình bày danh sách các ấn phẩm vừa được bổ sung vào kho tài liệu của thư viện. Mỗi thẻ sách cung cấp hình ảnh trực quan, trạng thái (thông báo số lượng bản sao khả dụng), tiêu đề, tên tác giả, đánh giá sao trung bình và nút điều hướng xem chi tiết.
- **Vùng 2 (Xu hướng - Được mượn nhiều):** Hiển thị bảng xếp hạng các quyển sách phổ biến nhất trong hệ thống. Giao diện nhấn mạnh thứ hạng (từ #1 trở đi) và công khai tổng số lượt mượn thực tế của mỗi tựa sách.

6.2.16.2 Đặc tả giao tiếp hệ thống (API)

Phân hệ khám phá cung cấp dữ liệu trực quan thông qua hai điểm cuối công khai, cho phép mọi người dùng (kể cả chưa đăng nhập) đều có thể truy cập.

1. Truy xuất danh sách Sách mới cập nhật

- **Điểm cuối (Endpoint):** GET /api/v1/publications/newest
- **Cấu hình dữ liệu:** Hệ thống mặc định trả về 10 ấn phẩm mới nhất (có thể thay đổi qua tham số limit).
- **Logic xử lý:** Máy chủ thực hiện truy vấn nhóm (GROUP BY) trên cơ sở dữ liệu để tổng hợp toàn diện thông tin. Quá trình này đồng thời đếm số lượng bản sao đang ở trạng thái khả dụng (AVAILABLE), tính điểm đánh giá trung bình và đếm tổng số lượt nhận xét từ cộng đồng. Kết quả trả về được sắp xếp nghiêm ngặt theo thời gian tạo mới nhất (createdAt DESC).

2. Truy xuất danh sách Xu hướng

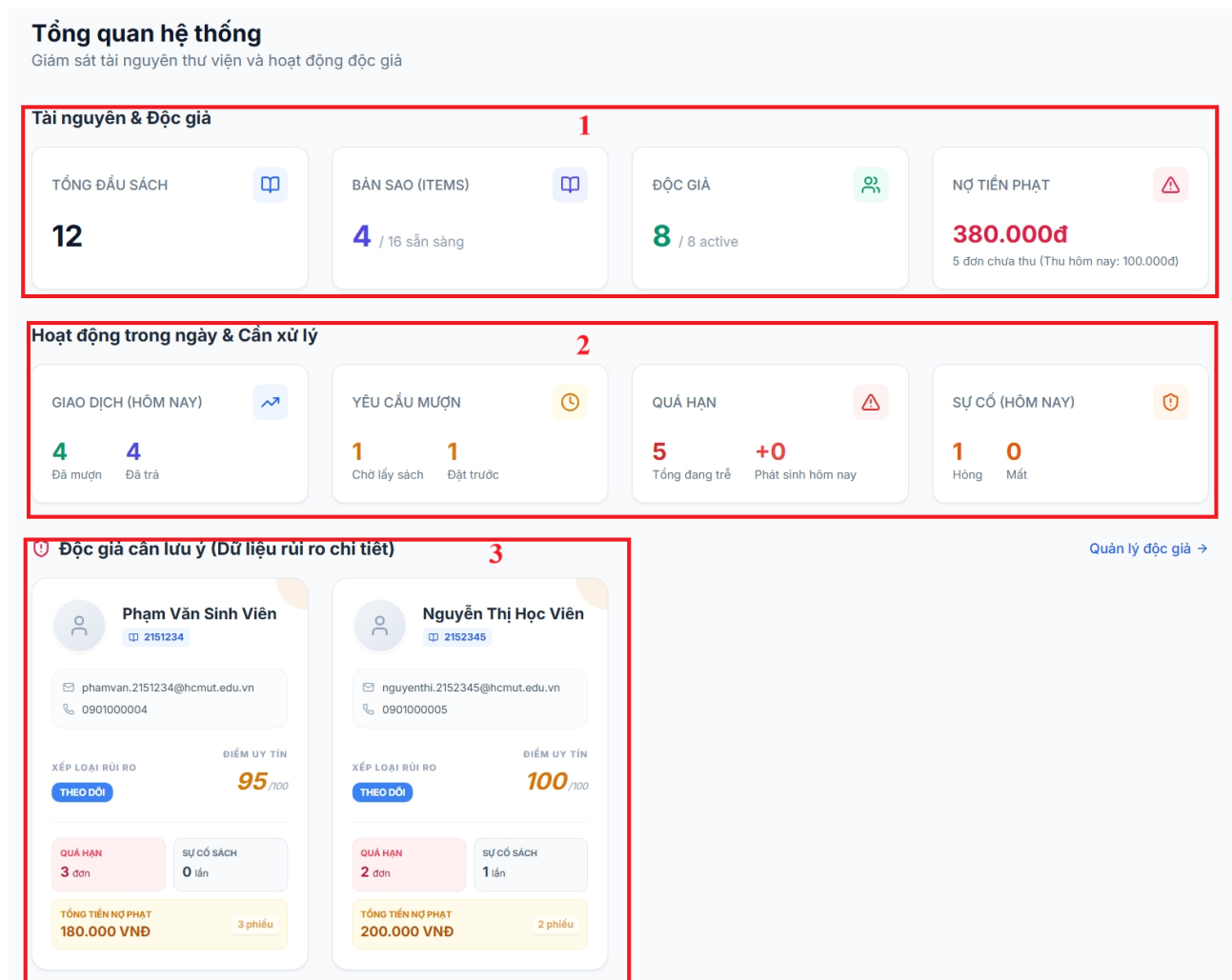
- **Điểm cuối (Endpoint):** GET /api/v1/publications/most-borrowed
- **Logic định nghĩa "Lượt mượn":** Nhằm phản ánh chính xác sức hút thực tế của một quyển sách, thuật toán chỉ đếm các giao dịch mượn trả đã hoàn tất (tức là có trạng thái RETURNED). Các giao dịch đang diễn ra hoặc đang bị trễ hạn sẽ bị loại trừ hoàn toàn khỏi công thức xếp hạng.
- **Sắp xếp:** Dữ liệu danh sách được sắp xếp giảm dần dựa trên tổng số lượt mượn (borrowCount DESC).

6.2.16.3 Kỹ thuật Tối ưu hóa hiệu suất

- **Chống bùng nổ dữ liệu (Tránh Cartesian Product):** Vì một quyển sách có thể có nhiều tác giả, hệ thống không thực hiện liên kết (JOIN) bảng tác giả trực tiếp vào câu lệnh truy vấn chính vốn đã chứa nhiều hàm tính toán phức tạp. Thay vào đó, kỹ thuật xử lý được chia làm hai bước: (1) Lấy danh sách mã định danh của ấn phẩm, sau đó (2) thực hiện một truy vấn độc lập để lấy toàn bộ tên tác giả tương ứng và ánh xạ ngược lại vào kết quả trả về.
- **Dữ liệu theo thời gian thực:** Toàn bộ số liệu báo cáo về số bản sao khả dụng và lượt mượn đều được truy vấn trực tiếp từ cơ sở dữ liệu tại thời điểm gọi, hoàn toàn không sử dụng bộ nhớ đệm (cache) nhằm đảm bảo hệ thống luôn cung cấp thông tin chính xác nhất cho độc giả.

6.2.17 Bảng điều khiển Thủ thư

6.2.17.1 Giao diện Quản trị tổng thể



Hình 6.51: Giao diện trung tâm điều hành dành cho Thủ thư

Chú thích các vùng chức năng:

- **Vùng 1 (Hệ thống thẻ thống kê):** Cung cấp số liệu tổng hợp theo thời gian thực về quy mô kho tài liệu và người dùng, bao gồm: Tổng số đầu sách (ấn phẩm), tổng số bản sao vật lý hiện có, số lượng độc giả đang hoạt động và tổng số giao dịch mượn trả trong tháng.

- **Vùng 2 (Biểu đồ xu hướng mượn sách):** Trình bày dữ liệu dưới dạng biểu đồ cột, giúp thủ thư theo dõi biến động lượng mượn sách theo từng tháng trong năm. Đây là cơ sở quan trọng để đánh giá hiệu suất vận hành và nhu cầu đọc của sinh viên.
- **Vùng 3 (Phân tích trạng thái bản sao):** Biểu đồ tròn thể hiện tỷ lệ phân bổ trạng thái của các bản sao trong kho (Có sẵn, Đang mượn, Đang bảo trì, Thất lạc).
- **Vùng 4 (Hoạt động gần đây):** Danh sách các giao dịch hoặc yêu cầu đặt trước mới nhất vừa phát sinh trên hệ thống, giúp thủ thư nắm bắt nhanh các công việc cần xử lý ưu tiên.

6.2.17.2 Đặc tả kỹ thuật và Xử lý dữ liệu

Để hiển thị được các chỉ số tổng hợp trên, hệ thống sử dụng các truy vấn tính toán phức tạp (Aggregate Functions) để gom nhóm dữ liệu từ nhiều phân hệ khác nhau.

1. Đặc tả API Thống kê tổng quát

- **Điểm cuối (Endpoint):** GET /api/v1/librarian/dashboard/statistics
- **Quyền truy cập:** Dành riêng cho quyền Thủ thư.
- **Mô tả logic:** Máy chủ thực hiện lệnh đếm (COUNT) đồng thời trên các bảng publications, items, users và transactions. Để tối ưu hiệu suất, các kết quả này có thể được lưu vào bộ nhớ đệm (Cache) và cập nhật định kỳ sau mỗi 30 phút.

2. Đặc tả API Biểu đồ xu hướng

- **Điểm cuối (Endpoint):** GET /api/v1/librarian/dashboard/borrow-trends
- **Logic xử lý:** Hệ thống truy vấn bảng transactions, lọc theo trạng thái BORROWING và RETURNED trong khoảng thời gian 12 tháng gần nhất. Dữ liệu được nhóm theo tháng (GROUP BY month) để trả về mảng số liệu phục vụ việc vẽ biểu đồ tại giao diện.

3. Cơ chế cập nhật thời gian thực

- Bảng điều khiển được tích hợp với hệ thống thông báo đẩy. Khi có một giao dịch mượn sách hoặc một yêu cầu đặt trước mới được thực hiện bởi độc giả, hệ thống sẽ gửi tín hiệu qua WebSocket để tự động cập nhật lại các con số thống kê trên màn hình của thủ thư mà không cần tải lại trang, giúp việc quản lý luôn ở trạng thái cập nhật nhất.

6.2.19.1 Giao diện Danh sách Ấn phẩm

SmartLibrary Admin Portal

MENU CHÍNH

- Trang chủ
- Quản lý mượn - trả sách
- Đầu sách 842
- Bản sao
- Yêu cầu 12
- HỆ THỐNG
- Cài đặt
- Thông báo
- Đăng xuất

Hồ Sỹ Thắng
Librarian Admin

Danh sách ấn phẩm
Quản lý danh mục ấn phẩm thư viện

1 + Thêm đầu sách mới

TÌM KIẾM **CHỦ ĐỀ** **NĂM XUẤT BẢN**

Tìm theo tiêu đề / tác giả / ISBN... **Tất cả chủ đề** **Tất cả năm**

2 ☒ Tất cả ☐ Có bản sao ☐ Chưa có bản sao **Xóa bộ lọc** **Áp dụng**

3

☐	TIÊU ĐỀ	TÁC GIẢ	NĂM	SỐ LƯỢNG	NGÀY TẠO ↓	THAO TÁC
☐	ha	Martin Fowler	2022	0 / 0	11:02 10/05/2026	
☐	ấn phẩm 5	Eric Evans	2022	0 / 0	00:02 10/05/2026	
☐	ấn phẩm 4	Nguyễn Văn Linh	2024	0 / 0	23:54 09/05/2026	
☐	ấn phẩm 3	Eric Evans	2020	0 / 0	23:47 09/05/2026	
☐	đầu sách 3	Eric Evans	2016	0 / 0	21:10 09/05/2026	
☐	đầu sách 2	Eric Evans	2016	0 / 0	21:09 09/05/2026	
☐	đầu sách 1	Eric Evans	2016	0 / 0	21:08 09/05/2026	
☐	Clean Code: A Handbook of Agile Software Craftsmanship	Robert C. Martin	2008	0 / 4	14:44 09/05/2026	
☐	Lập Trình Java Hướng Đối Tượng	Trần Thị Minh Hoa	2021	0 / 2	14:44 09/05/2026	
☐	Giáo Trình Cơ Sở Dữ Liệu	Nguyễn Văn Linh	2020	0 / 3	14:44 09/05/2026	

Hiện thị 1-10 trong tổng số 12 đầu sách

< **1** 2 >

Hình 6.53: Giao diện quản lý danh mục ấn phẩm tập trung

Các khu vực chức năng chính (Hình 6.53):

- **Tìm kiếm và Thao tác:** Thanh tìm kiếm hỗ trợ tra cứu đa năng theo tiêu đề, tác giả hoặc mã ISBN. Nút "Thêm đầu sách mới" cho phép khởi tạo nhanh một ấn phẩm vào hệ thống.
- **Bộ lọc thông minh:** Hệ thống cung cấp các bộ lọc nâng cao theo trạng thái sở hữu (Có bản sao / Chưa có bản sao), chủ đề và năm xuất bản để tối ưu hóa việc quản lý.
- **Bảng dữ liệu:** Hiển thị chi tiết tiêu đề, tác giả, năm xuất bản, tỷ lệ bản sao khả dụng trên tổng số và mốc thời gian cập nhật gần nhất.

6.2.19.2 Xem chi tiết và Cập nhật Ấn phẩm

Đầu sách / Clean Code

←

Chi Tiết Đầu Sách

Tạo Bản Sao Mới

Publication Metadata

Quản lý các thông tin cơ bản về ấn phẩm

CHỈNH SỬA

Tiêu đề *

Clean Code

ISBN *

9780132350884

Mã xếp giá (DDC) VD: 813.083 M6237M

Nhập hoặc tra cứu từ ISBN

Số trang

431

Năm xuất bản *

2008

Ngôn ngữ *

English

Tái bản lần thứ

1

Kích cỡ (A x A x A cm)

23×18cm

Khối lượng (g)

0,72

Đặc tả

Hướng dẫn viết code sạch, dễ bảo trì và mở rộng. Cuốn sách kinh điển của Robert C. Martin.

Đối tượng độc giả (Khoa)

-- Chưa phân loại --

Khoa / ngành phù hợp với nội dung ấn phẩm này

Nhà xuất bản *

O'Reilly Media

Tác giả *

Robert C. Martin

Danh mục

Lập Trình

Kiến Trúc Phần Mềm

Tags

Clean Code

Architecture

Xem Trước

Clean Code

Robert C. Martin • 2008

★★★★☆ 5 (1 đánh giá)

Available: 0 bản sao

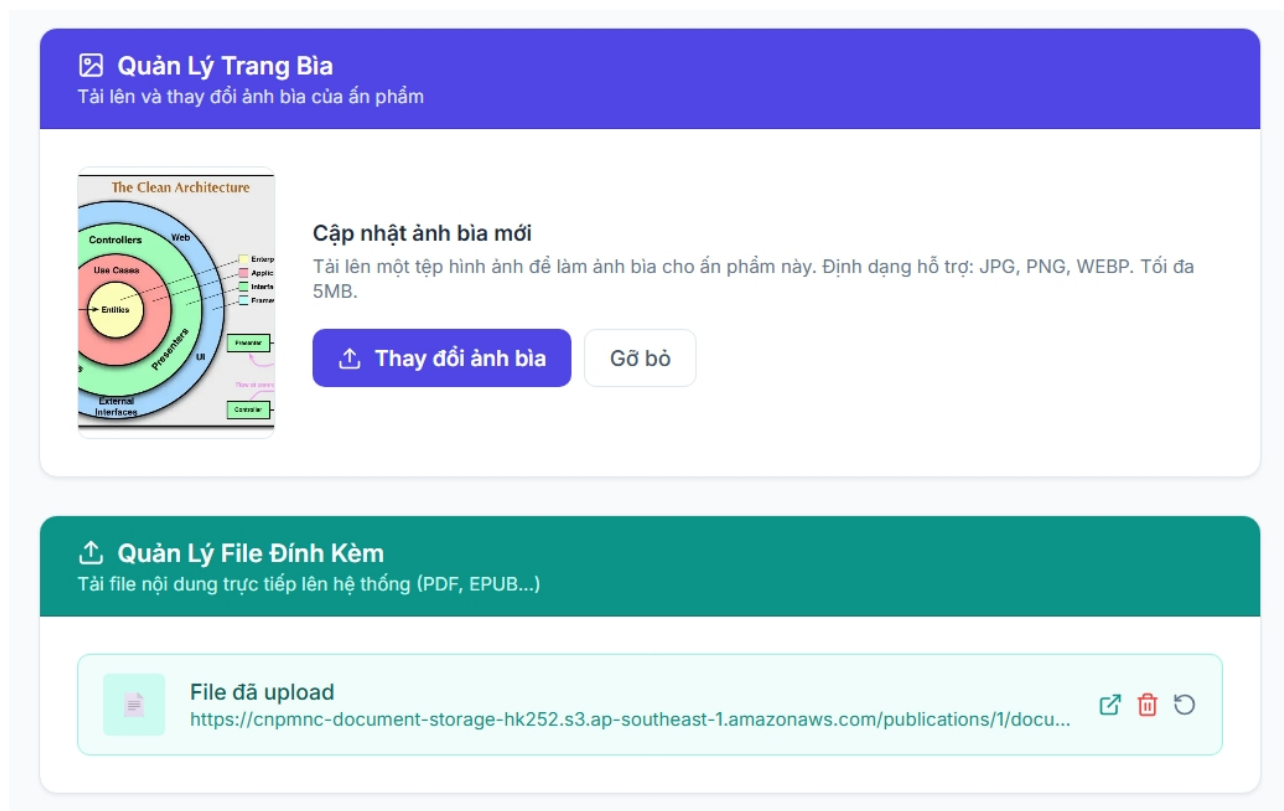
On Loan: 3 bản sao

Total: 4 bản sao

Xem Trang Công Khai

Hình 6.54: Giao diện hiệu chỉnh siêu dữ liệu của ấn phẩm

Quản lý siêu dữ liệu (Hình 6.54): Thủ thư có thể hiệu chỉnh các thông tin định danh (ISBN), thông số vật lý (số trang, kích thước, khối lượng) và các thuộc tính phân loại (Tác giả, Danh mục, Tags). Hệ thống cũng cho phép thiết lập "Đối tượng độc giả mục tiêu" (Khoa/Ngành) để phục vụ thuật toán gợi ý của AI.



Hình 6.55: Quản lý ảnh bìa và tệp tin tài liệu số

Quản lý tệp tin (Hình 6.55): Phân hệ này cho phép cập nhật ảnh bìa mới (hỗ trợ JPG, PNG, WEBP) và quản lý tài liệu số đi kèm (PDF/EPUB). Các tệp tin được lưu trữ an toàn trên dịch vụ S3.

6.2.19.3 Đặc tả kỹ thuật và Xử lý dữ liệu

1. Truy xuất danh sách quản trị (GET API)

- **Cơ chế:** Sử dụng Native SQL với EntityManager để tối ưu hiệu suất khi truy vấn dữ liệu liên kết phức tạp.
- **Kỹ thuật:** Sử dụng hàm STRING_AGG để gộp danh sách tác giả/danh mục và mệnh đề FILTER để đếm số bản sao khả dụng (AVAILABLE). Bộ lọc hasItems được xử lý bằng truy vấn con EXISTS để không làm sai lệch kết quả phân trang.

2. Đồng bộ hóa dữ liệu cập nhật (PUT API)

- **Quy trình:** Sau khi cập nhật các thông tin cơ bản qua JPA saveAndFlush, hệ thống sẽ thực hiện đồng bộ các bảng trung gian.
- **Xử lý:** Toàn bộ các liên kết cũ (Tác giả, Thể loại, Tags) sẽ bị xóa và chèn mới hàng loạt (Batch Insert) bằng JdbcTemplate kết hợp với trình tạo định danh TsIdGenerator để đảm bảo tốc độ xử lý.

6.2.20 Tạo mới ấn phẩm

Quy trình khởi tạo một ấn phẩm mới trong hệ thống được thiết kế theo mô hình đa bước nhằm đảm bảo tính toàn vẹn của dữ liệu và tối ưu hóa băng thông máy chủ khi xử lý các tệp tin dung lượng lớn.

6.2.20.1 Thiết lập thông tin siêu dữ liệu

←

Đầu sách / Thêm mới

Thêm đầu sách

Publication Metadata

Quản lý các thông tin cơ bản về ấn phẩm

Tiêu đề *

ISBN *

Mã xếp giá (DDC) VD: 813.083 M6237M

Nhập hoặc tra cứu từ ISBN

Tra cứu DDC

Số trang

Năm xuất bản *

Ngôn ngữ *

Chọn ngôn ngữ

Tái bản lần thứ

Kích cỡ (A x A x A cm)

Khối lượng (g)

Đặc tả

Đối tượng độc giả (Khoa)

-- Chưa phân loại --

Khoa / ngành phù hợp với nội dung ấn phẩm này

Nhà xuất bản *

Chọn hoặc nhập nhà xuất bản

Tác giả *

Chọn hoặc nhập tác giả

Danh mục

Chọn hoặc nhập danh mục

Tags

Chọn hoặc nhập tag

Hình 6.56: Giao diện nhập thông tin chi tiết ấn phẩm

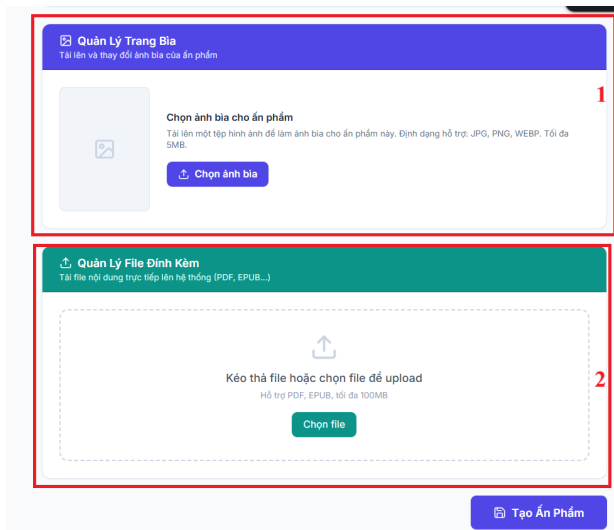
Chú thích các vùng chức năng:

- **Vùng 1 (Thông tin định danh):** Các trường dữ liệu cốt lõi bao gồm mã ISBN, Tiêu đề ấn phẩm và Nhà xuất bản.
- **Vùng 2 (Hệ thống phân loại):** Cho phép liên kết ấn phẩm với danh sách tác giả, danh mục thể loại

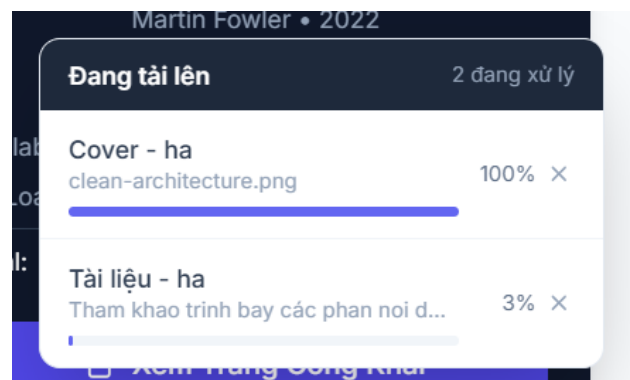
và các nhãn (tags) từ khóa để phục vụ việc tra cứu.

- **Vùng 3 (Hỗ trợ AI):** Khu vực nhập tóm tắt nội dung do AI trích xuất và xác định đối tượng độc giả mục tiêu (ví dụ: Khoa Cơ khí) nhằm phục vụ thuật toán gợi ý sách.
- **Vùng 4 (Thông số kỹ thuật):** Các trường thông tin bổ sung như số trang, năm xuất bản và ký hiệu xếp giá (Call Number) để quản lý vị trí vật lý.

6.2.20.2 Quản lý tệp tin đính kèm và Ảnh bìa



Hình 6.57: Giao diện chọn tệp đính kèm



Hình 6.58: Bảng điều khiển trạng thái tải lên

Mô tả nghiệp vụ: Hệ thống hỗ trợ đính kèm ảnh bìa (Cover) và tài liệu số (PDF/Ebook). Để giảm tải cho máy chủ Backend, quy trình tải lên tài liệu số được thực hiện trực tiếp từ trình duyệt lên dịch vụ lưu trữ đám mây S3 thông qua một đường dẫn được cấp quyền tạm thời (Presigned URL).

6.2.20.3 Đặc tả kỹ thuật và Luồng xử lý API

Quy trình tạo mới được thực hiện thông qua chuỗi 4 điểm cuối API tuần tự nhằm tối ưu hóa hiệu suất và khả năng mở rộng của hệ thống.

Giai đoạn 1: Khởi tạo siêu dữ liệu (Metadata)

- **Điểm cuối:** POST /api/v1/publications (Quyền: Thủ thư).
- **Logic xử lý:** Hệ thống sử dụng TsidGenerator để tạo định danh duy nhất, lưu trữ vào bảng ấn phẩm và thực hiện chèn hàng loạt (batch insert) vào các bảng trung gian liên kết với tác giả và thể loại.

Giai đoạn 2: Tải lên ảnh bìa

- **Điểm cuối:** POST /api/v1/publications/{id}/cover
- **Mô tả:** Tải lên tệp tin dạng multipart/form-data. Máy chủ xác thực định dạng ảnh (jpeg, png, webp) và dung lượng (tối đa 5MB) trước khi đẩy lên S3 tại đường dẫn covers/{publicationId}/.

Giai đoạn 3: Cấp quyền tải tài liệu trực tiếp (S3 Bypass)

- **Điểm cuối:** GET /api/v1/publications/{id}/document-upload-url

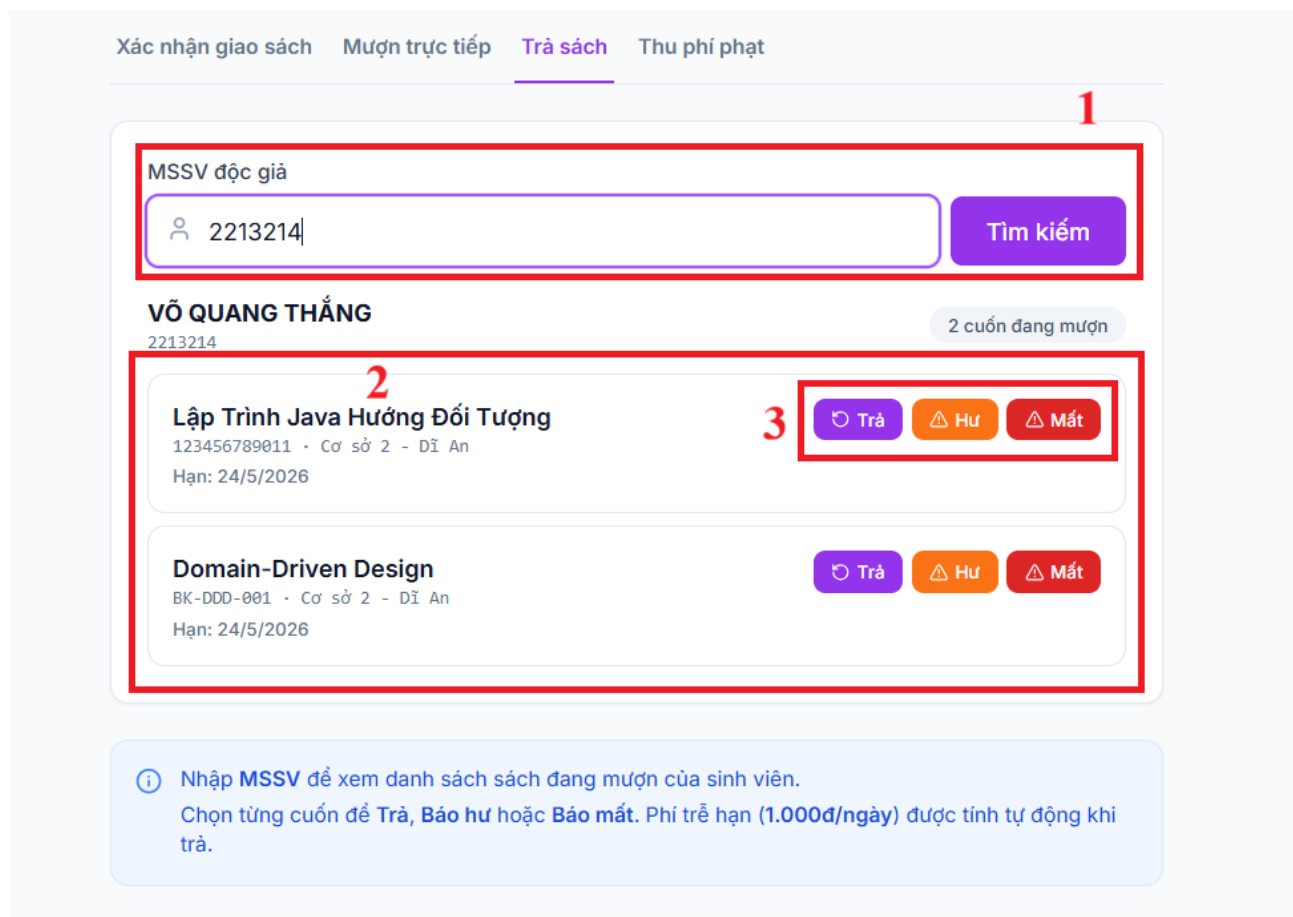
- **Mô tả:** Máy chủ Backend tạo ra một mã S3 Key duy nhất và gọi S3 SDK để cấp một đường dẫn PUT tạm thời có hiệu lực trong 15 phút.
- **Hành động tại Giao diện:** Hệ thống Frontend nhận đường dẫn này và thực hiện lệnh PUT trực tiếp lên S3, giúp tiết kiệm băng thông và tài nguyên CPU của máy chủ ứng dụng.

Giai đoạn 4: Hoàn tất liên kết tài liệu

- **Điểm cuối:** PUT /api/v1/publications/{id}/document-url
- **Mô tả:** Sau khi tệp đã nằm an toàn trên S3, giao diện gửi lại mã S3 Key để Backend lưu trữ vào cơ sở dữ liệu, chính thức hoàn tất việc liên kết tài liệu số với ấn phẩm.

6.2.21 Nghiệp vụ Trả sách và Xử lý sự cố

6.2.21.1 Giao diện Tra cứu và Trả sách



Xác nhận giao sách Mượn trực tiếp **Trả sách** Thu phí phạt

MSSV độc giả

2213214

Tìm kiếm

VÕ QUANG THẮNG
2213214

2 cuốn đang mượn

Lập Trình Java Hướng Đối Tượng
123456789011 · Cơ sở 2 - Dĩ An
Hạn: 24/5/2026

Domain-Driven Design
BK-DDD-001 · Cơ sở 2 - Dĩ An
Hạn: 24/5/2026

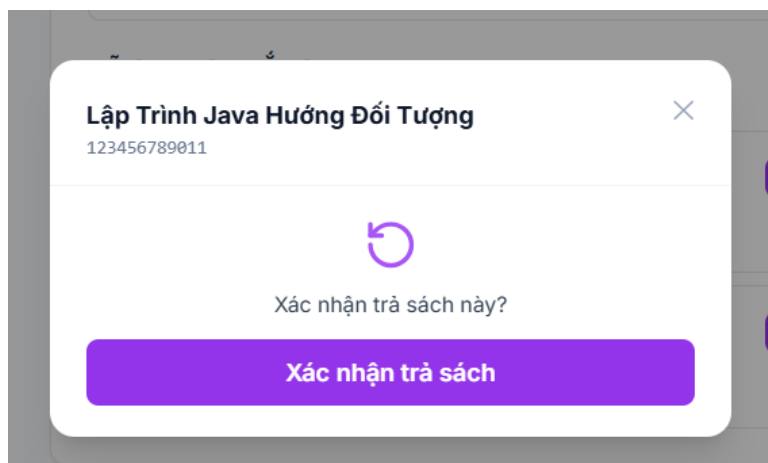
Trả Hư Mất

Trả Hư Mất

Thông tin: Nhập MSSV để xem danh sách sách đang mượn của sinh viên.
Chọn từng cuốn để Trả, Báo hư hoặc Báo mất. Phí trễ hạn (1.000đ/ngày) được tính tự động khi trả.

Hình 6.59: Giao diện nhập thông tin để tra cứu sách đang mượn

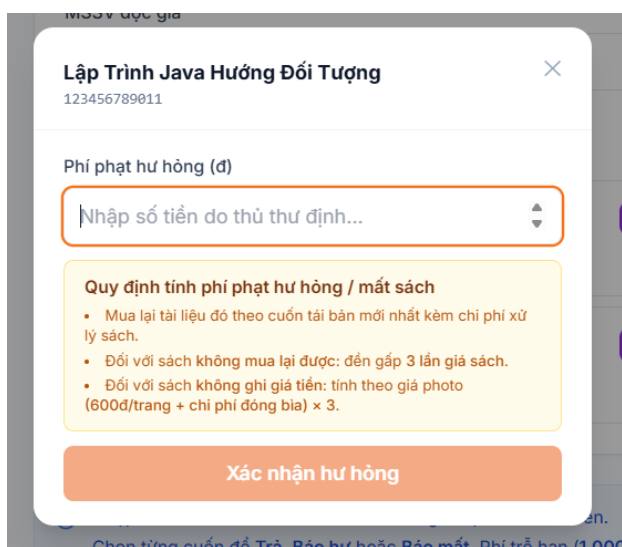
Chú thích giao diện: Tại phân hệ dành cho thủ thư, hệ thống cho phép tra cứu nhanh các giao dịch đang mượn thông qua Mã số sinh viên (MSSV). Danh sách trả về hiển thị rõ các ấn phẩm mà độc giả đang giữ, đi kèm với các nút thao tác tương ứng: "Trả" (hoàn trả bình thường), "Hư" (báo cáo hư hỏng) và "Mất" (báo cáo thất lạc).



Hình 6.60: Hộp thoại xác nhận hoàn trả tài liệu

Nghiệp vụ hoàn trả cơ bản: Khi thủ thư chọn "Trả", một hộp thoại xác nhận sẽ xuất hiện để đối chiếu thông tin sách và mã vạch lần cuối. Nếu giao dịch hoàn tất thành công, trạng thái của bản sao sẽ ngay lập tức được chuyển về "Có sẵn" (AVAILABLE). Trong trường hợp độc giả trả sách quá hạn, hệ thống sẽ tự động tính toán phí phạt dựa trên công thức số ngày trễ nhân với 1.000 VNĐ/ngày.

6.2.21.2 Xử lý sự cố Hư hỏng và Mất sách



Hình 6.61: Hộp thoại xử lý báo cáo hư hỏng sách



Hình 6.62: Hộp thoại xử lý báo cáo mất sách

Quy trình xử lý ngoại lệ: Khi xảy ra sự cố vật lý với bản sao, thủ thư sẽ sử dụng tính năng "Hư" hoặc "Mất". Hệ thống cung cấp một biểu mẫu để thủ thư nhập số tiền phạt cụ thể dựa trên quy định bồi thường của thư viện (ví dụ: đến gấp 3 lần giá sách đối với sách không mua lại được). Trạng thái bản sao sẽ lập tức bị khóa, chuyển thành Đang bảo trì (IN_MAINTENANCE) hoặc Thất lạc (LOST), đảm bảo không có giao dịch đặt trước nào được phép khởi tạo trên bản sao này.

6.2.21.3 Hệ thống thông báo và Xác nhận



Hình 6.63: Thư điện tử xác nhận giao dịch trả sách thành công

Mô tả luồng thông báo: Ngay sau khi thủ thư xác nhận trả sách trên hệ thống, một sự kiện (event) sẽ được kích hoạt thông qua Kafka. Độc giả sẽ lập tức nhận được một thư điện tử biên nhận, ghi rõ thời gian hoàn tất giao dịch để lưu trữ và đối soát.

6.2.21.4 Đặc tả API và Kỹ thuật điều phối

Phân hệ trả sách yêu cầu xử lý logic phức tạp để đảm bảo tính nhất quán dữ liệu, đặc biệt là cơ chế khóa chống tranh chấp (Pessimistic Lock).

1. Truy xuất thông tin giao dịch

- **Điểm cuối (Endpoint):** GET /api/v1/transactions/active
- **Mô tả:** Hệ thống tiến hành liên kết (JOIN) các bảng dữ liệu để tìm kiếm giao dịch đang ở trạng thái Đang mượn (BORROWING) hoặc Quá hạn (OVERDUE) tương ứng với mã vạch được cung cấp.

2. Hoàn trả tài liệu bình thường

- **Điểm cuối (Endpoint):** POST /api/v1/transactions/return
- **Cơ chế xử lý (Logic):** Máy chủ áp dụng khóa dữ liệu (Pessimistic Lock) trên bản sao để ngăn chặn các tiến trình đồng thời. Sau khi chuyển trạng thái giao dịch thành Đã trả (RETURNED), hệ thống sẽ gọi Dịch vụ Điều phối (ReservationAssignmentService) để tự động phân bổ quyền sách này cho độc giả tiếp theo đang xếp hàng đợi.

3. Báo cáo và xử lý sự cố

- **Điểm cuối (Endpoint):** POST /api/v1/transactions/{id}/report-issue
- **Mô tả:** API tiếp nhận yêu cầu cập nhật sự cố kèm theo mức phạt bồi thường do thủ thư quyết định. Nếu độc giả đồng thời trả trễ hạn và làm mất sách, hệ thống sẽ gộp cả hai khoản phạt: phí trễ hạn (OVERDUE_RETURN) và phí bồi thường (LOST_BOOK) thành danh sách nợ (UNPAID) trên hồ sơ sinh viên. Quy trình phân bổ sách tự động sẽ bị chặn lại hoàn toàn đối với ấn phẩm này.

4. Bảng tổng kết trạng thái

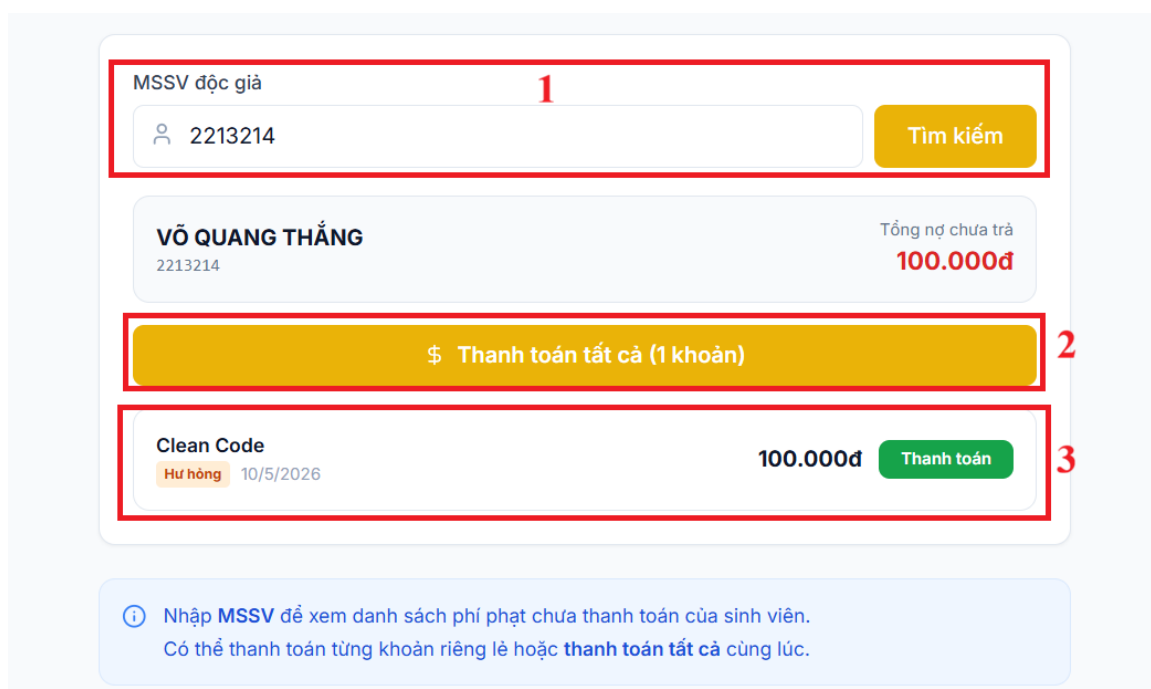
Bảng 6.13: Trạng thái bản sao và Phí phạt tương ứng sau khi trả sách

Trường hợp thực tế	Trạng thái bản sao	Loại phí phạt phát sinh
Trả sách đúng hạn	AVAILABLE	Không có
Trả sách trễ hạn	AVAILABLE	1 khoản: OVERDUE_RETURN
Báo hỏng (Đúng hạn)	IN_MAINTENANCE	1 khoản: DAMAGED_BOOK
Báo mất (Đúng hạn)	LOST	1 khoản: LOST_BOOK
Báo hỏng hoặc mất + Trễ hạn	IN_MAINTENANCE / LOST	2 khoản: Phí sự cố + OVERDUE_RETURN

6.2.22 Nghiệp vụ Thu phí phạt

Phân hệ này cung cấp cho thủ thư công cụ để tra cứu và thanh toán các khoản phí phát sinh từ vi phạm của độc giả. Việc hoàn tất thanh toán sẽ giải phóng các ràng buộc, cho phép sinh viên tiếp tục sử dụng toàn bộ dịch vụ của thư viện.

6.2.22.1 Giao diện Tra cứu và Thanh toán



Hình 6.64: Giao diện quản lý thu phí phạt tại quầy

Chú thích các vùng chức năng:

- **Vùng 1 (Khung tìm kiếm):** Thủ thư nhập Mã số sinh viên (MSSV) để hệ thống tự động truy xuất hồ sơ nợ đọng hiện tại của độc giả.
- **Vùng 2 (Thanh toán hàng loạt):** Nút thao tác nhanh cho phép thủ thư xác nhận thanh toán tất cả các khoản phạt cùng một lúc, tối ưu hóa thời gian xử lý khi sinh viên có nhu cầu đóng góp nhiều khoản.
- **Vùng 3 (Danh sách chi tiết):** Liệt kê minh bạch từng vi phạm (ví dụ: Hư hỏng ấn phẩm) kèm theo mức phạt tương ứng và nút thanh toán cho từng khoản riêng lẻ.

6.2.22.2 Đặc tả Giao tiếp API và Xử lý dữ liệu

Quy trình thu phí phạt được tích hợp chặt chẽ với hệ thống thông báo thời gian thực nhằm đảm bảo tính minh bạch về mặt tài chính cho sinh viên.

1. Truy xuất danh sách nợ phạt theo MSSV

- **Điểm cuối:** GET /api/v1/fines/student
- **Tham số bắt buộc:** studentId
- **Logic xử lý:** Máy chủ truy vấn thông tin người dùng. Nếu hợp lệ, hệ thống thực hiện lệnh liên kết dữ liệu (Native SQL JOIN) qua 4 bảng để lấy toàn bộ danh sách các vi phạm đang ở trạng thái chưa thanh toán (UNPAID).

2. Thanh toán khoản phạt đơn lẻ

- **Điểm cuối:** PUT /api/v1/fines/{id}/pay
- **Logic xử lý:** Hệ thống kiểm tra định danh khoản phạt. Nếu hợp lệ, trạng thái được cập nhật thành đã thanh toán (PAID) cùng mốc thời gian thực thi. Ngay sau đó, một thông điệp được đẩy vào nền tảng Kafka để kích hoạt dịch vụ gửi thư điện tử (Email) xác nhận biên lai cho sinh viên.

3. Thanh toán gộp (Bulk Update)

- **Điểm cuối:** PUT /api/v1/fines/pay-all
- **Tham số bắt buộc:** studentId
- **Logic xử lý tối ưu:** Thay vì lặp qua từng bản ghi gây hao tổn tài nguyên, hệ thống thực thi một câu lệnh UPDATE hàng loạt trực tiếp tại cơ sở dữ liệu. Đồng thời, hệ thống được thiết kế để chỉ kích hoạt một thông báo duy nhất tóm tắt việc thanh toán gộp, tránh tình trạng gửi thư rác (spam) làm phiền sinh viên.

6.2.22.3 Quản lý Phí phạt độc giả

Phân hệ quản lý phí phạt đóng vai trò quan trọng trong việc duy trì tính kỷ luật và bảo vệ tài sản của thư viện. Hệ thống tự động ghi nhận các khoản phí phát sinh từ vi phạm mượn trả và cung cấp giao diện minh bạch để độc giả theo dõi tình trạng tài chính cá nhân.

6.2.22.3.1 Giao diện Thống kê và Danh sách phí phạt

Phí phạt

Quản lý các khoản phí phạt của bạn

Chưa thanh toán

100.000đ

Đã thanh toán

0đ

Tổng cộng

100.000đ

Chưa thanh toán

2

Đã thanh toán

Clean Code

Hư hỏng 10/05/2026

100.000đ

Chưa thanh toán

Hướng dẫn thanh toán

- Thanh toán trực tiếp:** Đến quầy thủ thư tại thư viện trong giờ hành chính.
- Lưu ý:** Phí phạt chưa thanh toán sẽ khóa quyền mượn sách mới.

Thông báo phí phạt

Sách "Clean Code" đã được ghi nhận hư hỏng. Vui lòng đến thư viện thanh toán phí phạt.

Hình 6.65: Giao diện Thống kê tổng quát và danh sách các khoản phí chưa thanh toán

Chú thích các vùng chức năng (Hình 6.65):

- Vùng 1 (Thẻ thống kê tổng hợp):** Hiển thị tóm tắt tình hình phí phạt thông qua các chỉ số: Tổng tiền chưa thanh toán (cảnh báo đỏ), Tổng tiền đã thanh toán (xanh lá) và Tổng cộng tất cả các khoản phí đã từng phát sinh.
- Vùng 2 (Bộ lọc trạng thái):** Độc giả có thể chuyển đổi linh hoạt giữa các tab "Chưa thanh toán" và "Đã thanh toán". Khi thực hiện chuyển tab, hệ thống tự động thiết lập lại phân trang và truy vấn dữ liệu tương ứng.
- Vùng 3 (Bảng danh sách chi tiết):** Mỗi dòng thông tin bao gồm tên ấn phẩm liên quan, nhãn loại vi phạm (Trễ hạn, Hư hỏng hoặc Mất sách), số tiền và ngày phát sinh.

Chưa thanh toán

Đã thanh toán

✓

Chưa có lịch sử thanh toán.

Hình 6.66: Giao diện hiển thị lịch sử các khoản phí đã hoàn tất thanh toán

Mô tả nghiệp vụ: Khi độc giả hoàn tất nộp phạt tại quầy, thủ thư cập nhật trạng thái trên hệ thống quản trị. Ngay lập tức, khoản phí sẽ được chuyển sang tab "Đã thanh toán" (Hình 6.66) cùng với ghi nhận ngày thanh toán thực tế để độc giả đối soát.

6.2.22.3.2 Đặc tả kỹ thuật và Luồng xử lý

Hệ thống quản lý phí phạt được tích hợp chặt chẽ với mô-đun lưu thông tài liệu để đảm bảo mọi vi phạm đều

được ghi nhận tức thời.

1. Cơ chế khởi tạo phí phạt

- **Tự động (Batch Job):** Hệ thống định kỳ quét các giao dịch quá hạn vào lúc 00:00 mỗi ngày để cập nhật phí trễ hạn (mặc định 1.000 VNĐ/ngày).
- **Thủ công (Librarian Action):** Khi thủ thư tiếp nhận sách trả bị hư hỏng hoặc mất, các khoản phí bồi thường sẽ được khởi tạo trực tiếp từ giao diện quản trị và ánh xạ vào hồ sơ của độc giả.

2. Đặc tả giao tiếp API

- **Điểm cuối (Endpoint):** GET /api/v1/fines/my-fines
- **Tham số truy vấn:** status (UNPAID/PAID), page, size.
- **Logic xử lý Backend:** Hệ thống thực hiện liên kết (JOIN) giữa các bảng fines, transactions, items và publications để lấy đầy đủ thông tin tên sách và loại vi phạm dựa trên danh tính người dùng trích xuất từ mã định danh (JWT).

3. Tối ưu hóa trải nghiệm người dùng (Frontend)

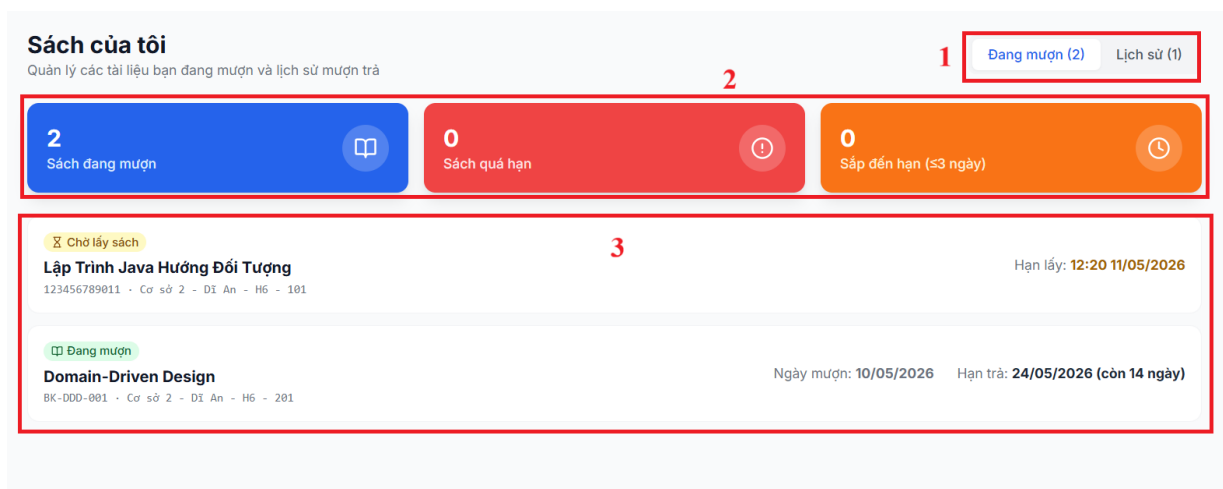
- **Tính toán tổng quan:** Ngay khi truy cập trang, hệ thống thực hiện gọi đồng thời hai yêu cầu truy vấn (cho cả trạng thái chưa trả và đã trả) để tính toán tổng số tiền hiển thị trên các thẻ thống kê phía trên, đảm bảo số liệu luôn đồng nhất.
- **Ràng buộc nghiệp vụ:** Một ràng buộc quan trọng được thiết lập tại tầng dịch vụ: Nếu người dùng có bất kỳ khoản phạt nào ở trạng thái chưa thanh toán (UNPAID), hệ thống sẽ tự động chặn mọi yêu cầu mượn sách hoặc đặt trước sách mới để đảm bảo nghĩa vụ bồi hoàn.

6.2.23 Quản lý và Xem danh sách mượn trả

Hệ thống cung cấp giao diện theo dõi hoạt động mượn trả chuyên biệt cho cả độc giả và thủ thư, giúp minh bạch hóa lộ trình lưu thông tài liệu và tình trạng của các giao dịch.

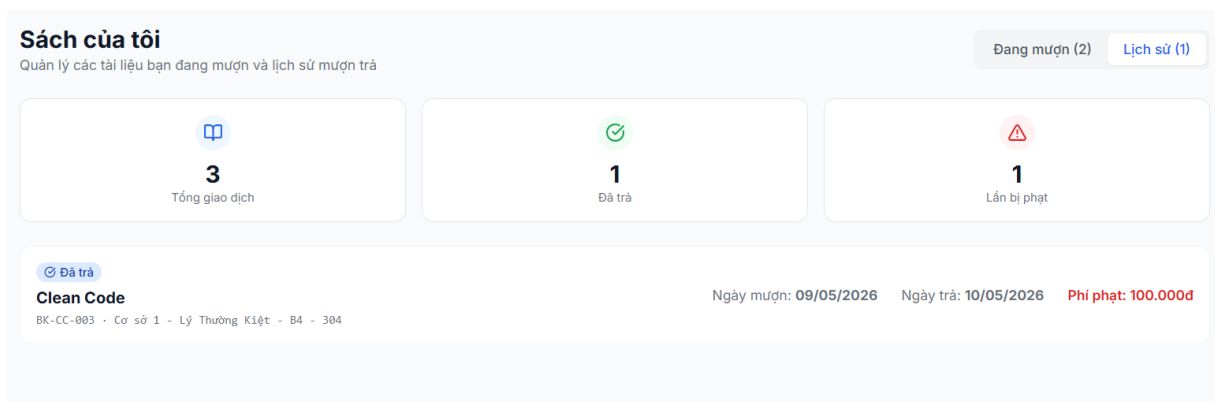
6.2.23.1 Giao diện dành cho Độc giả

Giao diện của độc giả được chia làm hai phân hệ chính thông qua hệ thống thẻ (tabs) để tối ưu hóa việc quản lý các giao dịch hiện thời và tra cứu quá khứ.



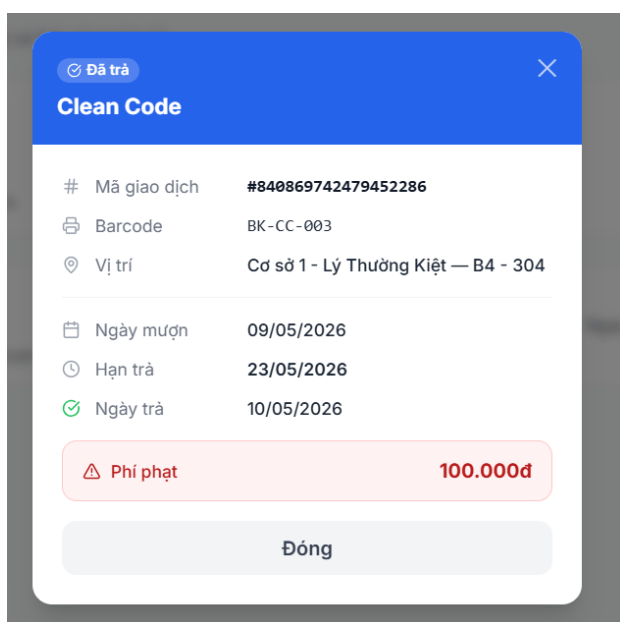
Hình 6.67: Giao diện danh sách sách đang mượn và chờ lấy

Tab Đang mượn (Hình 6.67): Tập trung hiển thị các giao dịch có trạng thái WAITING_FOR_PICKUP (Chờ lấy sách), BORROWING (Đang mượn) và OVERDUE (Quá hạn). Hệ thống cung cấp các thẻ thống kê tóm tắt (Summary Cards) giúp độc giả nắm bắt nhanh số lượng sách thực tế đang giữ, số sách đã quá hạn và số sách sắp đến hạn trong vòng 3 ngày tới.

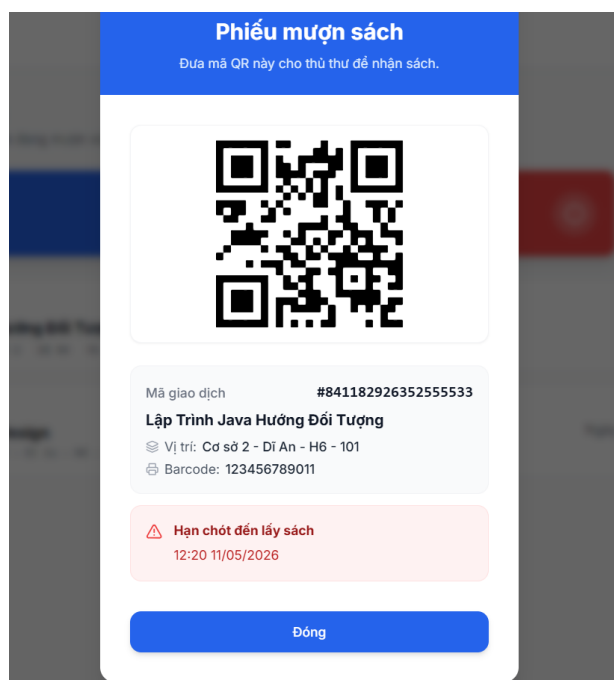


Hình 6.68: Giao diện lịch sử mượn trả tài liệu

Tab Lịch sử (Hình 6.68): Lưu trữ toàn bộ các giao dịch đã hoàn tất với trạng thái RETURNED (Đã trả) hoặc CANCELLED (Đã hủy). Phần thống kê tại đây tập trung vào tổng số lượt giao dịch, số lần trả sách thành công và tổng kết số lần phát sinh phí phạt (nếu có).



Hình 6.69: Chi tiết trạng thái giao dịch



Hình 6.70: Thông tin biên nhận mượn sách

Xem chi tiết giao dịch: Khi nhấn vào từng mục, độc giả có thể xem thông tin chi tiết về bản sao (barcode, vị trí), mốc thời gian mượn/trả thực tế và lịch trình thay đổi trạng thái của phiếu mượn (Hình 6.69, 6.70).

6.2.23.2 Giao diện dành cho Thủ thư

Lịch sử giao dịch
Tất cả giao dịch mượn sách trong hệ thống

MÃ GD	SINH VIÊN	MSSV	NGÀY MƯỢN ↑↓	HẠN TRẢ ↑↓	NGÀY TRẢ ↑↓	PHÍ PHẠT	TRẠNG THÁI
841182926352555533	VÕ QUANG THẮNG	2213214	10/05/2026	24/05/2026	10/05/2026	—	Đã trả
841155302033071579	VÕ QUANG THẮNG	2213214	10/05/2026	24/05/2026	—	—	Đang mượn
840869742479452286	VÕ QUANG THẮNG	2213214	09/05/2026	23/05/2026	10/05/2026	100.000đ	Đã trả
9	Nguyễn Thị Học Viên	2152345	24/04/2026	08/05/2026	09/05/2026	40.000đ	Đã trả
10	Phạm Văn Sinh Viên	2151234	25/04/2026	09/05/2026	—	50.000đ	Quá hạn
11	Nguyễn Thị Học Viên	2152345	25/04/2026	09/05/2026	—	50.000đ	Quá hạn
12	Phạm Văn Sinh Viên	2151234	21/04/2026	05/05/2026	—	80.000đ	Quá hạn
13	Nguyễn Thị Học Viên	2152345	14/04/2026	28/04/2026	—	150.000đ	Quá hạn

Hình 6.71: Giao diện quản lý danh sách mượn trả tổng thể dành cho Thủ thư

Thủ thư được cung cấp một bảng quản trị tập trung để theo dõi mọi biến động mượn trả trong thư viện. Bảng dữ liệu hỗ trợ hiển thị đầy đủ thông tin định danh độc giả (MSSV, Họ tên), tên ấn phẩm, thời hạn trả và mức phí phạt hiện thời của từng giao dịch (Hình 6.71).

6.2.23.3 Đặc tả API và Logic nghiệp vụ

1. Truy xuất danh sách mượn trả tổng quát

- **Điểm cuối (Endpoint):** GET /api/v1/transactions (Quyền: Thủ thư).
- **Cơ chế truy vấn:** Sử dụng JPQL để liên kết (LEFT JOIN) giữa bảng giao dịch (BorrowingTransaction), người dùng (User) và phí phạt (Fine) nhằm tổng hợp thông tin phản hồi đầy đủ cho giao diện danh sách. Kết quả luôn được sắp xếp theo thời gian tạo mới nhất (createdAt DESC).

2. Truy xuất lịch sử theo bản sao cụ thể

- **Điểm cuối (Endpoint):** GET /api/v1/transactions/items/{id}.
- **Mô tả:** Phục vụ tính năng xem lịch sử mượn trả của duy nhất một quyền sách vật lý, giúp thủ thư đánh giá tần suất sử dụng và tình trạng của bản sao đó qua thời gian.

3. Hệ thống Nhân trạng thái (Status Badges) Hệ thống sử dụng bảng mã màu chuẩn để biểu thị trạng thái giao dịch một cách trực quan:

Bảng 6.15: Quy ước hiển thị trạng thái mượn trả

Trạng thái	Màu sắc	Ý nghĩa nghiệp vụ
WAITING_FOR_PICKUP	Vàng	Sách đang chờ độc giả đến nhận.
BORROWING	Xanh lá	Độc giả đang giữ sách trong thời hạn cho phép.
OVERDUE	Đỏ	Giao dịch đã vượt quá thời hạn hoàn trả.
RETURNED	Xanh dương	Sách đã được trả về kho thành công.
CANCELLED	Xám	Yêu cầu mượn đã bị hủy bỏ.

Chương 7: Kiểm thử

7.1 Kiểm thử phần mềm và chiến lược kiểm thử

Kiểm thử phần mềm là quá trình đánh giá và xác minh có hệ thống rằng một ứng dụng phần mềm thực hiện đúng các chức năng đã định, đáp ứng các yêu cầu kỹ thuật và không có các lỗi nghiêm trọng [11]. Ngoài việc phát hiện lỗi, kiểm thử phần mềm hiện đại còn đóng vai trò là cơ chế đảm bảo chất lượng, xác nhận độ tin cậy, tính bảo mật và hiệu năng của hệ thống trong nhiều điều kiện khác nhau.

Để đảm bảo tính ổn định của Hệ thống quản lý thư viện, dự án áp dụng phương pháp kiểm thử phân tầng. Việc kiểm thử được thực hiện qua nhiều tầng—from các thành phần riêng lẻ đến các luồng tích hợp hoàn chỉnh—đảm bảo rằng mỗi tầng của kiến trúc được xác minh trước khi tiến đến tầng tiếp theo [12]. Các phần tiếp theo sẽ trình bày chi tiết các chiến lược kiểm thử cụ thể được áp dụng trong dự án.

7.1.1 Kiểm thử đơn vị

Kiểm thử đơn vị (Unit Testing) là tầng kiểm thử đầu tiên, nơi các phần nhỏ nhất có thể kiểm thử được của ứng dụng—gọi là “đơn vị”—là các thành phần hoặc hàm riêng lẻ được cô lập và xác minh tính đúng đắn [13]. Bằng cách phát hiện lỗi ngay ở tầng này, nhóm phát triển đảm bảo rằng logic nghiệp vụ cốt lõi của ứng dụng luôn ổn định trong suốt quá trình phát triển và tái cấu trúc. Các bài kiểm thử đơn vị được thiết kế để chạy nhanh và độc lập, với tất cả các phụ thuộc bên ngoài như cơ sở dữ liệu, message broker và API ngoài được thay thế bằng đối tượng giả lập (mock).

7.1.2 Kiểm thử tích hợp

Sau khi các đơn vị riêng lẻ được xác minh, kiểm thử tích hợp (Integration Testing) được thực hiện để đánh giá sự tương tác và luồng dữ liệu giữa các module phần mềm khác nhau [14]. Tầng kiểm thử này tập trung vào giao diện giữa các thành phần, xác minh rằng các module đã tách rời của hệ thống hoạt động cùng nhau như một đơn vị thống nhất. Không giống kiểm thử đơn vị, kiểm thử tích hợp sử dụng hạ tầng thực như cơ sở dữ liệu để xác nhận toàn bộ luồng dữ liệu hoạt động đúng từ đầu đến cuối.

7.1.3 Kiểm thử API

Kiểm thử API xác nhận hợp đồng giao tiếp giữa frontend và backend bằng cách gửi các yêu cầu HTTP đến các endpoint REST và kiểm tra mã trạng thái phản hồi, header và payload JSON [15]. Hình thức kiểm thử này nằm giữa kiểm thử tích hợp và kiểm thử hệ thống, xác nhận rằng máy chủ xử lý đúng các tình huống khác nhau—bao gồm yêu cầu hợp lệ, lỗi xác thực và vi phạm quy tắc nghiệp vụ—mà không cần giao diện người dùng đầy đủ.

7.1.4 Kiểm thử hiệu năng

Kiểm thử hiệu năng (Performance Testing) đánh giá khả năng phản hồi, tính ổn định và khả năng mở rộng của hệ thống dưới các tải trọng khác nhau [16]. Nó được sử dụng để xác định các điểm nghẽn trong pipeline xử lý và đảm bảo rằng các yêu cầu từ nhiều người dùng đồng thời không làm suy giảm trải nghiệm tổng thể. Các chỉ số chính được thu thập bao gồm thời gian phản hồi trung bình, phân vị thứ 95 (p95), số lượng yêu cầu mỗi giây (RPS) và tỷ lệ lỗi dưới tải.

7.2 Kiểm thử đơn vị

7.2.1 Framework kiểm thử Backend (JUnit 5 + Mockito)

Đối với kiến trúc backend xây dựng trên Java 21 và Spring Boot 3.3.5, bộ kiểm thử sử dụng JUnit 5 kết hợp với framework giả lập Mockito.

7.2.1.1 JUnit 5

JUnit 5 là framework kiểm thử tiêu chuẩn cho các ứng dụng Java. Framework cung cấp tập hợp phong phú các annotation—như `@Test`, `@BeforeEach` và `@DisplayName`—để cấu trúc việc thực thi kiểm thử, đồng thời tích hợp liền mạch với vòng đời build Maven thông qua `maven-surefire-plugin` [17].

7.2.1.2 Mockito

Mockito là framework giả lập (mocking) cho Java, cho phép tạo và cấu hình các đối tượng mock, thay thế phụ thuộc thực bằng các thay thế có kiểm soát. Trong dự án này, Mockito được sử dụng để giả lập `ItemStatusPort`, `KafkaTemplate` và các interface repository, đảm bảo mỗi use case có thể được kiểm thử hoàn toàn độc lập [18].

7.2.2 Framework kiểm thử Frontend (Jest)

Đối với kiến trúc frontend xây dựng trên React 19 và TypeScript, bộ kiểm thử sử dụng Jest kết hợp với bộ chuyển đổi `ts-jest`.

7.2.2.1 Jest

Jest là framework kiểm thử JavaScript tập trung vào sự đơn giản [19]. Nó đóng vai trò là test runner, thư viện assertion và framework giả lập cốt lõi. Jest được tối ưu cho môi trường Node.js và thực thi kiểm thử nhanh chóng bằng cách sử dụng `jsdom` để mô phỏng môi trường trình duyệt.

7.2.2.2 ts-jest

Do dự án được viết bằng TypeScript, `ts-jest` được sử dụng như một bộ tiền xử lý biên dịch các file TypeScript trước khi Jest thực thi. Cấu hình được khai báo trong `jest.config.cjs` (định dạng CommonJS, bắt buộc vì `package.json` của dự án khai báo `"type": "module"`).

7.2.3 Thiết kế test case

Các test case trên cả frontend và backend đều tuân thủ nghiêm ngặt mẫu Arrange-Act-Assert (AAA) để đảm bảo khả năng đọc và bảo trì:

- **Arrange (Chuẩn bị):** Thiết lập trạng thái ban đầu, khởi tạo các đối tượng mock và cấu hình giá trị trả về bằng `when(...).thenReturn(...)`.
- **Act (Thực thi):** Gọi phương thức hoặc hàm cụ thể đang được kiểm thử.
- **Assert (Kiểm tra):** Xác minh rằng kết quả thực tế khớp với kết quả mong đợi bằng `AssertJ` (backend) hoặc `Jest expect` (frontend).

7.2.3.1 Trọng tâm kiểm thử Backend

Chiến lược kiểm thử backend nhắm vào ba use case nghiệp vụ cốt lõi của module circulation. Tất cả các phụ thuộc bên ngoài—bao gồm cơ sở dữ liệu thông qua `ItemStatusPort`, Kafka message broker thông qua

KafkaTemplate và JPA repository—được tách biệt hoàn toàn bằng Mockito để đảm bảo thực thi xác định và độc lập với mạng. Mỗi use case được kiểm thử với ít nhất một kịch bản happy-path và một kịch bản error-path xác nhận việc ném ApplicationException đúng lỗi.

7.2.3.2 Trọng tâm kiểm thử Frontend

Chiến lược kiểm thử frontend tập trung vào các hàm tiện ích thuần túy (pure functions) đóng gói logic nghiệp vụ của trang đặt trước, cụ thể là module `reservationUtils.ts`. Việc kiểm thử các pure function—những hàm không có tác dụng phụ hay gọi API bên ngoài—đảm bảo cô lập tối đa và thực thi nhanh.

7.2.4 Danh sách test case

Bảng 7.1 tổng hợp các test case kiểm thử đơn vị backend được triển khai trong module `library-circulation-module`.

Bảng 7.1: Danh sách test case kiểm thử đơn vị Backend

Class	Tên test	Kịch bản	Kết quả mong đợi
BorrowRequest	borrowSuccess	Item AVAILABLE, không có fine	WAITING_FOR_PICKUP
	itemNotAvailable	Item đang ở trạng thái BORROWED	AppException
	unpaidFines	User có khoản phạt chưa thanh toán	AppException
	duplicateBorrow	Cùng ấn phẩm đang được mượn	AppException
ReturnBook	returnSuccess_noFine	Trả trước hạn	RETURNED, không tạo fine
	returnSuccess_withFine	Trả trễ 3 ngày	Fine = 3.000 VND
	transactionNotFound	Không có giao dịch đang mượn	AppException
CreateReservation	reservationSuccess	Item không có sẵn, không có fine	PENDING
	unpaidFines	User có khoản phạt chưa thanh toán	AppException
	limitExceeded	Đã có 2 đặt trước đang hoạt động	AppException
	itemAvailable	Item đang AVAILABLE tại cơ sở	AppException
	duplicateReservation	Đã đặt trước ấn phẩm này	AppException

Bảng 7.2 tổng hợp các test case kiểm thử đơn vị frontend được triển khai trong `reservationUtils.test.ts`.

7.2.5 Kết quả

Bộ kiểm thử đơn vị tự động cho cả frontend và backend đã được thực thi thành công, thể hiện tính ổn định cao của logic nghiệp vụ cốt lõi trong hệ thống.

7.2.5.1 Kết quả Backend

Kiểm thử đơn vị backend xác nhận ba use case circulation chính trong môi trường hoàn toàn cô lập. Tất cả phụ thuộc bên ngoài được giả lập hoàn toàn bằng Mockito để đảm bảo thực thi xác định.

- Tổng số class kiểm thử: 3
- Tổng số test case: 12
- Passed: 12

Bảng 7.2: Danh sách test case kiểm thử đơn vị Frontend

Nhóm	Tên test	Kết quả
STATUS_CONFIG	Hiển thị đúng nhãn cho trạng thái PENDING	Pass
	Hiển thị đúng màu cho trạng thái READY_FOR_PICKUP	Pass
	Hiển thị đúng nhãn cho trạng thái BORROWING	Pass
	Hiển thị đúng nhãn cho trạng thái COMPLETED	Pass
	Hiển thị đúng nhãn cho trạng thái CANCELLED	Pass
	Hiển thị đúng nhãn cho trạng thái OVERDUE	Pass
fmtDate	Định dạng đúng chuỗi ngày ISO hợp lệ	Pass
	Trả về gạch ngang khi đầu vào là null	Pass
	Trả về gạch ngang khi đầu vào là undefined	Pass
isActiveCancellable	Trả về true cho trạng thái PENDING	Pass
	Trả về true cho trạng thái READY_FOR_PICKUP	Pass
	Trả về false cho trạng thái COMPLETED	Pass
	Trả về false cho trạng thái CANCELLED	Pass
	Trả về false cho trạng thái EXPIRED	Pass
calcHoldDeadlineLabel	Trả về số giờ còn lại khi còn hơn 1 giờ	Pass
	Trả về “sắp hết hạn” khi còn dưới 1 giờ	Pass
	Trả về “đã hết hạn” khi quá thời hạn	Pass
	Trả về gạch ngang khi deadline là null	Pass

- Failed: 0
- Thời gian thực thi: $\approx$ 4 giây

```
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 12, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 12.282 s
[INFO] Finished at: 2026-05-10T23:43:41+07:00
[INFO] -----
```

Hình 7.1: Kết quả kiểm thử đơn vị Backend (mvn test -pl library-circulation-module)

7.2.5.2 Kết quả Frontend

Kiểm thử đơn vị frontend xác nhận các hàm tiện ích đặt trước đóng gói logic nghiệp vụ UI quan trọng.

- Tổng số test suite: 1
- Tổng số test case: 18
- Passed: 18

- Failed: 0
- Thời gian thực thi: ≈ 2.3 giây

```
PASS src/utils/reservationUtils.test.ts
STATUS_CONFIG
  ✓ PENDING có nhãn "Đang chờ" và màu vàng (5 ms)
  ✓ READY_FOR_PICKUP có nhãn "Sẵn sàng nhận" và màu xanh (3 ms)
  ✓ CANCELLED có nhãn "Đã hủy" và màu xám (2 ms)
  ✓ COMPLETED có nhãn "Hoàn thành" và màu xanh dương (2 ms)
  ✓ EXPIRED có nhãn "Hết hạn" và màu đỏ (3 ms)
fmtDate
  ✓ trả về "-" khi input là null (2 ms)
  ✓ trả về "-" khi input là chuỗi không hợp lệ (3 ms)
  ✓ format ISO string thành dạng HH:mm dd/MM/yyyy (5 ms)
  ✓ kết quả chứa năm 2026 (1 ms)
isActiveCancellable
  ✓ PENDING có thể hủy (1 ms)
  ✓ READY_FOR_PICKUP có thể hủy (1 ms)
  ✓ COMPLETED không thể hủy (1 ms)
  ✓ CANCELLED không thể hủy (1 ms)
  ✓ EXPIRED không thể hủy (1 ms)
calcHoldDeadlineLabel
  ✓ trả về chuỗi rỗng khi holdExpirationTime là null (1 ms)
  ✓ trả về "Đã hết hạn" khi thời gian đã qua (3 ms)
  ✓ trả về "Còn X giờ" khi còn dưới 24 giờ (2 ms)
  ✓ trả về "Còn X ngày" khi còn trên 24 giờ (1 ms)

Test Suites: 1 passed, 1 total
Tests:       18 passed, 18 total
Snapshots:   0 total
Time:        4.014 s
Ran all test suites.
```

Hình 7.2: Kết quả kiểm thử đơn vị Frontend (npm test)

7.3 Kiểm thử tích hợp

7.3.1 Chiến lược kiểm thử

Chiến lược kiểm thử tích hợp cho backend tập trung vào việc xác nhận toàn bộ vòng đời mượn-trả sách qua tất cả các tầng hạ tầng thực—bao gồm cơ sở dữ liệu PostgreSQL, tầng persistence JPA và các use case domain—trong khi cô lập các dịch vụ bên thứ ba không liên quan đến luồng nghiệp vụ đang kiểm thử.

7.3.1.1 SpringBootTest và Testcontainers

Class kiểm thử được đánh dấu với `@SpringBootTest(webEnvironment = NONE)`, khởi động đầy đủ application context của Spring mà không khởi chạy HTTP server. Điều này cho phép inject trực tiếp các bean use case và JPA repository trong khi vẫn giữ tốc độ thực thi. Một instance PostgreSQL 15 thực sự được cung cấp theo yêu cầu bằng Testcontainers [20]—một thư viện Java quản lý container Docker trong vòng đời JUnit. URL kết nối cơ sở dữ liệu được inject động qua `@DynamicPropertySource`, ghi đè cấu hình datasource production cho profile kiểm thử.

7.3.1.2 Cô lập các dịch vụ bên ngoài

Các dịch vụ không thuộc phạm vi kiểm thử được thay thế bằng các instance `@MockBean` để ngăn kết nối mạng và tác dụng phụ ngoài ý muốn:

- `KafkaTemplate` và `KafkaAdmin` — ngăn kết nối đến Kafka broker.

- S3Client và S3Presigner — ngăn AWS SDK xác thực thông tin đăng nhập.
- ClientRegistrationRepository — ngăn tải cấu hình OAuth2 provider.
- Bốn bean scheduler — ngăn các background job can thiệp vào kết quả kiểm thử.

7.3.1.3 Cơ lập kiểm thử qua @BeforeEach

Để đảm bảo mỗi phương thức kiểm thử bắt đầu từ trạng thái sạch đã biết trước, một phương thức setup @BeforeEach thực hiện các thao tác SQL sau trên cơ sở dữ liệu kiểm thử:

1. Xóa tất cả khoản phạt liên quan đến người dùng kiểm thử (ràng buộc khóa ngoại yêu cầu thứ tự này).
2. Xóa tất cả giao dịch mượn sách không ở trạng thái RETURNED của người dùng kiểm thử để đặt lại số lượng mượn đang hoạt động.
3. Xóa tất cả đặt trước PENDING cho ấn phẩm kiểm thử để ngăn dịch vụ gán sách tự động sau thao tác trả sách.
4. Đặt lại trạng thái item kiểm thử về AVAILABLE.

7.3.2 Thiết kế test case

Bộ kiểm thử tích hợp bao gồm vòng đời mượn-trả hoàn chỉnh của Hệ thống quản lý thư viện, được triển khai trong class BorrowFlowIntegrationTest. Dữ liệu kiểm thử được seed bởi migration Flyway V2__seed_data.sql, khởi tạo người dùng, ấn phẩm, items và các giao dịch mẫu trước khi application context được tải.

Bảng 7.3: Danh sách test case kiểm thử tích hợp — Luồng mượn sách

Thứ tự	Tên test	Kịch bản	Kết quả mong đợi
T1	borrow_should CreateTransaction AndReserveItem	Người dùng đặt mượn một item đang AVAILABLE, không có khoản phạt	Transaction = WAITING_FOR_PICKUP; Item = RESERVED
T2	return_should MarkTransaction ReturnedAnd- FreeItem	Thủ thư trả sách đang BORROWING theo barcode trước hạn	Transaction = RETURNED; Item = AVAILABLE
T3	borrow_shouldFail_ whenUserHasUn- paid FineInDB	Người dùng cố mượn khi có khoản phạt UNPAID trong DB	Ném AppException; không tạo giao dịch mới

7.3.3 Kết quả

Bộ kiểm thử tích hợp đã thực thi thành công, xác nhận rằng use case mượn sách, use case trả sách và quy tắc nghiệp vụ chặn mượn khi có phạt tương tác đúng đắn với cơ sở dữ liệu PostgreSQL thực.

- Tổng số test case: 3
- Passed: 3
- Failed: 0
- Thời gian thực thi: ≈37 giây (bao gồm thời gian khởi động Testcontainers)

Surefire Report

Summary

[\[Summary\]](#) [\[Package List\]](#) [\[Test Cases\]](#)

Tests	Errors	Failures	Skipped	Success Rate	Time
3	0	0	0	100%	60.42 s

Note: failures are anticipated and checked for with assertions while errors are unanticipated.

Package List

[\[Summary\]](#) [\[Package List\]](#) [\[Test Cases\]](#)

Package	Tests	Errors	Failures	Skipped	Success Rate	Time
com.library.integration	3	0	0	0	100%	60.42 s

Hình 7.3: Kết quả kiểm thử tích hợp — Surefire HTML Report (`library-bootstrap/target/reports/surefire.html`)

7.4 Kiểm thử API

7.4.1 Giới thiệu công cụ

Postman là nền tảng kiểm thử API phổ biến cung cấp giao diện đồ họa trực quan để soạn thảo, gửi và kiểm tra các yêu cầu HTTP [21]. Postman cho phép tổ chức các request thành Collection có cấu trúc, định nghĩa biến môi trường dùng chung và viết script kiểm tra JavaScript tự động sau mỗi request. Tính năng **Collection Runner** của Postman cho phép thực thi toàn bộ collection theo thứ tự, tự động truyền giá trị giữa các request thông qua biến môi trường, và hiển thị báo cáo tổng hợp kết quả passed/failed ngay trong giao diện.

7.4.2 Bộ test collection

Bộ test collection được tổ chức thành tám thư mục, mỗi thư mục đại diện cho một luồng API riêng biệt của Hệ thống quản lý thư viện. Bảng 7.4 trình bày phạm vi kiểm thử của từng thư mục.

Bảng 7.4: Tổng quan Postman Collection

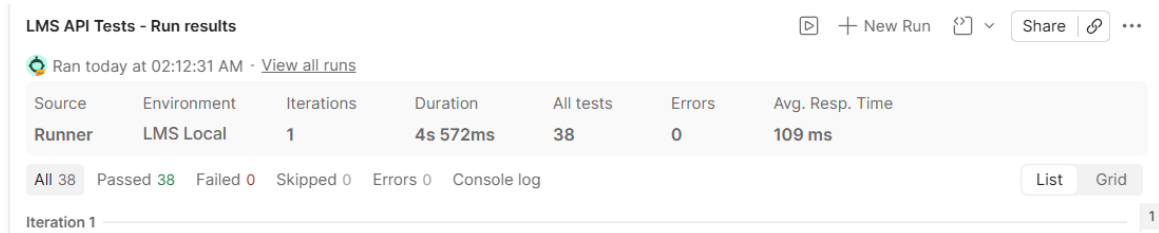
Thư mục	Số request	Nội dung kiểm tra
00 - Xác thực	2	Đăng nhập student + librarian, token lưu vào biến môi trường
01 - Ấn phẩm (công khai)	4	Tìm kiếm, sách mới nhất, chi tiết, danh sách item AVAILABLE
02 - Luồng mượn sách	4	Đặt mượn (201), tra cứu (200), xác nhận lấy sách (200), lịch sử
03 - Luồng trả sách	2	Tra cứu giao dịch đang mượn theo barcode, trả sách
04 - Luồng đặt trước	3	Tạo đặt trước (201), danh sách (200), hủy (200)
05 - Quản lý phạt	1	Xem danh sách phạt của sinh viên hiện tại
06 - Dashboard	2	Tổng hợp dashboard, biểu đồ theo tuần
07 - Trường hợp lỗi	2	Mượn không có token (401), đăng nhập sai mật khẩu (4xx)
Tổng	20	

Mỗi request bao gồm một script kiểm tra JavaScript trong tab Tests để xác nhận mã trạng thái HTTP và kiểm tra các trường quan trọng trong body JSON phản hồi. Các biến môi trường (`studentToken`,

`librarianToken, transactionId, reservationId, itemId)` được tự động truyền giữa các request thông qua `pm.environment.set()`, cho phép collection thực thi như một luồng tuần tự có trạng thái mà không cần can thiệp thủ công.

7.4.3 Kết quả

Postman Collection được thực thi trực tiếp trong giao diện Postman thông qua tính năng **Collection Runner** với máy chủ backend đang chạy cục bộ tại `http://localhost:8080`. Tất cả 20 request được thực thi theo thứ tự, kết quả kiểm tra hiển thị trực tiếp trong giao diện Runner.



Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	LMS Local	1	4s 572ms	38	0	109 ms

All 38 Passed 38 Failed 0 Skipped 0 Errors 0 Console log

Iteration 1

Hình 7.4: Kết quả kiểm thử API — Postman Collection Runner

7.5 Kiểm thử hiệu năng

7.5.1 Giới thiệu công cụ

k6 là công cụ kiểm thử hiệu năng mã nguồn mở được viết bằng Go, được thiết kế để lập trình các kịch bản kiểm thử tải bằng JavaScript [22]. Công cụ hỗ trợ định nghĩa các người dùng ảo (Virtual Users - VU) thực thi các kịch bản kiểm thử đồng thời, phù hợp để đánh giá hành vi của REST API dưới tải đồng thời thực tế. k6 tự động báo cáo các chỉ số hiệu năng quan trọng bao gồm phân vị thời gian phản hồi, thông lượng (RPS) và tỷ lệ lỗi.

7.5.2 Cấu hình kiểm thử

Kiểm thử hiệu năng nhắm vào ba endpoint công khai được truy cập thường xuyên nhất của hệ thống: API tìm kiếm ấn phẩm, API lấy danh sách sách mới nhất và API sách được mượn nhiều nhất. Các endpoint này được kỳ vọng chịu tải đọc cao nhất trong môi trường thư viện thực tế, vì chúng không yêu cầu xác thực và được gọi mỗi khi người dùng tải trang chủ hoặc trang tìm kiếm công khai.

Kiểm thử được cấu hình theo mô hình ramp-up với ba giai đoạn:

- **Warm-up (10 giây):** Tăng dần từ 0 lên 5 VU để làm nóng hệ thống.
- **Load (20 giây):** Duy trì 10 VU đồng thời — tải chính của bài kiểm thử.
- **Cool-down (10 giây):** Giảm dần về 0 VU.
- **Thời gian chờ:** 1 giây giữa mỗi lần lặp của mỗi VU.
- **Ngưỡng chấp nhận (thresholds):**
 - `http_req_duration p(95) < 1000ms` — 95% yêu cầu hoàn thành trong vòng 1 giây.
 - `http_req_failed rate < 5%` — Tỷ lệ lỗi dưới 5%.
 - `search_duration p(90) < 800ms` — 90% yêu cầu tìm kiếm hoàn thành trong 800ms.
 - `newest_duration p(90) < 500ms` — 90% yêu cầu sách mới hoàn thành trong 500ms.

7.5.3 Kết quả

Kiểm thử tải k6 được thực thi với máy chủ backend đang chạy cục bộ bằng lệnh:

```
k6 run tests/performance/search_load_test.js
```

Bảng 7.5 tổng hợp các chỉ số hiệu năng thu thập được trong toàn bộ 40 giây kiểm thử (113 iterations, 339 HTTP requests).

Bảng 7.5: Kết quả kiểm thử hiệu năng k6

Chỉ số	Kết quả	Ngưỡng	Trạng thái
Thời gian phản hồi trung bình	5.53 ms	—	—
Phân vị p90	6.49 ms	—	—
Phân vị p95	7.08 ms	< 1000ms	Đạt ✓
Số yêu cầu mỗi giây (RPS)	8.30 req/s	—	—
Tỷ lệ lỗi HTTP	0.00%	< 5%	Đạt ✓
Kiểm tra (checks)	100%	—	Đạt ✓

Kết quả cho thấy hệ thống xử lý tốt tải đồng thời 10 người dùng với thời gian phản hồi trung bình chỉ 5.53ms và tỷ lệ lỗi 0%. Tất cả 5 ngưỡng chấp nhận (thresholds) đều đạt yêu cầu, xác nhận hiệu năng của các API công khai đáp ứng tiêu chuẩn vận hành trong điều kiện thực tế.

```

checks.....: 100.00% ✓ 791      X 0
data_received.....: 451 kB  11 kB/s
data_sent.....: 40 kB  970 B/s
✓ error_rate.....: 0.00% ✓ 0      X 226
group_duration.....: avg=258.87ms min=4.01ms med=5.92ms max=1.02s p(90)=1.01s p(95)=1.01s
http_req_blocked.....: avg=20.34µs min=0s med=0s max=1.06ms p(90)=0s p(95)=0s
http_req_connecting.....: avg=15.75µs min=0s med=0s max=951.6µs p(90)=0s p(95)=0s
✓ http_req_duration.....: avg=5.53ms min=3.6ms med=5.6ms max=13.7ms p(90)=6.49ms p(95)=7.08ms
  { expected_response:true }...: avg=5.53ms min=3.6ms med=5.6ms max=13.7ms p(90)=6.49ms p(95)=7.08ms
✓ http_req_failed.....: 0.00% ✓ 0      X 339
http_req_receiving.....: avg=250.42µs min=0s med=0s max=743.9µs p(90)=626.05µs p(95)=638.17µs
http_req_sending.....: avg=6.19µs min=0s med=0s max=574.5µs p(90)=0s p(95)=0s
http_req_tls_handshaking.....: avg=0s min=0s med=0s max=0s p(90)=0s p(95)=0s
http_req_waiting.....: avg=5.28ms min=3.43ms med=5.27ms max=13.16ms p(90)=6.19ms p(95)=6.6ms
http_reqs.....: 339 8.296806/s
iteration_duration.....: avg=2.01s min=2.01s med=2.01s max=2.02s p(90)=2.02s p(95)=2.02s
iterations.....: 113 2.765602/s
✓ newest_duration.....: avg=5.83524s min=4.5361 med=5.6888 max=10.1804 p(90)=6.54408 p(95)=6.95474
✓ search_duration.....: avg=4.742312 min=3.6019 med=4.5028 max=7.3086 p(90)=5.6191 p(95)=5.9343
vus.....: 2 min=1 max=10

running (0m40.9s), 00/10 VUs, 113 complete and 0 interrupted iterations
default ✓ [=====] 00/10 VUs 40s

```

Hình 7.5: Kết quả kiểm thử hiệu năng k6 — tất cả thresholds đạt yêu cầu

7.6 Tổng kết

Chiến lược kiểm thử được áp dụng trong dự án bao gồm bốn tầng bổ sung cho nhau, cùng xác nhận tính đúng đắn, tích hợp, hợp đồng API và đặc tính hiệu năng của Hệ thống quản lý thư viện. Bảng 7.6 cung cấp tổng quan tổng hợp về tất cả kết quả kiểm thử.

Bảng 7.6: Tổng hợp kết quả kiểm thử

Loại kiểm thử	Công cụ	Tổng	Passed	Failed
Kiểm thử đơn vị (BE)	JUnit 5 + Mockito	12	12	0
Kiểm thử đơn vị (FE)	Jest + ts-jest	18	18	0
Kiểm thử tích hợp	SpringBootTest + Testcontainers	3	3	0
Kiểm thử API	Postman Collection Runner	20	20	0
Kiểm thử hiệu năng	k6	Xem Bảng 7.5		

Các bài kiểm thử đơn vị backend và frontend tự động, cùng với bộ kiểm thử tích hợp, đạt tổng cộng 33 test case tự động với tỷ lệ pass 100%. Các kết quả này xác nhận tính ổn định của logic nghiệp vụ cốt lõi—cụ thể

là luồng đặt mượn sách, trả sách và tạo đặt trước—và cho thấy kiến trúc Spring Boot đa module lưu trữ đúng đắn trạng thái domain vào cơ sở dữ liệu PostgreSQL thực.

Bộ test collection Postman gồm 20 request tổ chức thành 8 luồng chức năng xác nhận toàn bộ hợp đồng HTTP giữa client và server. Kiểm thử hiệu năng k6 với 10 người dùng ảo đồng thời cho thấy thời gian phản hồi trung bình 5.53ms và tỷ lệ lỗi 0%, tất cả ngưỡng chấp nhận đều đạt yêu cầu. Cùng nhau, bốn tầng kiểm thử này tạo thành một khung đảm bảo chất lượng toàn diện, phù hợp với một hệ thống quản lý thư viện sẵn sàng đưa vào môi trường sản xuất.

Chương 8: Triển khai hệ thống

8.1 Container hóa với Docker

8.1.1 Giới thiệu Docker

Docker là một nền tảng mã nguồn mở được thiết kế để phát triển, đóng gói và chạy các ứng dụng [23]. Docker giải quyết bài toán “chạy được trên máy tôi nhưng không chạy được trên máy bạn” bằng cách đóng gói ứng dụng cùng toàn bộ phụ thuộc—thư viện runtime, biến môi trường, file cấu hình—vào một đơn vị tiêu chuẩn gọi là *container*. Đối với dự án này, Docker là thành phần thiết yếu vì nó đảm bảo tính nhất quán tuyệt đối giữa môi trường phát triển cục bộ, kiểm thử và sản xuất, đồng thời đơn giản hóa đáng kể quy trình triển khai trên máy chủ.



Hình 8.1: Docker

8.1.2 Cấu hình môi trường phát triển cục bộ

Trong giai đoạn phát triển, Docker Compose được sử dụng để quản lý kiến trúc đa container của hệ thống. File `docker-compose.yml` định nghĩa hai dịch vụ hạ tầng cốt lõi:

- **PostgreSQL 15:** Cơ sở dữ liệu quan hệ chính của hệ thống, được cấu hình với health check để đảm bảo sẵn sàng trước khi ứng dụng kết nối. Dữ liệu được lưu trữ bền vững qua Docker volume `postgres_data`.
- **Kafka 4.0.0 (KRaft mode):** Message broker xử lý giao tiếp bất đồng bộ cho các luồng email và thông báo. Dự án sử dụng chế độ KRaft (không cần ZooKeeper), giảm đáng kể độ phức tạp vận hành. Dữ liệu Kafka được lưu trong volume `kafka_data`.

Ứng dụng Spring Boot và giao diện React được chạy trực tiếp trên máy của nhà phát triển (không trong container) để tận dụng tính năng hot-reload, giúp chu kỳ phát triển nhanh hơn.

```
# Khởi động hạ tầng
docker compose up -d

# Chạy Backend
cd LMS_be && mvn spring-boot:run -pl library-bootstrap

# Chạy Frontend
cd LMS_fe && npm run dev
```

8.2 Dockerfile đa giai đoạn

Để triển khai lên môi trường sản xuất, mỗi thành phần được đóng gói thành Docker image riêng sử dụng kỹ thuật *multi-stage build*—tách biệt giai đoạn biên dịch và giai đoạn chạy nhằm tối thiểu hóa kích thước image cuối cùng [24].

8.2.1 Backend Dockerfile

Dockerfile của Spring Boot bao gồm hai giai đoạn:

- **Giai đoạn Build:** Sử dụng image `maven:3.9-eclipse-temurin-21-alpine`. Các file `pom.xml` được sao chép trước để Docker cache lớp phụ thuộc Maven—chỉ tải lại khi `pom.xml` thay đổi, không phải mỗi lần build. Sau đó toàn bộ mã nguồn được sao chép và build với lệnh `mvn package -DskipTests -pl library-bootstrap -am`.
- **Giai đoạn Runtime:** Sử dụng image `eclipse-temurin:21-jre-alpine`—chỉ chứa Java Runtime Environment, không có Maven hay mã nguồn. File JAR được sao chép từ giai đoạn build sang. Kích thước image cuối cùng giảm đáng kể so với image chứa đầy đủ JDK và Maven.

8.2.2 Frontend Dockerfile

Dockerfile của React cũng sử dụng hai giai đoạn:

- **Giai đoạn Build:** Sử dụng `node:20-alpine`, cài đặt phụ thuộc qua `npm ci` và chạy `npm run build` (Vite) để tạo ra thư mục `/dist` chứa các file tĩnh đã được tối ưu. URL API được truyền vào lúc build qua build argument `VITE_API_BASE_URL=/api/v1`.
- **Giai đoạn Runtime:** Sử dụng `nginx:alpine` phục vụ các file tĩnh từ `/usr/share/nginx/html`. Cấu hình Nginx của frontend được thiết lập để chuyển hướng tất cả các route về `index.html`, hỗ trợ React Router hoạt động đúng khi người dùng truy cập trực tiếp vào URL.

8.3 Nginx — Reverse Proxy

8.3.1 Vai trò của Nginx

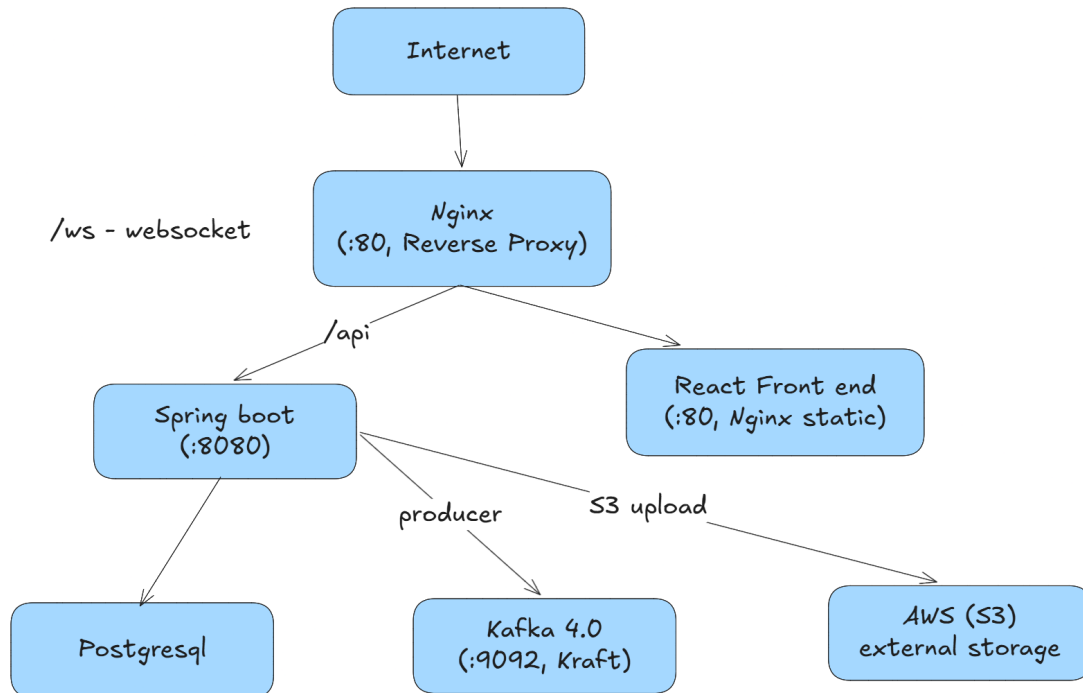
Nginx đóng vai trò là reverse proxy trung tâm, tiếp nhận toàn bộ traffic từ bên ngoài trên cổng 80 và định tuyến đến đúng dịch vụ nội bộ [25]. Kiến trúc này mang lại hai lợi ích chính: thứ nhất, các cổng nội bộ của từng container (8080 cho Backend, 80 cho Frontend) không bị lộ ra internet; thứ hai, client chỉ cần giao tiếp với một địa chỉ duy nhất.

8.3.2 Cấu hình định tuyến

File cấu hình `nginx/nginx.conf` định nghĩa ba quy tắc định tuyến:

- `/ws` → Backend (WebSocket): Định tuyến các kết nối WebSocket STOMP/SockJS đến Spring Boot. Cần thiết lập `proxy_http_version 1.1` và header `Upgrade` để nâng cấp kết nối HTTP thành WebSocket. Thời gian chờ `proxy_read_timeout` được đặt là 3600 giây để duy trì kết nối real-time lâu dài.
- `/api/` → Backend (REST API): Tất cả các yêu cầu REST API được chuyển tiếp đến Spring Boot tại `http://spring-boot:8080`. Header `X-Real-IP` và `X-Forwarded-For` được thêm vào để Spring Boot ghi nhận địa chỉ IP thực của client.

- / → Frontend (React SPA): Toàn bộ các yêu cầu còn lại được chuyển đến container React. Frontend Nginx nội bộ xử lý việc phục vụ file tĩnh và hỗ trợ React Router.



Hình 8.2: Kiến trúc triển khai hệ thống

8.4 Môi trường sản xuất

8.4.1 Docker Compose sản xuất

File `docker-compose.prod.yml` mở rộng cấu hình phát triển bằng cách thêm hai container mới—spring-boot và frontend—vào cùng mạng Docker nội bộ với PostgreSQL, Kafka và Nginx. Tất cả năm dịch vụ giao tiếp với nhau qua tên service (DNS nội bộ của Docker), không cần địa chỉ IP cố định.

Một điểm khác biệt quan trọng so với cấu hình phát triển là giá trị `KAFKA_CFG_ADVERTISED_LISTENERS` được thay đổi từ `PLAINTEXT://localhost:9092` thành `PLAINTEXT://kafka:9092`. Trong môi trường phát triển, Spring Boot chạy trên máy host nên dùng `localhost`; trong môi trường container, Spring Boot nằm trong một container khác và phải kết nối đến Kafka qua tên service Docker.

```
# Triển khai toàn bộ hệ thống
docker compose -f docker-compose.prod.yml up -d --build
```

8.4.2 Quản lý biến môi trường và bảo mật

Tất cả thông tin nhạy cảm của hệ thống được quản lý thông qua file `.env` và không bao giờ được đưa vào mã nguồn (được liệt kê trong `.gitignore`). Bảng 8.1 liệt kê các nhóm biến môi trường chính.

Khi triển khai lên máy chủ, file `.env` được sao chép trực tiếp lên server và được tham chiếu qua chỉ thị `env_file` trong Docker Compose. Cách tiếp cận này đảm bảo không có thông tin đăng nhập nào được nhúng vào Docker image hoặc lộ trong lịch sử git.

Bảng 8.1: Các nhóm biến môi trường của hệ thống

Nhóm	Biến	Mục đích
Cơ sở dữ liệu	DATABASE_URL DATABASE_USERNAME DATABASE_PASSWORD	Kết nối PostgreSQL
JWT	EXP_TOKEN EXP_REFRESH_TOKEN	Thời hạn Access/Refresh token
OAuth2 Google	GOOGLE_CLIENT_ID GOOGLE_CLIENT_SECRET OAUTH2_REDIRECT_URI	Đăng nhập Google
Email	MAIL_SENDER_USERNAME MAIL_SENDER_PASSWORD	Gửi email qua SMTP
AWS S3	AWS_ACCESS_KEY AWS_SECRET_KEY	Lưu trữ file (ảnh bìa, tài liệu)

8.4.3 Flyway Migration tự động

Hệ thống sử dụng Flyway để quản lý phiên bản schema cơ sở dữ liệu. Khi container Spring Boot khởi động, Flyway tự động thực thi các file migration chưa được áp dụng theo thứ tự phiên bản trước khi ứng dụng nhận request. Điều này đảm bảo schema cơ sở dữ liệu luôn đồng bộ với phiên bản ứng dụng mà không cần can thiệp thủ công.

```
db/migration/  
+-- V1__create_schema.sql      # Toàn bộ schema hệ thống  
\-- V2__seed_data.sql          # Dữ liệu mẫu ban đầu
```

8.4.4 Sơ đồ kiến trúc triển khai

Hình 8.3 mô tả chi tiết luồng traffic từ người dùng đến từng thành phần của hệ thống trong môi trường sản xuất.

Toàn bộ năm container chạy trong cùng một Docker network nội bộ, giao tiếp qua DNS tên service. Chỉ có Nginx container được ánh xạ cổng ra ngoài (cổng 80), đảm bảo các dịch vụ nội bộ như PostgreSQL (5432) và Kafka (9092) không bị truy cập trực tiếp từ internet.

8.5 Quy trình triển khai

Quy trình triển khai hệ thống lên một máy chủ Linux mới bao gồm các bước sau:

1. **Chuẩn bị máy chủ:** Cài đặt Docker Engine và Docker Compose Plugin trên máy chủ Ubuntu.

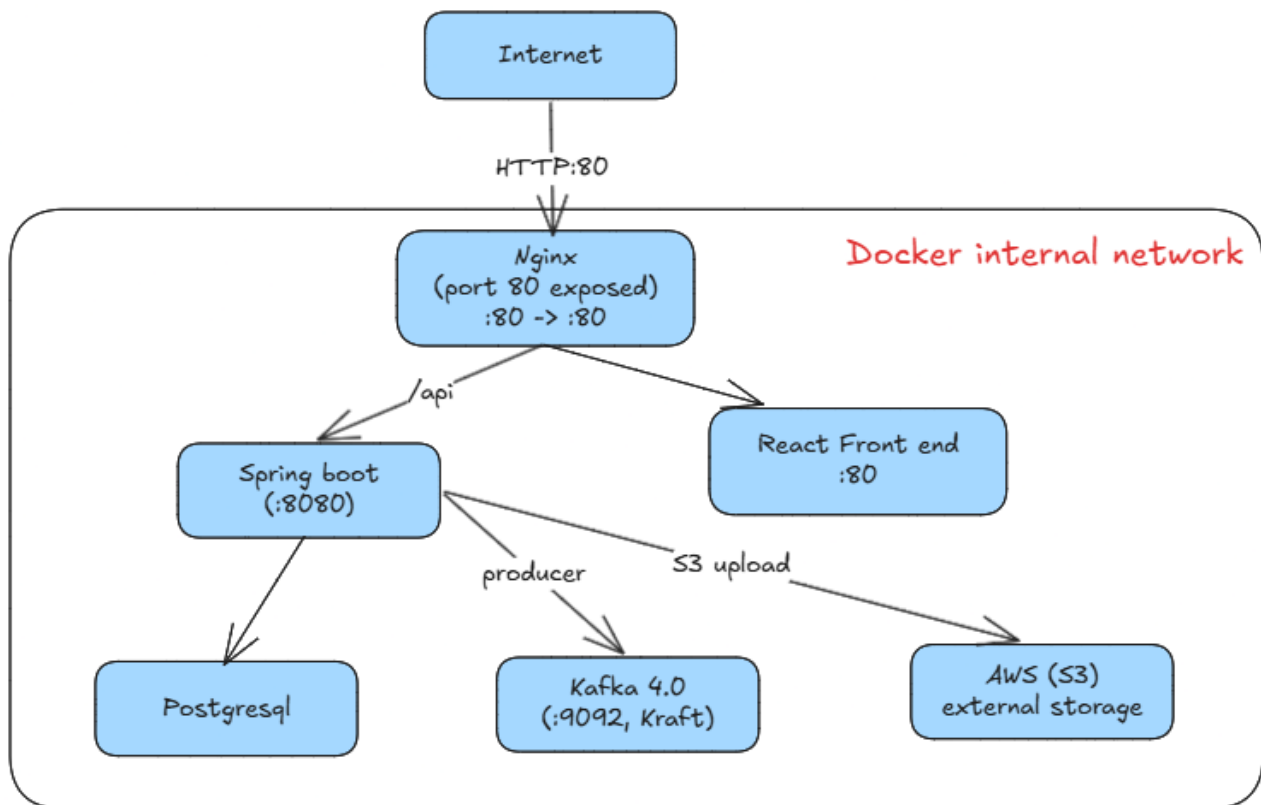
```
curl -fsSL https://get.docker.com | sh
```

2. **Sao chép mã nguồn:** Clone repository về máy chủ.

```
git clone <repository-url>  
cd doan
```

3. **Cấu hình môi trường:** Tạo file .env từ template và điền các giá trị thực tế (database credentials, API keys, v.v.).

```
cp LMS_be/.env.example LMS_be/.env
```



Hình 8.3: Luồng traffic qua các container trong môi trường sản xuất

Chỉnh sửa LMS_be/.env với giá trị thực tế

4. Khởi động hệ thống: Build image và khởi động tất cả container.

```
docker compose -f docker-compose.prod.yml up -d --build
```

5. Kiểm tra trạng thái: Xác nhận tất cả container đang chạy.

```
docker compose -f docker-compose.prod.yml ps
docker compose -f docker-compose.prod.yml logs spring-boot
```

Sau khi khởi động, Flyway tự động chạy các migration, sau đó hệ thống sẵn sàng phục vụ tại địa chỉ `http://<server-ip>`.

8.6 Hạ tầng mạng: VPS, Domain và HTTPS

8.6.1 Máy chủ ảo (VPS)

Để một ứng dụng web có thể truy cập từ internet, hệ thống cần được triển khai trên một máy chủ có địa chỉ IP công cộng hoạt động liên tục 24/7. Giải pháp phổ biến nhất là thuê *Virtual Private Server* (VPS)—một máy chủ ảo được cấp phát tài nguyên CPU, RAM và ổ đĩa riêng biệt trên hạ tầng vật lý của nhà cung cấp dịch vụ đám mây [26].

Đối với hệ thống thư viện này, cấu hình tối thiểu được đề xuất là **2GB RAM** do Kafka yêu cầu khoảng 512MB

bộ nhớ để hoạt động ổn định cùng với Spring Boot và PostgreSQL. Bảng 8.2 so sánh một số nhà cung cấp VPS phổ biến.

Bảng 8.2: So sánh các nhà cung cấp VPS phổ biến

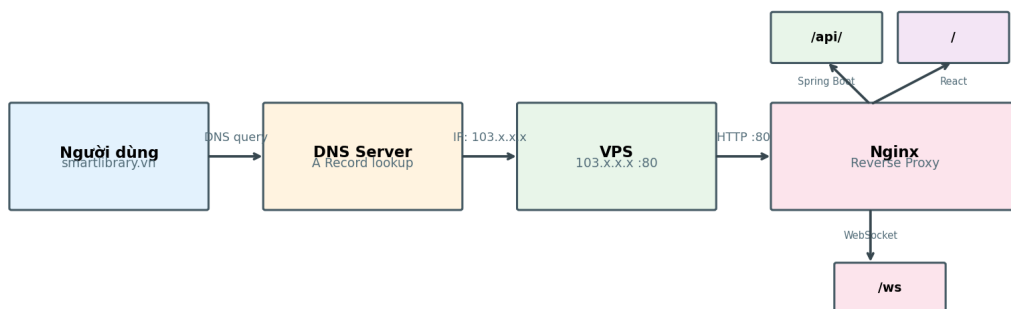
Nhà cung cấp	RAM	Giá	Ghi chú
Hetzner Cloud (CX22)	4 GB	€4/tháng	Giá rẻ, datacenter EU
DigitalOcean (Basic)	1 GB	\$6/tháng	Phổ biến, dễ sử dụng
Vultr (Regular)	1 GB	\$5/tháng	Nhiều khu vực
Oracle Cloud Free Tier	1 GB	Miễn phí	Vĩnh viễn, ARM architecture

8.6.2 Tên miền (Domain)

Sau khi có VPS với địa chỉ IP công cộng (ví dụ: 103.45.67.89), hệ thống đã có thể truy cập trực tiếp qua IP. Tuy nhiên, để có địa chỉ thân thiện và dễ nhớ, cần mua thêm một *tên miền*—ví dụ smartlibrary.vn hoặc lms-hcmut.com. Tên miền được đăng ký thông qua các nhà cung cấp như GoDaddy, Namecheap hoặc các nhà đăng ký tên miền Việt Nam.

Sau khi mua tên miền, cần cấu hình bản ghi DNS loại **A record** trỏ tên miền về địa chỉ IP của VPS. Quá trình phân giải DNS diễn ra tự động, thông thường có hiệu lực trong vòng vài phút đến 24 giờ. Hình 8.4 minh họa luồng kết nối từ người dùng đến hệ thống.

Luồng kết nối từ tên miền đến hệ thống



Hình 8.4: Luồng kết nối từ tên miền đến hệ thống

8.6.3 HTTPS và SSL/TLS Certificate

Các trình duyệt hiện đại cảnh báo người dùng khi truy cập trang web qua HTTP không mã hóa. Để bảo mật dữ liệu truyền tải—đặc biệt là JWT token, thông tin đăng nhập và dữ liệu cá nhân—hệ thống cần được cấu hình HTTPS thông qua SSL/TLS certificate [27].

Let's Encrypt là tổ chức phi lợi nhuận cung cấp SSL certificate hoàn toàn miễn phí và tự động. Công cụ **Certbot** tích hợp trực tiếp với Nginx, tự động xác minh quyền sở hữu tên miền, cấp certificate và cập nhật cấu hình Nginx để lắng nghe trên cổng 443 (HTTPS):

```
# Cài đặt Certbot
apt install certbot python3-certbot-nginx

# Cấp certificate và tự động cấu hình Nginx
certbot --nginx -d smartlibrary.vn -d www.smartlibrary.vn
```

Certificate Let's Encrypt có hiệu lực 90 ngày và được Certbot tự động gia hạn qua cron job, không cần can thiệp thủ công.

8.6.4 Tổng chi phí vận hành

Bảng 8.3 tổng hợp chi phí ước tính để vận hành hệ thống trong môi trường sản xuất với tên miền và HTTPS đầy đủ.

Bảng 8.3: Chi phí vận hành hệ thống ước tính

Thành phần	Chi phí	Chu kỳ	Ghi chú
VPS (Hetzner CX22)	90.000đ	Tháng	4GB RAM, 2 vCPU
Tên miền .com	300.000đ	Năm	Qua Namecheap, GoDaddy
SSL Certificate	Miễn phí	90 ngày	Let's Encrypt, tự gia hạn
AWS S3 (lưu trữ)	23.000đ	Tháng	1GB storage + transfer
Tổng	≈140.000đ	Tháng	Chưa tính domain năm đầu

Chi phí vận hành thấp này cho thấy hệ thống hoàn toàn khả thi để triển khai thực tế trong môi trường trường đại học với ngân sách hạn chế.

Chương 9: Tổng kết

Dự án “Hệ thống Quản lý Thư viện Thông minh tích hợp AI” (Mã dự án: HK251-DAGD1-512) được xây dựng với mục tiêu hiện đại hóa hoạt động quản lý thư viện truyền thống, đồng thời nâng cao trải nghiệm tra cứu và gợi ý sách cho người dùng thông qua các kỹ thuật Trí tuệ nhân tạo. Sau toàn bộ quá trình nghiên cứu, thiết kế, hiện thực và kiểm thử, nhóm đã hoàn thành một hệ thống phần mềm toàn diện từ lớp kiến trúc đến lớp triển khai.

9.1 Kết quả đạt được

9.1.1 Kiến trúc phần mềm

Nhóm đã thiết kế và hiện thực hóa thành công kiến trúc **Modular Monolith** kết hợp **Clean Architecture** cho lõi Backend (Spring Boot). Mã nguồn được phân chia theo ranh giới nghiệp vụ (Bounded Contexts) theo nguyên tắc Domain-Driven Design gồm các module: Quản lý sách, Đọc giả, Lưu thông, Đặt chỗ, Phạt vi phạm và Tương tác người dùng. Bên trong mỗi module, các tầng *Domain*, *Application*, *Infrastructure* và *Presentation* được tổ chức nghiêm ngặt theo Dependency Rule, giúp logic nghiệp vụ độc lập hoàn toàn với Framework và Cơ sở dữ liệu [28].

Giao diện người dùng được xây dựng trên **Next.js** với mô hình Backend-for-Frontend (BFF), đảm nhiệm việc Server-Side Rendering và tổng hợp API calls. Dịch vụ AI được triển khai như một **Autonomous Satellite Microservice** độc lập bằng Python FastAPI, giao tiếp với Backend chính qua REST và Message Queue (RabbitMQ), tách biệt hoàn toàn vòng đời phát triển và mở rộng.

9.1.2 Tích hợp Trí tuệ nhân tạo

- **Tìm kiếm ngữ nghĩa:** Tích hợp thành công mô hình `bkai-foundation-models/vietnamese-bi-encoder` (dựa trên SBERT) để mã hóa văn bản tiếng Việt thành vector 768 chiều. Kết hợp với chỉ mục HNSW trên PostgreSQL (pgvector), hệ thống đạt tốc độ tìm kiếm láng giềng gần nhất (ANN) vượt trội so với phương pháp quét toàn bảng [29].
- **Gợi ý sách cá nhân hóa:** Xây dựng pipeline Gợi ý lai ghép (Hybrid Recommender) sử dụng thư viện LightFM, kết hợp Collaborative Filtering và Content-based Filtering. Hệ thống tự động cập nhật vector người dùng dựa trên lịch sử tương tác và đánh giá, trả về danh sách gợi ý phù hợp theo ngữ cảnh (Trang chủ, Chi tiết sách) [30].
- **Xử lý tài liệu bất đồng bộ:** Xây dựng pipeline OCR và Embedding tự động thông qua Celery Workers, cho phép nhập sách mới kèm file PDF và tự động vector hóa nội dung vào Vector Database mà không gián đoạn dịch vụ.

9.1.3 Kiểm thử

Nhóm áp dụng chiến lược kiểm thử phân tầng nhằm đảm bảo chất lượng xuyên suốt toàn bộ hệ thống:

- **Kiểm thử đơn vị (Unit Testing):** Viết bộ test case cho các Use Case và Domain Service cốt lõi bằng JUnit 5 và Mockito (Backend), Jest (Frontend). Mọi phụ thuộc ngoài đều được thay thế bằng đối tượng giả lập (mock) để đảm bảo tính độc lập và tốc độ thực thi.

- **Kiểm thử tích hợp (Integration Testing):** Sử dụng Testcontainers để khởi tạo môi trường cơ sở dữ liệu thực (PostgreSQL, Redis) trong container tạm thời, kiểm tra toàn bộ luồng nghiệp vụ từ Repository đến Service mà không cần môi trường bên ngoài cố định.
- **Kiểm thử API:** Xây dựng bộ Collection Postman bao phủ các endpoint chính, kiểm tra tính đúng đắn của Request/Response, cơ chế xác thực JWT và xử lý ngoại lệ.
- **Kiểm thử hiệu năng (Performance Testing):** Sử dụng k6 để mô phỏng tải trọng đồng thời, đo đặc Throughput (RPS) và Latency của hệ thống dưới các kịch bản tải trọng khác nhau, xác nhận kiến trúc đáp ứng yêu cầu về hiệu năng trong điều kiện vận hành thực tế.

9.1.4 Triển khai hệ thống

Toàn bộ các dịch vụ (Backend, AI Service, PostgreSQL, Redis, RabbitMQ, NGINX) được đóng gói thành Docker images và điều phối thông qua Docker Compose. Áp dụng kỹ thuật **Multi-stage Build** để tối ưu kích thước image, tách biệt môi trường build và runtime. Hệ thống được triển khai thành công lên máy chủ VPS, sử dụng NGINX làm Reverse Proxy và cấu hình SSL/TLS qua Let's Encrypt, đảm bảo kết nối HTTPS an toàn.

9.2 Nhận xét và đánh giá

Qua quá trình thực hiện, nhóm nhận thấy việc lựa chọn kiến trúc Modular Monolith kết hợp Clean Architecture là phù hợp với quy mô đồ án: đủ nghiêm ngặt để duy trì chất lượng mã nguồn, nhưng không đánh đổi sự phức tạp vận hành của Microservices thuần túy. Mô hình SBERT tiếng Việt chứng tỏ hiệu quả rõ rệt so với tìm kiếm từ khóa truyền thống, đặc biệt với các truy vấn ngữ nghĩa và từ đồng nghĩa.

Hạn chế lớn nhất của hệ thống hiện tại là chất lượng gợi ý phụ thuộc nhiều vào lượng dữ liệu tương tác, dẫn đến vấn đề Cold Start đối với người dùng và đầu sách mới. Ngoài ra, chi phí tính toán của việc mã hóa (encoding) toàn bộ kho sách khi cập nhật lớn đòi hỏi cơ chế cập nhật gia tăng (incremental update) trong tương lai.

Nhìn chung, dự án đã đạt được mục tiêu đề ra: xây dựng một hệ thống quản lý thư viện hoàn chỉnh, có khả năng vận hành thực tế, tích hợp thành công các kỹ thuật AI hiện đại phù hợp với đặc thù tiếng Việt, và được kiểm thử nghiêm túc trước khi triển khai.

Chương 10: Tài nguyên và Mã nguồn dự án

Để phục vụ cho quá trình thiết kế, kiểm thử và đánh giá, nhóm thực hiện xin đính kèm các đường dẫn đến tài nguyên của dự án:

Loại tài nguyên	Đường dẫn truy cập
Mã nguồn (Source Code)	frontend: https://github.com/imsythang/LMS_fe backend: https://github.com/quangthangk4/LMS_BE
Thiết kế giao diện (Figma)	https://www.figma.com/design/aSHkyebIWfWN7LMgTeY0be/Library-Management-System?node-id=0-1&t=mf5nzMKPeV3M6Vyp-1

Bảng 10.1: Danh sách các tài nguyên trực tuyến của hệ thống

Tài liệu tham khảo

- [1] M. Richards and N. Ford, *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media, Inc., 2020.
- [2] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Pearson Education, 2018.
- [3] L. N. Thanh, "Clean architecture - giả không lồ sinh sau đẻ muộn," <https://tech.cybozu.vn/clean-architecture-ga-khong-lo-sinh-sau-de-muon-phan-1-51c96/>, 2025, truy cập: 17-06-2025.
- [4] A. Kane, *pgvector: Open-source vector similarity search for PostgreSQL*, 2024, accessed: 2026-05-01. [Online]. Available: <https://github.com/pgvector/pgvector>
- [5] J. Pennington, R. Socher, and C. D. Manning, "GloVe: Global vectors for word representation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics (ACL), 2014, pp. 1532–1543. [Online]. Available: <https://nlp.stanford.edu/pubs/glove.pdf>
- [6] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," *arXiv preprint arXiv:1301.3781*, 2013.
- [7] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-Networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 3982–3992.
- [8] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional Transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Minneapolis, Minnesota: Association for Computational Linguistics, 2019, pp. 4171–4186.
- [9] R. Burke, "Hybrid recommender systems: Survey and experiments," *User Modeling and User-Adapted Interaction*, vol. 12, no. 4, pp. 331–370, 2002.
- [10] Y. Hu, Y. Koren, and C. Volinsky, "Collaborative filtering for implicit feedback datasets," in *8th IEEE International Conference on Data Mining (ICDM 2008)*. IEEE, 2008, pp. 263–272.
- [11] G. J. Myers, C. Sandler, and T. Badgett, *The Art of Software Testing*, 3rd ed. John Wiley & Sons, 2011.
- [12] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley, 2010.
- [13] G. Meszaros, *xUnit Test Patterns: Refactoring Test Code*. Addison-Wesley, 2007.
- [14] M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed. Addison-Wesley, 2018.
- [15] Postman Inc., "What is API testing?" <https://www.postman.com/api-platform/api-testing/>, 2024, accessed: 2026-05-01.
- [16] I. Molyneaux, *The Art of Application Performance Testing*, 2nd ed. O'Reilly Media, 2014.
- [17] JUnit Team, "JUnit 5 user guide," <https://junit.org/junit5/docs/current/user-guide/>, 2024, accessed:

2026-05-01.

- [18] Mockito Contributors, “Mockito framework site,” <https://site.mockito.org/>, 2024, accessed: 2026-05-01.
- [19] Meta Platforms, Inc., “Jest: Delightful JavaScript testing,” <https://jestjs.io/>, 2024, accessed: 2026-05-01.
- [20] Testcontainers Contributors, “Testcontainers for Java,” <https://java.testcontainers.org/>, 2024, accessed: 2026-05-01.
- [21] Postman Inc., “Postman API platform,” <https://www.postman.com/>, 2024, accessed: 2026-05-01.
- [22] Grafana Labs, “k6: Open source load testing tool,” <https://k6.io/>, 2024, accessed: 2026-05-01.
- [23] Docker Inc., “Docker: Accelerated container application development,” <https://www.docker.com/>, 2024, accessed: 2026-05-01.
- [24] —, “Use multi-stage builds,” <https://docs.docker.com/build/building/multi-stage/>, 2024, accessed: 2026-05-01.
- [25] F5, Inc., “NGINX documentation,” <https://nginx.org/en/docs/>, 2024, accessed: 2026-05-01.
- [26] DigitalOcean, LLC, “What is a VPS?” <https://www.digitalocean.com/resources/articles/what-is-a-vps>, 2024, accessed: 2026-05-01.
- [27] Internet Security Research Group, “Let’s encrypt: Free, automated, and open certificate authority,” <https://letsencrypt.org/>, 2024, accessed: 2026-05-01.
- [28] S. Newman, *Building microservices: designing fine-grained systems*. " O'Reilly Media, Inc.", 2015.
- [29] N. Reimers and I. Gurevych, “Sentence-bert: Sentence embeddings using siamese bert-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019, pp. 3982–3992.
- [30] C. C. Aggarwal, *Recommender systems*. Springer, 2016.
- [31] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE transactions on pattern analysis and machine intelligence*, vol. 42, no. 4, pp. 824–836, 2018.
- [32] E. Evans, *Domain-driven design: tackling complexity in the heart of software*. Addison-Wesley Professional, 2004.
- [33] D. Q. Nguyen and N. A. Tuan, “Phobert: Pre-trained language models for vietnamese,” in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020, pp. 1037–1042.